

SAPIENZA UNIVERSITY OF ROME

DOCTORAL THESIS

Model Merging: Foundations and Algorithms

Author:

Donato CRISOSTOMI

Supervisor:

Prof. Emanuele RODOLÀ

Prof. Pietro LIÒ

*A thesis submitted in fulfillment of the requirements
for the degree of Doctor of Philosophy*

in the

GLADIA research lab
Department of Computer Science
Faculty of Information Engineering, Informatics, and Statistics



Acknowledgements

First and foremost, I thank my wife, Jovita. Throughout a journey often marked by challenges and uncertainty, you have been my constant source of strength. Your patience, love, and support have meant more than I can express.

I am deeply grateful to my advisor, Prof. Emanuele Rodolà, and to the GLADIA Lab as a whole. Thank you for your trust, your vision, and for fostering an environment in which I could grow from a student into a researcher. The intellectual freedom, guidance, and inspiration I found under your mentorship have shaped me in lasting ways.

I also sincerely thank my ELLIS co-advisor, Prof. Pietro Liò, and everyone at the Computer Lab. Thank you for welcoming me so warmly during my visits and for the many memories I will carry with me long after this chapter closes.

A research journey is often defined by those who take the time to guide it. In this respect, I am especially grateful to Prof. Iacopo Masi and Prof. Simone Scardapane. Your mentorship, advice, and example were invaluable when I was first finding my way in academia.

To my fellow GLADIA lab members, Daniele Baieri, Irene Cannistraci, Irene Tallini, Andrea Santilli, Luca Moschella, Marco Pegoraro, Filippo Maggioli, Emiliano Postolache, Antonio Ricciardi, Antonio Norelli, Silvio Severino, Michele Mancusi, Michele Miranda, Valentino Maiorca, Emanuele Rossi, Marco Fumero, Luca Cosmo, Riccardo Marin, Simone Melzi, Giorgio Strano, Francesco Pappone, Simone Facchiano, and Giorgio Mariani: thank you. Your camaraderie, conversations, and shared moments made the lab a lively and rewarding place to work.

Finally, I thank the colleagues with whom I had the privilege of working on model merging research: Antonio Gargiulo, Tommaso Mencattini, Giorgos Nikolaou, Alessandro Zirilli, Luca Zhou, Daniele Solombrino, Davide Marincione, Adrian Robert Minut, and Maria Sofia Bucarelli. I also thank Prof. Florian Bernard and Prof. Fabrizio Silvestri for the insightful exchanges and collaborations we shared across several of these projects. Working alongside all of you, exchanging ideas, and helping push this field forward has been one of the most rewarding parts of my PhD.

SAPIENZA UNIVERSITY OF ROME

Abstract

Faculty of Information Engineering, Informatics, and Statistics
Department of Computer Science

Doctor of Philosophy

Model Merging: Foundations and Algorithms

by Donato CRISOSTOMI

Modern deep learning typically treats models as separate artifacts: trained independently, specialized for particular purposes, and replaced when improved versions appear. This thesis studies an alternative paradigm, *model merging*: combining independently trained neural networks into a single model directly in weight space, without access to additional training data and with little or no optimization.

The thesis is organized around two regimes. In the *single-task setting*, where models share a common objective but differ in initialization, we introduce C^2M^3 , a cycle-consistent merging algorithm grounded in Frank-Wolfe optimization. C^2M^3 aligns collections of networks into a shared parameter space that serves as a reference-free aggregation point, making weight averaging meaningful without privileging any one model as the anchor.

In the *multi-task setting*, where models are fine-tuned for distinct downstream tasks, we first develop a theoretical account of *task vectors*, the parameter differences between a fine-tuned model and its pretrained initialization. We show that task vectors admit a gradient-based interpretation under standard assumptions, clarifying both the success and the limits of task arithmetic. This gradient view has a direct consequence: gradients are known to exhibit low-rank structure, and task vectors inherit this property. We formalize and exploit this low-rank structure through *Task Singular Vectors* (TSV), a decomposition that supports both model compression and interference reduction in TSV-Merge. We then present *MASS*, an input-adaptive routing mechanism that uses TSV geometry to direct inference through task-relevant subspaces. Finally, we introduce *MERGE*³, an evolutionary merging framework that incorporates Item Response Theory to reduce evaluation costs by up to $50\times$ while preserving solution quality.

Taken together, these contributions place model merging on firmer theoretical and algorithmic foundations, advancing a paradigm in which learned capabilities can be composed, reused, and extended across models.

Contents

Acknowledgements	iii
Abstract	v
Notation	xv
I Introduction	1
1 Introduction	3
1.1 Motivation	3
1.1.1 Instant knowledge transfer	3
1.1.2 Towards a model algebra	3
1.1.3 Weights as a data modality	4
1.1.4 The need for reuse	4
1.1.5 A democratic alternative to fine-tuning	4
1.1.6 Model merging as a pre-training regularizer	5
1.2 Landscape of merging approaches	5
1.2.1 Merging of models independently trained on the same task	5
1.2.2 Merging of models independently fine-tuned on the same task starting from the same base model	6
1.2.3 Merging of models independently fine-tuned on different tasks starting from the same base model	6
1.2.4 Merging to mitigate catastrophic forgetting	6
1.2.5 Merging during pre-training	7
1.3 Thesis outline and contributions	7
II Single-Task Model Merging	9
2 Foundations of Single-Task Merging	11
2.1 Mode connectivity	11
2.1.1 Linear mode connectivity	12
2.2 Weight-space symmetries	13
2.3 Neuron matching	14
3 Cycle-Consistent Alignment of Multiple Models	17
3.1 From pairwise to multi-model alignment	17
3.2 Designing the cycle-consistent objective	19
3.3 Empirical evaluation	20
3.4 Summary and outlook	25

III	Multi-Task Model Merging	27
4	Foundations of Multi-Task Merging	29
4.1	Background on multi-task merging	29
4.1.1	Task arithmetic	29
4.1.2	Mitigating multi-task interference	29
4.1.3	Routing-based merging	30
4.1.4	Optimization-based model merging	31
4.1.5	Understanding why task arithmetic works	31
5	Task Vectors as Approximate Gradients	33
5.1	Motivation and Overview	33
5.2	Formalizing the task vector–gradient equivalence	33
5.2.1	Dominance of the first epoch	35
5.2.2	Proof sketch	35
5.2.3	Bounding the second-order deviation term	37
5.3	Scope and limitations	38
5.4	Implications for merging practice	38
5.4.1	Parameter space trajectory	39
5.4.2	Task proficiency is not mergeability	39
5.5	Summary and outlook	39
6	Exploiting the Low-Rank Structure of Task Vectors	41
6.1	From flat vectors to layer-wise structure	41
6.2	Task singular vectors and their properties	42
6.2.1	Layer-wise task matrices and their SVD	42
6.2.2	Low-rank nature of layer task matrices	43
6.2.3	Singular task interference	44
6.3	Compression and interference reduction	44
6.3.1	TSV for compression	45
6.3.2	TSV for model merging	45
6.4	Experimental results	46
6.4.1	Model merging results	46
6.4.2	Compression results	47
6.5	Ablations and analysis	47
6.5.1	Ablation study	47
6.5.2	Per-layer task interference	49
6.5.3	Choice of the scaling coefficient	49
6.5.4	Effect of task interference reduction	49
6.6	Summary and outlook	50
7	Input-Adaptive Merging via Subspace Routing	51
7.1	Beyond static merging	51
7.2	Adaptive subspace selection	51
7.2.1	Fixed merging	53
7.2.2	Integrating routing	53
	Projection-based routing	53
	Accounting for redundant directions	54
7.2.3	Adaptive merging and inference	54
7.2.4	Residual minimization as maximum a posteriori estimation	55
7.3	Experimental results	55

7.3.1	Merging performance	55
7.3.2	Choosing a routing layer	57
7.3.3	Comparison with other routers	57
7.3.4	Interpreting task singular vectors	58
7.4	Summary and outlook	58
8	Democratizing Evolutionary Merging	61
8.1	The computational bottleneck of evolutionary merging	61
8.2	Evolutionary algorithms and item response theory	62
8.3	The MERGE ³ framework	63
8.3.1	Extract & estimate	63
8.3.2	Evolve: Performance estimator	64
8.3.3	Evolve: Evolutionary search	66
8.4	Experimental results	66
8.4.1	Validating estimators	67
8.4.2	Cross-lingual transfer of mathematical skills	68
8.4.3	Ablation study: self-merging	69
8.4.4	Evolving a multilingual model	69
8.5	Theoretical guarantees	70
8.5.1	Part I: ϵ -stable estimators and ϵ -optimality preservation	70
8.5.2	Part II: Theoretical guarantees for MP-IRT	71
8.5.3	Part III: Performance preservation via MP-IRT	71
8.6	Computational cost	72
8.7	Summary and outlook	72
IV	Applications, Outlook, and Conclusions	75
9	Applications and Impact	77
10	Future Directions	81
11	Conclusions	85
11.1	Summary of Contributions	85
11.2	Overarching Themes	87
11.3	Limitations and Open Questions	88
V	Appendix	91
A	Additional material on Chapter 3	93
A.1	Additional details	93
A.1.1	Extended related work	93
A.1.2	Pairwise Frank-Wolfe algorithm	94
A.1.3	MergeMany algorithm	94
A.1.4	Convergence and efficiency	95
A.1.5	Architectural details	96
A.1.6	Datasets, hyperparameters and hardware details	97
A.1.7	Proofs	98
A.2	Additional experiments	99
A.2.1	Pairwise model matching and merging	99
	ResNet with BatchNorm	99

A.2.2	Initialization strategies	100
A.2.3	Variance of the results in Git Re-Basin	100
A.2.4	Large-scale matching: ResNet50s trained over ImageNet	100
A.2.5	Federated learning	101
A.3	Additional analysis	103
A.3.1	Similarity of models	103
	Representation-level similarity	103
	Weight-level similarity	103
A.3.2	Merging different subsets	103
A.4	Discussion	105
A.4.1	On the cycle-consistency of C^2M^3	105
A.4.2	Limitations	105
A.4.3	Societal impact and broader vision	105
B	Additional material on Chapter 5	109
B.1	Proofs	109
B.1.1	Proofs of Proposition 5.2.3 and Theorem 5.2.1	109
B.1.2	Proofs of Theorem 5.2.6	112
B.1.3	Upper bounding the multi-task loss	116
C	Additional material on Chapter 6	119
C.1	Illustrative example: merging two tasks with rank-1 approximation	119
C.2	Additional details	120
C.2.1	Implementation details and computational resources	120
C.2.2	Hyperparameter settings	121
C.2.3	Storage cost calculation	122
C.3	Proofs	123
C.3.1	Characterization of the similarity matrices	123
C.3.2	Observations	125
C.3.3	Proof of Theorem 6.5.1	125
C.4	Additional experimental results	127
C.4.1	Per-dataset performance metrics	127
C.4.2	Extended analysis	127
	Whitening vs. SVD	127
	Impact of rank	128
	Extended ablation study	128
	Effect of task interference	130
	Detailed per-layer task interference	130
	Compression analysis	130
	Test-time adaptation	130
C.5	Theoretical motivations and analysis	131
C.5.1	Theoretical foundation – empirical design	131
C.5.2	Heuristic interference measure	131
D	Additional material on Chapter 7	135
D.1	Extended related work	136
D.1.1	Model merging	136
D.1.2	MoErging	136
D.2	Additional details	136
D.2.1	Storage and compute overhead	137
D.2.2	Implementation	137

D.2.3	Benchmarks and datasets	137
D.2.4	Evaluation measures	137
D.2.5	Architectures	138
D.2.6	Adapting oracle MoErging methods	138
D.3	Additional experiments and results	138
D.3.1	Hyperparameter settings	138
D.4	Propositions and proofs	142
E	Additional material on Chapter 8	147
E.1	Mergenetic	147
E.2	Additional Details	147
E.2.1	IRT Fitting Details	147
E.2.2	Experimental Details	150
Cross-lingual transfer	150	
Multi-lingual transfer	151	
Ability and Performance Estimator	151	
E.3	Additional Experiments	151
E.3.1	Extract step	151
IRT clustering	151	
Representation clustering	151	
Experiments	153	
E.3.2	Estimation step	154
Additional experiment for ability estimator	154	
Additional experiment for performance estimator	154	
Hyperparameter analysis for performance estimator	154	
Spearman correlation	158	
E.3.3	Evolve step	158
Comparison with in-context learning	158	
Analyzing negative transfer	160	
Time & FLOPs requirements for evolutionary merging	161	
E.4	Mathematical proofs	162
E.4.1	Proof of Theorem 8.5.2	162
E.4.2	Proof of Theorem 8.5.4	163
E.4.3	Proof of Proposition 8.5.5	164
E.4.4	Proof of Theorem 8.5.2 via Proposition 8.5.5	165

List of Figures

1.1	Growth of models hosted on Hugging Face over time.	5
2.1	Progressive understanding of neural network loss surfaces	11
2.2	Loss barrier between two modes θ_A and θ_B	13
2.3	Schematic of <code>Git Re-Basin</code> neuron alignment	14
3.1	Cycle-consistent multi-model merging via universe permutations	18
3.2	Cycle consistency comparison of permutation methods	18

3.3	Loss landscape projection when matching three modes	21
3.4	Interpolation accuracy variance across optimization seeds	22
3.5	Weight cosine similarity before and after universe mapping	22
3.6	Interpolation curves of VGG models in the universe.	23
3.7	Accuracy and loss vs. number of merged models	24
3.8	Effect of architecture width on model merging	24
3.9	Merging accuracy: fixed reference model vs. universe space	24
3.10	Linear mode connectivity before and after universe mapping	25
5.1	Task vectors as negative gradients under single-epoch GD	34
5.2	Task arithmetic accuracy: 1 epoch vs. converged.	35
5.3	Analysis of first-epoch gradients.	36
5.4	Checkpoint projection of different strategies.	39
6.1	TSV-M merging accuracy on ViT-L-14	42
6.2	Accuracy vs. fraction of retained singular components	43
6.3	Task interference similarity matrices for 8 tasks	44
6.4	ViT-B-32 merging accuracy across task sets	45
6.5	Procrustes orthogonalization approximation error	48
6.6	Singular Task Interference across layers	49
6.8	Effect of interference reduction on accuracy	49
6.7	Normalized accuracy for different scaling coefficients	50
7.1	Overview of the MASS adaptive merging framework	52
7.2	Projection of activations onto task singular vector spans	54
7.3	Per-dataset accuracies.	56
7.4	Per-layer task accuracies for ViT-B-32	57
7.5	Batched accuracy.	57
7.6	Decoding task singular vectors as text	58
8.1	Accuracy vs. fitness evaluation FLOPs on Japanese GSM8K	62
8.2	Performance estimator error vs. sample size	64
8.3	Ability estimator cosine similarity comparison	66
8.4	Cross-lingual math skill transfer via model merging	67
8.5	Accuracy of merged models for Japanese GSM8K.	68
8.6	Self-merging ablation on Italian GSM8K	69
A.1	Objective values during optimization	95
A.2	Step sizes during the optimization.	96
A.3	First 6 steps of Frank-Wolfe for one permutation matrix	97
A.4	Pairwise merging accuracies for ResNet50 models	102
A.5	CKA and Euclidean distances of RESNET20 representations	104
A.6	Euclidean distance of RESNET20 weights	104
A.7	Accuracy when merging different model subsets	106
C.1	Absolute accuracy of merged ViT-B-16 over 8, 14, and 20 tasks	120
C.2	Absolute accuracy of merged ViT-L-14 over 8, 14, and 20 tasks	121
C.3	Mean accuracy of ViT-B-16 by fraction of retained singular components	122
C.4	Mean accuracy of ViT-L-14 by fraction of retained singular components	123
C.5	STI and normalized accuracy for ViT-B-16	127
C.6	Per-layer Singular Task Interference in ViT-B-32	128
C.7	STI and normalized accuracy for ViT-L-14	129

C.8	Accuracy vs. compression budget for ViT-L-14	130
C.9	Per-task accuracy of ViT-B-32 by retained singular components	132
C.10	Confusion matrices for merged ViT-L-14 over 20 tasks	133
D.1	ViT-L-14 per-layer task accuracies.	138
D.3	Normalized task accuracies for the 8 tasks benchmark	140
D.2	Hyperparameter sensitivity.	140
D.4	Normalized task accuracies for the 14 tasks benchmark	141
D.5	Per-layer task accuracies for ViT-B-32 (first 4 datasets)	141
D.6	Per-layer task accuracies for ViT-B-32 (second 4 datasets)	141
D.7	Decoded task singular vectors for additional datasets	142
E.1	Sample extractor error across languages	155
E.2	Sample extractor error across merges	155
E.3	Ability estimator Euclidean distance across languages	156
E.4	Ability estimator cosine similarity across languages	156
E.5	Ability estimator cosine similarity across tasks	157
E.6	Performance estimator error on WINOGRANDE and HELLASWAG	157
E.7	Spearman rank correlations for performance estimators	159
E.8	Negative Transfer Rate across languages	161

List of Tables

3.1	Accuracy when merging five models with different initializations	23
6.1	Model merging benchmark results	46
6.2	TSV-C compression benchmark results	47
6.3	Ablation of low-rank approximation and interference reduction	47
7.1	MASS model merging benchmark results	56
7.2	Merging results on GLUE benchmark	56
7.3	Average normalized accuracy for different routers.	57
8.1	Evolved multilingual model accuracy on ARC	69
8.2	Comparison of Evolve methods by cost and accuracy	72
9.1	Applicability and open challenges for the deployment scenarios discussed in Chapter 9.	80
A.1	Wall-clock time for merging ResNet20 models	96
A.3	Pairwise loss barriers on CIFAR10	99
A.2	Pairwise loss barriers on CIFAR100	99
A.4	Loss barriers for ResNet20 with <i>BatchNorm</i>	100
A.5	Test barrier with different initialization strategies	100
A.6	Variance of <i>Git Re-Basin</i> across optimization seeds	101
A.7	Merging ResNet50 models on ImageNet	101
A.8	Federated learning accuracy over 50 rounds with 20 local epochs	102

A.9	Federated learning accuracy over 10 rounds with 30 local epochs	102
C.1	Ablation study for TSV-Merge on ViT-B-16	124
C.2	Ablation study for TSV-Merge on ViT-L-14	125
D.1	Relative parameter increase per method	137
D.2	Dataset image sizes and sample counts	144
D.3	ViT architecture comparison	144
D.4	Average normalized accuracy for different routers.	145
E.1	Supported merging methods in Mergenetic	147
E.2	Supported evolutionary algorithms in Mergenetic	148
E.3	Mistral-based models used in experiments	149
E.4	Notation used in chapter 8.	152
E.5	Absolute estimation error with adaptive interpolation	158
E.6	Multilingual ARC accuracy: ICL vs. evolutionary merging	160
E.7	Comparison of Evolve methods by cost and accuracy	161
E.8	Evaluation throughput by sample size on GSM8K	162
E.9	Evaluation and merge time across GPU models	162

Notation

General mathematics

$\mathbb{R}^d$	Euclidean space of dimension d
$\ x\ _2$	Euclidean norm of vector x
$\ W\ _F$	Frobenius norm of matrix W
$\ W\ _2$	Spectral norm of matrix W
$\langle A, B \rangle$	Frobenius inner product between matrices A and B
I	Identity matrix
$\mathbb{E}[\cdot]$	Expectation
$\text{std}(\cdot)$	Standard deviation
$\text{vec}(A)$	Vectorization of matrix A (stacking columns)
$\mathbb{1}_{[\cdot]}$	Indicator function
$\text{span}(V)$	Subspace spanned by the columns of V

Data and tasks

$\mathcal{X}$	Input space
$\mathcal{Y}$	Output space
(x_i, y_i)	Training example (input, label) pair
$\mathcal{D}$	Dataset
T	Number of tasks, or set of tasks
t	Task index
n_t	Number of training samples for task t

Neural network

$\mathcal{W}$	Weight space
$f(\cdot; \theta)$	Neural network parameterized by θ
θ	Model parameters (weights)
θ_{pre}	Pretrained (base) model parameters
θ_t	Parameters fine-tuned on task t
L	Number of layers
ℓ	Layer index
d_ℓ	Width of layer ℓ
$W^{(\ell)}$	Weight matrix at layer ℓ
b_ℓ	Bias vector at layer ℓ
ϕ	Activation function
$\mathbf{z}_\ell$	Intermediate embedding (hidden representation) at layer ℓ

Loss and optimization

$\ell(x, y, \theta)$	Per-sample loss function
$\mathcal{L}(\theta)$	Loss function
$\bar{L}_t(\theta)$	Empirical loss for task t : $\frac{1}{n_t} \sum_{i=1}^{n_t} \ell(x_i, y_i, \theta)$
$\nabla L_t(\theta)$	Gradient of loss for task t
$\nabla^2 \bar{L}_t$	Hessian of empirical loss for task t
η	Learning rate

Task vectors and model merging

τ_t	Task vector for task t : $\tau_t := \theta_t - \theta_{\text{pre}}$
α	Scaling coefficient for task vectors
λ	Interpolation parameter between two models
θ_{MT}	Merged multi-task model parameters
Δ_i	Per-layer task matrix for task i : $\theta_i^{(\ell)} - \theta_{\text{pre}}^{(\ell)}$
$\hat{\Delta}_i$	Low-rank approximation of task matrix Δ_i

Permutation alignment

P_ℓ	Permutation matrix at layer ℓ
P^{pq}	Permutation mapping from model q to model p
P^p	Object-to-universe permutation for model p
$\mathbb{P}$	Set of all permutation matrices
$\text{LAP}(\cdot)$	Linear Assignment Problem solver

SVD and low-rank structure

U_i, Σ_i, V_i	SVD of task matrix: $\Delta_i = U_i \Sigma_i V_i^\top$
σ_j^i	j -th singular value of task i
u_j^i, v_j^i	j -th left and right singular vectors of task i
k	Number of retained singular components (rank)
STI	Singular Task Interference measure

Routing and adaptive merging

$g_i(\mathbf{x})$	Per-task gating function for input $\mathbf{x}$
Ω	Subset of relevant tasks selected by the router
r_i	Euclidean residual: $\ \mathbf{z}_\ell - \text{Proj}_{V_i}(\mathbf{z}_\ell)\ _2$
h_i	Classification head for task i
C_i	Number of classes for task i
$\text{Proj}_V(\mathbf{x})$	Orthogonal projection of $\mathbf{x}$ onto subspace spanned by columns of V

Item Response Theory

γ_m	Latent ability vector for model m
a_i	Discrimination (ability dimensions needed to answer example i)
β_i	Difficulty parameter for example i
Y_{im}	Binary correctness of model m on example i
ξ	Interpolation coefficients for latent abilities
$F(\theta; D)$	Fitness/performance of parameters θ on dataset D

Part I

Introduction



“The whole is other than the sum of the parts.”

— Kurt Koffka

Chapter 1

Introduction

1.1 Motivation

The broad applicability of model merging has driven its rapid adoption across diverse domains, spurred by a variety of compelling motivations. This section explores these fundamental drivers, progressing from high-level conceptual inspirations to immediate, practical use cases.

1.1.1 Instant knowledge transfer

In one of his most popular talks¹, Prof. G. Hinton likened human learning to knowledge distillation in neural networks: as a student listens to a teacher, they adapt their internal cognitive circuitry to eventually reproduce the teacher’s knowledge. Regardless of the accuracy of this biological analogy, it highlights a shared limitation of both human learning and traditional model distillation: the process is inherently slow, effortful, and computationally expensive.

In contrast, distributed training enables the instantaneous transfer of information (in the form of a gradient) across different model instances exposed to varying data distributions. This mechanism is precisely what permits modern Large Language Models (LLMs) to effectively train on internet-scale datasets. Ultimately, neural models possess a distinct advantage over human cognition: *they allow instant knowledge transfer*. Because neural networks can share the same underlying architecture, it is trivial to transfer learned information between instances, a feat that is biologically impossible for human brains due to their intrinsic structural differences. Hinton identifies this capability as a critical advantage that could eventually propel silicon-based intelligence past its biological counterpart. From this perspective, gradient aggregation in distributed training is itself a rudimentary form of model merging: each worker’s update encodes knowledge learned from its local data shard, and aggregating them yields a model that synthesizes knowledge from all shards simultaneously. Model merging is thus not an exotic post-hoc technique, but *a primitive built into the very fabric of how modern neural networks are trained*, with gradient averaging being just one of its many possible implementations.

1.1.2 Towards a model algebra

“Knowledge transfer” remains a somewhat ambiguous term, prompting deeper questions: What specific knowledge is actually being transferred, and how should “knowledge” be formally defined in this context? While a rigorous definition is deferred to later chapters, we can intuitively explore this concept. If two distinct models possess different task-solving abilities (for instance, one excels at MNIST classification while the other is specialized for CIFAR10), the ideal merged model would seamlessly perform classification across the union of both

¹Prof. Geoffrey Hinton: *Two Paths To Intelligence*. <https://www.youtube.com/watch?v=rGgGOccMEiY>

datasets (MNIST $\cup$ CIFAR10). Intuitively, this process resembles an arithmetic addition or an averaging of the two models.

A natural follow-up is whether a “subtraction” operation between models makes conceptual sense. In specific scenarios, the answer is yes. For example, if a model is fine-tuned on toxic data, subtracting it² from the base model actively reduces toxicity in the resulting network [78]. More generally, model difference can be leveraged for targeted machine unlearning. As will be demonstrated in part III, these operations can even facilitate complex analogies. Taken together, these capabilities point toward the development of a cohesive “model algebra,” where models can be dynamically added and subtracted on the fly to synthesize entirely new functional networks.

1.1.3 Weights as a data modality

By enabling meaningful algebraic operations between model weights, model merging shares profound connections with the emerging field of weight-space learning [139]. This paradigm shift treats model parameters not merely as static configurations, but as first-class citizens that act as raw data within novel learning pipelines. As evidence of this synergy, weight-space techniques are already being utilized to facilitate model merging [116]. Furthermore, theoretical insights pioneered in the model merging literature, such as the study of neuron permutation symmetries, have since become foundational pillars of weight-space methodologies [87, 90, 100].

1.1.4 The need for reuse

Stepping away from theoretical conceptualization, model merging is heavily driven by immediate, practical necessities. The sheer volume of models hosted on platforms like Hugging Face³ has grown exponentially since their inception (fig. 1.1). Even accounting for identical re-uploads and inactive repositories, this proliferation is staggering. In the face of such abundance, we must ask: is it still necessary to train new models from scratch?

While fine-tuning remains essential to inject current knowledge and recent facts into models, many practical applications do not require constant data updates. Instead, these use cases can be more efficiently addressed by reusing and repurposing existing models through merging, circumventing the need for computationally expensive tuning. This shift towards reuse has also begun to reshape how model artifacts are versioned and shared: frameworks such as *Git-Theta* [88] extend distributed version control to neural network parameters, exposing familiar abstractions (commits, diffs, merges) over checkpoints rather than source files. Under this lens, model merging is no longer a post-hoc operation but a first-class primitive in the open-source model lifecycle, on par with branching and integration in software development.

1.1.5 A democratic alternative to fine-tuning

In the early days of deep learning, standard practice dictated training models entirely from scratch. With the advent of massive pre-trained networks, the community shifted toward fine-tuning these robust backbones for specialized downstream tasks [44, 73, 132, 154, 182]. This paradigm significantly lowered the entry barrier, as fine-tuning a large model is substantially cheaper and more effective than training a comparable architecture from scratch.

Model merging lowers this accessibility barrier further still. End users can synthesize custom, state-of-the-art models entirely on consumer-grade hardware, sometimes without even

²Actually, subtracting its task vector; we will give the necessary formalism in part III.

³<https://huggingface.co/models>

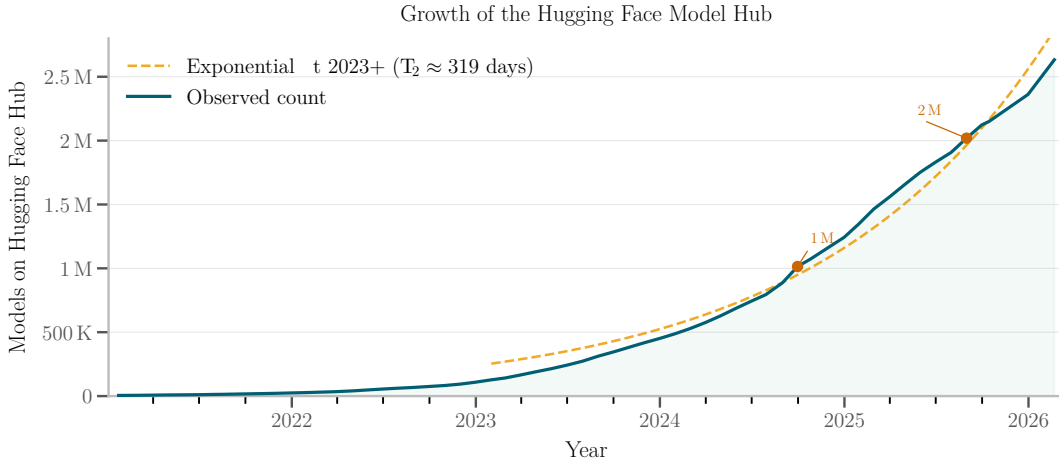


FIGURE 1.1: Growth of models hosted on Hugging Face over time.

requiring a GPU. Furthermore, merging drastically reduces the required technical expertise. Unlike fine-tuning, which demands custom code and delicate hyperparameter tuning, model merging can be as straightforward as selecting simple interpolation coefficients. This simplicity has sparked a proliferation of community-driven models; at the time of writing, roughly 30% of the models on the Hugging Face Open LLM leaderboard are the direct result of merging. As the resource divide between major AI labs and smaller research teams continues to widen, model merging offers a practical path toward reducing this disparity.

1.1.6 Model merging as a pre-training regularizer

While model merging is frequently framed as a lightweight alternative to traditional training, these techniques also share rich, underexplored connections with general optimization and find notable applications in the pre-training of foundation models. For instance, established optimization strategies like Polyak averaging [130] and Exponential Moving Average (EMA) [158] can be reinterpreted as specific instances of model merging, where consecutive checkpoints are simply aggregated according to a predefined weighting schedule. Viewed through this lens, it is unsurprising that model merging has shown promise in the pre-training phases of large language models [99, 138, 160]. We argue that the deeper theoretical connection between model merging and optimization remains largely untapped, a topic that will be further explored in part IV.

1.2 Landscape of merging approaches

The field of model merging has developed rapidly, producing a diverse set of techniques that address different needs; these can be broadly organized into five categories. This thesis revolves around the first and the third family, presented in part II and part III respectively.

1.2.1 Merging of models independently trained on the same task

Mostly originating from the study of linear mode connectivity [50, 53, 184], this setting studies how to merge different instances of the same architecture trained on the same task starting from different initializations. From a theoretical standpoint, this sheds light on interesting properties of the loss landscape: effective merging through a simple averaging signals that the region of the loss landscape containing the different modes might be more convex than usually believed for deep neural networks. From a practical perspective, this makes it possible to

perform efficient ensembling without maintaining separate models: given two models, one can simply interpolate between them to obtain additional well-performing models [13, 59]. Part II shows that the study of neuron permutation symmetries is key for this family of approaches, as they stand as a primary obstacle to linear connectivity: different sets of weights become incompatible even when the models compute the same function.

1.2.2 Merging of models independently fine-tuned on the same task starting from the same base model

The difficulty of connecting models trained from different seeds, even when sharing the same task, seems to leave no chance of merging models trained on different tasks. However, in the modern pre-train and fine-tune regime, models are far more often fine-tuned than trained from scratch. Sharing the same initialization (the pre-trained base model), fine-tuned models have been observed to be much more linearly connected, partially due to neuron configurations being fixed by the pre-training process [118, 173]. This opens up new merging possibilities: when fine-tuning a foundation model onto some downstream task, practitioners often run multiple configurations, characterized by different hyperparameters or data splits, and eventually pick the one that maximizes some criterion on a validation split. Several works, originating from model soups [173], argue that this selection procedure is suboptimal, and that a stronger model can be obtained by merging (often simply averaging) the different resulting models [77, 80, 173]. Unlike the first family, this approach is as simple as it is practical, and is often used in the industry to produce the best-performing models.

1.2.3 Merging of models independently fine-tuned on different tasks starting from the same base model

The ease with which foundation models can be cheaply and quickly specialized to downstream tasks has spurred an abundance of fine-tuned models on public model repositories such as Hugging Face. This abundance has paved the way for a new use case: collecting multiple models, fine-tuned on different downstream tasks, and aggregating them into a single, all-encompassing one [35, 36, 58, 78, 176, 183, 191]. While the previous category of approaches found the most adoption in industry, this third family is by far the most adopted in the open-source community: practitioners can produce new models starting from publicly available fine-tunings, often without having access to their tuning data. Most interestingly, this is the first category in which compositionality appears: a model that can code and one that can speak Japanese can now be merged into one that can code in Japanese [3, 109]. Part III treats this family of approaches in depth.

1.2.4 Merging to mitigate catastrophic forgetting

A perhaps less systematically studied but commonly employed practice is to use merging to recover general skills of a base model that were lost in favour of specialized ones during fine-tuning. In effect, this amounts to mitigating catastrophic forgetting post hoc, induced by aggressive fine-tuning. For this scenario, model merging offers a practical solution by aggregating the base and the fine-tuned models, recovering general skills such as instruction following while maintaining strong performance on the downstream fine-tuning task(s). Examples include recovering instruction-following capabilities for a large language model fine-tuned on bioacoustic tasks [106] or robotics tasks [178].

1.2.5 Merging during pre-training

Finally, merging has direct implications for optimization and the pre-training of foundation models [99, 138, 160]. Classical techniques such as Polyak averaging and EMA can already be reinterpreted as merging consecutive checkpoints along the training trajectory. Examples include using snapshot merging to replace learning rate decay [160], or to improve convergence in the early stages of pre-training [99, 138].

1.3 Thesis outline and contributions

This thesis is organized into four parts, progressing from single-task to multi-task merging and concluding with applications, future directions, and a synthesis of the contributions. The five core contributions are:

- **C²M³** [34] (chapter 3): cycle-consistent multi-model merging via Frank-Wolfe optimization over a shared universe space, with accuracy gains up to 20% when merging five models simultaneously.
- **Task vector theory** [190] (chapter 5): formal gradient equivalence showing that task vectors are scaled negative loss gradients under single-epoch full-batch GD, with a curvature-controlled $O(\eta^2)$ deviation for multi-epoch fine-tuning.
- **TSV** [58] (chapter 6): low-rank compression reducing task vector storage by a factor of T (number of tasks) while retaining 99% accuracy, combined with Procrustes-based interference reduction yielding up to +15 percentage points on 20-task merging benchmarks.
- **MASS** [33] (chapter 7): data-free input-adaptive routing via projection-residual distances, MAP-optimal under an isotropic Gaussian noise model, achieving state of the art on 8 out of 9 benchmarks.
- **MERGE³** [109] (chapter 8): efficient evolutionary merging with a $50\times$ cost reduction through IRT-based performance estimation, enabling cross-lingual transfer with +10–20% gains over all endpoint models.

We now provide an overview of each part.

Part II: Single-Task Model Merging. Chapter 2 introduces mode connectivity and the neuron matching problem, showing how permutation symmetries of neurons create a vast number of functionally equivalent networks that hinder the aggregation of independently trained models. Existing alignment methods, such as `Git Re-Basin` [2], tackle this problem in a pairwise fashion, solving for the optimal permutations one layer at a time. However, when merging more than two models, a setting that naturally arises in distributed learning and federated optimization, these pairwise approaches offer no guarantees on the consistency of the permutations across the set. In chapter 3, which presents the first contribution of this thesis, we address this limitation by introducing a cycle-consistent alignment algorithm that matches any number of models simultaneously through a shared universal space, and we leverage it to perform principled multi-model merging.

Part III: Multi-Task Model Merging. Having addressed the problem of merging models trained on the same task but from different initializations, where the central challenge was aligning neurons across networks, we turn to a fundamentally different and arguably more impactful setting: merging models that have been fine-tuned on different tasks from a shared

pre-trained backbone. In this multi-task regime, the dominant paradigm is *task arithmetic*: constructing a multi-task model by summing the weight differences between each fine-tuned model and the common base. Despite its widespread adoption and surprising effectiveness, a rigorous understanding of *why* task arithmetic works has remained elusive. Chapter 5 lays the theoretical groundwork for the rest of this part by establishing a formal connection between task vectors and the gradients of the task losses, reframing task arithmetic as approximate multi-task learning.

Building on this theoretical foundation, chapter 6 departs from the flattened view of networks as monolithic parameter vectors and instead examines task vectors at the *layer* level, preserving their natural matrix form. Through singular value decomposition, we uncover that per-layer task matrices are inherently low-rank, and we exploit this structure both for compression and for a more fine-grained understanding of task interference, ultimately yielding a merging method that significantly outperforms prior approaches.

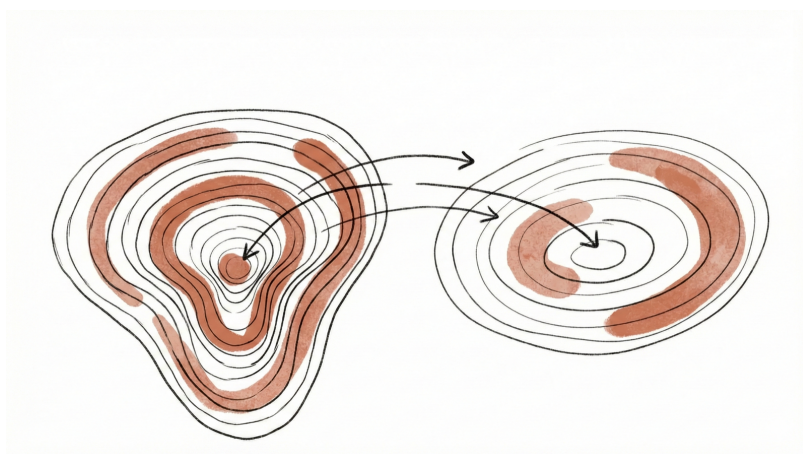
A natural follow-up question is whether this low-rank structure can be leveraged not just at merge time, but also at *inference* time, to adapt the merged model to each individual input. All methods discussed so far produce a single, static merged model that is applied identically regardless of the input, a limitation that leaves performance on the table. In chapter 7, we explore a different paradigm: rather than committing to a fixed merge, we route each input to the most relevant task subspaces, effectively making the merging process *input-dependent*.

Finally, a complementary line of work, *evolutionary merging*, sidesteps the design of hand-crafted merging rules altogether by using evolutionary search to optimize merging coefficients directly. While this approach has produced models of unprecedented quality, its computational demands (thousands of fitness evaluations, each involving full inference over large datasets) place it out of reach for practitioners working on consumer hardware. Chapter 8 addresses this accessibility gap, introducing an efficient evolutionary merging framework that reduces the computational cost by $50\times$ and brings evolutionary merging to single-GPU setups. In doing so, we also extend the scope of this thesis from vision models to Large Language Models, demonstrating cross-lingual knowledge transfer through merging.

Part IV: Applications, Outlook, and Conclusions. The final part of the thesis broadens the perspective from individual methods to their practical impact, open problems, and the future of the field. Chapter 9 examines how the methods developed in parts II and III transfer to concrete deployment scenarios: the cycle-consistent alignment of chapter 3 naturally extends to federated learning, where it compensates for the drift induced by prolonged local training; the low-rank structure of task vectors yields practical compression of multi-task expert libraries; and the evolutionary framework of chapter 8 enables cross-lingual knowledge transfer on consumer hardware. Chapter 10 identifies the most promising avenues for advancing the field, including more expressive alignment frameworks based on quadratic assignment and functional maps, heterogeneous merging across architectures and tokenizers, the emerging connections between structured merging and spectral optimizers such as `Muon`, scaling the layer-wise SVD paradigm to large language models, and the development of a predictive theory of mergeability. Finally, chapter 11 synthesizes the theoretical and algorithmic insights developed across the thesis, distilling three overarching themes: that weight space is legible and interpretable, that model merging admits formal structure and guarantees, and that exploiting this structure yields methods that are simultaneously more accurate, more efficient, and more accessible.

Part II

Single-Task Model Merging



“Symmetry, as wide or as narrow as you may define its meaning, is one idea by which man through the ages has tried to comprehend and create order.”

— Hermann Weyl

Chapter 2

Foundations of Single-Task Merging

The simplest incarnation of model merging, introduced in section 1.2.1, considers multiple instances of the same architecture trained independently on the same task from different random initializations. The goal is to combine them into a single model, ideally by a simple weight average, that matches or exceeds the performance of the individual networks. This is appealing both theoretically, as it implies that the loss landscape is more benign than traditionally believed, and practically, as it enables efficient ensembling without retraining. A major obstacle, however, is that two networks that have learned the same function may appear far apart in weight space: the permutation symmetries of neurons create a combinatorial number of functionally equivalent parameterizations, and independently trained models will in general converge to different elements of the same equivalence class. Naively averaging such misaligned models destroys the learned representations.

This part of the thesis addresses exactly this challenge. The present chapter reviews the necessary foundations: we first trace how the community’s understanding of the loss landscape evolved from isolated minima to connected low-loss manifolds; then formalize the permutation symmetries that obscure this connectivity; and finally present the alignment algorithms that resolve these symmetries, enabling the merging methods developed in chapter 3.

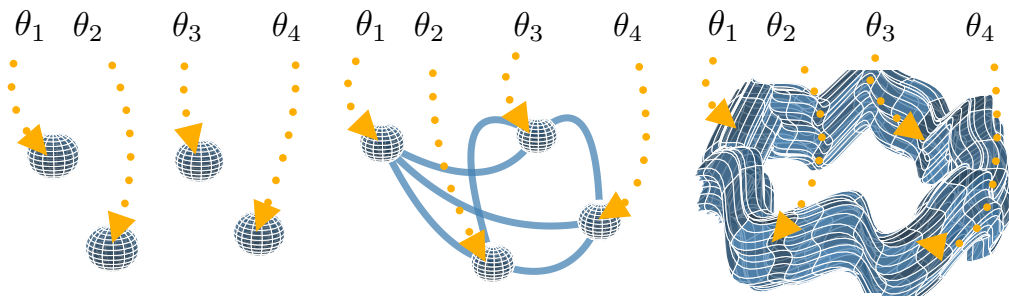


FIGURE 2.1: Progressive understanding of neural network loss surfaces. **Left:** the traditional view, where low-loss regions are thought to be isolated [64]. **Center:** low-loss curves can connect apparently distinct modes [46, 59]. **Right:** SGD solutions may populate a broader connected low-loss region, suggesting a higher-dimensional low-loss volume rather than isolated paths alone [13]. Figure from [13].

2.1 Mode connectivity

Mode connectivity studies the geometry of the loss landscape, focusing on the regions corresponding to local minima. It was long believed that the highly non-convex loss landscape of deep networks makes optimization difficult, with many poor local minima and saddle points. Goodfellow et al. [64] were among the first to challenge this view, evaluating the loss along

linear paths $\theta = (1 - \lambda)\theta_A + \lambda\theta_B$ from initialization to convergence and finding the path to be approximately convex. This suggests that optimization is more well-behaved than previously thought.

Daniel Freeman and Bruna [37] further studied the topology of the loss landscape through the lens of its sublevel sets $\Omega_\lambda = \{\theta : \mathcal{L}(\theta) \leq \lambda\}$. When sublevel sets are connected, the landscape cannot fragment into isolated bad valleys within the regime under study, since one can move within a low-loss region without encountering an unavoidable barrier. They show that half-rectified single-layer networks have *asymptotically connected* sublevel sets, and empirically find near-convex behavior in several architectures. This suggests a loss landscape with no spurious valleys in the regimes they study, though not a general theorem for arbitrary deep networks.

Moving from topology to the relationship between different modes, Garipov et al. [59] investigate whether distinct solutions can be connected through a high-accuracy path. They propose a training procedure to find such paths and show they can be well approximated as a piecewise-linear path composed of two segments, a finding they exploit to construct efficient ensembles. Concurrently and independently, Draxler et al. [46] find *Minimum Energy Paths* (MEPs) between modes, showing they are essentially flat in both training and test loss. Taken together, these works suggest that many apparent minima are not isolated in practice, but can instead be joined by continuous paths that remain in regions of low loss (fig. 2.1).

2.1.1 Linear mode connectivity

Frankle et al. [53] ask a finer question: are the modes found by two independent training runs of the same network, from the same initialization but with different SGD noise (random data order and augmentation), linearly connected? If so, merging models by weight interpolation within that region incurs no loss.

Instability analysis. To study how SGD noise affects the optimization outcome, Frankle et al. [53] propose *instability analysis*. A network is initialized at θ_{init} and two copies are trained with different noise samples. The loss barrier along the linear interpolation between the two resulting models (defined below) quantifies how stable the outcome is to SGD noise. The analysis can also start from an intermediate checkpoint θ_k , revealing *when* the trajectory commits to a linearly connected minimum; empirically, this happens early in training.

The key metric used throughout this analysis is the *loss barrier*:

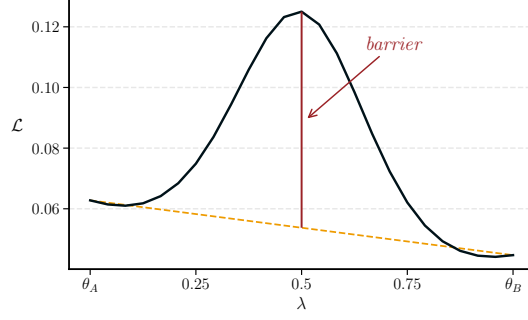
Definition 2.1.1 (Loss barrier). Given two points θ_A, θ_B and a loss function $\mathcal{L}$ such that $\mathcal{L}(\theta_A) \approx \mathcal{L}(\theta_B)$, the *loss barrier* is defined as

$$\max_{\lambda \in [0,1]} \mathcal{L}((1 - \lambda)\theta_A + \lambda\theta_B) - \frac{1}{2}(\mathcal{L}(\theta_A) + \mathcal{L}(\theta_B)).$$

A barrier of zero means that linear interpolation never rises above the average endpoint loss, whereas larger values indicate that the line segment passes through a higher-loss region. An intuitive illustration is given in fig. 2.2.

Frankle et al. [53] find that standard vision models become stable to SGD noise early in training, after which the outcome of optimization belongs to a fixed, linearly connected region. This property has important implications for pruning: the *Lottery Ticket Hypothesis* (LTH) [52] posits that dense networks contain sparse subnetworks (“winning tickets”) that, when trained in isolation from the original initialization, match the full network’s test accuracy.

Frankle et al. [53] establish a tight connection between LTH and mode connectivity, showing that solutions found from the same initialization that are linearly connected share the same winning ticket, and that the sparsest subnetworks identified by Iterative Magnitude

FIGURE 2.2: Loss barrier between two modes θ_A and θ_B .

Pruning can only be trained to full accuracy when the full network is robust to SGD noise, the very condition that determines whether two training runs land in the same linearly connected region.

Going further, Benton et al. [13] provide empirical evidence that many independently trained SGD solutions can be connected through multi-dimensional low-loss simplicial complexes, and propose a technique to discover such structures for ensembling. More generally, these low-loss paths and manifolds translate directly into efficient ensembling strategies: rather than training many independent models, one can sample diverse checkpoints along the low-loss geometry and average their predictions. Garipov et al. [59] sample along the piecewise-linear curve connecting two modes, obtaining ensemble-quality predictions from a single extended training run, while Benton et al. [13] scale this to the full simplicial complex, achieving further improvements in accuracy, calibration, and robustness over standard independently trained ensembles.

2.2 Weight-space symmetries

From the previous section, linear connectivity appears easiest to observe when models share not only the same architecture and task, but also the same initialization and an initial portion of the training trajectory. This raises a natural question: if these conditions are not met, what makes the corresponding modes appear isolated in weight space? A key part of the answer lies in *weight-space symmetries*. Even when two networks implement essentially the same function, they need not occupy nearby points in parameter space, because hidden units can be permuted without changing the network’s input-output map. As a result, independently trained models may converge to parameterizations that are functionally equivalent yet separated by large Euclidean distance, making naive interpolation fail. To make this precise, consider a Multi-Layer Perceptron (MLP) where the computation at layer ℓ is $\mathbf{z}_{\ell+1} = \phi(W^{(\ell)}\mathbf{z}_\ell + b_\ell)$, with ϕ an element-wise activation function. Setting $b_\ell = 0$ for clarity and applying a permutation matrix $P \in \mathbb{P}$ to the rows (neurons) of $W^{(\ell)}$ gives $\mathbf{z}'_{\ell+1} = \phi(PW^{(\ell)}\mathbf{z}_\ell)$. Since ϕ is element-wise, it commutes with P , so the effect can be fully cancelled by permuting the columns of the next layer by $P^\top$:

$$\mathbf{z}'_{\ell+2} = W^{(\ell+1)}P^\top\mathbf{z}'_{\ell+1} = W^{(\ell+1)}\underbrace{P^\top P}_I W^{(\ell)}\mathbf{z}_\ell = \mathbf{z}_{\ell+2}.$$

Hence any two models differing only by a neuron permutation are functionally equivalent. For a network with L hidden layers $W_1, \dots, W_L$ with widths $d_1, \dots, d_L$, there are $\prod_{\ell=1}^L d_\ell!$ such symmetries in total. For a parameter set Θ , the full equivalence class of functionally equivalent reparameterizations is denoted $\pi(\Theta)$. This combinatorial explosion of symmetries is a major factor in the apparent isolation of modes: two models that learned the same function

may appear far apart in parameter space simply because they converged to different elements of the same equivalence class.

2.3 Neuron matching

If permutation symmetries are indeed a primary source of the apparent isolation between modes, then connectivity may be hidden rather than absent: two solutions can fail to be linearly connected in their raw parameterizations, yet become connected after an appropriate reordering of their neurons. Under this view, the key operation is to find a symmetry transformation that maps one model into the representation used by the other. Doing so effectively maps one parameterization into the basin of the other, after which a simple linear interpolation can remain in low loss (fig. 2.3).

This intuition is formalized by the following conjecture of Entezari et al. [50], which posits that, once permutation symmetries are accounted for, most modes lie in a common convex basin:

Conjecture 2.3.1 (Entezari et al. 50). *Most modes belong to a set S whose elements can be permuted such that there is no loss barrier on the linear interpolation between any two permuted elements of S .*

Building on Conjecture 2.3.1, Ainsworth et al. [2] propose three algorithms to find a permutation π that aligns a model B with a reference model A . Once such a permutation is found, model B can be re-expressed in the basin of model A , making weight interpolation possible.

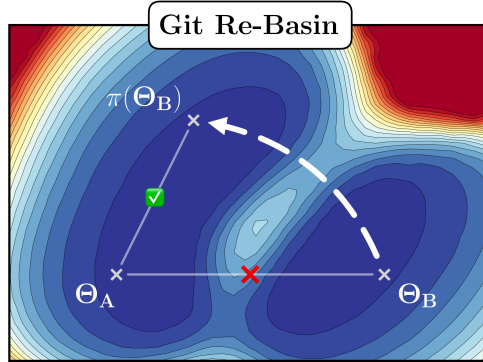


FIGURE 2.3: Schematic of Git Re-Basin: a permutation is found that maps model B into the basin of model A , enabling low-barrier linear interpolation [2]. Figure from [2].

Activation matching. A natural way to align units across two models is to compare their activations and match those that respond similarly over the data. For layer ℓ , let $\mathbf{Z}^{(A)}, \mathbf{Z}^{(B)} \in \mathbb{R}^{d \times n}$ denote the d -dimensional activations across all n training points. The optimal permutation solves

$$\mathbf{P}_\ell = \arg \min_{\mathbf{P} \in \mathbb{P}} \sum_{i=1}^n \left\| \mathbf{Z}_{:,i}^{(A)} - \mathbf{P} \mathbf{Z}_{:,i}^{(B)} \right\|^2 = \arg \max_{\mathbf{P} \in \mathbb{P}} \left\langle \mathbf{P}, \mathbf{Z}^{(A)} \left(\mathbf{Z}^{(B)} \right)^\top \right\rangle_F, \quad (2.1)$$

a Linear Assignment Problem (LAP) solvable in polynomial time (e.g., via the Hungarian algorithm). The permuted weights are set as $\mathbf{W}'_\ell = \mathbf{P}_\ell \mathbf{W}_\ell^{(B)} \mathbf{P}_{\ell-1}^\top$.

Weight matching. Alternatively, units can be matched directly from the model weights, without requiring any data. The objective is to find permutations $\{P_\ell\}$ maximizing the weight overlap across all layers:

$$\arg \max_{\{P_\ell \in \mathbb{P}\}} \sum_{\ell=1}^L \langle W_A^{(\ell)}, P_\ell W_B^{(\ell)} P_{\ell-1}^\top \rangle, \quad (2.2)$$

with $P_0 := I$. Since eq. (2.2) is NP-hard globally, Ainsworth et al. [2] solve it one layer at a time, relaxing each bilinear problem to a LAP that can be solved efficiently.

Straight-through estimator. A third approach learns the permutation by directly minimizing the loss of the interpolated model:

$$\min_{\tilde{\Theta}_B} \mathcal{L} \left(\frac{1}{2} (\Theta_A + \text{proj}(\tilde{\Theta}_B)) \right), \quad \text{proj}(\Theta) \triangleq \arg \max_{\pi} \text{vec}(\Theta) \cdot \text{vec}(\pi(\Theta_B)), \quad (2.3)$$

where $\tilde{\Theta}_B \approx \pi(\Theta_B)$ and $\text{proj}(\cdot)$ finds the nearest realizable permutation. Because the projection onto valid permutations is discrete and therefore non-differentiable, optimization uses a straight-through approximation [12]: the projected parameters are used to evaluate the loss, while gradients are propagated through the underlying continuous variables. Among the three methods, the straight-through estimator performs best, though it requires an additional training phase to learn the permutation.

Renormalizing the activations. Even with high-quality alignment, loss barriers can remain elevated due to mismatches in activation statistics between the endpoint networks. REPAIR [85] observes that interpolation can induce discrepancies in the mean and standard deviation of intermediate activations, and demonstrates that restoring these statistics greatly improves the quality of the merged network. Concretely, given endpoint activations X_1 and X_2 , the merged model’s activations X_α are renormalized to have:

$$\mathbb{E}[X_\alpha] = (1 - \alpha) \cdot \mathbb{E}[X_1] + \alpha \cdot \mathbb{E}[X_2], \quad (2.4)$$

$$\text{std}(X_\alpha) = (1 - \alpha) \cdot \text{std}(X_1) + \alpha \cdot \text{std}(X_2). \quad (2.5)$$

Alignment through neuron permutations directly enables weight-space model merging: once two models are brought into the same loss basin, their weights can be meaningfully interpolated, often yielding a single network with comparable performance. This geometric perspective—first align, then merge—forms the basis of the single-task methods developed in this part of the thesis. At the same time, most permutation-based approaches are inherently pairwise, aligning one model to another chosen reference. The next chapter asks how this picture changes when the number of models grows beyond two: can one align an entire collection of networks in a globally consistent way, without depending on an arbitrary anchor? In chapter 3, we answer this question by studying N -way matching and merging in the single-task setting. After that, in part III, we move to a different but equally fundamental regime, where models are fine-tuned for distinct downstream tasks from a shared pre-trained backbone and the central challenge becomes task interference rather than geometric alignment.

Chapter 3

Cycle-Consistent Alignment of Multiple Models

In this chapter, we extend the pairwise neuron-matching problem to more than two models. We present a data-free matching and merging technique that enforces cycle consistency of the permutations when merging $n \geq 3$ models, preventing error accumulation in circular compositions. We demonstrate the benefits of this constraint across varying architectures and datasets.

3.1 From pairwise to multi-model alignment

As established in chapter 2, neuron permutation symmetries are the primary obstacle to weight-space merging: two models may have learned the same function yet appear arbitrarily far apart in parameter space simply because their neurons are ordered differently. The key step toward low-barrier interpolation is therefore to solve the alignment problem: finding the permutations that bring independently trained models into a shared basin. Existing solutions such as `Git Re-Basin` [2] tackle this pairwise and layer-by-layer, which suffices when only two models are involved. However, real-world scenarios (distributed training, federated optimization, or simply combining a pool of independently trained checkpoints) naturally involve $n > 2$ models, and here a subtler issue emerges. When permutations are computed independently for each pair, the composition along any cycle generally does *not* reduce to the identity (see fig. 3.1). As shown in fig. 3.2b and fig. 3.2a, mapping a model A to C through B and then back to A yields a different model altogether, one that lands in a completely different basin. This lack of *cycle consistency* also manifests in the $n = 2$ case: the permutations optimized to align model A to model B are not guaranteed to be the inverse of those mapping B to A , making the alignment pipeline brittle and dependent on an arbitrary choice of mapping direction.

This chapter addresses the problem by introducing an alignment algorithm that works for the general case with $n \geq 2$ models while *guaranteeing* cycle consistency. The key idea is to factorize each pairwise permutation P^{AB} as $P^A(P^B)^\top$, where $(P^B)^\top$ maps B to a common space denoted as the *universe*, and P^A maps from the universe back to A . This formulation ensures cycle consistency by design, as any cyclic composition of such permutations equals the identity.

The numerical implementation is based on the Frank-Wolfe algorithm [51] and optimizes for the permutations of *all* layers simultaneously at each step, naturally accounting for inter-layer dependencies. This is in contrast with Ainsworth et al. [2], who seek the optimal permutations for each layer separately and thus cannot ensure coherence across the entire network. The universe space also serves as a natural aggregation point for merging: once all models are mapped into a shared basin, their weights can be meaningfully averaged.

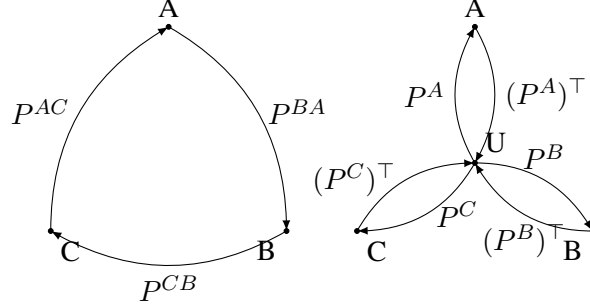
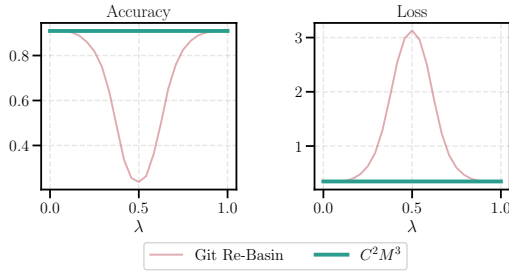


FIGURE 3.1: Cycle-Consistent Multi-Model Merging over three models A, B, C . **Left:** existing methods seek pairwise permutations that map between models; note that $P^{AC} \circ P^{CB} \circ P^{BA} \neq I$ in general, unless this is explicitly enforced. **Right:** our method computes permutations P^A, P^B, P^C from each model to a *universe* U , such that a pairwise permutation P^{BA} mapping A to B can be obtained as $P^{BA} = P^B(P^A)^\top$. This way, cycle-consistency is enforced by design and $P^{AC} \circ P^{CB} \circ P^{BA} = I$.



(A) Loss and accuracy curves for a model A and the model mapped back after a cyclic permutation. Models cyclically permuted with Git Re-Basin end up in a different basin than the one they started from.

Permutation	Git Re-Basin	C^2M^3
$d(A, P_{A \rightarrow B \rightarrow C \rightarrow A}(A))$	41.07	0.0
$d(B, P_{B \rightarrow C \rightarrow A \rightarrow B}(B))$	41.18	0.0
$d(C, P_{C \rightarrow A \rightarrow B \rightarrow C}(C))$	41.19	0.0

(B) Accumulated error obtained when cyclically permuting models A, B and C as in fig. 3.1. $P_{A \rightarrow B \rightarrow C \rightarrow A}$ refers to the composition $P_{AC} \circ P_{CB} \circ P_{BA}$ and $d(\cdot)$ is the ℓ_2 loss.

FIGURE 3.2: Existing methods accumulate error when cyclically mapping a model through a series of permutations, while C^2M^3 correctly maps the model back to the starting point.

3.2 Designing the cycle-consistent objective

We now propose a novel algorithm to tackle the weight matching problem, first introducing its formulation in the pairwise case and then generalizing it to match and merge a larger number n of models in a cycle-consistent fashion.

Pairwise matching As we have seen, the NP-hardness of eq. (2.2) demands a relaxation of the problem to be tackled. Differently from Ainsworth et al. [2], we opt to maintain the objective global with respect to the layers and instead, iteratively optimize its linear approximation via the Frank-Wolfe algorithm [51]. This procedure requires the computation of the gradient of eq. (2.2) with respect to each permutation P_i , thus we have to account for two contributions for each ∇_{P_i} , i.e., its gradient from permuting the rows of W_i and the one from permuting the columns of W_{i+1} :

$$\nabla_{P_i} f = \underbrace{W_i^A P_{i-1} (W_i^B)^\top}_{\text{from permuting rows}} + \underbrace{(W_{i+1}^A)^\top P_{i+1} W_{i+1}^B}_{\text{from permuting columns}}. \quad (3.1)$$

The Frank-Wolfe algorithm then uses the gradient to iteratively update the solution by linearly interpolating between the current solution and the projected gradient. We refer to Lacoste-Julien [94] for theoretical guarantees of convergence. The full algorithm is reported in section A.1.2.

Generalization to n models To generalize to n models, we jointly consider all pairwise problems

$$\arg \max_{P_i^{pq} \in \mathbb{P}} \sum_{p=1}^n \sum_{\substack{q=1 \\ q \neq p}}^n \sum_{i=1}^L \langle W_i^p, P_i^{pq} W_i^q (P_{i-1}^{pq})^\top \rangle, \quad (3.2)$$

where the superscript pq indicates that the permutations maps model q to model p , with $P_0^{pq} := I$. To ensure cycle consistency by construction, we replace the quadratic polynomial by a fourth-order polynomial. Dropping the layer subscript for clarity, we replace the pairwise matchings P^{pq} in the objective of eq. (3.2) by factorizing the permutations into *object-to-universe matchings* $P^{pq} = P^p \circ (P^q)^\top$ so that each model q can be mapped back and forth to a common universe u with a permutation and its transpose, allowing us to map model q to model p by composition of $(P^q)^\top$ ($q \rightarrow u$) and P^p ($u \rightarrow p$). This way, the objective of eq. (3.2) becomes

$$\sum_{p \neq q} \sum_{i=1}^L \langle W_i^p, P_i^p (P_i^q)^\top W_i^q (P_{i-1}^p (P_{i-1}^q)^\top)^\top \rangle = \sum_{p \neq q} \sum_{i=1}^L \langle (P_i^p)^\top W_i^p P_{i-1}^p, (P_i^q)^\top W_i^q P_{i-1}^q \rangle. \quad (3.3)$$

As stated by Theorem 3.2.1, the permutations we obtain using eq. (3.3) are cycle consistent. We refer the reader to Bernard et al. [15] for the proof and a complete discussion of the subject.

Theorem 3.2.1 (Restated from Bernard et al. 15). *Given a set of n models $p_1, \dots, p_n$ and object-to-universe permutations $P_i^{p_j}$ computed via eq. (3.3), the pairwise correspondences defined by $P_i^{p_l p_j} = P_i^{p_l} \circ (P_i^{p_j})^\top$ are cycle-consistent, i.e.,*

$$P_i^{p_1 p_j} \circ \dots \circ P_i^{p_3 p_2} \circ P_i^{p_2 p_1} = I$$

for all layer indices i , $2 \leq j \leq n$.

Similarly to the pairwise case, the approach requires computing the gradients for the linearization. This time, however, each $\nabla_{P_\ell^A} f$ has four different contributions: one from permuting the rows of its corresponding layer, one from anti-permuting the columns of the subsequent layer, and two other contributions that arise from the symmetric case where A becomes B . In detail,

$$\nabla_{P_\ell^A} = \nabla_{P_\ell^A}^{\text{rows}} + \nabla_{P_\ell^A}^{\text{cols}} + \nabla_{P_\ell^A}^{\text{rows}, \leftrightarrow} + \nabla_{P_\ell^A}^{\text{cols}, \leftrightarrow} \quad (3.4)$$

where

$$\begin{aligned} \nabla_{P_\ell^A}^{\text{rows}} &= W_\ell^A P_{\ell-1}^A (P_{\ell-1}^B)^\top (W_\ell^B)^\top P_\ell^B & \nabla_{P_\ell^A}^{\text{cols}} &= (W_{\ell+1}^A)^\top P_{\ell+1}^A (P_{\ell+1}^B)^\top W_{\ell+1}^B P_\ell^B \\ \nabla_{P_\ell^A}^{\text{rows}, \leftrightarrow} &= W_\ell^B P_{\ell-1}^B (P_{\ell-1}^A)^\top (W_\ell^A)^\top P_\ell^A & \nabla_{P_\ell^A}^{\text{cols}, \leftrightarrow} &= (W_{\ell+1}^B)^\top P_{\ell+1}^B (P_{\ell+1}^A)^\top W_{\ell+1}^A P_\ell^A \end{aligned}$$

See algorithm 1 for a complete description of the procedure.

Algorithm 1 Frank-Wolfe for n -Model Matching

Require: Weights of n models $M_{i=1}^n$

tolerance $\epsilon > 0$

Ensure: Approximate solution to eq. (3.3)

```

1:  $\mathbf{P}^k \leftarrow$  identity matrices
2: repeat
3:   for  $(p, q) \in [1, \dots, n] \times [1, \dots, n]$  do
4:     for  $i = 1$  to  $L$  do
5:        $P_i^{p,k}, P_{i-1}^{p,k} \leftarrow$  permutations over rows and columns of  $W_i^p$  respectively
6:        $P_i^{q,k}, P_{i-1}^{q,k} \leftarrow$  permutation over rows and columns of  $W_i^q$  respectively
7:        $\nabla_{P_i^{p,k}} f \mathrel{+}= (W_{i+1}^p)^\top P_{i+1}^p (P_{i+1}^q)^\top W_{i+1}^q P_i^q$ 
8:        $\nabla_{P_{i-1}^{p,k}} f \mathrel{+}= (W_{i+1}^p)^\top P_{i+1}^p (P_{i+1}^q)^\top W_{i+1}^q P_i^q$ 
9:     end for
10:  end for
11:  for  $P_i^k \in \mathbf{P}^k$  do
12:     $\Pi_i \leftarrow \text{LAP}(\nabla_{P_i^k} f)$ 
13:  end for
14:   $\alpha \leftarrow \text{line search}(f, \mathbf{P}^k, \Pi)$ 
15:  for  $P_i^k \in \mathbf{P}^k$  do
16:     $P_i^{k+1} = (1 - \alpha)P_i^k + \alpha \Pi_i$ 
17:  end for
18: until  $\|f(A, B, \mathbf{P}^{k+1}) - f(A, B, \mathbf{P}^k)\| < \epsilon$ 
19: return  $\mathbf{P}^k$ 

```

Merging in the universe space Looking at the loss landscape resulting from interpolating models in fig. 3.3, we see that the loss curves are much lower when the models are interpolated in the universe space. In fact, the originally disconnected modes end up in the same basin when mapped onto the universe, making it suitable to average the models. Therefore, our merging method aggregates the models by taking the mean of the weights in the universe space, as detailed in algorithm 2.

3.3 Empirical evaluation

This section evaluates the cycle-consistent framework on both the matching and the subsequent merging operation. Approaches suffixed with a $\dagger$ indicate the application of REPAIR.

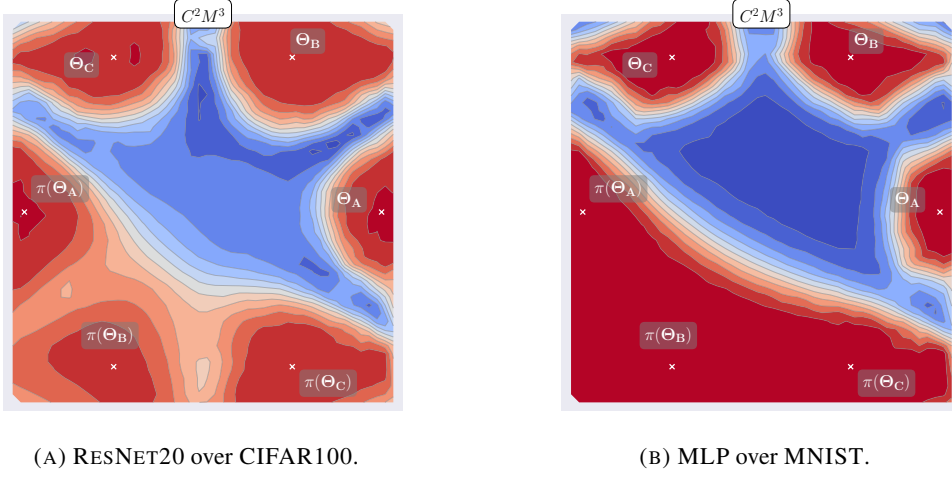


FIGURE 3.3: 2D projection of the loss landscape when matching three modes $\theta_A, \theta_B, \theta_C$; the models $\pi(\theta_A), \pi(\theta_B), \pi(\theta_C)$ are their resulting images in the universe, and lie in the same basin. Red zones indicate low-loss regions (typically basins), while blue zones indicate high-loss ones.

Algorithm 2 C^2M^3 : Cycle-Consistent Multi Model Merging

Require: n models $A_1, \dots, A_n$ with L layers

Ensure: merged model M

- 1: $\{P_1, \dots, P_n\} \leftarrow \text{Frank-Wolfe}(A_1, \dots, A_n)$
 - 2: **for** $i = 1$ to n **do**
 - 3: $M_i^{\text{uni}} \leftarrow \text{map_to_universe}(A_i, P_i)$
 - 4: **end for**
 - 5: $M^{\text{uni}} \leftarrow \frac{1}{n} \sum_{i=1}^n M_i^{\text{uni}}$
 - 6: **return** M^{uni}
-

Matching and merging two models As described in section 3.2, our formalization can readily be used to match $n = 2$ models. In this case, the energy is given by eq. (2.2) and the permutations are not factorized. We compare the performance of our approach against the `Git Re-Basin` algorithm [2] and the naive baseline that aggregates the models by taking an unweighted mean on the original model weights without applying any permutation. In this setting, our method performs on par with the state of the art. Unlike the latter, however, we do not depend on the random choice of layers, as the optimization is performed over all layers simultaneously. As presented in fig. 3.4, this results in `Git Re-Basin` exhibiting variations of up to 10% in accuracy depending on the optimization seed, while our method shows zero variance. We refer the reader to section A.2.1 for a thorough evaluation of C^2M^3 over a set of different datasets and architectures. The approach thus matches two models with the same accuracy as the state of the art, while being deterministic and independent of the random choice of layers.

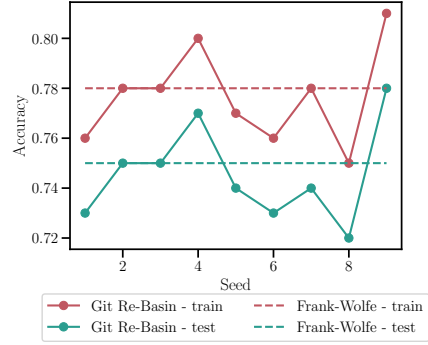


FIGURE 3.4: Accuracy of the interpolated model using `Git Re-Basin` [2] over different optimization seeds.

Matching and merging n models We now evaluate C^2M^3 in matching and merging n models. The matching is given by the factorized permutations obtained by algorithm 1. We compare against two baselines: the simple approach of naively averaging the weights without any matching, and the `MergeMany` approach proposed by Ainsworth et al. [2]. The latter is reported in section A.1 for convenience. As reported in table 3.1, C^2M^3 obtains far superior results in terms of accuracy and loss in all considered settings, with accuracy gains as high as +20%. Moreover, our approach natively yields cycle-consistent permutations: fig. 3.2b shows that `Git Re-Basin` [2] accumulates significant error when computing the distance between the source model and the model obtained by applying a cyclic series of permutations, while our approach is able to perfectly recover the source model. This is further confirmed in fig. 3.2a, where we show the loss and accuracy curves when interpolating between a model A and the model mapped back after a cyclic permutation. Models cyclically permuted with `Git Re-Basin` end up in a different basin than the one they started from, while our cycle-consistent approach ensures that the target model is exactly the same as the source. The approach thus matches and merges n models with a significant improvement in performance over the state of the art, while ensuring cycle-consistent permutations.

Model similarity before and after mapping As we can see in fig. 3.5, the cosine similarity of the weights of the models is $3\times$ higher after mapping the latter to the universe. This suggests that the initial quasi-orthogonality of models is at least partially due to neuron permutation symmetries. We also report in section A.3.1 the similarity of the representations between pairs of models. Interestingly, the latter does not change before and after mapping to the universe, but only if we consider a similarity measure that is invariant to orthogonal transformations such as CKA [91]. When using a measure that does not enjoy this property, such as the Euclidean distance, the representations become much

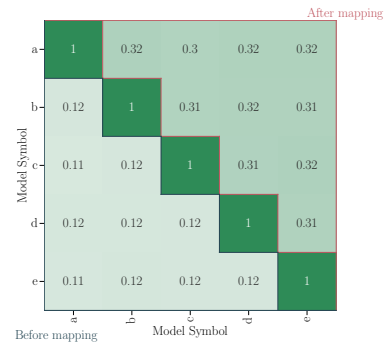


FIGURE 3.5: Cosine similarity of the weights of 5 ResNet20 trained on CIFAR10 with $2\times$ width.

Matcher	EMNIST					CIFAR10					CIFAR100			
	Accuracy ($\uparrow$)		Loss ($\downarrow$)			Accuracy ($\uparrow$)		Loss ($\downarrow$)			Accuracy ($\uparrow$)		Loss ($\downarrow$)	
	train	test	train	test		train	test	train	test		train	test	train	test
Naive	0.03	0.03	3.28	3.28	ResNet _{2x}	0.10	0.10	3.07	3.07	ResNet _{4x}	0.01	0.01	5.30	5.30
MergeMany	0.88	0.86	1.11	1.13		0.38	0.37	2.08	2.06		0.31	0.28	3.01	2.76
MergeMany [†]	0.88	0.86	1.11	1.13		0.50	0.50	2.34	2.30		0.24	0.22	3.31	3.12
C^2M^3	0.89	0.87	1.07	1.10		0.42	0.40	2.11	2.05		0.34	0.30	2.94	2.63
C^2M^3 [†]	0.89	0.87	1.07	1.10		0.72	0.69	1.26	1.12		0.53	0.46	2.13	1.67
Naive	0.04	0.04	4.04	4.04	ResNet _{2x}	0.10	0.10	2.31	2.31	ResNet _{10x}	0.01	0.01	6.22	6.22
MergeMany	0.03	0.03	7.17	7.18		0.10	0.10	2.36	2.36		0.45	0.38	2.32	3.06
MergeMany [†]	0.03	0.03	4.74	4.72		0.60	0.57	1.43	1.32		0.41	0.35	2.27	2.68
C^2M^3	0.27	0.27	3.43	3.47		0.11	0.11	2.34	2.34		0.46	0.39	2.25	3.03
C^2M^3 [†]	0.60	0.60	1.32	1.34		0.64	0.62	1.34	1.23		0.60	0.49	1.43	2.23

TABLE 3.1: Accuracy of the merged model when merging 5 models trained with different initializations. The best results are highlighted in bold. [†] denotes models after the REPAIR operation.

more similar in the universe space. The models are thus approximately $3\times$ more similar in the universe space, and the mapping affects the representations as an orthogonal transformation.

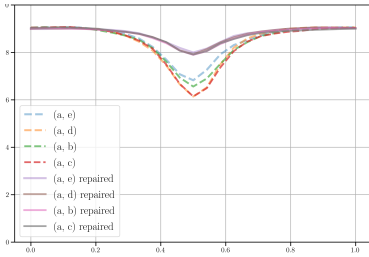


FIGURE 3.6: Interpolation curves of VGG models in the universe.

Effect of activation renormalization Our empirical evidence also points out the benefits of the REPAIR operation [85] that is performed after the merging. In fact, the detrimental effect of model averaging on the activation statistics [85] still applies when taking the mean of n models instead of two. Our results clearly show the benefit of REPAIR, making it a key ingredient of our overall framework. Requiring meaningful interpolation endpoints to be effective, REPAIR has lower benefit when employed on the MergeMany algorithm of Ainsworth et al. [2]. In fact,

iteratively taking means of different random model subsets and aligning the left-out models to the mean is a more complex process than interpolating between some endpoint models. By taking the mean of models in the universe space, we are instead effectively interpolating between endpoint models that can be used for the computation of the statistics in eq. (2.5). Figure 3.6 shows the benefit of using the repair operation on 5 VGG models trained on CIFAR10 mapped to the universe space. Specifically, we fix one model “a” and we linearly interpolate in the universe space with respect to the other models, measuring accuracy before and after applying REPAIR. Beyond boosting performance, we observe that the latter reduces the variance over interpolation paths, resulting in the interpolation curves of all the models overlapping. Using the models in the universe as meaningful endpoints to gather activation statistics, the approach can fully leverage activation renormalization techniques such as REPAIR.

Increasing n In this experiment, we show how the merged model behaves when increasing the number of aggregated models. As we can see in fig. 3.7a, increasing the number of MLPs up to 20 causes the performance to slightly deteriorate in a relative sense, but remains stable in an absolute sense, as it does not fall below 98%. More surprisingly, fig. 3.7b shows that for a ResNet20 architecture with $4\times$ width the loss and accuracy are not monotonic, but rather they seem to slightly fluctuate. This may hint at the merging process being more influenced by the composition of the model set than by its cardinality. Intuitively, a model that is difficult to match with the others will induce a harder optimization problem, possibly resulting in a worse merged model. We dive deeper into the effect of the composition of the set of models in

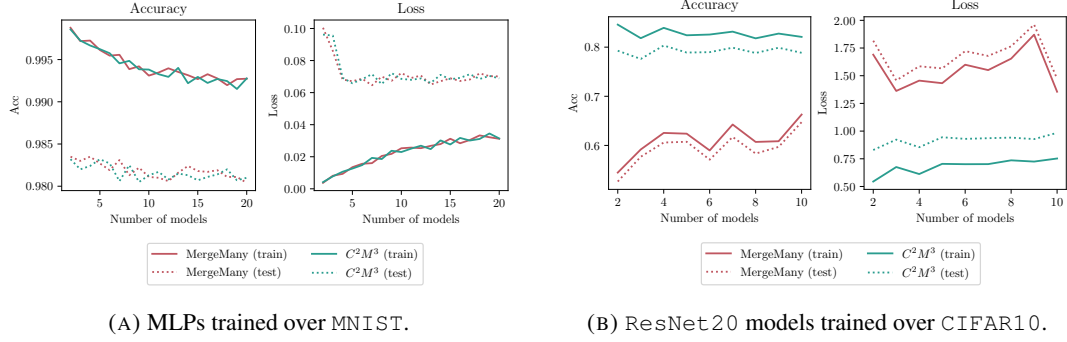


FIGURE 3.7: Accuracy and loss when increasing the number n of models to match and merge.

section A.3.2. The approach is effective in merging a larger number of models, suggesting promise in federated settings.

Varying widths We now measure how architectural width affects model merging, taking into consideration ResNet20 architectures with width $W \in \{1, 2, 4, 8, 16\}$. As we can see in fig. 3.8, *width greatly increases the performance of the merged model*, reaching the zero-loss barrier first observed in [2] when $W = 16$. This is in line with the observations relating linear mode connectivity and network widths [2, 50], and confirms the intuition that the merging is only effective when modes *can* be linearly connected.

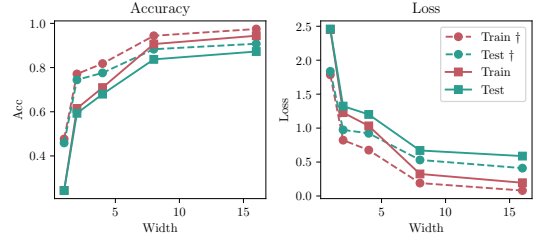


FIGURE 3.8: Accuracy and loss when merging 3 ResNet20s trained over CIFAR10 with different widths. † indicates models after applying REPAIR.

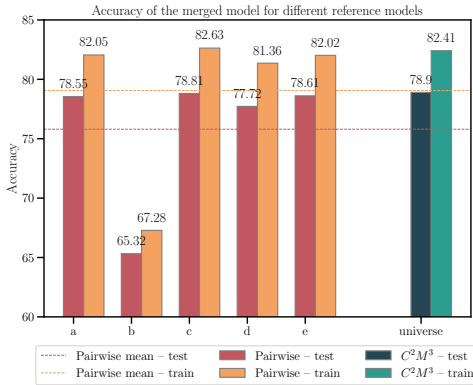


FIGURE 3.9: Accuracy of the merged model when mapping towards one arbitrary model (a, b, c, d, e) versus using our universe space.

a multi-task setting, models optimally mapped to a reference basin would only be able to solve the task solved by the reference model. This would prevent merging from being used for models containing complementary information, such as knowledge fusion [83] or multi-task merging [149]. In our setting, instead, the universe model must by design be a function of all the models and act as a midpoint, hence aggregating information from all the models.

Alternative: fixing one model as universe Alternatively, one could achieve cycle consistency by using one of the source models as a reference and learning pairwise maps towards this one. This, however, would require arbitrarily selecting one of the models, making the overall merging dependent on an arbitrary choice. To see why this matters, we merged 5 ResNet20-4 $\times$ by choosing one model as reference and aggregating the models in its basin. Figure 3.9 shows severe oscillations in the results, with one model reaching an accuracy as low as 65%, while our approach performs as well as the best possible reference. This approach, moreover, does not address multi-model merging, as it is intrinsically pairwise: in

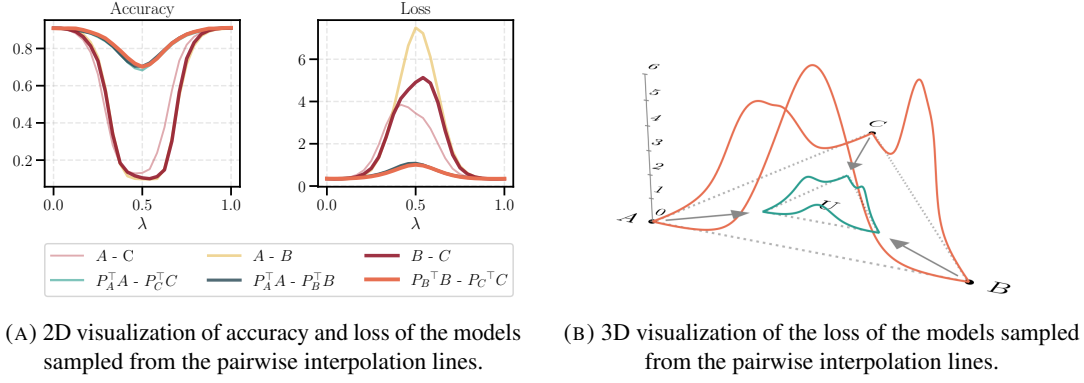


FIGURE 3.10: Linear mode connectivity before and after mapping to the universe for 3 ResNet20-2 $\times$ models trained over CIFAR10 according to algorithm 1.

Linear mode connectivity in the universe Figure 3.10 shows that the loss curves of models interpolated in the universe are much lower than those interpolated in the original space, suggesting that the models are more connected in the former. These results, together with the loss landscape observed in fig. 3.3, encourage merging the models in the universe space due to the lower loss barrier.

3.4 Summary and outlook

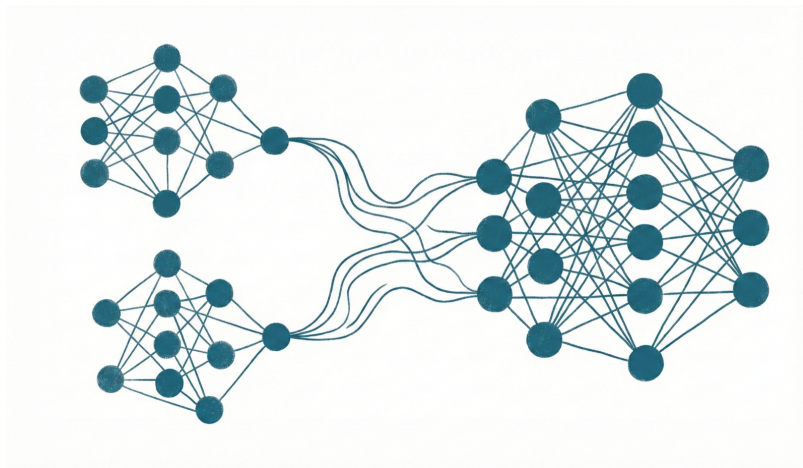
The cycle-consistent framework developed in this chapter solves the multi-model matching and merging problem through a single, principled abstraction: factorizing all pairwise permutations through a shared universe space. The Frank-Wolfe-based optimization jointly handles all layers, guaranteeing cycle consistency and eliminating the need for an arbitrary reference model. Across architectures and datasets, the approach consistently outperforms existing multi-model merging methods, and the formalism elegantly satisfies the requirement for the merging operation to unify different models into a cohesive whole, rather than mapping all of them to one arbitrarily chosen member of the set.

The framework is also general beyond the permutation group: by relaxing permutations to orthogonal matrices, the same universe-factorization principle can be applied to align latent representations across models, as explored in Acharya et al. [1].

The methods developed in part II have addressed the single-task setting, where the central challenge is geometric: aligning independently trained models to overcome permutation symmetries and reduce loss barriers. The approach succeeds precisely because the models share the same objective and differ only in how their neurons are organized. In part III, we turn to a fundamentally different and more practically impactful regime: merging models that have been fine-tuned on *distinct tasks* from a shared pre-trained backbone. Here the challenge is no longer one of geometric alignment (the shared initialization already makes models compatible) but rather one of *task interference*: understanding why aggregating task-specific weight updates works, and how to do so with minimal loss of per-task performance.

Part III

Multi-Task Model Merging



“Perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away.”

— Antoine de Saint-Exupéry

Chapter 4

Foundations of Multi-Task Merging

4.1 Background on multi-task merging

As introduced in section 1.2, this part focuses on merging models that have been fine-tuned on different downstream tasks from a shared pre-trained backbone. The central question is: *can the knowledge acquired by individually fine-tuned models be combined into a single multi-task model, without retraining?* The shared initialization makes these models far more compatible than those trained from different random seeds, but simply averaging their parameters introduces destructive interference that grows with the number of tasks. This section reviews the key ideas and methods that form the technical foundation for the contributions in the following chapters.

4.1.1 Task arithmetic

Among multi-task merging methods, *task arithmetic* [78] stands out for its effectiveness and simplicity. The central abstraction is the *task vector*: given a pre-trained model with parameters θ_0 and a fine-tuned copy θ_t^* specialized on task t , the task vector is defined as the displacement

$$\tau_t = \theta_t^* - \theta_0. \quad (4.1)$$

This displacement isolates the direction in weight space that encodes the knowledge acquired during fine-tuning. The key insight is that these vectors can be composed through elementary arithmetic to edit the model’s behavior. Two primitive operations define the framework. *Addition* constructs a multi-task checkpoint by summing task vectors from different tasks:

$$\theta_{\text{merge}} = \theta_0 + \lambda \sum_{t=1}^T \tau_t, \quad (4.2)$$

where λ is a scaling coefficient chosen on validation. This yields a single model that inherits competencies from all constituent tasks at no additional inference cost. *Negation* subtracts a task vector, $\theta_0 - \lambda\tau_t$, to suppress a specific behavior (e.g., reducing toxicity) while approximately preserving the model’s remaining capabilities.

Empirically, Ilharco et al. [78] demonstrated that negating task vectors can sharply reduce targeted capabilities while addition yields functional multi-task models. However, performance degrades as the number of merged tasks grows, since naive summation treats all parameters uniformly and does not explicitly manage conflicts between task vectors.

4.1.2 Mitigating multi-task interference

While task arithmetic is simple and often effective, standard nonlinear fine-tuning inevitably introduces inter-task interference when task vectors are summed. A natural line of work tackles

this interference through post-hoc interventions on precomputed task vectors, identifying and removing the parameter-level conflicts responsible for performance degradation.

Yadav et al. [176] diagnosed two concrete failure modes of naive task arithmetic: *redundant parameters*, whose small magnitudes contribute noise rather than signal under averaging, and *sign conflicts*, where different task vectors push the same parameter in opposite directions, causing destructive cancellation. Their TIES-Merging framework (Trim, Elect Sign, Disjoint merge) addresses both issues through a three-step pipeline: (i) *trim* each task vector by retaining only the top- $k\%$ of parameters by absolute magnitude; (ii) *elect* a dominant sign per coordinate via a majority vote weighted by magnitude; and (iii) *disjoint merge* by averaging only those entries whose sign agrees with the elected sign, discarding conflicting updates. Empirically, up to 80% of delta parameters could be trimmed without statistically significant performance loss, and enforcing sign consistency substantially improved multi-task coherence, with gains growing as more tasks are merged.

A complementary perspective on parameter redundancy was offered by Yu et al. [183] with DARE (Drop And REscale). For models obtained via supervised fine-tuning (SFT), the delta parameters tend to be remarkably small in magnitude, implying that the vast majority of individual updates are individually dispensable. DARE exploits this by randomly dropping a fraction p of delta parameters (routinely $p \in [0.9, 0.99]$) and rescaling the survivors by $1/(1 - p)$ to preserve the expected activation magnitude. The extreme sparsification dramatically reduces the probability of two task vectors interfering at the same coordinate, and the sparsified vectors can then be combined using standard task arithmetic or TIES. An important caveat is that DARE relies on the SFT deltas being small; when fine-tuning amounts to substantial continued pre-training, even moderate dropping rates cause catastrophic performance collapse.

As the number of merged tasks scales, even interference-aware methods like TIES exhibit degradation. Wang et al. [166] investigated whether task information is truly destroyed by merging or merely hidden by interference, finding support for the latter view. Their TALL-masks framework constructs per-task binary masks m_t that approximate each single-task vector from the dense multi-task vector $\tau_{\text{MTL}} = \sum_t \tau_t$, using a closed-form criterion that identifies coordinates where a task’s own contribution dominates over the aggregate of all other tasks. Analyzing the intersection patterns of these masks reveals three regimes: *selfish* weights critical to one task but harmful to others, *catastrophic* weights that no task considers critical, and *general* weights selected by multiple tasks forming a consensus backbone. The resulting *Consensus Merging* strategy retains only general weights, yielding substantial improvements.

An alternative perspective is offered by Tam et al. [153], who formulate merging as finding a solution in the *task parameter subspace* (the span of the individual task vectors) that simultaneously approximates all single-task models. This reduces the problem to solving a linear system, which the authors tackle with the conjugate gradient method, achieving strong results in both multi-task and intermediate-task merging.

Daheim et al. [35] framed merging as minimizing the mismatch between the gradients of the individual models and those of the merged model, proposing an uncertainty-based scheme that is more robust to hyperparameter choices than standard task arithmetic.

These methods share a common principle: multi-task interference can be substantially reduced by identifying and removing the specific parameter coordinates responsible for conflicts, or by finding better combinations within the space spanned by the task vectors. Chapter 6 presents an alternative view rooted in spectral properties of the layer-wise weight updates.

4.1.3 Routing-based merging

Rather than committing to a single static merge, routing-based methods [68, 104, 155, 177] make composition input-dependent: at inference time, a gating mechanism selects or weights

task-specific components, in the spirit of Mixture-of-Experts models. This preserves specialization while still sharing most backbone parameters. Existing approaches span a broad design space. In LLMs, several methods train routers over LoRAs or adapters [24, 81, 115], or softly merge expert modules via learned weighted combinations [114, 123]. Muqeeth et al. [114] proposed SMEAR, which replaces discrete expert routing with a single merged expert formed by a differentiable weighted average of parameters, enabling standard gradient-based training. Ostapenko et al. [123] extended the idea of adapter libraries by introducing model-based clustering to organize LoRAs by similarity and a zero-shot routing mechanism (ARROW) that selects relevant adapters for new inputs without retraining. These approaches can require additional supervision or optimization. Other methods target post-hoc composition from existing checkpoints: some still depend on modified fine-tuning pipelines (e.g., *Transformer*² [151]), whereas others are explicitly data- and training-free, routing low-rank task updates at the layer level while leaving the pre-trained backbone unchanged. Chapter 7 presents one such approach.

4.1.4 Optimization-based model merging

A complementary line of work casts model merging as an explicit optimization problem, rather than relying on fixed interpolation rules such as uniform averaging. Instead of retraining weights, these methods directly optimize a merging objective using information available at the endpoints, either in parameter space or through low-dimensional combination rules.

A canonical example is Fisher-weighted merging [107], which adopts a Bayesian perspective on model combination. Each fine-tuned model is approximated by a Gaussian posterior obtained via a second-order (Laplace) expansion of the loss around its optimum, with the diagonal Fisher information acting as a precision matrix. Merging then amounts to maximizing the joint posterior, yielding a closed-form solution in which each parameter is set to a Fisher-weighted average across models. This induces a geometry-aware merge that preserves parameters deemed important for each task while allowing flexibility along directions of low curvature.

More recently, black-box optimization strategies have been proposed to directly search for high-performing merges. Evolutionary approaches [3] treat model merging as a fitness-driven process, using validation performance to guide exploration over merging strategies. While this enables highly flexible, non-differentiable search and can discover strong merges in challenging settings, it incurs substantial computational cost due to repeated model evaluations. In chapter 8, we show how this cost can be drastically reduced while preserving the benefits of adaptive, objective-driven merging.

4.1.5 Understanding why task arithmetic works

The methods reviewed so far operate on task vectors to improve merging performance, but a deeper question remains: under what conditions does task arithmetic succeed in the first place?

Ortiz-Jiménez et al. [122] formalized a sufficient condition: task arithmetic succeeds when the model satisfies a *locality property*, meaning each task vector affects the model’s output only on the data support of its own task and leaves outputs unchanged elsewhere. When this non-interference condition holds, task vectors can be summed without mutual corruption.

The authors identified *weight disentanglement* as the core enabling mechanism: when approximately orthogonal directions in parameter space govern separate regions of function space. Crucially, this disentanglement is not present at random initialization; it is an emergent consequence of large-scale pre-training. Standard nonlinear fine-tuning, however, introduces curvature that warps these directions, causing task vectors to interfere when summed.

To amplify disentanglement, the authors proposed performing fine-tuning in the *tangent space* of the pre-trained model, i.e., linearizing the network via a first-order Taylor expansion around θ_0 :

$$f(x; \theta) \approx f(x; \theta_0) + \nabla_{\theta} f(x; \theta_0)^{\top} (\theta - \theta_0). \quad (4.3)$$

This linearization ensures that the resulting task vectors remain strictly in a regime where superposition is benign. The theoretical justification is grounded in an eigenfunction localization property: if the Neural Tangent Kernel eigenfunctions associated with different tasks have non-overlapping supports, the corresponding weight updates are orthogonal and non-interfering.

In practice, linear fine-tuning of CLIP ViT models improved absolute accuracy on multi-task addition by up to 5 points and task negation by up to 13 points compared to standard nonlinear fine-tuning. While this confirms the centrality of disentanglement, the approach has a practical limitation: it requires modifying the fine-tuning procedure itself, and thus cannot be applied post-hoc to existing checkpoints.

In this regime, parameter updates are entirely determined by first-order loss gradients evaluated at θ_0 , with higher-order curvature effects explicitly suppressed. If linearization around θ_0 is the regime where task arithmetic is most principled, then task vectors should be closely related to the accumulation of early training gradients evaluated near θ_0 . Chapter 5 makes this connection precise, showing that task vectors from short fine-tuning are approximate negative gradients, and that this viewpoint yields both theoretical guarantees and practical improvements.

Chapter 5

Task Vectors as Approximate Gradients

In this chapter, we provide a theoretical foundation for task arithmetic. We show that, under full-batch gradient descent, a single-epoch task vector is exactly the scaled negative gradient of the task loss, so that summing task vectors is equivalent to one step of joint multi-task training. Beyond one epoch, the two approaches diverge by a curvature-controlled $O(\eta^2)$ term, which we bound explicitly for feed-forward networks.

5.1 Motivation and Overview

Having addressed single-task merging in part II, we now turn to the multi-task regime introduced in section 4.1. The shared pre-trained backbone makes independently fine-tuned models far more compatible than those trained from different random seeds, eliminating the need for explicit alignment. Yet a new difficulty arises: when multiple task vectors are simply summed, they interfere with one another in ways that degrade per-task performance. Understanding this interference calls for a theoretical account of what task vectors actually encode.

As introduced in section 4.1.1, task arithmetic constructs a multi-task model by summing the task vectors of individually fine-tuned models. Despite its widespread adoption, a rigorous understanding of *why* task arithmetic works has remained elusive. Prior work [71, 104, 191] has observed that task vectors from shorter fine-tuning, although less performant on the individual task, are better suited for merging, but without a formal explanation.

This chapter bridges the gap by connecting task vectors to gradient descent dynamics. Under full-batch GD, a task vector from one epoch of fine-tuning is precisely the negative loss gradient scaled by the learning rate (fig. 5.1). Summing these vectors therefore recovers a single GD step on the aggregated multi-task loss, with the scaling factor α acting as an effective learning rate. For multi-epoch fine-tuning the equivalence holds approximately, with an $O(\eta^2)$ deviation controlled by loss curvature.

These results carry a practical implication: merging models fine-tuned for a single epoch can match or exceed the performance of merging fully converged models, since first-epoch task vectors are the most faithful gradient approximations. More broadly, they reframe task arithmetic as approximate multi-task gradient descent and highlight the outsized role of early training dynamics.

5.2 Formalizing the task vector–gradient equivalence

This section formalizes the relationship between task vectors and multi-task loss gradients. For analytical clarity, we model learning as *full-batch* gradient descent with a fixed step size η ,

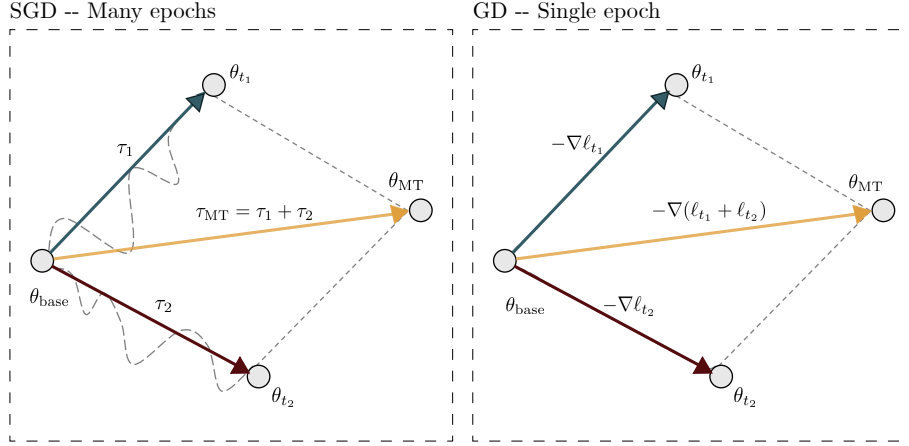


FIGURE 5.1: Left: endpoint models are fine-tuned with SGD for more than one epoch. Right: endpoint models are fine-tuned with GD for a single epoch, making task vectors equivalent to negative gradients.

an assumption common in theoretical studies of convergence and generalization [6, 120] that simplifies the derivations while preserving key insights into the optimization dynamics.

Recall that vanilla task arithmetic (TA) [78] constructs a multi-task model by (i) fine-tuning the base network separately on each task, and (ii) adding the resulting task vectors, scaled by a coefficient α , to the pre-trained weights. In the full-batch setting, we show that TA is *exactly* equivalent to one epoch of joint training; thereafter, the two approaches diverge by a curvature-controlled $O(\eta^2)$ term.

We first introduce the notation used throughout this chapter.

Notation. Let T denote the set of tasks, with $|T|$ its cardinality. The pre-trained weights are θ_{pre} . For a task $t \in T$, let $\theta_t^{(k)}$ be the parameters after k epochs of fine-tuning on t , and define the *task vector* $\tau_t^{(k)} := \theta_t^{(k)} - \theta_{\text{pre}}$, i.e., the parameter displacement induced by fine-tuning [78]. Fine-tuning on task t minimizes the empirical loss

$$\bar{L}_t(\theta) := \frac{1}{n_t} \sum_{i=1}^{n_t} \ell(x_i, y_i, \theta),$$

where n_t is the number of training samples for task t and ℓ is the per-sample loss. We write $\theta_{\text{MT}}^{(k)}$ for the parameters after k epochs of *joint* training on the combined loss $\sum_{t \in T} \bar{L}_t(\theta)$, and define the *multi-task vector* $\tau_{\text{MT}}^{(k)} := \sum_{t \in T} \tau_t^{(k)}$.

Theorem 5.2.1. Let $\theta_{\text{TA}}^{(k)} = \theta_{\text{pre}} + \alpha \sum_{t \in T} \tau_t^{(k)}$ be the model obtained using vanilla task arithmetic with parameter α . Let $\{\theta_t^{(k)}\}_{t \in T}$ be produced by running k full-batch GD epochs with step size η on each task, and let $\theta_{\text{MT}}^{(k)}$ be the model obtained from k GD epochs with step size $\alpha\eta$ on the aggregated loss $\sum_{t \in T} \bar{L}_t$. Then

$$\theta_{\text{TA}}^{(1)} = \theta_{\text{MT}}^{(1)}, \quad (5.1)$$

$$\theta_{\text{TA}}^{(k)} = \theta_{\text{MT}}^{(k)} + \eta^2 C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^{k-2}) + O(\eta^3), \quad k > 1, \quad (5.2)$$

where the curvature term C and the per-task residual r_t are defined as

$$C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h) = \sum_{t \in T} \sum_{e=0}^h \nabla^2 \bar{L}_t(\theta_{\text{MT}}^{(e)}) \sum_{m=0}^e r_t(\theta_{\text{MT}}^{(m)}), \quad (5.3)$$

$$r_t(\theta) := \alpha \sum_{\substack{t' \neq t \\ t' \in T}} \nabla \bar{L}_{t'}(\theta) + (\alpha - 1) \nabla \bar{L}_t(\theta). \quad (5.4)$$

Remark 5.2.2. Theorem 5.2.1 assumes full-batch gradient descent with a fixed step size. Extending it to mini-batch SGD with momentum, Adam, or AdamW remains open: stochastic noise, adaptive step sizes, and momentum break the algebraic cancellations exploited in the proof. Nonetheless, the empirical evidence in this chapter suggests that the qualitative insights (first-epoch dominance, curvature-controlled divergence) carry over to practical training regimes.

Equation (5.1) states that after a single epoch, TA matches multi-task training *exactly*: at $k=1$ the task vectors are precisely the negative gradients of the individual task losses, and summing them recovers the joint gradient step with no approximation error (fig. 5.1).

For $k > 1$ the exact equivalence breaks down. The leading $O(\eta)$ terms cancel, and the gap reduces to a curvature-controlled residual $\eta^2 C(\cdot)$ plus higher-order corrections $O(\eta^3)$. The term $C(\cdot)$ accumulates Hessian–gradient interactions along the joint training trajectory: TA and multi-task training diverge whenever the loss landscape curves differently across tasks. This perturbative view pinpoints the source of the gap and shows that the approximation degrades *gradually* with training, rather than failing abruptly after the first epoch.

5.2.1 Dominance of the first epoch

Even beyond the first epoch, task vectors remain effective because much of the fine-tuning trajectory is dictated by the initial gradient. We verify this with a controlled experiment.

We compare TA performance when merging models fine-tuned to convergence versus after a single epoch (fig. 5.2). Remarkably, the single-epoch variant performs competitively across all tasks, consistent with our theoretical finding that early task vectors are the most faithful gradient approximations.

To understand why, we examine how much each epoch contributes to the overall optimization. Figure 5.3a plots the epoch-wise normalized gradient norm $\frac{\|\nabla_{\theta}^{(k)} L\|}{\sum_{k'=1}^K \|\nabla_{\theta}^{(k')} L\|}$: the first epoch contributes the largest share across all tasks. Even on datasets where this dominance is less pronounced (e.g., Cars, SVHN), the cumulative gradient through k epochs remains well-aligned with the first-epoch gradient: fig. 5.3b shows that $\cos(\tau_1, \tau_k)$ exceeds 0.8 at epoch 2 for all tasks and remains above 0.6 through epoch 10, decaying gradually as training progresses. Fully converged task vectors favor task-specific performance, while first-epoch vectors are better gradient surrogates; both yield similar multi-task accuracy after merging, but the single-epoch setting is considerably cheaper.

Our formal results assume GD, but treating adaptive optimizers as approximate GD preserves the core intuition: reducing task interference in the merged model corresponds to minimizing gradient conflicts in multi-task learning.

5.2.2 Proof sketch

The full derivation is in section B.1.1; we outline the key ideas here. The technical backbone is the following proposition, which gives a term-by-term Taylor expansion of the multi-task vector; Theorem 5.2.1 follows as a corollary.

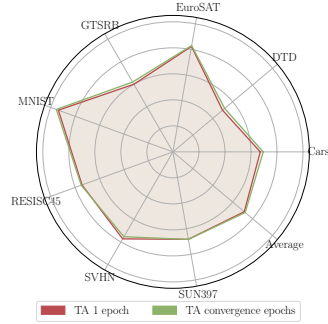
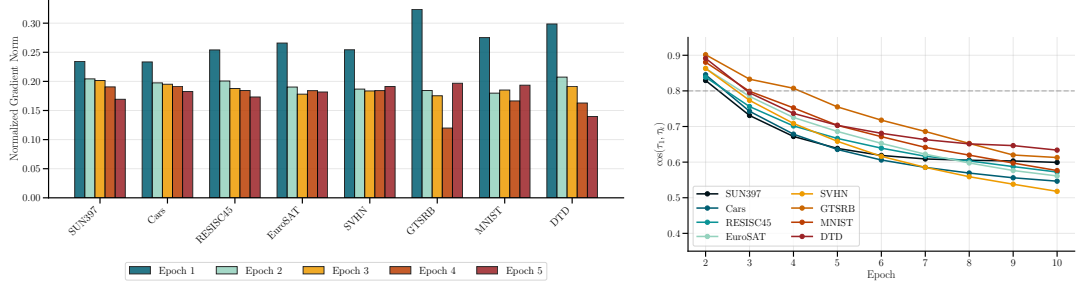


FIGURE 5.2: Task arithmetic accuracy: 1 epoch vs. converged.



(A) Normalized gradient norms for the first 5 epochs of fine-tuning. (B) Cosine similarity of cumulative gradients with respect to the first-epoch gradient.

FIGURE 5.3: Analysis of first-epoch gradients.

Proposition 5.2.3. Let $\{\theta_t^{(k)}\}_{t \in T}$ be produced by fine-tuning the base model for k full-batch GD epochs with step size η for each task, and let $\theta_{MT}^{(k)}$ be the model obtained from k GD epochs with step size $\alpha\eta$. It holds that

$$\tau_{MT}^{(1)} = -\eta \nabla \sum_{t \in T} \bar{L}_t(\theta_{pre}) \quad (5.5)$$

$$\tau_{MT}^{(k)} = -\eta \sum_{t \in T} \sum_{j=0}^{k-1} \nabla \bar{L}_t(\theta_{MT}^{(j)}) + \eta^2 C(\{\theta_{MT}^{(j)}\}_{j=1}^{k-2}) + O(\eta^3) \quad \forall k \geq 2 \quad (5.6)$$

Remark 5.2.4. In the single-task case, Proposition 5.2.3 immediately gives $\tau_t = -\eta \nabla \bar{L}_t(\theta_{pre})$ after one epoch: the task vector is exactly the scaled negative gradient, and adding it to the pre-trained model is equivalent to a single gradient descent step on the task loss.

The proof compares multi-task training and vanilla task arithmetic, showing that they differ only by a curvature-controlled second-order term. It proceeds in two stages.

Stage 1: Decomposing Single-Task Error TA fine-tunes each task separately and then adds the resulting task vectors $\tau_t^{(k)} = \theta_t^{(k)} - \theta_{pre}$. The key question is how far each single-task iterate $\theta_t^{(k)}$ drifts from the multi-task iterate $\theta_{MT}^{(k)}$. Lemma 5.2.5 shows that this distance decomposes into (i) a linear drift term, (ii) a quadratic curvature correction, and (iii) higher-order remainders negligible for small η .

The lemma is expressed in terms of three auxiliary quantities r_t , p_t^k , and s_t^k , each capturing a distinct effect when comparing single-task and multi-task trajectories:

- $r_t(\cdot)$ quantifies the mismatch between the two gradients at one step.

$$r_t(\theta) = \alpha \sum_{\substack{t' \neq t \\ t' \in T}} \nabla \bar{L}_{t'}(\theta) + (\alpha - 1) \nabla \bar{L}_t(\theta) = \alpha \sum_{t' \in T} \nabla \bar{L}_{t'}(\theta) - \nabla \bar{L}_t(\theta) \quad (5.7)$$

- $p_t^k(\cdot)$ represents the accumulation of those mismatches over many steps.

$$p_t^k(\theta_{pre}, \theta_{MT}^{(1)}, \dots, \theta_{MT}^{(k)}) = \sum_{j=0}^k r_t(\theta_{MT}^{(j)}) \quad (5.8)$$

- $s_t^k(\cdot)$ accounts for the extra bending introduced by curvature once the paths have parted.

$$s_t^k(\theta_{pre}, \dots, \theta_{MT}^{(k)}) = \sum_{j=0}^k \nabla^2 \bar{L}_t(\theta_{MT}^{(j)}) [p_t^j(\theta_{pre}, \dots, \theta_{MT}^{(j-1)})]. \quad (5.9)$$

Lemma 5.2.5. *Using the notation of Proposition 5.2.3, for $m = 1$:*

$$\theta_t^{(1)} = \theta_{\text{MT}}^{(1)} + \eta p_t^0(\theta_{\text{pre}}), \quad (5.10)$$

and for $m \geq 2$:

$$\theta_t^{(m+1)} = \theta_{\text{MT}}^{(m+1)} + \eta p_t^m(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m)}) - \eta^2 s_t^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3). \quad (5.11)$$

The case $m = 1$ is immediate; for $m \geq 2$ the proof proceeds by induction (see section B.1.1). We sketch the base step $m = 2$: Taylor-expanding the single-task update around the multi-task iterate yields (i) a first-order linear term (p_t^1), accumulating the gradient mismatch from previous epochs, and (ii) a second-order curvature correction (s_t^0), capturing how the loss surface of task t bends around the multi-task trajectory. Rearranging recovers the lemma’s structure for the second epoch.

Stage 2: Error Aggregation across Tasks With the per-task decomposition in hand, we sum over all tasks to obtain $\theta_{\text{TA}} - \theta_{\text{MT}}$. The linear terms in η cancel, and the quadratic curvature terms accumulate into the coefficient C (eq. (5.3)).

Epoch 1. A direct computation shows $\theta_{\text{TA}}^{(1)} = \theta_{\text{MT}}^{(1)}$:

$$\theta_{\text{TA}}^{(1)} = \theta_{\text{pre}} - \eta \alpha \sum_{t \in T} \nabla \bar{L}_t(\theta_{\text{pre}}) = \theta_{\text{MT}}^{(1)},$$

where the right-hand side uses learning rate $\alpha\eta$ for the joint loss.

Epochs $k \geq 2$. Applying Lemma 5.2.5, each single-task parameter decomposes into the multi-task parameter plus a first-order drift (ηp_t^k) and a second-order curvature term ($-\eta^2 s_t^{k-1}$). When these are summed over all tasks to form the task arithmetic aggregate, the linear drift terms cancel by symmetry ($\sum_t r_t(\theta) = 0$ for $\alpha = 1/|T|$), leaving only the quadratic block C . This yields exactly the $O(\eta^2)$ gap in Theorem 5.2.1.

5.2.3 Bounding the second-order deviation term

The error term C defined in eq. (5.3) captures the second-order gap between TA and multi-task training. To characterize how large this gap can grow, we derive explicit, uniform bounds on $\|C\|_2$ under mild structural assumptions: feed-forward networks with bounded weights, bounded inputs, and activations with controlled first and second derivatives. These assumptions are standard in the neural network theory literature [9, 62, 181].

Theorem 5.2.6 (Uniform bound on the coefficient vector $C(\{\theta_{\text{MT}}^{(j)}\})$). *Assume the hypothesis of Proposition 5.2.3 holds and let $C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h)$ be the error term defined in eq. (5.3). We assume the tasks are all classification tasks optimized with cross-entropy loss. We also add a few structural constraints on the network itself. Specifically the model is a depth- L feed-forward network with weight matrices $W^{(1)}, \dots, W^{(L)}$. For every layer, there exist positive constants s_ℓ such that $\|W^{(\ell)}\|_2 \leq s_\ell$. The inputs are also bounded $\|x\|_2 \leq M_x$. Finally, we require the activation functions to have bounded first and second derivatives, i.e., there are $\beta_\phi, \gamma_\phi > 0$ such that $\sup_z |\phi'(z)| \leq \beta_\phi$ and $\sup_z |\phi''(z)| \leq \gamma_\phi$. Setting $\Pi = \prod_{\ell=1}^L s_\ell$, we obtain the following.*

(i) **For general activations:**

$$\|C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h)\|_2 \leq T \binom{h+2}{2} |\alpha T - 1| H_{\text{max}}^\phi G_{\text{max}}^\phi,$$

$$\text{with } H_{\text{max}}^\phi \leq 2 \gamma_\phi M_x^2 \Pi^2 \beta_\phi^{2L-2} \text{ and } G_{\text{max}}^\phi \leq \sqrt{2} M_x \Pi \beta_\phi^{L-1}.$$

(ii) **For ReLU activations** ($\gamma_\phi = 0, \beta_\phi = 1$):

$$\|C(\{\theta_{MT}^{(j)}\}_{j=1}^h)\|_2 \leq \frac{T}{2} \binom{h+2}{2} |\alpha T - 1| H_{\max}^{\text{ReLU}} G_{\max}^{\text{ReLU}}$$

$$\text{with } H_{\max}^{\text{ReLU}} \leq \frac{1}{2} \sqrt{2} M_x^3 \Pi^3 \beta_\phi^{3L-3} \text{ and } G_{\max}^{\text{ReLU}} \leq \sqrt{2} M_x \Pi.$$

The bound separates into a *task-dependent* factor $T \frac{(h+1)(h+2)}{2} |\alpha T - 1|$ and a *network-dependent* factor controlled by $\{M_x, \Pi, \beta_\phi, \gamma_\phi\}$. For ReLU activations the bound tightens because the piecewise-linear activation has zero second derivative almost everywhere.

The complete derivation is in section B.1.2; the proof proceeds in three steps:

(i) We provide a bound over the ℓ_2 norm of $C(\{\theta_{MT}^{(j)}\}_{j=1}^h)$:

$$\|C\|_2 \leq T \frac{(h+1)(h+2)}{2} |\alpha T - 1| H_{\max}^\phi G_{\max}^\phi$$

where we define the uniform bounds:

- **Hessian bound** $H_{\max}^\phi := \max_{t,\ell} \|\nabla^2 \bar{L}_t(\theta_{MT}^{(\ell)})\|_2$
- **Gradient bound** $G_{\max}^\phi := \max_{t,m} \|\nabla \bar{L}_t(\theta_{MT}^{(m)})\|_2$

(ii) Then, we bound the Hessian bound $H_{\max}^\phi$ and the gradient bound $G_{\max}^\phi$ as follows:

- $H_{\max}^\phi \leq 2 \gamma_\phi M_x^2 \Pi^2 \beta_\phi^{2L-2}$ (for general activations)
- $G_{\max}^\phi \leq \sqrt{2} M_x \Pi \beta_\phi^{L-1}$

where we assumed that $\|x\|_2 \leq M_x$, $\|W^{(\ell)}\|_2 \leq s_\ell$, $|\phi'(z)| \leq \beta_\phi$.

(iii) Finally, plugging $H_{\max}^\phi$ and $G_{\max}^\phi$ explicitly into the inequality reproduces exactly the two cases stated in Theorem 5.2.6.

5.3 Scope and limitations

The analysis above assumes full-batch gradient descent (GD), whereas in practice task vectors are produced by stochastic optimizers (SGD, Adam, AdamW). The noise, momentum, and adaptive step sizes of these methods break the algebraic cancellations on which the proof relies, and extending the results to stochastic settings remains open. Likewise, the closed-form bound on C applies to feed-forward networks; CNNs and Transformers introduce architectural elements (weight sharing, attention) that the current framework does not cover. A formal treatment under SGD and extensions to modern architectures are natural next steps; we discuss them further in part IV.

5.4 Implications for merging practice

The formal results above have direct implications for merging practice. We examine how the parameter-space trajectory is shaped by the merging strategy and why task proficiency does not necessarily correlate with mergeability.

5.4.1 Parameter space trajectory

Most task-arithmetic-based approaches construct a merged model in a single shot by aggregating task vectors extracted from the pretrained initialization. From the gradient perspective developed above, this corresponds to taking a single large step in parameter space along the sum of task gradients evaluated at θ_{pre} . While effective in many cases, this strategy relies on a linear approximation that is only guaranteed to be accurate locally. When task gradients are misaligned or curvature effects accumulate, a single extrapolation can move the parameters away from low-loss regions.

Several recent works implicitly address this issue by spreading task arithmetic over time rather than performing it in a single step. In particular, iterative task arithmetic [191] alternates short fine-tuning phases with repeated merging operations, effectively re-estimating task vectors around the current parameters at each iteration. While originally proposed as a practical heuristic, this procedure can be interpreted as tracking a trajectory in parameter space through a sequence of small, locally valid updates.

Figure 5.4 provides a geometric illustration of this effect via a 2D PCA projection of the resulting checkpoints. TIES and Model Breadcrumbs converge to similar basins, while DARE diverges due to stochastic pruning. In contrast, the trajectory produced by iterative merging progresses smoothly through parameter space, remaining closer to low-loss regions throughout optimization. This empirical behavior is consistent with the gradient interpretation of task vectors developed in this chapter: smaller, repeated steps better respect the local geometry of the loss landscape and mitigate task interference.

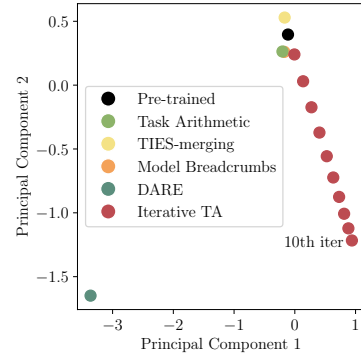


FIGURE 5.4: Checkpoint projection of different strategies.

5.4.2 Task proficiency is not mergeability

One might expect that more specialized, higher-performing models would produce better merged results, but the evidence points the other way. As shown in fig. 5.2, merging models fine-tuned to convergence yields no gains over merging single-epoch models.

From the gradient perspective, the explanation is direct: the longer a model fine-tunes, the more its task vector deviates from the true multi-task gradient. The first epoch’s task vector is an exact scaled negative gradient; subsequent epochs introduce a curvature-controlled $O(\eta^2)$ error. Highly specialized models also diverge further in parameter space [104], making naive aggregation less effective (fig. 5.4).

These findings complement the *weight disentanglement* view of Ortiz-Jiménez et al. [122] (see section 4.1.5), which shows that task vectors are more reliable when models remain close to the pre-trained initialization, where functional components are more linearly separable. The gradient perspective offers an orthogonal rationale grounded in optimization dynamics: *models that stay closer to the pre-trained base are more mergeable* because their task vectors are more faithful gradient approximations.

5.5 Summary and outlook

This chapter established a formal connection between task vectors and loss gradients. After a single epoch of gradient descent, a task vector is exactly the scaled negative gradient; applying task arithmetic is therefore equivalent to one step of joint multi-task training. For

multi-epoch fine-tuning the equivalence holds approximately, with a quantifiable $O(\eta^2)$ error bounded explicitly for feed-forward networks. Empirical investigation across multiple vision benchmarks corroborates this picture: the first-epoch gradient dominates the fine-tuning trajectory in both magnitude and direction, and merging single-epoch models matches the performance of merging fully converged ones.

The gradient interpretation also carries a structural implication. If a task vector approximates the negative gradient of the task loss, it encodes a specific *direction* in weight space pointing toward lower loss for that task. Does this directional information have further exploitable structure? In chapter 6, we answer affirmatively, showing that per-layer task matrices are intrinsically low-rank and leveraging this structure to simultaneously compress task vectors and reduce inter-task interference.

Chapter 6

Exploiting the Low-Rank Structure of Task Vectors

In this chapter, we study the layer-wise matrix structure of weight updates rather than treating them as flat vectors. We decompose per-layer task matrices via SVD, showing they are intrinsically low-rank, and leverage this structure for both compression (TSV-Compress) and interference reduction (TSV-Merge) during merging.

6.1 From flat vectors to layer-wise structure

The gradient perspective established in chapter 5 reveals that task vectors encode structured, directional information about each task’s loss landscape, information that a flat-vector treatment discards. Indeed, most multi-task merging methods treat entire networks as high-dimensional vectors, overlooking the layer-wise matrix structure of weight updates [35, 39, 83, 107, 134, 176, 183]. This flattened view limits these approaches to coarse-grained measures like cosine similarity for assessing inter-task interactions.

The approach developed here instead preserves the natural matrix structure of neural network layers. The starting point is to examine per-layer task *matrices*, which represent weight changes for each task, through singular value decomposition (SVD). This decomposition yields singular vectors and singular values that capture the most significant directions of variation within each layer. We term these singular vectors *Task Singular Vectors* (TSVs), as they provide an interpretable basis for assessing task-specific contributions at each layer. Since task vectors approximate gradients, they may inherit a well-documented property of gradients: low-rank structure. Gradients in deep networks have been repeatedly observed to be low-rank [147, 187], and the effectiveness of LoRA [72], which reparametrizes weight updates as low-rank products, further supports this. Our analysis confirms that task matrices are indeed low-rank: only a small fraction of TSVs suffices to represent each layer’s function with high fidelity.

Building on this insight, we introduce TSV-Compress (TSV-C), a simple yet effective procedure to compress task vectors by a factor proportional to the number of tasks while maintaining 99% of the original accuracy. Focusing on only a fraction of the most relevant TSVs for each task, TSV-Compress enables efficient storage and processing of task-specific information without sacrificing performance.

Beyond compression, examining how TSVs from different tasks align or diverge within each layer provides a fine-grained, geometrically grounded view of task interference, going beyond global similarity metrics like cosine similarity. Building on this, we introduce TSV-Merge (TSV-M), which combines compression and interference reduction: it discards the least relevant singular vectors per task and then decorrelates the remaining ones via a

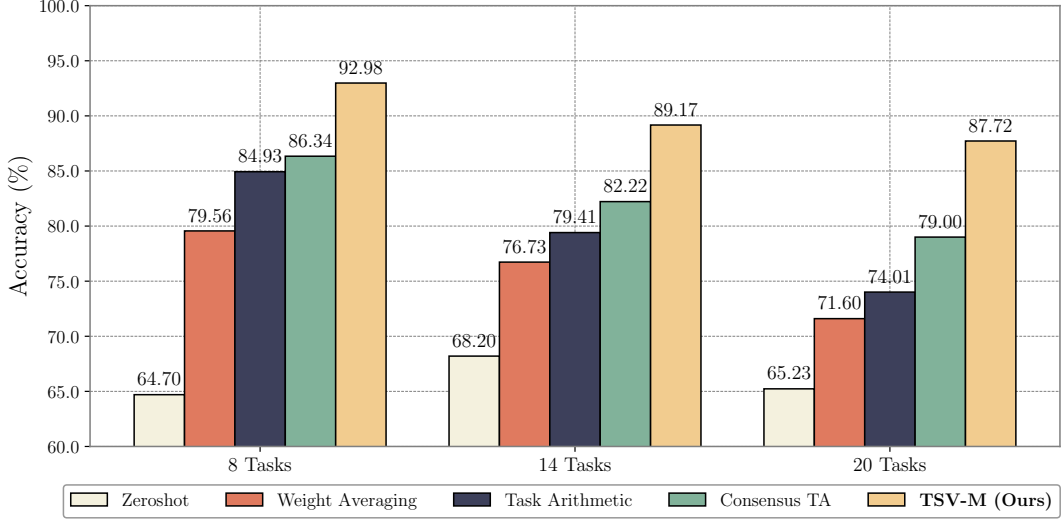


FIGURE 6.1: Mean accuracy of a ViT-L-14 merged over 8, 14, and 20 tasks respectively. By significantly surpassing existing methods, TSV-M establishes the new state of the art in model merging.

whitening transformation. Across several benchmarks, TSV-M outperforms existing methods by an average of ~ 15 percentage points. All code is publicly available¹.

6.2 Task singular vectors and their properties

6.2.1 Layer-wise task matrices and their SVD

As introduced in section 4.1.1, task arithmetic operates on entire task vectors $\tau_t = \theta_t - \theta_{\text{pre}}$, treating them as flat vectors in parameter space. A finer-grained view considers these operations at the individual layer level. The per-layer *task matrix* for task i at layer ℓ is $\Delta_i^{(\ell)} = \theta_i^{(\ell)} - \theta_{\text{pre}}^{(\ell)}$, and the merged weights become:

$$\theta_{\text{MT}}^{(\ell)} = \theta_{\text{pre}}^{(\ell)} + \alpha \frac{\sum_{i=1}^T \Delta_i^{(\ell)}}{T}. \quad (6.1)$$

When layers have a matrix structure (e.g., linear projections in attention or MLP blocks), this formulation preserves structural information that is lost in the flat-vector view. For brevity, we omit the layer index and refer to $\Delta_i^{(\ell)}$ simply as Δ_i .

Decomposing layer task matrices

Treating layer-wise weights as matrices rather than flattened vectors enables us to analyze their SVD. Given two tasks i, j , we consider the SVD of their task matrices Δ_i and Δ_j at a generic layer:

$$\Delta_i = U_i \Sigma_i V_i^\top, \quad \Delta_j = U_j \Sigma_j V_j^\top$$

where U_i, U_j and V_i, V_j are the matrices of left and right singular vectors, respectively, and Σ_i, Σ_j are diagonal matrices of singular values. We term the singular vectors *Task Singular Vectors* (TSVs).

We can equivalently write the aggregated task matrices in eq. (6.1) in terms of their SVDs. Defining $U = [U_1 \cdots U_T]$ as the column-wise concatenation of all the left TSVs,

¹https://github.com/AntoAndGar/task_singular_vectors

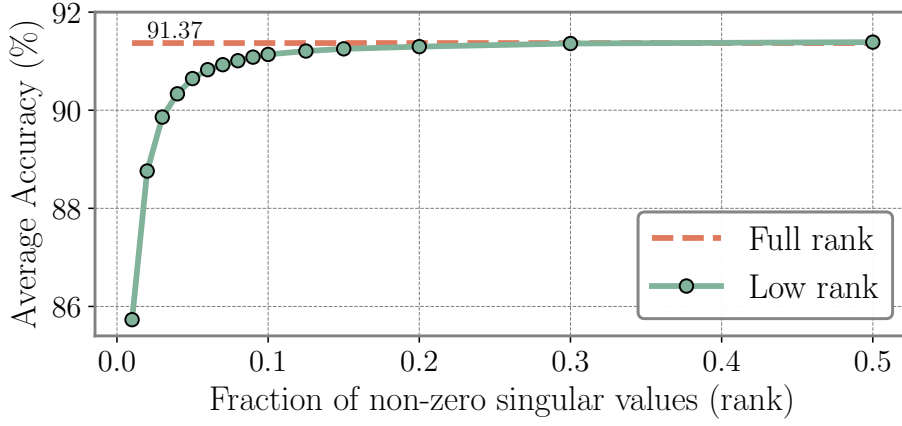


FIGURE 6.2: Mean absolute accuracy of the ViT-B-32 model across increasing fractions of retained singular components, averaged over 20 tasks. The red line represents the average accuracy of the original fine-tuned models with full-rank task matrices, while the green line shows the accuracies using low-rank approximations.

$V = [V_1 \cdots V_T]$ as the column-wise concatenation of the right TSVs, and Σ as the block diagonal matrix with $\{\Sigma_i\}_{i=1}^T$ along its diagonal:

$$M = U\Sigma V^\top = \sum_{i=1}^T U_i \Sigma_i V_i^\top = \sum_{i=1}^T \Delta_i. \quad (6.2)$$

In the simple case where $T = 2$, M would be given by:

$$M = \begin{bmatrix} U_1 & U_2 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} V_1^\top \\ V_2^\top \end{bmatrix} = \Delta_1 + \Delta_2.$$

Note that concatenating singular components from different tasks violates certain SVD properties: the matrices U and V are no longer orthogonal, because singular vectors from different tasks may overlap. Additionally, the singular values Σ_i from different tasks can vary significantly in magnitude, potentially biasing the merge toward tasks with larger singular values.

6.2.2 Low-rank nature of layer task matrices

By Eckart–Young’s theorem [47], the best rank- k approximation (in Frobenius norm) of each task matrix Δ_i is obtained by retaining only the top- k singular values and their corresponding vectors:

$$\hat{\Delta}_i = \sum_{j=1}^k \sigma_j^i u_j^i v_j^{i\top}. \quad (6.3)$$

As shown in fig. 6.2, task matrices are inherently low-rank: preserving only 3% of the singular components per task reduces mean accuracy by merely 1.5%. Building on this, we propose in section 6.3.1 a compression algorithm that maintains 99% of accuracy while shrinking task vectors by a factor of T .

Given this low-rank structure, it is natural to aggregate task matrices within their subspaces. We, therefore, obtain a reduced version of the aggregation matrix M (eq. (6.2)) as:

$$\hat{M} = \sum_{i=1}^T \hat{U}_i \hat{\Sigma}_i \hat{V}_i^\top = \sum_{i=1}^T \sum_{j=1}^k \sigma_j^i u_j^i v_j^{i\top}, \quad (6.4)$$

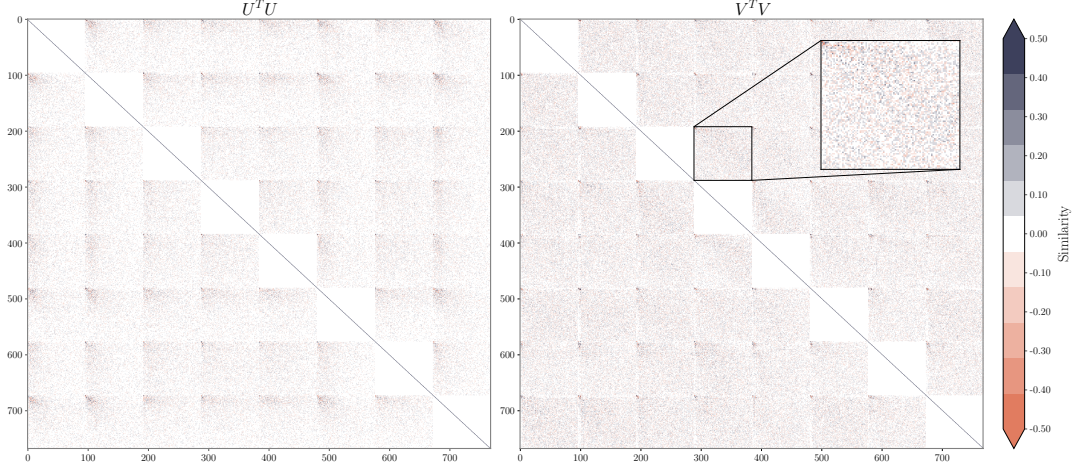


FIGURE 6.3: Visualization of task interference among 8 tasks computed on the first attention layer of a ViT-B-32. The diagonal blocks display intra-task similarities, while the off-diagonal blocks illustrate inter-task similarities. The zoomed-in section highlights the interaction between the right singular vectors of the 3rd and 4th tasks.

where $\hat{U}_i$ and $\hat{V}_i$ contain the top- k left and right singular vectors for task i , respectively, and $\hat{\Sigma}_i$ is the diagonal matrix of the top- k singular values σ_j^i .

6.2.3 Singular task interference

We now introduce a measure of task interference based on TSVs, which we term *Singular Task Interference* (STI).

If all tasks occupied *orthogonal* subspaces, the concatenated matrices U and V would have orthonormal columns, so $U^\top U = I$ and $V^\top V = I$. Deviations of these Gram matrices from the identity measure the degree of subspace overlap. We weight these deviations by Σ to reflect each direction's importance (directions with large singular values matter more), and aggregate with the element-wise ℓ_1 norm:

$$\text{STI} \left(\{\Delta_i\}_{i=1}^T \right) = \|(U^\top U - I)\Sigma(V^\top V - I)\|_1, \quad (6.5)$$

where U , Σ and V are obtained by concatenating the singular value decompositions ($\{U_i\}_{i=1}^T$, $\{\Sigma_i\}_{i=1}^T$, $\{V_i\}_{i=1}^T$) of per-layer task matrices $\{\Delta_i\}_{i=1}^T$ as detailed in section 6.2.1. By construction, $\text{STI} = 0$ when all task subspaces are mutually orthogonal, and grows as tasks share more directions in weight space.

Overlapping singular vectors indicate shared features in weight space across tasks; such overlap introduces interference when models are merged. Figure 6.3 visualizes this for eight tasks.

In transformer-based architectures, each transformer block contains multiple weight matrices (W_Q , W_K , W_V , W_O , and the two MLP projection matrices). Each of these is treated as a separate per-layer task matrix $\Delta_i^{(\ell)}$, with an independent SVD applied to each.

6.3 Compression and interference reduction

We now show how TSVs can be used for both compression and interference reduction.

Algorithm 3 TSV-Merge.**Require:** Task matrices $\Delta_1, \dots, \Delta_T$, scaling factor α **Ensure:** Merged model weights θ_{MT}

```

1: for  $i = 1$  to  $T$  do
2:   Compute SVD:  $\Delta_i = U_i \Sigma_i V_i^\top$ 
3:   Retain first  $\frac{1}{T}$  singular components of  $U_i, \Sigma_i$ , and  $V_i$ 
4: end for
5: Concatenate the matrices:
6:    $U \leftarrow [U_1 | U_2 | \dots | U_T]$ 
7:    $\Sigma \leftarrow \text{block-diag}(\Sigma_1, \Sigma_2, \dots, \Sigma_T)$ 
8:    $V \leftarrow [V_1 | V_2 | \dots | V_T]$ 
9: Compute the SVD of  $U$  and  $V$ :
10:   $U = P_U D_U Q_U^\top$     $V = P_V D_V Q_V^\top$ 
11: Obtain the orthogonal matrices:
12:   $U_\perp = P_U Q_U^\top$     $V_\perp = P_V Q_V^\top$ 
13: Reconstruct the merged matrix:
14:   $\hat{M} \leftarrow U_\perp \Sigma V_\perp^\top$ 
15: Construct merged model weights:
16:   $\theta_{\text{MT}} \leftarrow \theta_{\text{pre}} + \alpha \hat{M}$ 
17: return  $\theta_{\text{MT}}$ 

```

Method	ViT-B-32			ViT-B-16			ViT-L-14		
	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks
Zero-shot	48.26 _(53.59)	57.21 _(63.69)	56.10 _(62.41)	55.34 _(59.34)	61.28 _(66.19)	59.73 _(64.52)	64.70 _(68.00)	68.20 _(72.15)	65.23 _(68.99)
Weight Averaging	66.34 _(72.13)	64.34 _(71.12)	61.04 _(67.53)	72.22 _(76.60)	69.46 _(74.82)	65.31 _(70.36)	79.56 _(83.15)	76.73 _(81.10)	71.60 _(75.60)
Task Arithmetic	70.79 _(76.55)	65.32 _(72.09)	60.52 _(66.79)	75.41 _(79.58)	70.52 _(75.89)	65.78 _(70.76)	84.93 _(88.65)	79.41 _(83.95)	74.01 _(78.07)
Consensus TA	75.03 _(80.84)	70.39 _(77.36)	65.43 _(71.98)	79.39 _(83.86)	74.39 _(79.92)	69.76 _(74.93)	86.34 _(90.08)	82.22 _(86.94)	79.00 _(83.22)
TSV-M (Ours)	85.86 _(92.31)	80.06 _(87.88)	77.07 _(84.29)	89.01 _(93.94)	84.58 _(91.01)	80.57 _(86.45)	92.98 _(96.98)	89.17 _(94.43)	87.72 _(92.50)

TABLE 6.1: Average absolute accuracy results on model merging benchmarks; subscript (in parentheses) is the normalized average accuracy.

6.4 Experimental results

We evaluate our approaches over three different suites of tasks having cardinality 8, 14, and 20, respectively. The first one, introduced in [78], consists of datasets: CARS [92], DTD [26], EURO SAT [69], GTSRB [148], MNIST [97], RESISC45 [23], SUN397 [175], and SVHN [117]. The benchmark with 14 tasks builds on the preceding one, incorporating six additional datasets: CIFAR100 [93], STL10 [29], FLOWERS102 [121], OXFORDIIITPET [126], PCAM [164], and FER2013 [63]. Finally, the 20-tasks benchmark includes the preceding 14 plus the following six: EMNIST [31], CIFAR10 [93], FOOD101 [19], FASHIONMNIST [174], RENDEREDSST2 [146], and KMIST [27].

6.4.1 Model merging results

We evaluate on three CLIP [132] variants with different ViT [45] encoders: ViT-B-32, ViT-B-16, and ViT-L-14. Benchmarks involve merging 8, 14, and 20 tasks, following the setup of Wang et al. [166]. We compare against weight averaging, Task Arithmetic [78], and Consensus Merging [166]. Zero-shot performance (Table 6.1) and individually fine-tuned models (Table 6.2) serve as lower and upper bounds, respectively. We report both average absolute and normalized accuracy (see Section C.2.1).

As shown in Table 6.1, TSV-M achieves state-of-the-art results across all benchmarks, regardless of model size or number of tasks. The largest gains appear on ViT-B-32, where it

Method	ViT-B-32			ViT-B-16			ViT-L-14		
	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks
Finetuned	92.83 ₍₁₀₀₎	90.88 ₍₁₀₀₎	91.37 ₍₁₀₀₎	94.64 ₍₁₀₀₎	92.76 ₍₁₀₀₎	93.17 ₍₁₀₀₎	95.81 ₍₁₀₀₎	94.29 ₍₁₀₀₎	94.73 ₍₁₀₀₎
TALL-Mask+TIES	93.13 _(100.37)	90.92 _(100.04)	91.11 _(99.70)	94.68 _(100.04)	92.69 _(99.90)	93.05 _(99.87)	95.96 _(100.15)	93.40 _(99.09)	93.91 _(99.16)
TSV-C (Ours)	92.62 _(99.74)	90.29 _(99.28)	90.64 _(99.14)	94.47 _(99.79)	92.25 _(99.41)	92.53 _(99.27)	95.68 _(99.85)	94.04 _(99.72)	94.42 _(99.66)

TABLE 6.2: Average absolute accuracy results across all compression benchmarks for different models and varying number of tasks, subscript (in parentheses) the normalized average accuracies. Our TSV-C compression method consistently achieves over 99% of the original fine-tuned models’ accuracy while significantly reducing storage requirements.

Low-rank approx.	Interf. reduction	ViT-B-32		
		8 tasks	14 tasks	20 tasks
×	×	76.5 (+0.0)	72.1 (+0.0)	66.8 (+0.0)
✓	×	75.2 (-1.3)	71.0 (-1.1)	66.3 (-0.5)
×	✓	82.6 (+7.4)	75.7 (+4.7)	69.9 (+3.6)
✓	✓	92.3 (+9.7)	87.9 (+12.2)	84.3 (+14.4)

TABLE 6.3: Comparison of different versions of Task Arithmetic, comprising either the low-rank approximation step, the interference reduction step, or both. The method performing both corresponds to the proposed TSV-Merge.

outperforms Task Arithmetic and Consensus TA by +15.45% and +10.71% absolute accuracy on average. On ViT-L-14 with 8 tasks, the method attains 96.98% normalized accuracy, meaning a single merged model incurs only a 3.02% accuracy reduction compared to eight individual fine-tuned models. Per-dataset accuracies are reported in fig. 6.4.

6.4.2 Compression results

Table 6.2 reports the results for TSV-C, compared with TALL-masks [166], which stores sparse binary masks for each task. TSV-C consistently retains more than 99% of the original accuracy across all benchmarks and models. The two methods perform comparably (within 1% on average), with TSV-C slightly trailing on smaller ViTs but ahead on ViT-L-14. A key advantage of TSV-C is predictable storage: it stores only the top $\frac{1}{T}$ singular vectors per task, yielding a fixed footprint of approximately twice the original model size regardless of the number of tasks. TALL-masks, by contrast, has storage that grows with the number of tasks and depends on mask compressibility.

6.5 Ablations and analysis

We ablate the contributions of interference reduction and low-rank compression, study how task interference varies across layers, and show that the method does not require tuning the scaling coefficient.

6.5.1 Ablation study

TSV-M combines low-rank approximation with interference reduction. Table 6.3 summarizes the contribution of each component. Low-rank approximation alone slightly hurts performance compared to vanilla Task Arithmetic (first row). Interference reduction alone yields

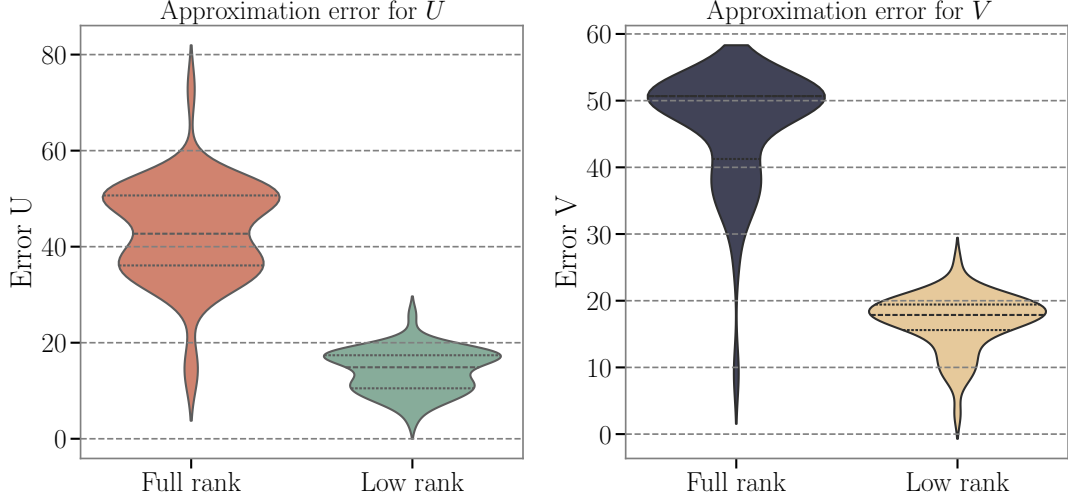


FIGURE 6.5: Approximation error from the orthogonalization of the TSVs through Procrustes for the ViT-B-32 model across 8 tasks. The violin plots represent layer-wise approximation error distributions for U and V in both full-rank and low-rank cases.

gains of +3.1% to +6.1%. Combining both steps produces improvements of +15.8% to +17.5%, exceeding the sum of the individual contributions.

This superadditive synergy arises from the interaction between the two steps. Low-rank approximation alone retains the dominant singular directions but leaves inter-task conflicts intact. Interference reduction alone must orthogonalize the concatenated *full-rank* matrices: when each U_i has full ambient dimension, the concatenated matrix $[U_1 \cdots U_T]$ is wide and highly correlated, and forcing it to be nearly orthogonal via Procrustes incurs large approximation error. Applying interference reduction to the low-rank approximations breaks this cycle: with only $k \ll n$ columns per task, Procrustes operates on a compact matrix and achieves tighter orthogonality with less distortion. Compression thus acts as a prerequisite that makes interference reduction effective, and Theorem 6.5.1 formalizes this.

Figure 6.5 confirms this empirically: orthogonalizing full-rank matrices incurs significantly larger approximation errors than their low-rank counterparts, with wider and higher error distributions across layers. The following theorem shows that this is a general property, not specific to the chosen model (proof in Section C.3.3).

Theorem 6.5.1. *Let $T \in \mathbb{N}$ such that $T > 4$. Define $U = [U_1, \dots, U_T]$ as the matrix obtained by concatenating T orthogonal matrices U_i , each of shape $n \times n$. Let $\hat{U} = [\hat{U}_1, \dots, \hat{U}_T]$ be the matrix formed by truncating each U_i to its first k columns. Denote by X and $\hat{X}$ the matrices resulting from Procrustes orthonormalization of U and $\hat{U}$, respectively. If $k \leq n \frac{T-2\sqrt{T}}{T}$, then*

$$\|U - X\|_F \geq \|\hat{U} - \hat{X}\|_F.$$

In words, Procrustes orthogonalization introduces less error when applied to truncated (low-rank) matrices than to full-rank ones, provided k is small enough. To verify applicability, consider our most demanding setting, $T = 20$. The bound becomes $k \leq \frac{20-2\sqrt{20}}{20} n \approx 0.55 n$, and we choose $k = \lfloor n/T \rfloor = \lfloor n/20 \rfloor$, which satisfies it. For $T = 8$ and $T = 14$ the bound is even more permissive.

Note that Gram–Schmidt would be ineffective here, as it sequentially orthogonalizes vectors without minimizing deviation from the original set, potentially introducing large distortions.

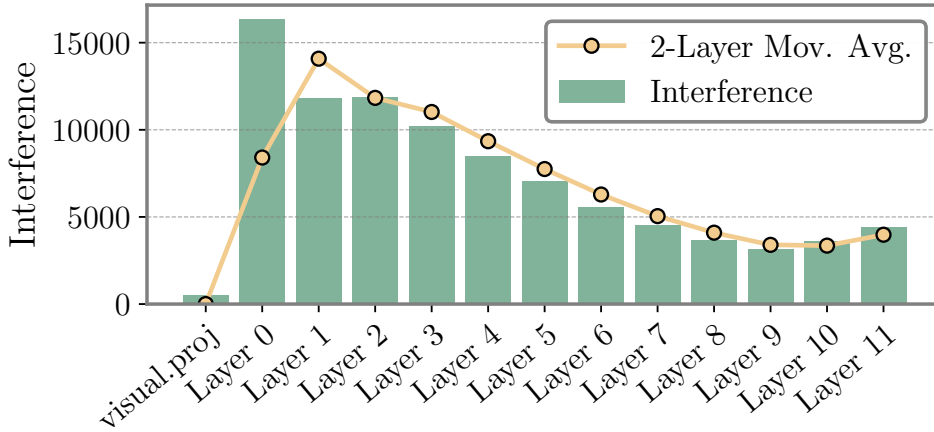


FIGURE 6.6: Singular Task Interference (STI) across layers in a ViT-B-32 for 20 tasks. STI is high in early layers sharing common knowledge and lower in more specialized ones.

Beyond their quantitative role in merging and compression, TSVs also carry an interpretability benefit: because they capture the principal directions of weight change induced by each task, they can be viewed as a *semantic basis* that summarizes what a task “does” to a given layer. In section 7.3.4, we confirm this intuition by decoding individual TSVs into natural-language descriptions via TEXTSPAN, showing that mid-layer singular vectors recover semantically meaningful concepts aligned with each task’s visual domain.

6.5.2 Per-layer task interference

Using eq. (6.5), we analyze task interference on a per-layer basis. As shown in fig. 6.6, interference is highest in the initial transformer layers and decreases in deeper ones. This aligns with the well-known pattern that early layers capture shared features across tasks while deeper layers specialize. For brevity, we grouped layers within each transformer block in our analysis; each “Layer n ” in fig. 6.6 includes two attention matrices and two MLP matrices. A layer-by-layer analysis is provided in fig. C.6. This layer-wise interference profile motivates the adaptive routing strategy of chapter 7, which selects different task subspaces per input rather than committing to a single static combination.

6.5.3 Choice of the scaling coefficient

The aggregation in eq. (6.1) involves a scaling coefficient α , typically tuned on a validation set. As shown in fig. 6.7, our approach consistently achieves the best results with $\alpha = 1.0$, eliminating the need for hyperparameter tuning and validation data.

6.5.4 Effect of task interference reduction

Figure 6.8 shows the interference of a ViT-B-32 model before and after applying TSV-M, across task sets of different cardinalities. In all cases, the reduction in interference is accompanied by a significant accuracy gain, confirming that STI captures a quantity directly relevant to merging quality.

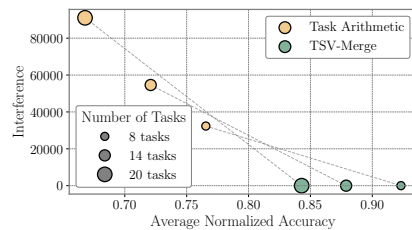


FIGURE 6.8: Singular Task Interference (STI) and average normalized accuracy on the ViT-B-32 model, evaluated across merges of 8, 14, and 20 tasks.

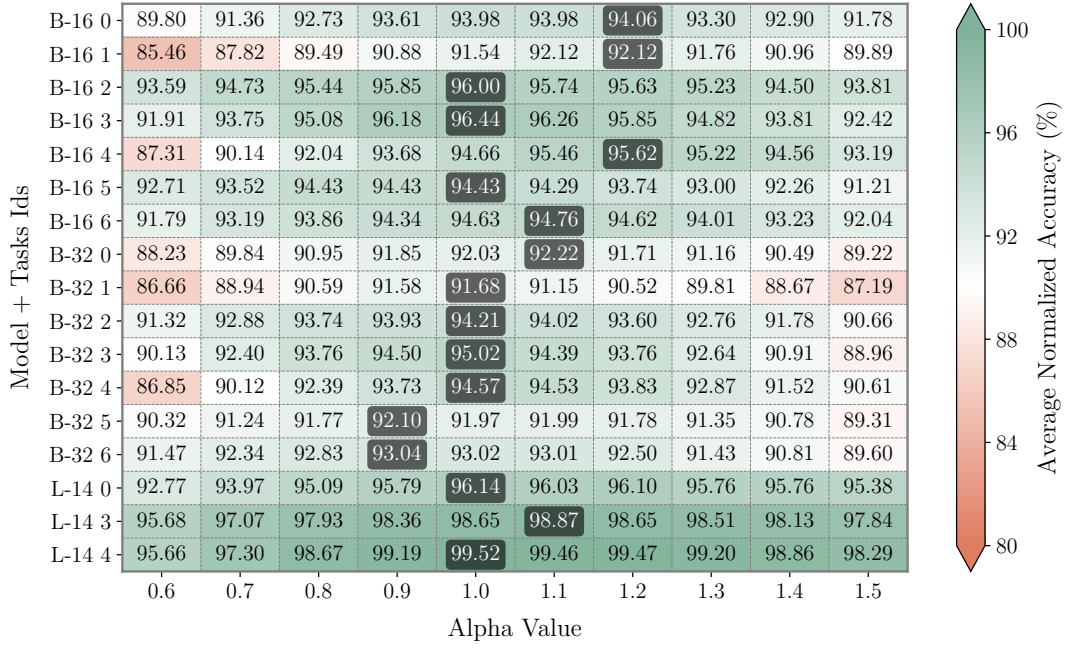


FIGURE 6.7: Best average normalized accuracy for different alpha values.
The vertical labels indicate different sets of 8 tasks.

6.6 Summary and outlook

This chapter showed that per-layer task matrices are inherently low-rank and introduced a geometrically grounded measure of task interference based on the alignment of singular vectors across tasks.

These insights yield two practical tools. TSV-Compress reduces task vectors by a factor of T while retaining 99% of accuracy, with constant storage independent of the number of tasks. TSV-Merge combines low-rank approximation with Procrustes-based interference reduction, achieving up to 97% normalized accuracy. The ablation study reveals a superadditive synergy between the two steps: compression makes interference reduction effective by reducing the dimensionality of the Procrustes problem.

Exploring alternative methods for task importance and rank approximation could be a valuable direction for future work. Currently, we use a uniform rank across tasks, approximating each task vector to $\frac{1}{T}$ of the layer’s dimensionality. However, techniques like those from Idelbayev and Carreira-Perpiñán [76] or optimal hard thresholds from Gavish and Donoho [60] could be applied to select the optimal rank for each task individually.

The low-rank structure of task matrices, established in this chapter, also opens a new question beyond static merging: if each task occupies a distinct low-dimensional subspace of the weight space, can we exploit this geometry at *inference time*? All the methods studied so far produce a single, fixed merged model that is applied identically regardless of the input, a limitation that leaves per-task accuracy on the table. In chapter 7, we relax this constraint and explore an input-adaptive paradigm: rather than committing to one static combination of task subspaces, the model routes each input to the most relevant task singular vectors at inference time.

Chapter 7

Input-Adaptive Merging via Subspace Routing

In this chapter, we move beyond static merging by introducing MASS (*MoErging through Adaptive Subspace Selection*), which adapts the merged model to each input at inference time. Building on the low-rank structure identified in chapter 6, MASS uses a training-free router to select the most relevant task subspaces for a given input, recovering up to $\sim 98\%$ of fine-tuned accuracy at a fraction of the storage cost.

7.1 Beyond static merging

The previous chapter showed that preserving the layer-wise structure of task vectors leads to stronger merging results. However, all methods considered so far share a common limitation: they assume that the task identity is known at inference time. When this oracle is available, the problem reduces to compression, and the low-rank method TSV-C (chapter 6) already achieves 99.5% normalized accuracy with only $2\times$ storage, effectively solving it.

Mixture-of-Experts Merging (MoErging) methods [177] such as SMILE [155], WeMoE [156], and TwinMerging [104] incorporate a router into the merging process, yet *still assume that the correct classification head is provided at test time*, inheriting the same unrealistic constraint.

The setting considered here is more challenging: the task is *not* known at inference time, and both the encoder subspaces and the classification head must be determined *automatically*. To address it, we introduce MASS (*MoErging through Adaptive Subspace Selection*), which conditions the merging process on the input itself (fig. 7.1). MASS routes each input to the most relevant task subspaces, as defined by the task singular vectors of chapter 6, and jointly selects the corresponding classification head, all without requiring training data or oracle supervision.

Evaluated on ViT-B-32, ViT-B-16, and ViT-L-14 across up to 20 vision tasks and 8 language tasks, MASS achieves up to 98% of fine-tuned accuracy for a modest overhead ($\sim 2\times$ forward passes and $\sim 2\times$ storage, regardless of the number of tasks), while handling the full union of label spaces without oracle guidance.

7.2 Adaptive subspace selection

Our approach consists of a one-time pre-processing step followed by an inference-time step. The former produces an encoder model θ_{MT} via algorithm 9. We refer to this as the “fixed” merging step, as it is performed only once and remains independent of the input. During inference, θ_{MT} is used in a dynamic process, outlined in algorithm 4, consisting of 4 steps:

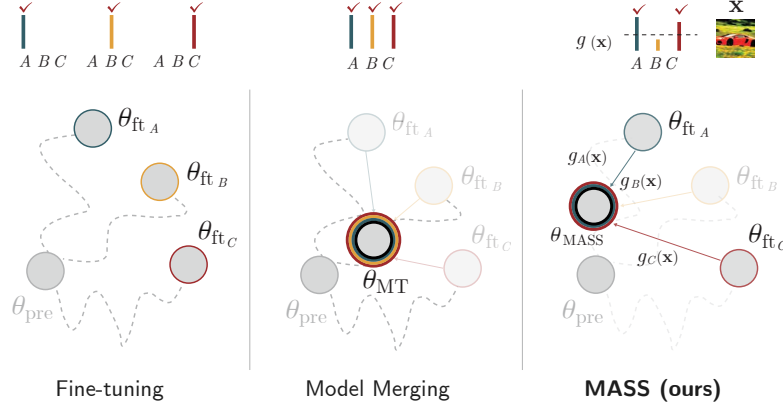


FIGURE 7.1: (left) Fine-tuning yields three separate models on tasks A, B and C. (middle) Model merging produces a single model incorporating the task vectors using a constant function of the input. (right) MASS stores the pre-trained model θ_{pre} and the task singular vectors $V^\top$ across tasks. At test time, MASS adaptively performs merging using a routing mechanism that chooses appropriate task vectors for the input $\mathbf{x}$, using a thresholded gating function $g(\mathbf{x})$. The gate is the residual between the activations of $\mathbf{x}$ and their projections onto the span of the right singular vectors $V_\perp$.

- (i) **First pass:** forward the input through θ_{MT} and extract its embedding $\mathbf{z}_\ell$ at a chosen layer ℓ ;
- (ii) **Routing:** project $\mathbf{z}_\ell$ onto the task subspaces, selecting those having lowest projection error;
- (iii) **Adaptive merge:** merge the selected task subspaces into Δ_{ada} ;
- (iv) **Second pass:** classify the input using the final merged model $\theta_{MT} = \theta_{pre} + \alpha \Delta_{ada}$.

Algorithm 4 Adaptive Merging Step

Require: Pretrained model weights θ_{pre} , task-specific updates $\{\Delta_i\}_{i=1}^T$, fixed merged model θ_{MT} , top- k parameter k , threshold η , task-specific classification heads $\{h_i\}_{i=1}^T$, sample $\mathbf{x}$

Ensure: Predicted class c^*

- 1: $\mathbf{z}_\ell \leftarrow \text{ForwardPass}(\theta_{MT}, \mathbf{x})$ # first pass
 - 2: **for** $i = 1, \dots, T$ **do**
 - 3: $r_i \leftarrow \|\mathbf{z}_\ell - V_i V_i^\top \mathbf{z}_\ell\|_2$ # residual as per eq. (7.2)
 - 4: **end for**
 - 5: $w \leftarrow \text{softmax}(-r)$
 - 6: $\Omega \leftarrow \{i : w_i \geq \eta\}$ # Select tasks above threshold
 - 7: $\Omega \leftarrow \text{TopK}(\Omega, w, k)$ # Keep only top- k weighted tasks
 - 8: **Merge selected subspaces:** $\Delta_{ada} \leftarrow \sum_{i \in \Omega} U_i \Sigma_i V_i^\top$
 - 9: **Compute adaptive model:** $\theta_{MASS} \leftarrow \theta_{pre} + \alpha \Delta_{ada}$
 - 10: **Classification procedure**
 - 11: $\mathbf{z}_{L-1} \leftarrow \text{ForwardPass}(\theta_{MASS}, \mathbf{x})$ # Compute shared representation
 - 12: $\mathbf{z}_i \leftarrow h_i(\mathbf{z}_{L-1})$ # Evaluate each head
 - 13: $(i^*, c^*) \leftarrow \arg \max_{(i,c) \in \Omega \times \{1, \dots, C_i\}} z_i[c]$ # Highest logit across heads
 - 14: **return** c^*
-

7.2.1 Fixed merging

The fixed step produces a model capable of task discrimination, whose intermediate activations feed the router in the first pass. We use TSV-M (chapter 6) for this step due to its subspace-aware aggregation; table 7.3 confirms that routing from the pre-trained backbone yields lower task classification accuracy.

Algorithm 5 Fixed Merging Step

Require: Pretrained model weights θ_{pre} , task-specific updates $\{\Delta_i\}_{i=1}^T$, user-specified threshold ε

Ensure: Fixed merged model weights θ_{MT}

```

1: Accounting for redundant directions (section 7.2.2)
2:  $\mathcal{M} = \{\}$ 
3: for  $i = 1, \dots, T$  do
4:    $\delta_i \leftarrow \text{vec}(\Delta_i)$ 
5:   if  $\max_{j \in \mathcal{M}} \text{sim}(\delta_i, \delta_j) < \varepsilon$  then
6:      $\mathcal{M} \leftarrow \mathcal{M} \cup \{i\}$ 
7:   end if
8: end for
9: Merging step using TSV-M (chapter 6) on the  $\{\Delta_i\}_{i \in \mathcal{M}}$ 
10: for  $i \in \mathcal{M}$  do
11:    $\Delta_i = U_i \Sigma_i V_i^\top$ 
12:    $\tilde{U}_i \leftarrow U_{i[:,1:k]}, \tilde{\Sigma}_i \leftarrow \Sigma_{i[1:k,1:k]}, \tilde{V}_i \leftarrow V_{i[:,1:k]}$ 
13: end for
14:  $U \leftarrow [\tilde{U}_1 \mid \tilde{U}_2 \mid \dots \mid \tilde{U}_T]$ 
15:  $\Sigma \leftarrow \text{block\_diag}(\tilde{\Sigma}_1, \tilde{\Sigma}_2, \dots, \tilde{\Sigma}_T)$ 
16:  $V \leftarrow [\tilde{V}_1 \mid \tilde{V}_2 \mid \dots \mid \tilde{V}_T]$ 
17:  $U_\perp \leftarrow \text{orthogonalize}(U)$ 
18:  $V_\perp \leftarrow \text{orthogonalize}(V)$ 
19:  $\hat{\Delta} \leftarrow U_\perp \Sigma V_\perp^\top$ 
20:  $\theta_{\text{MT}} \leftarrow \theta_{\text{pre}} + \alpha \hat{\Delta}$ 
21: return  $\theta_{\text{MT}}$ 

```

7.2.2 Integrating routing

We extend the aggregation step in eq. (4.2) to include a routing mechanism. Given an input $\mathbf{x}$, the merged model can be adaptively determined by:

$$\theta_{\text{MT}} = \theta_{\text{pre}} + \alpha \sum_{i=1}^T \mathbb{1}_{[g_i(\mathbf{x})=1]}(\mathbf{x}) \tau_i = \theta_{\text{pre}} + \alpha \sum_{i=1}^T \mathbb{1}_{[g_i(\mathbf{x})=1]}(\mathbf{x}) \sum_{j=1}^k \sigma_j^i u_j^i v_j^{i\top}, \quad (7.1)$$

where $g_i(\mathbf{x})$ is a per-task gating function that adaptively selects which task subspaces to activate, and subsequently merge, depending on the input at hand.

Routers typically require task-specific *data* (for non-parametric methods such as nearest-neighbor routing) or both data and *training* (for parametric routers). This clashes with the merging scenario, where the fine-tuning data is generally unavailable because models may be downloaded from public repositories. We therefore introduce a completely **data- and tuning-free** approach.

Projection-based routing

Given an input intermediate representation $\mathbf{z}_\ell$ for a pre-determined layer ℓ , we want to identify which task subspace (or set of subspaces) is the most relevant. Concretely, one way to do this is to compute the Euclidean residual of $\mathbf{z}_\ell$ after projecting onto $\text{span}(V_i^{(\ell)})$:

$$r_i = \|\mathbf{z}_\ell - \text{Proj}_{V_i^{(\ell)}}(\mathbf{z}_\ell)\|_2, \quad (7.2)$$

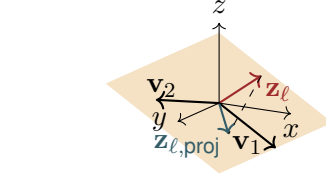


FIGURE 7.2: Projection of the activations $\mathbf{z}_\ell$ onto the span of TSVs $\mathbf{v}_1, \mathbf{v}_2$.

where $\text{Proj}_{V_i^{(\ell)}}(\mathbf{z}_\ell) = V_i^{(\ell)}(V_i^{(\ell)})^\top \mathbf{z}_\ell$ is the optimal

L_2 projector (see Proposition D.4.1). At this point, the additive inverse of the residuals is normalized through a softmax to obtain the coefficients. The router then picks those exceeding a predetermined threshold η , limiting the selection to the top- k when more tasks surpass it. For details regarding the choice of the layer used to compute the residual, see section 7.3.2. The effectiveness of this projection-based routing rests on the assumption that different tasks occupy geometrically distinct subspaces. The gradient perspective of chapter 5 provides a theoretical basis for this: since task vectors approximate scaled gradients of their respective task losses (Theorem 5.2.1), tasks with distinct loss landscapes produce task vectors that point in different directions, making their singular-vector subspaces separable by projection.

Accounting for redundant directions

For projection-based routing to be effective, no task should overshadow the others. Consider, for example, three tasks: MNIST , EMNIST , and KMNIST . Being trained on very similar datasets covering the same classes, Δ_{MN} and Δ_{EMN} share a large portion of their right-singular directions: $\text{span}(V_{\text{MN}}^{(\ell)}) \approx \text{span}(V_{\text{EMN}}^{(\ell)})$, while KMNIST has some distinct directions $V_{\text{KM}}^{(\ell)}$ capturing more Japanese *kana*-like shapes. However, all three tasks may agree on certain generic “black background, white glyph” features. Because MNIST and EMNIST partially *reinforce* these directions (they both include them), the *union* of their subspaces can appear “wider” or more dominant in that region of feature space. Consequently, for many test samples $\mathbf{z}_\ell$ with black backgrounds and centered shapes:

$$\|\mathbf{z}_\ell - \text{Proj}_{V_{\text{MN}} \cup V_{\text{EMN}}}(\mathbf{z}_\ell)\|_2 < \|\mathbf{z}_\ell - \text{Proj}_{V_{\text{KM}}}(\mathbf{z}_\ell)\|_2.$$

Hence, the router sees a smaller residual for MNIST/EMNIST, declaring those tasks more suitable even if the glyph belongs to KMNIST.

During the fixed merging step, instead of aggregating all task matrices, we only keep those that are *sufficiently distinct*. Starting from a single task matrix, each remaining candidate Δ_i is accepted only if its cosine similarity with every previously accepted update stays below a threshold ε . Concretely, we flatten each update as $\delta_i = \text{vec}(\Delta_i)$ and discard Δ_i whenever

$$\max_{1 \leq m \leq r} \text{sim}(\delta_i, \delta_{a_m}) > \varepsilon,$$

where $\{\Delta_{a_1}, \dots, \Delta_{a_r}\}$ are the already-accepted updates and ε is a user-specified threshold (e.g., $\varepsilon = 0.3$). This prevents highly similar subspaces from overshadowing less common ones (see lines 2–8 of algorithm 9).

7.2.3 Adaptive merging and inference

Once the router has selected a subset Ω of relevant tasks, we merge their subspaces via TSV-M into a single model θ_{MASS} . Because the task is unknown, we evaluate every selected head:

after obtaining the shared representation $\mathbf{z}_{L-1} \in \mathbb{R}^d$ from θ_{MASS} , we compute the logits of each head $h_i : \mathbb{R}^d \rightarrow \mathbb{R}^{C_i}$ for $i \in \Omega$:

$$\mathbf{z}_i = h_i(\mathbf{z}_{L-1}), \quad \mathbf{z}_i \in \mathbb{R}^{C_i}.$$

We then pick the highest logit among all heads in Ω . Formally:

$$(i^*, c^*) = \arg \max_{(i, c) \in \Omega \times \{1, \dots, C_i\}} z_i[c].$$

That is, we pick the most confident head i^* and its predicted class c^* , allowing the model to determine both the label space and the prediction on a per-input basis. Because the cost scales with $|\Omega|$ rather than the total number of tasks, inference overhead remains modest even with large task libraries.

7.2.4 Residual minimization as maximum a posteriori estimation

The task selection process in MASS admits a probabilistic interpretation as *maximum a posteriori* (MAP) estimation. If residuals follow an isotropic Gaussian, the likelihood of a feature vector given a task decays exponentially with the squared ℓ_2 reconstruction error, so choosing the task with the smallest residual is equivalent to the MAP estimate. This parallels probabilistic PCA [161], where reconstruction-error minimization corresponds to maximum likelihood under the same Gaussian assumption. Because MASS lacks training data to fit richer distributions, this least-informative model is a natural choice: the isotropic prior treats all directions equally, avoiding bias toward any particular task.

Proposition 7.2.1. *Let $\mathbf{z}_\ell \in \mathbb{R}^d$ be a feature vector, and for each task i , decompose it as*

$$\mathbf{z}_\ell = V_i V_i^\top \mathbf{z}_\ell + \varepsilon_i, \quad \varepsilon_i = (I - V_i V_i^\top) \mathbf{z}_\ell.$$

Assume $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$ and a uniform prior over tasks: $p(\text{task} = i) = \frac{1}{K}$ for all $i \in \{1, 2, \dots, K\}$. Then the maximum a posteriori estimate of the task reduces to

$$\hat{i}_{\text{MAP}} = \arg \max_i p(\text{task} = i \mid \mathbf{z}_\ell) = \arg \min_i \|\varepsilon_i\|_2^2.$$

7.3 Experimental results

7.3.1 Merging performance

Models, baselines and datasets We experiment with three CLIP [132] variants using ViT-B-32, ViT-B-16, and ViT-L-14 visual encoders [45]. Baselines include weight averaging, Task Arithmetic [78], and Consensus Merging [166]; zero-shot performance and the mean accuracy of individually fine-tuned models serve as lower and upper reference points, respectively. We evaluate on collections of 8, 14, and 20 tasks (see section D.2.3 for details), reporting both absolute and normalized accuracy.

MoErging results table 7.1 reports the average absolute accuracy and the corresponding normalized accuracy (subscript, also in percentage) for each method, model size, and number of vision tasks. To adapt SMILE and WeMoE to our general setting, we employ a majority-voting-based heuristic to route the classification head.

	Method	ViT-B-32			ViT-B-16			ViT-L-14		
		8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks
Base	Zero-shot	48.1 _(54.8)	56.9 _(64.5)	57.5 _(65.2)	55.3 _(60.6)	61.9 _(67.9)	62.5 _(68.3)	64.9 _(69.2)	69.1 _(73.8)	68.2 _(72.7)
	Fine-tuned	90.3 ₍₁₀₀₎	89.0 ₍₁₀₀₎	89.5 ₍₁₀₀₎	92.4 ₍₁₀₀₎	91.3 ₍₁₀₀₎	91.9 ₍₁₀₀₎	94.2 ₍₁₀₀₎	93.4 ₍₁₀₀₎	94.0 ₍₁₀₀₎
Fixed	Weight Averaging	67.1 _(74.6)	65.5 _(73.8)	64.4 _(72.6)	73.0 _(78.8)	70.8 _(77.3)	69.2 _(75.3)	80.5 _(85.2)	78.5 _(83.8)	76.1 _(80.9)
	Task Arithmetic	68.8 _(75.7)	64.6 _(72.5)	64.0 _(71.9)	73.0 _(78.3)	70.6 _(77.0)	69.0 _(75.0)	84.4 _(89.3)	80.4 _(85.8)	76.9 _(81.7)
	Consensus TA	72.6 _(80.1)	70.3 _(78.9)	68.5 _(76.8)	75.9 _(81.7)	74.9 _(81.7)	72.2 _(78.4)	85.5 _(90.5)	82.0 _(87.6)	78.9 _(83.8)
	TSV-M	83.2 _(91.8)	78.6 _(88.0)	75.6 _(84.3)	85.5 _(92.2)	81.4 _(88.8)	78.8 _(85.5)	91.2 _(96.7)	88.8 _(94.9)	87.5 _(93.0)
	Iso-C	82.8 _(91.7)	78.4 _(88.0)	73.2 _(81.9)	87.5 _(94.4)	79.8 _(87.0)	75.3 _(81.6)	92.6 _(98.2)	89.6 _(95.8)	86.8 _(92.3)
	Iso-CTS	82.0 _(90.9)	80.6 _(90.4)	77.0 _(86.2)	88.7 _(95.9)	84.1 _(91.8)	80.7 _(87.7)	92.8 _(98.5)	91.1_(97.4)	89.2 _(94.9)
MoE	WeMoE	88.8_(97.5)	74.3 _(82.8)	68.2 _(76.3)	89.1 _(96.4)	76.6 _(83.2)	65.0 _(70.5)	88.7 _(94.2)	72.3 _(76.8)	65.0 _(69.4)
	SMILE-1	83.2 _(92.1)	75.4 _(84.5)	72.8 _(82.3)	87.8 _(94.9)	81.7 _(89.5)	79.5 _(86.7)	91.2 _(96.7)	86.6 _(92.7)	84.9 _(90.5)
	SMILE-2	84.4 _(93.5)	76.4 _(85.6)	74.1 _(83.8)	89.0 _(96.2)	82.7 _(90.7)	80.4 _(87.7)	92.0 _(97.6)	87.1 _(93.4)	85.5 _(91.1)
	MASS	87.0 _(96.5)	82.9_(93.2)	81.1_(90.9)	90.6_(98.0)	87.8_(96.1)	81.1_(88.7)	92.9_(98.6)	90.9 _(97.3)	90.8_(96.6)

TABLE 7.1: Average absolute accuracy results on model merging benchmarks; subscript (in parentheses) is the normalized average accuracy.

Method	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	Avg.
Finetuned	75.0 ₍₁₀₀₎	83.4 ₍₁₀₀₎	87.5 ₍₁₀₀₎	91.5 ₍₁₀₀₎	85.4 ₍₁₀₀₎	85.9 ₍₁₀₀₎	93.6 ₍₁₀₀₎	88.7 ₍₁₀₀₎	86.4 ₍₁₀₀₎
Weight Averaging	69.1 _(92.1)	62.6 _(75.1)	79.4 _(90.7)	89.8 _(98.1)	83.9 _(98.2)	81.2 _(94.5)	91.7 _(97.9)	73.2 _(82.5)	78.9 _(91.3)
Task Arithmetic	70.5 _(94.0)	57.8 _(69.3)	78.4 _(89.6)	90.2 _(98.6)	83.6 _(97.9)	80.5 _(93.7)	92.3 _(98.6)	77.8 _(87.7)	78.9 _(91.3)
Ties-Merging	70.3 _(93.7)	65.0 _(77.9)	78.9 _(90.2)	90.2 _(98.6)	83.5 _(97.8)	81.6 _(95.0)	91.7 _(97.9)	78.3 _(88.3)	79.9 _(92.5)
Twin-Merging	74.9_(99.8)	83.7 _(100.3)	87.5_(100.0)	90.3 _(98.6)	84.6 _(99.0)	86.6_(100.8)	93.6 _(100.0)	88.8 _(100.1)	86.2_(99.7)
WeMoE	72.5 _(96.6)	79.0 _(94.7)	51.9 _(59.3)	89.3 _(97.5)	69.6 _(81.4)	81.5 _(94.8)	88.6 _(94.6)	82.1 _(92.5)	76.8 _(88.9)
SMILE-1	72.0 _(96.0)	84.2 _(101.0)	84.3 _(96.3)	91.3 _(99.8)	84.7 _(99.2)	84.1 _(97.9)	93.3 _(99.7)	87.0 _(98.1)	85.1 _(98.5)
SMILE-2	73.2 _(97.6)	84.2_(101.0)	85.0 _(97.1)	91.3_(99.8)	84.9 _(99.4)	84.8 _(98.7)	93.5 _(99.9)	87.3 _(98.4)	85.5 _(99.0)
MASS	74.1 _(98.8)	83.2 _(99.8)	85.8 _(98.1)	90.9 _(99.3)	85.1_(99.6)	84.9 _(98.8)	94.2_(100.6)	88.9_(100.2)	85.9 _(99.4)

TABLE 7.2: Accuracy when merging 8 fine-tuned models on the GLUE [165] benchmark; Normalized average accuracy in subscript.

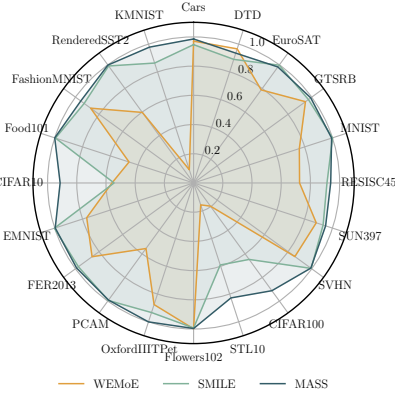
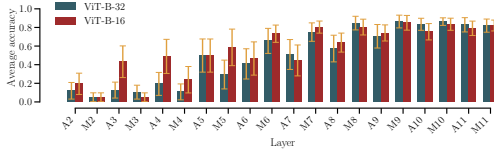


FIGURE 7.3: Per-dataset accuracies.

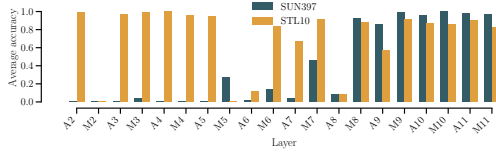
Details in section D.2.6. MASS sets a new state of the art for MoErging in 8 out of 9 benchmarks, with gains up to $\approx 6\%$ over the best-performing baseline. Figure 7.3 shows a per-dataset breakdown, confirming that accuracy is consistently high across all datasets rather than skewed toward easier ones. Storage-wise, MASS requires a constant $2\times$ parameter increase, whereas other MoErging baselines range from $\sim 2.5\times$ to $\sim 14\times$ (see section D.2.1).

For reference, we also report fixed merging methods that assume oracle access to the correct classification head. Despite operating in a strictly harder setting, MASS outperforms them by a consistent margin: compared with TSV-M, the routing mechanism yields an

$\approx 5\%$ accuracy increase. On the 8-task benchmark (the only vision setting with reported results from TwinMerging [104]), MASS achieves **97.6%** normalized accuracy versus 95.3% for TwinMerging, despite the latter assuming oracle knowledge of the classification head. Table 7.2 extends the evaluation to 8 FLAN-T5-BASE models [25] fine-tuned on GLUE [165] language tasks, where MASS again yields the best results, confirming that its benefits are modality-agnostic.



(A) Averaged across all tasks for two models.



(B) Focusing on SUN397 and STL10 for a ViT-B-32.

FIGURE 7.4: Per-layer task accuracies for ViT-B-32 on the 20-task benchmark. Layers starting with ‘A’ indicate attention layers, while those starting with ‘M’ refer to MLPs.

Batched inference The main experiments consider the hardest scenario, where each sample may belong to a different task. In practice, batched requests often share the same task, allowing the router to select a single merged model per batch. As Figure 7.5 shows, this closes the gap almost entirely: MASS reaches at least 97% normalized accuracy in 8 out of 9 settings, effectively matching individually fine-tuned models.

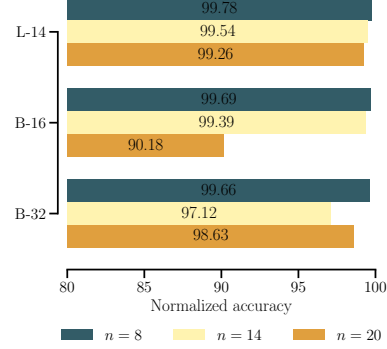


FIGURE 7.5: Batched accuracy.

7.3.2 Choosing a routing layer

Figure 7.4a shows task prediction accuracy when routing at different layers for ViT-B-32 and ViT-B-16. The best layer is consistent across architectures (layer 9 for both), with MLP layers slightly outperforming self-attention layers. However, the best layer varies substantially across tasks, with per-layer variance reaching 40%. Figure 7.4b illustrates this for a ViT-B-32: STL10 benefits from earlier layers ($\ell = 3-5$), while SUN397 peaks at later ones ($\ell = 9-11$). This suggests that adaptive, per-task layer selection is a promising direction for future work. An analogous analysis for ViT-L-14 is provided in the appendix.

7.3.3 Comparison with other routers

We compare our router with two alternatives that require progressively more supervision.

Nearest Neighbor (nn) stores a small support set of validation embeddings per task and classifies each test sample by cosine similarity to its nearest stored neighbor. It requires no learnable parameters but assumes access to per-task validation data.

MLP router (mlp) trains a small MLP f_θ on the union of per-task validation sets to predict task identity from $\mathbf{z}_\ell$ via cross-entropy (details in section D.2).

Results table 7.3 compares routing strategies. The nn router performs well but remains slightly below our best results. The mlp router achieves the highest accuracy overall, yet it relies on labeled validation data, contradicting the data-free premise of merging.

Among projection-based routers, routing from TSV-M ($\text{proj}_{\text{TSV-M}}$) outperforms routing from the pre-trained backbone (proj_{PRE}) as the number of tasks grows, with the gap widening to $\approx 10\%$ for a ViT-B-32 (table D.4). This highlights a key insight: because TSV-M embeds each task’s top singular vectors into distinct, orthogonal subspaces within a single

MASS	ViT-L-14		
	8 tasks	14 tasks	20 tasks
+			
nn	94.0	92.1	92.0
mlp	98.9	99.5	98.3
proj _{PRE}	99.1	97.7	91.9
proj _{TSV-M}	98.6	97.3	96.6

TABLE 7.3: Average normalized accuracy for different routers.

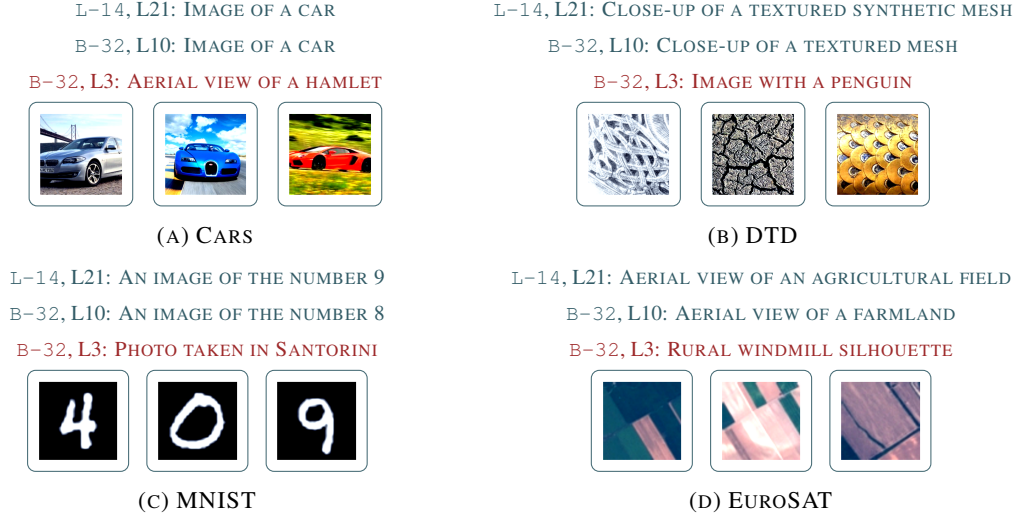


FIGURE 7.6: Captions obtained by decoding task singular vectors as text as described in section 7.3.4, accompanied by task representative images. Captions produced by the task singular vectors of predictive layers reflect the task content, those obtained by non-predictive ones do not.

model, a simple projection suffices to identify the relevant subspace, requiring no labels or additional training.

7.3.4 Interpreting task singular vectors

Finally, we interpret the task singular vectors used for routing. Since mid-layer embeddings in CLIP-like models are known to preserve semantic content, routing at these layers should rely on meaningful features. To verify this, we decode TSVs using TEXTSPAN [10, 55], which iteratively identifies the most influential text directions explaining how weight changes affect the representation. While originally designed for image embeddings, we apply it here to singular vectors derived from weight updates.

As Figure 7.6 shows, TSVs capture meaningful visual-language associations: the singular vectors for CARS [92] activate the description “IMAGE OF A CAR”, while those for DTD [26] align with “CLOSE-UP OF A TEXTURED MESH”. Notably, the decoded concepts are consistent across architectures: ViT-L-14 at layer 21 and ViT-B-32 at layer 10 yield nearly identical phrases for the same tasks, hinting at the possibility of transferring semantic information across architectures via text embeddings.

Interpretability depends strongly on the routing layer. Mid-to-late layers (e.g., layer 10 in ViT-B-32, layer 21 in ViT-L-14) produce semantically coherent descriptions, while early layers yield irrelevant concepts such as “IMAGE WITH A PENGUIN” for DTD or “PHOTO TAKEN IN SANTORINI” for MNIST. This confirms that early layers encode generic features, whereas deeper layers specialize toward domain-specific semantics.

These analyses validate the routing strategy: mid-layer TSVs capture precisely the semantic differences the router exploits for task discrimination. More broadly, they reveal a direct correspondence between TSVs and the semantic structure of the underlying data.

7.4 Summary and outlook

MASS demonstrates that the low-rank task subspaces identified in chapter 6 are useful not only at merge time but also at inference time: by routing each input to the most relevant subspaces and jointly selecting encoder and classification head, a single model handles all tasks without

test-time supervision. The projection-based router is entirely training- and data-free, making it directly applicable to checkpoints downloaded from public repositories, and recovers nearly the full accuracy of individually fine-tuned experts at a fraction of their combined storage cost.

Refining the router for more precise subspace selection and extending MASS to out-of-distribution settings, where unseen tasks could be composed on the fly from existing singular vectors, are promising directions for future work.

A complementary question, largely orthogonal to the merging mechanism itself, is how the merging *coefficients* should be chosen. The methods in chapters 5 and 6 and this chapter either fix coefficients analytically or derive them from structural properties of the task vectors. A more powerful alternative is to treat coefficient selection as an optimization problem solved via evolutionary search. Chapter 8 addresses the computational bottleneck that has made this approach impractical, enabling high-quality evolutionary merging on a single consumer GPU.

Chapter 8

Democratizing Evolutionary Merging

In this chapter, we address the computational bottleneck of evolutionary model merging, whose fitness evaluation requires repeated inference over large datasets. We introduce MERGE³, a three-stage framework that combines dataset reduction, Item Response Theory-based ability estimation, and evolutionary search to bring the cost of fitness computation down by roughly 50×. We provide theoretical guarantees on the resulting performance estimator and validate the approach on cross-lingual and multilingual merging tasks.

8.1 The computational bottleneck of evolutionary merging

As discussed in section 1.2, merging has significantly broadened access to the field, with ML practitioners producing competitive models on standard consumer GPUs. However, although computationally inexpensive, most existing approaches rely on ad-hoc, trial-and-error strategies for selecting merge coefficients, which ultimately limits their downstream performance [176, 183]. On the other hand, recent work has shown that evolutionary merging can produce models of unprecedented quality by automating the hyperparameter search for merging coefficients [3]. While this technique can incorporate any standard merging method, such models are absent from public leaderboards, likely due to a mismatch between the high computational demands of evolutionary merging and single-GPU setups typical of merging practitioners. Indeed, computing the fitness function requires generating and evaluating answers for each dataset element, for each candidate in every evolutionary step. As shown in fig. 8.1, the fitness computation alone in the 1,000-trial evolutionary merge from Akiba et al. [3] requires approximately 4×10^6 TFLOPs, with the full algorithm demanding well over a month of continuous computation if run on a single NVIDIA 4090 (section E.3.3) with 24 GB of VRAM. Because it requires repeated and costly forward passes through large language models, fitness evaluation is the primary bottleneck, making evolutionary merging out of reach on consumer hardware and potentially excluding the very users it was meant to empower.

The framework introduced here, MERGE³, addresses this computational barrier by reducing fitness evaluation costs by a factor of 50×, making evolutionary merging feasible on a single consumer GPU with competitive results (see fig. 8.1). Unlike the approach of Akiba et al. [3], MERGE³ operates with just 0.077×10^6 TFLOPs, bringing the computation from over a month down to roughly a day on a single NVIDIA 4090.

The approach rests on three steps. First, **Extract**: a reduced subset of the fitness evaluation dataset is sampled, significantly alleviating the computational bottleneck of fitness computation (fig. 5.1). This reduction, however, risks losing accuracy if the subset lacks diversity. To bridge the gap between reduced-dataset evaluations and full-dataset performance, we apply Item Response Theory (IRT) [103], a well-established statistical framework from psychometrics. Specifically, the second step, **Estimate**, uses IRT to infer the latent abilities of the endpoint

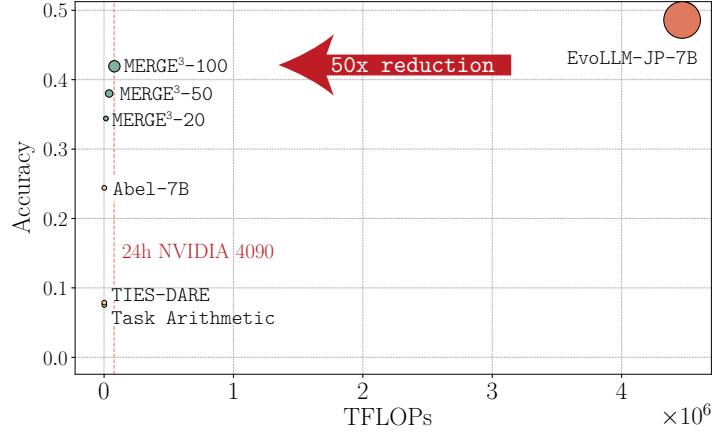


FIGURE 8.1: **Accuracy on Japanese GSM8K over fitness evaluation FLOPs.** MERGE³ is competitive with a model evolved on the full dataset by only using a consumer-grade GPU and 2% of the data (point size reflects data amount).

models, ensuring the merged models accurately reflect their components’ strengths. The third step, **Evolve**, optimizes merging coefficients with IRT-based performance estimators designed for model merging, assuming the merged model’s ability is a linear combination of those of the endpoints. This integration of merging-specific insights into performance estimation theory maintains high accuracy with drastically reduced datasets.

Experimental results show that MERGE³ effectively transfers mathematical skills by merging a strong math model with three language-specific models, achieving 10–20% higher accuracy than standard merging baselines in each language. The framework also evolves a single multilingual model by merging Italian, English, German, and Dutch models, outperforming individually fine-tuned models by up to 19% on ARC [28]. On Japanese GSM8K [30], MERGE³ approximates models evolved on the full dataset while maintaining high efficiency. A modular library for evolutionary merging on consumer GPUs and a suite of state-of-the-art models for several low-resource languages are released alongside this work.

8.2 Evolutionary algorithms and item response theory

Evolutionary Algorithms. Evolutionary Algorithms are black-box optimization algorithms operating on a population of potential solutions by evolving them through generations with operators such as selection, mutation, recombination, and crossover [38, 48, 128]. The fitness function evaluates the quality of each solution, guiding selection by favoring higher-scoring solutions for reproduction [48]. Closest to our work, Akiba et al. [3] apply evolutionary algorithms to optimize model merging recipes, eliminating the need for trial-and-error in selecting merge coefficients. In this context, the natural candidate for a fitness function is the performance of the resulting model on a held-out validation set.

Item Response Theory. Item Response Theory (IRT) [20, 21, 103, 162] is a paradigm to design, analyze, and score responses to tests such as SAT or GRE [5, 89, 127]. Based on the relationship between individuals’ performances on a test item and the test takers’ levels of performance on the corresponding required ability, IRT has recently spread from psychometrics to natural language processing. In this direction, Lalor et al. [95] leverage IRT’s latent dimensions to evaluate language models, while Vania et al. [163] use it to analyze benchmark saturation in NLP evaluations. More relevant to our work, Zhuang et al. [194] and

Polo et al. [129] employ IRT-driven adaptive testing to alleviate the computational burden of large-scale evaluations for LLMs. Although their focus is on general LLM evaluation, our work builds on these approaches to design IRT-based estimators specifically tailored for evolutionary model merging, exploiting the structure of the merging process to obtain more accurate fitness estimates from smaller data subsets.

8.3 The MERGE³ framework

Our method MERGE³ speeds up evolutionary model merging by reducing the computational cost of fitness evaluation. It achieves this by shrinking the fitness evaluation dataset and using IRT-based performance estimators to maintain full-dataset accuracy from subset evaluations. Figure 5.1 shows an overview of the method; the pseudocode for the end-to-end algorithm is given in algorithm 6.

Algorithm 6 The full MERGE³ algorithm.

Require: Dataset $\mathcal{D}$, models $\{M_1, M_2, \dots, M_n\}$, iterations T_{iter}

Ensure: Pareto-optimal merged models

```

1:  $\bar{\mathcal{D}} \leftarrow \text{RANDOMSAMPLE}(\mathcal{D}, k)$  # Sample  $k$  items from  $\mathcal{D}$ 
2:  $\{\gamma_1, \dots, \gamma_n\} \leftarrow \text{ESTIMATEABILITIES}(\{M_1, \dots, M_n\}, \mathcal{D})$ 
3:  $P \leftarrow \text{GenerateInitialPopulation}\{M_1, \dots, M_n\}$ 
4: for  $s \leftarrow 1$  to  $T_{\text{iter}}$  do
5:   for all  $M \in P$  do
6:      $\xi \leftarrow \text{FITXI}(M, \{\gamma_1, \dots, \gamma_n\}, \bar{\mathcal{D}})$ 
7:      $\text{preds} \leftarrow \text{GETPREDICTIONS}(M, \bar{\mathcal{D}}, \xi)$ 
8:      $\text{corr} \leftarrow \text{GETCORRECTNESS}(\text{preds}, \bar{\mathcal{D}})$ 
9:      $F(M) \leftarrow \text{ESTIMATEFITNESS}(\text{corr}, \xi)$ 
10:  end for
11:   $P \leftarrow \text{SELECTPARENTS}(P, f)$  # Select based on fitness
12:   $P \leftarrow \text{APPLYMUTATION}(P)$ 
13:   $P \leftarrow \text{APPLYCROSSOVER}(P)$  # Generate offspring
14: end for
15: return  $\text{PARETOFRONT}(P)$ 
```

8.3.1 Extract & estimate

Evaluating the fitness function involves generating and assessing answers for each data sample, repeated across all models in the population at every evolutionary step. Given the computational demands of evolutionary algorithms and LLMs, this process is highly intensive. To mitigate this, we reduce the dataset $\mathcal{D}$ to a smaller subset $\bar{\mathcal{D}} \subset \mathcal{D}$ with $|\bar{\mathcal{D}}| \ll |\mathcal{D}|$. After exploring various subsampling strategies, we found uniform random sampling to be as effective as more complex methods (see section E.3.1) and adopted it for simplicity.

Reducing the dataset speeds up evaluation but does not guarantee identical results, particularly when the subset is much smaller than the original, as in our setting. To bridge this gap, we build an IRT-based estimator that adjusts for this discrepancy, effectively estimating performance to reflect full-dataset results [103, 129].

IRT model. We first define an estimator to assess each endpoint model’s inherent abilities, derived from the latent variables of a Bayesian model. This ensures that merging preserves individual model strengths. In the Evolve step (section 8.3.2), the estimated latent abilities are fed to a *performance* estimator to compute the final fitness.

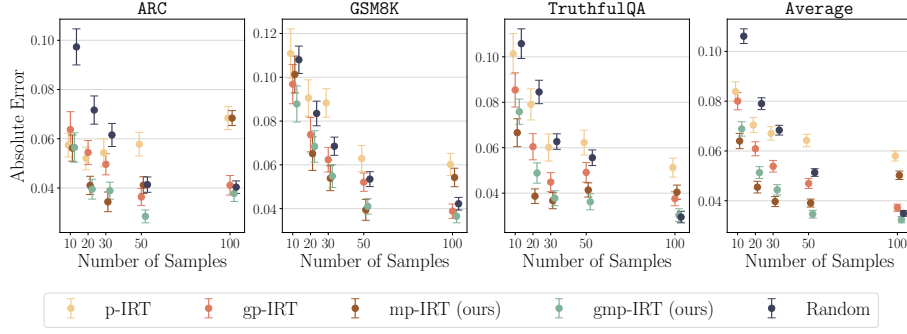


FIGURE 8.2: *Performance Estimators*: Absolute error of various estimators as a function of sample size (lower is better). Our MP-IRT and GMP-IRT estimators consistently achieve lower error across various sample sizes and datasets. Additional results available in Figure E.6.

To estimate LLM abilities, we build on Polo et al. [129], who applied IRT to evaluate LLM performance; however, while they used IRT for benchmarking, we extend it to estimate inherent abilities relevant for model merging, and explicitly use them to guide merging in the Evolve step.

In IRT, latent variables (γ) represent a model’s underlying abilities, while manifest variables (Y) indicate response correctness. The framework models the probability of a correct response based on model abilities and item characteristics (e.g., difficulty). IRT defines this probability as:

$$\mathbb{P}(Y_{im} = 1 \mid \gamma_m, a_i, \beta_i) = \frac{1}{1 + \exp(-a_i^\top \gamma_m + \beta_i)} \quad (8.1)$$

Here, $\gamma_m \in \mathbb{R}^d$ represents model m ’s latent abilities, $a_i \in \mathbb{R}^d$ defines the ability dimensions needed to answer example i , and β_i denotes its difficulty. A model is more likely to answer correctly when its abilities (γ_m) align with the example’s required traits (a_i) and less likely when the difficulty (β_i) is higher. Y_{im} is a binary variable indicating whether model m correctly predicts example i (1 if correct, 0 otherwise).

Crucially, this approach estimates a model’s likelihood of answering correctly *without directly analyzing the example’s content*, relying solely on the estimated IRT parameters (γ_m, a_i, β_i).

Fitting. We use variational inference to efficiently estimate both example-specific (a_i, β_i) and model-specific (γ_m) parameters within a hierarchical Bayesian model [96], initialized as detailed in section E.2.1. Following Polo et al. [129], we estimate a_i and β_i using correctness data (Y_{im}) from publicly available model evaluations, namely the Open LLM leaderboard. To estimate γ_m , each endpoint model generates answers for the full evaluation dataset, which are then used to assess correctness (Y_i) (see fig. 5.1). This procedure is repeated for each model m , producing the corresponding γ_m (γ_1 and γ_2 in the figure).

In contrast to prior work, where IRT latent abilities remain hidden variables used only for scoring, we explicitly extract γ_m as an *ability estimator* that quantifies each model’s strengths. Moreover, rather than estimating γ_m from a subset, we compute it using the *full* evaluation dataset, providing a more comprehensive measure of model ability that directly informs the merging process.

8.3.2 Evolve: Performance estimator

The *performance estimator*, a key part of the Evolve step, efficiently approximates the fitness function, which measures the merged model’s accuracy. Since the fitness function is evaluated

once per candidate per evolutionary step, reducing its cost is crucial. Instead of evaluating the full dataset, the estimator predicts performance using only the endpoint models' abilities and the reduced dataset from the previous steps, significantly accelerating the process.

We introduce two novel performance estimators for merging: merged performance Item Response Theory estimator (MP-IRT) and generalized merged performance Item Response Theory estimator (GMP-IRT). Since model merging linearly combines weights, we assume that the latent abilities of the merged model (e.g., problem-solving or linguistic capabilities) are also a linear combination of the endpoints' abilities. This assumption mirrors the linear task-vector framework established in chapter 5, where merged parameters are linear combinations of task-specific displacements. It also makes the approach substantially cheaper: we estimate only the interpolation coefficients ξ rather than recomputing the full ability vector γ of each merged candidate from scratch (as done in P-IRT and GP-IRT [129]).

Assumption 8.3.1 (Linear Combination of Latent Abilities). Let $\{m_0, m_1, \dots, m_n\}$ be endpoint models with latent ability vectors γ_i . If a new model $\tilde{m}$ is formed as a linear combination of their parameters, its ability vector $\gamma_{\tilde{m}}$ can be expressed as:

$$\gamma_{\tilde{m}} = \sum_{j=1}^n \xi_j \gamma_j = [\gamma_1, \dots, \gamma_n] \xi \quad (8.2)$$

where $\xi = (\xi_1, \dots, \xi_n)$ are the interpolation coefficients.

This assumption allows us to compute the multidimensional IRT model (eq. (8.1)) for model merging as a linear combination of the individual models' abilities:

$$\begin{aligned} p_{i\tilde{m}} &= \mathbb{P}(Y_{i\tilde{m}} = 1 \mid \xi_1 \gamma_1 + \xi_2 \gamma_2, a_i, \beta_i) \\ &= \frac{1}{1 + \exp(-a_i^\top (\xi_1 \gamma_1 + \xi_2 \gamma_2) + \beta_i)} \end{aligned} \quad (8.3)$$

Since the endpoint models' latent abilities γ_j were pre-estimated over the full dataset $\mathcal{D}$ in the Estimate step, we only need the subset $\bar{\mathcal{D}}$ to estimate the interpolation coefficients ξ_j via MLE.

Performance Estimators. To estimate the accuracy of the merged model $\tilde{m}$ using only the reduced dataset $\bar{\mathcal{D}}$ and $p_{i\tilde{m}}$, we define the *merged performance-IRT* (MP-IRT) estimator as:

$$\hat{Z}_{\tilde{m}}^{\text{mp-IRT}} = \frac{\hat{\tau}}{|\bar{\mathcal{D}}|} \sum_{i \in \bar{\mathcal{D}}} Y_{i\tilde{m}} + \frac{1 - \hat{\tau}}{|\mathcal{D} \setminus \bar{\mathcal{D}}|} \sum_{i \in \mathcal{D} \setminus \bar{\mathcal{D}}} \hat{p}_{i\tilde{m}} \quad (8.4)$$

where $\hat{\tau} = \frac{|\bar{\mathcal{D}}|}{|\mathcal{D}|}$ downweights smaller subsets that may be noisier. In practice, we are considering the observed correctness for the data points we have access to, while $\hat{p}_{i\tilde{m}}$ predictions are used for the rest of them, enabling accurate performance estimation across all examples despite evaluating only a subset, where $\hat{p}_{i\tilde{m}} = \mathbb{P}(Y_{i\tilde{m}} = 1 \mid \hat{\xi}_1 \hat{\gamma}_1 + \hat{\xi}_2 \hat{\gamma}_2, \hat{a}_i, \hat{\beta}_i)$ is the distribution defined by plugging into eq. (8.3) the parameters found via MLE.

Although designed for model merging, $\hat{Z}_{\tilde{m}}^{\text{mp-IRT}}$ inherits certain limitations of P-IRT [129], such as non-uniform weighting and imperfect IRT fits. To mitigate these, we define a *generalized* estimator. Let $c \in [0, 1]$ be a scalar that controls the balance between observed and predicted correctness, and let $w_i = 1/|\bar{\mathcal{D}}|$ be uniform per-sample weights. The generalized estimator interpolates between $\hat{Z}_{\tilde{m}}^{\text{mp-IRT}}$ and the observed correctness on $\bar{\mathcal{D}}$:

$$\hat{Z}_{\tilde{m}}^{\text{gmp-IRT}} = c \sum_{i \in \bar{\mathcal{D}}} w_i Y_{i\tilde{m}} + (1 - c) \hat{Z}_{\tilde{m}}^{\text{mp-IRT}} \quad (8.5)$$

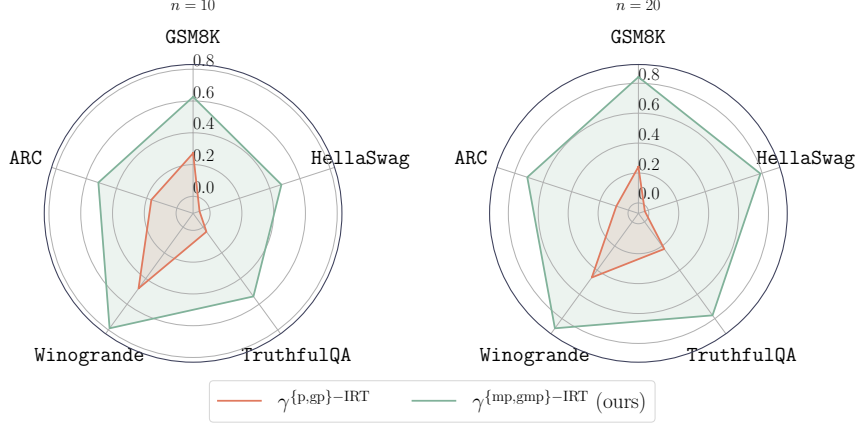


FIGURE 8.3: *Ability Estimator*: Cosine similarity between estimated and true abilities for different tasks (higher is better). Our estimated abilities $\gamma_{\{mp, gmp\}}-IRT$ better approximate true abilities.

We discuss in section E.3.2 the optimal choice for c . Although model merging can sometimes degrade performance due to weight interference, suggesting non-linear ability interactions, our assumption is empirically supported as we are interested only in evolved models that show a positive performance gain. As validated in our experiments (section 8.4.1), our custom estimators, designed around this assumption, outperform standard IRT estimators.

8.3.3 Evolve: Evolutionary search

The final step of our algorithm frames model merging as a multi-objective optimization problem. Each merging objective $F(\tilde{m}; D_i)$ represents the performance of the merged model $\tilde{m}$ on task i . In practice, we select a multi-objective evolutionary algorithm (e.g., NSGA-II [40]) and a merging strategy (e.g., TIES [176]), aiming to optimize the corresponding Pareto front, formally defined as:

$$P_{\overline{F}_D}(\Theta) = \{\theta_i \in \Theta : \nexists \theta_j \in \Theta \text{ s.t. } \theta_j \succ \theta_i\}$$

where $\succ$ denotes *Pareto-dominance*. A model m Pareto-dominates m' if:

$$\begin{aligned} \forall F \in \overline{F}_D : F(m; D) &\leq F(m'; D) \\ \text{and} \\ \exists F \in \overline{F}_D : F(m; D) &< F(m'; D) \end{aligned}$$

This means m is strictly better in at least one metric and no worse in all others. Models on the Pareto front are thus not dominated by any other model.

In our setting, to reduce computational costs, we approximate optimization using $\overline{F}_{\tilde{D}}$ instead of $\overline{F}_D$, where $\tilde{D} \subset D$ is obtained by the *extraction* step. Performance on $\tilde{D}$ is then estimated using the performance estimator.

8.4 Experimental results

In this section, we evaluate MERGE³, demonstrating its effectiveness in evolutionary model merging on consumer-grade GPUs. We first validate the proposed ability and performance estimators, assessing their accuracy in approximating full-dataset evaluations. Next, we examine cross-lingual transfer, where MERGE³ enables efficient merging of multilingual

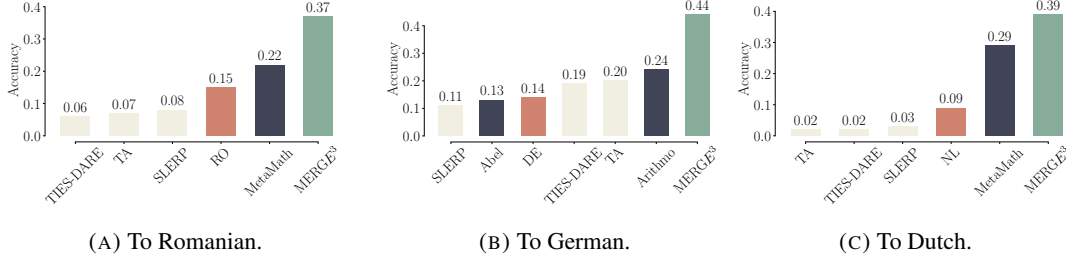


FIGURE 8.4: *Cross-lingual skill transfer*: merging math models (dark blue) with language-specific models (red) effectively transfers mathematical skills across languages (green - our method) compared to baselines (white). Accuracy on GSM8K for each target language.

models, improving mathematical reasoning across languages. Thereafter, we evaluate its ability to synthesize multilingual models, surpassing individual fine-tuned baselines while remaining computationally efficient. Finally, we analyze the performance of MERGE³ on different GPUs. All merging experiments were performed with our library *Mergetetic* [111] (see Appendix E.1) on an RTX 4090 GPU featuring 24 GB of VRAM, while employing a batch size of 8, 4-bit quantization, and models comprising ≈ 7 billion parameters (see Appendix E.2).

8.4.1 Validating estimators

In this section, we empirically validate our merged-performance estimators by comparing them against standard P-IRT and GP-IRT estimators [129] across five benchmark datasets: GSM8K [30], WINOGRANDE [137], TRUTHFULQA [101], HELLASWAG [186], and ARC [28]. Additional results are provided in section E.3.

Ability Estimators. To validate our ability estimators we compare their inferred latent ability vectors to the reference “ground-truth” vectors Γ . Specifically, we measure the cosine similarity and the Euclidean distance from the ground-truth Γ both for $\gamma^{\{\text{mp,gmp}\}-\text{IRT}}$, estimated with our merged-performance IRT approaches, and $\gamma^{\{\text{p,gp}\}-\text{IRT}}$, estimated with the P-IRT and GP-IRT estimators [129]. Here, Γ_m is computed by fitting the IRT model (as in section 8.3.1) to each merged model m using its entire set of responses on the full dataset D . Incorporating all available data, Γ_m serves as our best proxy for the model’s true ability. Conversely, both $\gamma_m^{\{\text{mp,gmp}\}-\text{IRT}}$ and $\gamma_m^{\{\text{p,gp}\}-\text{IRT}}$ are estimated using only a smaller subset $\bar{D} \subset D$ of size n . Figure 8.3 shows the results of this comparison for $n = 10$ and $n = 20$, while the results for $n = 15, 30, 50, 100$ are reported in Section E.3.2 along with the same experiment over different languages. Across all five benchmark tasks, our proposed ability estimator $\gamma_m^{\{\text{mp,gmp}\}-\text{IRT}}$ consistently yields ability vectors with higher cosine similarity to Γ than $\gamma_m^{\{\text{p,gp}\}-\text{IRT}}$. This trend is evident across both subset sizes, highlighting the robustness of our approach even with limited data. The superior performance of $\gamma_m^{\{\text{mp,gmp}\}-\text{IRT}}$ empirically validates Assumption 8.3.1, confirming that an IRT-based ability estimator designed around this assumption provides more accurate ability estimates than a general-purpose alternative.

Performance Estimators. To assess the accuracy of our proposed performance estimators, we measure their absolute estimation error across different sample sizes. Specifically, we evaluate the performance estimates of six merged models using random sampling, P-IRT, GP-IRT [129], MP-IRT, and GMP-IRT across various subset sizes. The resulting absolute

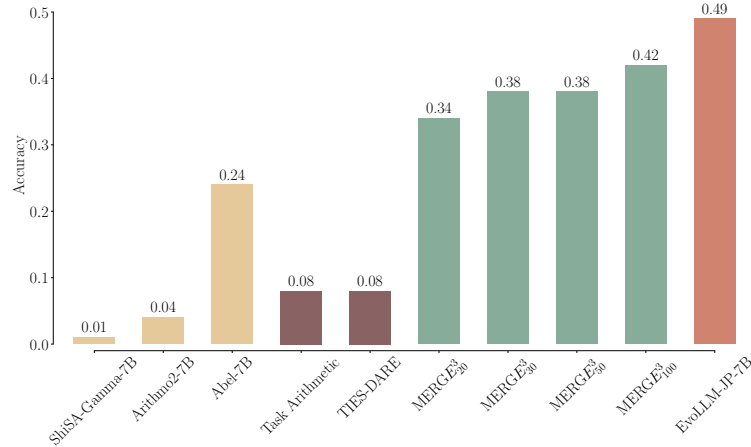


FIGURE 8.5: Accuracy of merged models for Japanese GSM8K.

errors shown in fig. 8.2 are reported for ARC, GSM8K, TRUTHFULQA, and an aggregate average across all five benchmarks.

As shown in the figure, our proposed estimators, MP-IRT and GMP-IRT, consistently achieve lower absolute error compared to GP-IRT and P-IRT. While all IRT-based methods outperform random sampling, the incorporation of merged-performance IRT significantly enhances estimation accuracy. Notably, both MP-IRT and GMP-IRT maintain low empirical error and reduced variance even when operating with very small subsets ($|\bar{D}| \approx 1.5\%$ of the full dataset). This highlights the robustness of our approach in low-data regimes.

Since lower empirical error often correlates with reduced *expected* error (as formalized in Section 8.5), we adopt MP-IRT and GMP-IRT as our primary estimators for evolving merged language models in subsequent experiments.

8.4.2 Cross-lingual transfer of mathematical skills

To assess the transfer of mathematical reasoning from English to other languages, we merge an English math-specialized model with a MISTRAL-7B [82] fine-tuned on each target language, then evaluate on the corresponding GSM8K translations [30]. Appendix E.2.2 provides details on the specific models used for merging. Following Akiba et al. [3], we label an answer correct only if it is both accurate and written in the target language. We benchmark our approach against three commonly used merging baselines: Task Arithmetic [78], TIES [176], and DARE [183]. Following standard practice in the merging community, we apply either TIES and DARE jointly, or SLERP [142].

As shown in fig. 8.4, merging a language-specific fine-tuned model with a math-specialized model consistently surpasses both endpoint models by 10–20% in accuracy on the translated GSM8K. In contrast, standard baselines often yield sub-optimal merges, performing worse than the endpoints themselves. This highlights the importance of optimized merging coefficients and motivates our evolutionary framework.

Next, we evaluate our method for transferring math skills from English to Japanese and compare it to EvoMerge [3], which serves as an upper bound by computing fitness on the full dataset. As illustrated in fig. 8.5, our approach confirms the significant gains seen for the other languages, greatly surpassing both the performance of the endpoint models and that of the merging baselines. While the accuracy is lower than that of the model obtained by computing the fitness on the full dataset as done by Akiba et al. [3], fig. 8.1 shows that our approximation yields a method that is $50\times$ more efficient, effectively making evolutionary

Model	Accuracy ($\uparrow$)							
	Italian		English		German		Dutch	
Finetuned	0.61	–	0.75	–	0.61	–	0.50	–
MERGE ³	0.69	($\uparrow$ 8%)	0.79	($\uparrow$ 4%)	0.72	($\uparrow$ 11%)	0.69	($\uparrow$ 19%)

TABLE 8.1: Evolving a multilingual model. For each language, we report the accuracy on the corresponding translated ARC of both the language-specific model and the evolved multilingual model.

merging feasible on a single NVIDIA 4090 GPU (see table E.9 for timings across different hardware).

8.4.3 Ablation study: self-merging

In this section, we present an ablation study to test whether the observed improvements in the merged models arise from genuine cross-lingual knowledge transfer or merely from fitting to the prompt template. To structure this analysis, we formalize our inquiry through two hypotheses:

Null Hypothesis (H_0). The improvements seen in the merged models are due to the model fitting itself on the prompt template, rather than any cross-lingual knowledge exchange.

Alternative Hypothesis (H_1). The improvements arise from actual cross-lingual knowledge transfer and are not merely the result of fitting the prompt template.

To evaluate these hypotheses, we propose a *self-merging* procedure. Concretely, we take the linguistic model and merge it with *itself* using the standard MERGE³ methodology outlined in algorithm 6. Under H_0 , if the improvements are solely due to the prompt template, merging the model with itself should lead to performance gains (i.e., the merged model would still “fit” the template). Conversely, under H_1 , if cross-lingual knowledge transfer is responsible for the enhanced performance, self-merging should *not* yield improvements. In fact, additional noise could even degrade performance relative to the baseline.

We conducted this self-merging experiment on the Italian model using the GSM8K dataset. The results, shown in fig. 8.6, reveal that performance actually *decreases* when the model is merged with itself. This observation strongly supports the alternative hypothesis (H_1): the performance gains in cross-lingual merges indeed stem from genuine knowledge transfer, rather than mere adaptation to a prompt template.

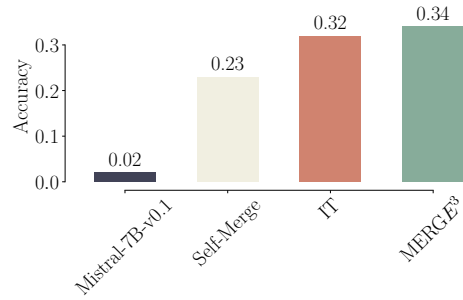


FIGURE 8.6: Accuracy of the base model (Mistral-7B), the Italian Endpoint (IT), Self-Merge and MERGE³ models on the Italian-translated version of GSM8K.

8.4.4 Evolving a multilingual model

We next combine individually fine-tuned models for {IT, EN, DE, NL} into a single multilingual model. Appendix E.2.2 provides details on the specific models used for each language. As shown in table 8.1, the resulting merged model surpasses each language-specific variant by up to 19% in accuracy on the ARC dataset [28]. Even more notably, it outperforms

all its constituent endpoints, demonstrating a clear positive transfer of knowledge across languages. Notably, the largest improvement occurs for Dutch (from 50% to 69%), suggesting that merging particularly benefits languages where baseline performance is lower. Even English, which starts from the highest baseline, gains by 4%, indicating that positive transfer is not limited to stronger endpoints. The fact that the merged model outperforms all individual fine-tunings (rather than landing between them) points to a genuine cross-lingual synergy. These conclusions are further strengthened by the self-merging ablation in section 8.4.3.

8.5 Theoretical guarantees

In this section, we provide theoretical guarantees for our *performance estimator*, analyzing its stability under dataset reduction and showing that it remains a reliable proxy for full-dataset accuracy.

The section is structured as follows: first (section 8.5.1), we establish a formal connection between the accuracy of the performance estimator and the quality of the minimum found when using it as the objective function; second (section 8.5.2), we study the asymptotic properties of the estimator, formalizing it as unbiased; and finally (section 8.5.3), we show that the estimator behaves in expectation within an ϵ -bound of the true optimum. The proofs are given in section E.4.

8.5.1 Part I: ϵ -stable estimators and ϵ -optimality preservation

We first consider a performance metric $F(\theta; \mathcal{D})$ for $\theta \in \Theta \subset \mathbb{R}^n$, where $\mathcal{D}$ is a dataset. If we choose a smaller subset $\bar{\mathcal{D}} \subset \mathcal{D}$ to approximate this metric, denoted $F(\theta; \bar{\mathcal{D}})$, we wish to control the loss in optimality incurred by replacing $F(\theta; \mathcal{D})$ with $F(\theta; \bar{\mathcal{D}})$.

Definition 8.5.1 (ϵ -Stability.). Given two datasets $\mathcal{D}$ and $\bar{\mathcal{D}}$, we say $F(\cdot; \bar{\mathcal{D}})$ is ϵ -stable with respect to $F(\cdot; \mathcal{D})$ if, for all $\theta \in \Theta$,

$$|F(\theta; \mathcal{D}) - F(\theta; \bar{\mathcal{D}})| \leq \epsilon$$

Under this condition, minimizing $F(\cdot; \bar{\mathcal{D}})$ yields an objective value within 2ϵ of minimizing $F(\cdot; \mathcal{D})$. Formally:

Theorem 8.5.2 (ϵ -Optimality Preservation). Let $\mathcal{D}$ be a dataset, let $\bar{\mathcal{D}} \subset \mathcal{D}$ be a subset, and let $F(\cdot; \bar{\mathcal{D}})$ be ϵ -stable with respect to $F(\cdot; \mathcal{D})$, with a fixed $\epsilon > 0$. Define

$$\theta^* = \operatorname{argmin}_{\theta \in \Theta} F(\theta; \mathcal{D}) \quad \text{and} \quad \hat{\theta} = \operatorname{argmin}_{\theta \in \Theta} F(\theta; \bar{\mathcal{D}})$$

Then

$$F(\hat{\theta}; \mathcal{D}) \leq F(\theta^*; \mathcal{D}) + 2\epsilon$$

Proof. By ϵ -stability applied to $\hat{\theta}$: $F(\hat{\theta}; \mathcal{D}) \leq F(\hat{\theta}; \bar{\mathcal{D}}) + \epsilon$. Since $\hat{\theta}$ minimizes $F(\cdot; \bar{\mathcal{D}})$: $F(\hat{\theta}; \bar{\mathcal{D}}) \leq F(\theta^*; \bar{\mathcal{D}})$. By ϵ -stability applied to θ^* : $F(\theta^*; \bar{\mathcal{D}}) \leq F(\theta^*; \mathcal{D}) + \epsilon$. Chaining these inequalities gives $F(\hat{\theta}; \mathcal{D}) \leq F(\theta^*; \mathcal{D}) + 2\epsilon$. $\square$

Thus, ϵ -stability ensures that any global minimizer on $\bar{\mathcal{D}}$ is 2ϵ -optimal on the true full dataset $\mathcal{D}$. Nevertheless, uniformly bounding $|F(\theta; \mathcal{D}) - F(\theta; \bar{\mathcal{D}})|$ for all θ may be too strong in practice. For this reason, we introduce:

Definition 8.5.3 (ϵ -Stability in expectation). Given two datasets $\mathcal{D}$ and $\bar{\mathcal{D}}$, we say $F(\cdot; \bar{\mathcal{D}})$ is ϵ -stable in expectation with respect to $F(\cdot; \mathcal{D})$ if

$$\mathbb{E}_{\bar{\mathcal{D}}} [|F(\theta; \mathcal{D}) - F(\theta; \bar{\mathcal{D}})|] \leq \epsilon$$

where the expectation is over the (random) choice of $\bar{\mathcal{D}}$

Under this relaxed notion, we still obtain a similar control on the *expected* suboptimality gap:

Theorem 8.5.4 (Expected ϵ -Stability of the Minimum). *Suppose $F(\cdot; \bar{\mathcal{D}})$ is ϵ -stable in expectation with respect to $F(\cdot; \mathcal{D})$. Let*

$$m^* := \min_{\theta \in \Theta} F(\theta; \mathcal{D}) \quad \text{and} \quad \hat{m}(\bar{\mathcal{D}}) := \min_{\theta \in \Theta} F(\theta; \bar{\mathcal{D}})$$

Then

$$|m^* - \mathbb{E}_{\bar{\mathcal{D}}}[\hat{m}(\bar{\mathcal{D}})]| \leq \epsilon$$

Hence, even if stability only holds *on average*, the expected gap between the global optimum on $\mathcal{D}$ and the optimum on $\bar{\mathcal{D}}$ remains at most ϵ .

8.5.2 Part II: Theoretical guarantees for MP-IRT

We now apply these ideas to our proposed MP-IRT estimator (cf. section 8.3.1). We first show that MP-IRT is asymptotically unbiased, and then combine this fact with Theorem 8.5.4 to argue that MP-IRT-based minimizers remain close to those that minimize the full-dataset performance measure.

Asymptotic unbiasedness. The following proposition establishes that, as $\bar{D}$ grows, $\hat{Z}^{\text{mp-IRT}}$ converges in probability to the true performance Z . Its proof relies on classical limit arguments for unbiased estimators.

Proposition 8.5.5 (Asymptotic unbiasedness of MP-IRT). *Assume: (i) $\hat{\xi} \rightarrow \xi$ in probability as $|\hat{I}| \rightarrow \infty$, (ii) for each $i \in I$, the true values $a_i, \beta_i, \theta_1, \theta_2$ are known, with $\sup_{i \in I} \|a_i\|_2 \leq c$ for a fixed c , (iii) linear inheritance of abilities (cf. Assumption 8.3.1) holds. Then, for all j, l ,*

$$\left| \mathbb{E}[\hat{Z}_{jl} \mid Y_{i_0l}, \dots, Y_{i_kl}] - \mathbb{E}[Z_{jl} \mid Y_{i_0l}, \dots, Y_{i_kl}] \right| \rightarrow 0$$

in probability as $|\hat{I}| \rightarrow \infty$.

Thus, for sufficiently large subsets $\bar{D}$, the discrepancy between $\hat{Z}_{\bar{m}}$ and $Z_{\bar{m}}$ can be made arbitrarily small with high probability.

8.5.3 Part III: Performance preservation via MP-IRT

We now conclude that MP-IRT preserves near-optimality when we optimize over a suitably large $\bar{\mathcal{D}} \subset \mathcal{D}$. Since Proposition 8.5.5 asserts that $\hat{Z}$ approximates Z well for large $|\bar{\mathcal{D}}|$, it follows (under mild conditions) that MP-IRT remains ϵ -stable in expectation. Hence, Theorem 8.5.4 shows that minimizing $\hat{Z}$ on $\bar{D}$ yields, on average, a solution within ϵ of the full-dataset optimum.

Theorem 8.5.6 (Asymptotic performance preservation of MP-IRT). *Let $\bar{\mathcal{D}} \subset \mathcal{D}$ be a random subset used to compute $\hat{Z}^{\text{mp-IRT}}$. Suppose that, as $|\bar{\mathcal{D}}| \rightarrow \infty$, $\hat{Z}^{\text{mp-IRT}}$ converges in probability to Z (the true performance on $\mathcal{D}$), and that $\hat{Z}^{\text{mp-IRT}}$ is ϵ -stable in expectation for sufficiently large $|\bar{\mathcal{D}}|$. Then the expected global optimum of $\hat{Z}^{\text{mp-IRT}}$ on $\bar{\mathcal{D}}$ differs from that of Z on $\mathcal{D}$ by at most ϵ . As $|\bar{\mathcal{D}}| \rightarrow \infty$, $\epsilon \rightarrow 0$.*

TABLE 8.2: Comparison of Evolve methods by number of trials, estimated total time on a single NVIDIA 4090, sample size used for Fitness computation, and final accuracy on GSM8K. The number of trials equals population size $\times$ iterations and represents the total number of merged models evaluated during the entire Evolve run.

Method	N_{models}	Estimated total time	Sample size	Accuracy
EvoLLM-JP-7B	1000	62 days	1000	0.49
MERGE ₁₀₀ ³	175	21h	100	0.42
MERGE ₅₀ ³	175	12h 20m	50	0.38
MERGE ₃₀ ³	175	10h 30m	30	0.38
MERGE ₂₀ ³	175	10h 15m	20	0.34

Finite-sample analysis via the Law of Large Numbers. In practice, we rarely have $|\bar{\mathcal{D}}| \rightarrow \infty$. Instead, one can appeal to *expected* ϵ -stability (Theorem 8.5.4) and then *estimate* the corresponding expectation empirically. For instance, one may draw multiple subsets $\bar{D}_1, \dots, \bar{D}_S$ at random from $\mathcal{D}$ and compute

$$\frac{1}{S} \sum_{s=1}^S |F(\theta; \mathcal{D}) - F(\theta; \bar{D}_s)|$$

as an empirical approximation to $\mathbb{E}_{\bar{\mathcal{D}}} [|F(\theta; \mathcal{D}) - F(\theta; \bar{\mathcal{D}})|]$. By the Law of Large Numbers, if this empirical average remains small (say, $\approx \tilde{\epsilon}$), then the true expectation is also small. Consequently, Theorem 8.5.4 implies that the optimal solution on each $\bar{D}_s$ is within $\tilde{\epsilon}$ of the global optimum on $\mathcal{D}$, on average.

Conclusion. In summary, MP-IRT inherits asymptotic consistency from p-IRT while requiring only a subset $\bar{\mathcal{D}} \subset \mathcal{D}$. By showing it is ϵ -stable (in expectation) for large $|\bar{\mathcal{D}}|$, we conclude that *optimizing on $\bar{\mathcal{D}}$ yields (on average) a solution close to the true optimum on $\mathcal{D}$* . In finite-sample regimes, multiple random draws of $\bar{\mathcal{D}}$ can be used to empirically verify that the discrepancy remains small, thereby justifying the practical use of MP-IRT on moderately sized subsets.

8.6 Computational cost

We summarize the GPU timing results for MERGE³ in table 8.2, comparing evaluation and merge times across different hardware setups. These findings highlight the practical feasibility of our approach even on older GPUs. For additional experimental details, refer to section E.3.3.

8.7 Summary and outlook

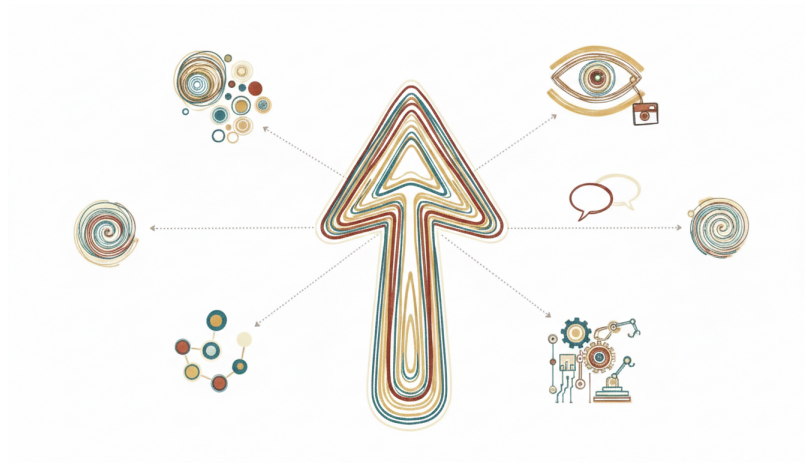
MERGE³ makes high-quality evolutionary merging feasible on a single GPU, requiring roughly one day on an NVIDIA 4090 with 24 GB of VRAM. By combining a subset-based approach with IRT-driven performance estimation, the framework reduces merging costs by up to $50\times$ compared to prior methods while maintaining competitive accuracy. The experiments demonstrate successful cross-lingual transfer in mathematics (e.g., from English to Japanese), as well as the synthesis of new multilingual models that outperform each of

their language-specific endpoints. MERGE³ thus expands the practical reach of evolutionary merging, allowing everyday practitioners to benefit from advanced multi-task and multilingual model merges at a fraction of the usual computational cost.

Together, the four chapters of part III form a coherent progression from theory to practice. Chapter 5 established *why* task arithmetic works by grounding task vectors in gradient theory. Chapter 6 revealed the *low-rank structure* of these vectors and showed how to exploit it for compression and interference reduction. Chapter 7 extended this structural insight to the inference stage, enabling *input-adaptive* routing through task subspaces. Finally, this chapter democratized *evolutionary search* over merging configurations: while chapter 6 reduces interference structurally at the parameter level, MERGE³ navigates interference by optimizing merging coefficients via evolution, removing the computational barrier that had confined evolutionary merging to large-scale compute environments. Taken together, these contributions advance the theoretical understanding, structural exploitation, adaptive deployment, and practical accessibility of multi-task model merging.

Part IV

Applications, Outlook, and Conclusions



“We can only see a short distance ahead, but we can see plenty there that needs to be done.”

— Alan Turing

Chapter 9

Applications and Impact

The methods developed throughout this thesis have applications that extend well beyond the benchmarks on which they were evaluated. We outline here the most immediate ones.

Federated learning. The single-task merging methods for models initialized from different checkpoints explored in part II lend themselves naturally to federated learning. In federated learning [108], clients collaboratively train a shared model without exchanging their private data: each client fine-tunes a local copy of the model on its own data, and a central server periodically aggregates the local updates into a global model that is redistributed to the clients. The standard aggregation strategy, FedAvg [108], simply averages the client parameters, which works well when clients start from the same initialization and perform few local steps, but degrades as local training diverges. The cycle-consistent alignment developed in chapter 3 directly addresses this failure mode: by mapping all client models into a shared universe space before averaging, it can compensate for the drift induced by prolonged local training or heterogeneous data distributions. Preliminary experiments (section A.2.5) already show that this approach outperforms FedAvg when clients are initialized from different random seeds, and that the gains increase with the number of local epochs, precisely the regime where standard averaging breaks down. While these results are limited to small-scale settings, they point to a promising direction in which permutation-based alignment could serve as a drop-in replacement for naive averaging in federated protocols.

Ensembling from the universe space. Beyond simple averaging, the universe space provides a geometrically principled sampling ground for ensembling. As discussed in chapter 2, prior work has shown that sampling models from low-loss connected regions and averaging their predictions yields ensemble-quality performance at a fraction of the cost of independently trained ensembles [13, 59]. The universe mapping from chapter 3 places all aligned models into a common low-loss basin (fig. 3.3), creating exactly the kind of connected manifold from which diverse checkpoints can be cheaply sampled for prediction averaging. This opens the possibility of obtaining ensemble-quality predictions from independently trained models without requiring a shared initialization, by first aligning them into the universe and then sampling along the interpolation paths within it.

Transferring fine-tuning across model releases. When a foundation model is updated or retrained, previously fine-tuned variants become obsolete, and practitioners must repeat the fine-tuning process from scratch. An emerging line of work investigates whether task vectors can instead be *transported* to a new backbone. Rinaldi et al. [135] propose a data-free, two-level permutation method for Transformers that re-basins task vectors onto an updated checkpoint by first permuting attention heads and then adjusting intra-head parameters. Rinaldi et al. [136] take a complementary approach, using a small number of target-model gradients to mask the source task vector so that it aligns with the loss landscape of the new backbone.

While both methods show promising initial results, fully closing the gap with native fine-tuning remains an open challenge, making this a natural application area for the alignment and merging techniques developed in this thesis.

State-of-the-art multi-task merging. The multi-task merging methods for models fine-tuned from the same foundation model developed in part III provide immediate practical utility. The structured, layer-wise approach introduced in chapter 6, which preserves the matrix structure of per-layer task vectors and exploits their low-rank geometry, established a new state of the art in vision model merging, surpassing prior methods by an average of ~ 15 percentage points. The input-adaptive extension MASS (chapter 7) pushes this further by routing each input to the most relevant task subspaces at inference time, recovering up to 98% of individual expert accuracy with a fixed $2\times$ storage overhead and no oracle knowledge of the task at hand.

Efficient compression of task-specific experts. From a complementary angle, TSV-C (chapter 6) provides a practical solution for scenarios where multiple task-specific experts must be stored and deployed. By retaining only the top $\frac{1}{T}$ singular vectors per task per layer, TSV-C compresses T task vectors down to a fixed footprint of approximately $2\times$ the size of the original pre-trained model, regardless of the number of tasks. Concretely, this means that 20 task-specific experts can be stored in the same space as 2 full models, while retaining over 99% of the original fine-tuned accuracy across all benchmarks (table 6.2). This is especially relevant for serving systems that need to maintain a library of specialized models: TSV-C either substantially reduces the storage required for a fixed set of experts, or allows many more experts to be stored within the same storage budget.

Democratizing evolutionary merging for new capabilities. The evolutionary merging framework MERGE³ (chapter 8) offers a practical and accessible way to obtain models exhibiting new skills by composing existing ones, without requiring any training. We demonstrated this by merging a math-specialized English model with language-specific fine-tuned variants of MISTRAL-7B, obtaining models that solve mathematical reasoning tasks in Romanian, Dutch, German, and Japanese, surpassing both endpoint models by 10–20% in accuracy on the translated GSM8K (fig. 8.4). We further showed that MERGE³ enables the synthesis of multilingual models that outperform all of their monolingual constituents: starting from individually fine-tuned models for Italian, English, German, and Dutch, the evolved multilingual model surpassed each language-specific baseline on ARC-CHALLENGE by up to 19% (table 8.1), demonstrating genuine cross-lingual positive transfer rather than a compromise between languages. Crucially, by introducing efficient performance estimators based on Item Response Theory, MERGE³ reduces the computational cost of evolutionary merging by $50\times$ compared to full-dataset evaluation, making it feasible in approximately one day on a single NVIDIA 4090 rather than requiring month-long runs on specialized hardware. This democratization of evolutionary merging puts model composition within reach of individual researchers and practitioners who lack access to large-scale compute infrastructure.

Applicability and challenges. The applications above differ substantially in their level of maturity. Some are immediate consequences of the methods developed in this thesis, while others should be understood as promising directions that require additional adaptation before deployment at the scale of current foundation models. Table 9.1 summarizes this distinction.

Relevance to current foundation models. A further qualification concerns the nature of modern foundation models. The clearest setting for multi-task merging is one in which several

task-specific experts are fine-tuned from the same pretrained backbone and then combined. This setting is central to much of the model-merging literature and to Part III of this thesis, but it is not an exact description of the strongest contemporary foundation models. Frontier base models and post-trained assistants are increasingly general-purpose systems rather than narrow task experts, and their capabilities are often shaped by broad instruction tuning, preference optimization, tool use, and large-scale data mixtures rather than by isolated task-specific fine-tuning. As a result, the literal “merge one expert per task” framing is less salient for monolithic frontier models than for open-source ecosystems of specialized checkpoints, adapters, and task vectors.

This does not make model merging obsolete for foundation models; rather, it changes the relevant unit of composition. In current practice, the objects being merged are often not narrow classifiers but model variants: instruction-tuned models, code models, reasoning models, multilingual models, safety-tuned models, domain-adapted models, or parameter-efficient deltas derived from a common base. In this regime, the methods studied in this thesis remain directly relevant. Task-vector methods provide a language for reusing and composing fine-tuning updates; TSV-style decompositions reduce the storage footprint of large collections of variants; routing methods such as MASS suggest ways to activate only the subspaces needed for a given input; and MERGE³ addresses the practical cost of searching over combinations of large language model variants. Thus, the immediate deployment target is not necessarily the internal training pipeline of a closed frontier model, but the broader foundation-model lifecycle: maintaining families of related checkpoints, transferring capabilities across releases, compressing specialized variants, and composing open-source models without retraining.

Scenario	Current applicability	Main challenges	Most relevant methods
Federated learning	Medium. Most natural when clients share an architecture but local training causes drift.	Scaling permutation alignment beyond small CN-N/MLP settings; handling transformer symmetries; communication overhead; privacy constraints.	C ² M ³
Universe-space ensembling	Medium-low. Conceptually natural, but currently closer to an algorithmic direction than a deployed use case.	Efficiently sampling diverse checkpoints; validating gains on large architectures; avoiding ensemble-time inference cost.	C ² M ³
Fine-tuning transfer across releases	Medium-high. Highly relevant when users have many fine-tuned variants of an older base model.	Backbone mismatch; tokenizer and architecture changes; incomplete recovery compared to native fine-tuning.	Alignment methods, task vectors
Multi-task expert merging	High for open-source and specialized model ecosystems; lower for monolithic frontier models.	Task interference; loss of specialist performance; dependence on a shared base model; evaluation of emergent combined capabilities.	TSV-Merge, MASS, MERGE ³
Expert-library compression	High when many task-specific deltas, adapters, or fine-tuned checkpoints must be stored.	Maintaining per-task fidelity; deciding the rank budget; extending compression reliably to very large language models.	TSV-C
Evolutionary merging	High for open-source LLM composition; medium for production settings.	Expensive evaluation; benchmark overfitting; robustness under prompt and distribution shifts; safety evaluation of merged models.	MERGE ³

TABLE 9.1: Applicability and open challenges for the deployment scenarios discussed in Chapter 9.

Chapter 10

Future Directions

We now outline several directions that, in our view, represent the most promising avenues for advancing the field.

More expressive alignment frameworks. The single-task alignment methods developed in part II rely on the Frank-Wolfe algorithm to solve a relaxation of the weight matching problem over the Birkhoff polytope of doubly stochastic matrices. While effective, this formulation treats the matching problem layer by layer and captures inter-layer dependencies only through gradient coupling. A natural extension is to formulate the problem directly as a *Quadratic Assignment Problem* (QAP) [169], which jointly optimizes all permutations while explicitly encoding the quadratic coupling between adjacent layers. QAP is NP-hard in general, but modern solvers based on semidefinite programming relaxations or learned graph matching networks [169] could yield tighter solutions than the current linear relaxation, particularly for deep architectures where inter-layer dependencies are strongest. An orthogonal and potentially complementary direction comes from *functional maps* [124], a framework from geometry processing that replaces explicit point-to-point correspondences with a small matrix mapping between spectral bases. Rather than finding a permutation between neurons, one could work in the spectral domain of the weight matrices, representing the correspondence as a compact linear map between the eigenbases of the two models. This perspective has already been brought into the representation alignment literature by Fumero et al. [54], who showed that spectral correspondences between neural representation spaces can be computed efficiently and used for model stitching and retrieval. Extending this framework to weight-space alignment could yield soft, differentiable, and naturally low-dimensional matching that sidesteps the combinatorial hardness of permutation search entirely.

Heterogeneous merging. Nearly all methods in this thesis, and in the broader literature, assume *homologous* source models: same architecture, same pre-trained initialization. This is a significant limitation in practice: one may wish to combine a vision transformer with a convolutional network, merge language models with different tokenizers and vocabulary sizes, or even bridge across fundamentally different layer types (e.g., Transformer and state-space models). Preliminary work has begun to address these settings. Stoica et al. [149] propose a “zipping” operation that merges internal features within and across models trained on disjoint tasks, even without a shared initialization, while Nguyen et al. [119] tackle the case of networks with different depths by aligning representations across layers. On the language side, Minixhofer et al. [110] further demonstrate that tokenizers can be transferred across models in a zero-shot fashion, removing one of the key obstacles to merging language models with mismatched vocabularies. Despite these advances, heterogeneous merging remains largely unsolved for large-scale models: merging models from different pre-training corpora or RLHF pipelines requires not only architectural alignment but also semantic alignment of the learned representations. The functional maps framework discussed above, which naturally

accommodates spaces of different dimensionality by working in a shared spectral basis, could provide a principled foundation for this line of work.

From merging to optimization. The connection between task vectors and gradients established in chapter 5 suggests that insights from model merging could inform optimizer design, and vice versa. A striking example is the `Muon` optimizer [84, 102], which applies an orthogonalization step to the momentum at each training step: it computes the SVD of the gradient matrix $G = U\Sigma V^\top$ and uses $UV^\top$ as the update direction, effectively discarding the singular values and retaining only the directional structure. This is remarkably close to the structured merging paradigm developed in chapter 6, where task vectors are decomposed via SVD and the singular vectors are identified as the structurally meaningful components, with interference reduction achieved through whitening of the singular value spectrum. Both approaches share the same insight: that the *directions* in weight space carry the essential information, while the *magnitudes* should be normalized or reweighted. `Muon` has already shown strong results at scale, achieving $2\times$ computational efficiency over AdamW when training 3B-parameter language models [102]. This convergence between the merging and optimization communities suggests a rich space for cross-pollination: structured merging techniques might yield better update rules for multi-task and continual learning, while insights from spectral optimizer design could inform how task vectors should be combined.

Scaling structured merging to large language models. The layer-wise SVD-based methods developed in chapters 6 and 7 have been validated extensively on vision transformers up to ViT-L-14 (300M parameters) across up to 20 tasks. Extending these methods to large language models with 8B parameters and beyond poses both computational and methodological challenges. On the computational side, the SVD of weight matrices at LLM scale is expensive, though recent advances in randomized and streaming SVD algorithms could make it tractable. Methodologically, the structure of LLM task vectors may differ from vision models: the attention and MLP blocks in transformer language models have different rank profiles and interference patterns than the vision encoder layers studied in this thesis. Understanding whether the low-rank structure and the effectiveness of interference reduction via whitening transfer to this regime is a necessary first step before the layer-wise structural paradigm can be applied at the frontier of language modeling.

Beyond classification: merging for generative models. The multi-task merging methods in this thesis have been evaluated primarily on vision classification benchmarks. However, model merging has already found enthusiastic adoption in the generative modeling community, particularly for diffusion models. Härmäläinen et al. [66] demonstrate that simple weight averaging of Stable Diffusion models trained on different data shards yields substantial improvements in image quality, and enables training-free continual learning by adding or removing models from the average. Gandikota et al. [56] introduce concept sliders, LoRA adapters for diffusion models that function as directional edits in weight space, directly analogous to task vectors: over 50 such sliders can be composed simultaneously without degrading output quality. These results suggest that the structured merging techniques from chapter 6, which reduce interference between task directions while preserving their individual contributions, could substantially improve upon the naive averaging currently used in practice. Extending the SVD-based interference analysis and the input-adaptive routing of `MASS` to diffusion model architectures, where “tasks” correspond to styles, concepts, or domains rather than classification categories, is a natural and high-impact direction.

Towards a predictive theory of mergeability. Finally, the field still lacks a *predictive theory of merge success*: there is no standard, cheaply computable “mergeability score” that generalizes across architectures and modalities, analogous to how scaling laws guide pre-training decisions. Current practice relies on grid search over scaling coefficients, pruning thresholds, and mask parameters. Recent work has begun to address this gap from complementary angles. Rahamim et al. [133] propose a concrete definition of mergeability and identify the base model’s existing knowledge as a dominant factor: models fine-tuned on instances that the base model already handles well are substantially more mergeable than those trained on difficult instances. Zhou et al. [192] take a method-aware perspective, showing that mergeability is not an intrinsic model property but depends jointly on the merging method and the partner tasks; nonetheless, subspace overlap and gradient alignment consistently emerge as method-agnostic prerequisites for compatibility, echoing the gradient analysis of chapter 5. On the practical side, Bolton et al. [18] introduce *SimMerge*, which learns to predict the best merge operator and merge order from cheap, task-agnostic similarity signals between checkpoints. These results are encouraging, but a unified theory that connects the structural, spectral, and gradient perspectives developed in this thesis into a single predictive framework remains an open challenge whose resolution would transform model merging from a largely empirical practice into a principled engineering discipline.

Chapter 11

Conclusions

This thesis began with a simple but consequential observation: the history of deep learning has been shaped by a quiet assumption that knowledge lives inside individual models, trained in isolation and discarded when no longer needed. The goal of this work has been to question that assumption and develop alternatives grounded in theory and algorithms. The result is a set of contributions that together advance the formal foundations of model merging, drawing on geometry, algebra, and provable guarantees.

11.1 Summary of Contributions

Single-task merging: geometry and cycle consistency. The first part of this thesis addressed the problem of merging independently trained instances of the same architecture. The central obstacle is well understood: the permutation symmetries of neurons create a combinatorial number of functionally equivalent parameterizations, causing independently trained models to occupy seemingly distant regions of weight space even when they compute the same function. Prior alignment methods such as Git Re-Basin resolved this pairwise and layer by layer, but offered no guarantees on consistency when more than two models were involved. Cyclic compositions of pairwise permutations would drift, accumulating error and placing the re-permuted model in a completely different basin from which it started.

Chapter 3 addressed this limitation by introducing C^2M^3 , a cycle-consistent multi-model merging algorithm. The key insight is a factorization of pairwise permutations through a shared *universe* space: rather than seeking a direct mapping between every pair of models, the method learns a single permutation from each model to a canonical reference, so that all pairwise correspondences are obtained by composition and cycle consistency is enforced by construction. The Frank-Wolfe-based optimization handles all layers simultaneously, capturing inter-layer dependencies that are invisible to layer-by-layer methods. Empirical evaluation across architectures and datasets demonstrated consistent and often substantial accuracy improvements over the prior state of the art, with accuracy gains as high as 20% when merging five models simultaneously. Crucially, the universe space acts not only as an alignment target but as a natural aggregation ground: models mapped into a shared low-loss basin can be meaningfully averaged, and the resulting merged model can serve as a principled midpoint from which diverse checkpoints are cheaply sampled for ensembling. This addresses a fundamental limitation of reference-based approaches, where the choice of reference introduces an arbitrary bias that the merged model cannot overcome.

Multi-task merging: theory. Having handled the geometric challenge of single-task alignment, the thesis turned to a fundamentally different and more practically impactful regime: merging models fine-tuned on distinct tasks from a shared pre-trained backbone. Here, the shared initialization already ensures weight-space compatibility, so the problem is not one of geometric alignment but of task interference: understanding why combining task-specific weight differences works at all, and when it fails.

Chapter 5 provided the theoretical foundation that had been largely missing from the literature. By analyzing full-batch gradient descent, the chapter established a formal equivalence: a task vector computed from a single epoch of fine-tuning is precisely the negative loss gradient, scaled by the learning rate. This makes the sum of task vectors in task arithmetic mathematically equivalent to a single gradient descent step on the aggregated multi-task loss. For models fine-tuned beyond a single epoch, the equivalence degrades gracefully, with a second-order deviation of $O(\eta^2)$ controlled by the curvature of the loss surface. These findings reframe task arithmetic as approximate multi-task learning, providing a principled explanation for both its effectiveness and its failure modes. A key practical implication follows directly: merging models fine-tuned for a single epoch can match or exceed the performance of merging fully converged models. This reflects the fact that early-epoch task vectors are the best approximations of the true task gradients, and that the fine-tuning trajectory is largely determined by the gradient information accrued in the first epoch. The result clarifies why task proficiency and mergeability are often at odds, and offers a principled justification for the widespread empirical observation that shorter fine-tuning intervals often produce better merged models.

Multi-task merging: low-rank structure and interference reduction. Chapter 6 departed from the convention of treating task vectors as flat parameter vectors and instead examined them at the layer level, preserving their natural matrix structure. Singular value decomposition of per-layer task matrices revealed a consistent structural pattern: task matrices are intrinsically low-rank. The effective rank of a task matrix is a small fraction of its ambient dimension, and the dominant singular vectors concentrate the task-relevant information. This empirical finding has two immediate algorithmic consequences.

The first is compression. By retaining only the top- k singular components per task per layer, TSV-COMPRESS stores T task-specific experts in the same space as approximately two full models, regardless of T . In practice, twenty fine-tuned experts can be compressed to twice the size of the base model while retaining over 99% of per-task accuracy, a result that significantly expands the practical scalability of multi-task model libraries.

The second consequence is interference reduction. The concept of Singular Task Interference (STI) quantifies the overlap between the singular vector subspaces of different tasks: high STI signals that two tasks share directions in weight space, making their task vectors prone to destructive interaction when summed. TSV-MERGE exploits this structure by disentangling task directions via a Procrustes-based whitening transformation before aggregation, producing a merged model that significantly outperforms prior methods. On multi-task vision benchmarks spanning up to twenty tasks across three ViT architectures, TSV-Merge established a new state of the art, with average accuracy improvements of approximately 15 percentage points over existing approaches.

Multi-task merging: input-adaptive routing. All methods discussed to this point produce a single, static merged model that is applied identically to every input regardless of its content. This uniformity leaves performance on the table: a merged model that is optimal on average may be suboptimal for any individual input if the input belongs clearly to one task’s distribution.

Chapter 7 introduced MASS (Merging via Adaptive Subspace Selection), which extends the low-rank structure of chapter 6 from merge time to inference time. The central observation is that the task singular vectors embedded into the merged model by TSV-MERGE form a set of orthogonal subspaces, each encoding the directional signature of a specific task. At inference, a projection-based router measures the reconstruction quality of each input’s intermediate representation against these subspaces and selects the task with minimal residual, a decision

that is equivalent, under an isotropic Gaussian noise model, to maximum a posteriori task estimation. The router requires no labels, no additional training, and no access to the original task data; it operates entirely from the geometry embedded in the merged model.

Experimentally, MASS sets a new state of the art for mixture-of-experts merging across eight out of nine benchmarks, recovering up to 98% of the accuracy of individually fine-tuned experts while requiring only a fixed $2\times$ storage overhead. The method generalizes across visual and language modalities, and a qualitative analysis of the task singular vectors confirms that they capture interpretable, domain-specific semantics aligned with the visual content of each task. The result is an adaptive merged model that is both principled and practical, deployable directly on top of any collection of fine-tuned checkpoints without any access to their original training data.

Efficient evolutionary merging at scale. A complementary strategy for model merging bypasses the design of hand-crafted merging rules entirely and instead optimizes merging coefficients through evolutionary search. This approach has been highly effective, but at a computational cost (thousands of fitness evaluations, each requiring full inference over large datasets) that has placed it far beyond the reach of practitioners working on standard hardware.

Chapter 8 introduced MERGE3, a framework that reduces the computational cost of evolutionary merging by $50\times$, making it feasible on a single consumer GPU in approximately one day rather than requiring month-long runs on specialized infrastructure. The approach combines three components: a dataset reduction strategy that selects a small subset for fitness evaluation; an Item Response Theory (IRT)-based ability estimator that infers the latent capabilities of endpoint models from their correctness patterns; and a novel performance estimator, MP-IRT, that predicts the fitness of a merged model as a linear combination of the endpoints' estimated abilities, exploiting the linearity of weight-space merging. Theoretical analysis shows that MP-IRT is asymptotically unbiased and ϵ -stable in expectation with respect to full-dataset evaluation, ensuring that evolutionary search on the reduced dataset yields solutions within ϵ of the true global optimum.

Experimentally, MERGE3 demonstrates successful cross-lingual transfer of mathematical reasoning skills: merging a math-specialized English model with language-specific fine-tunings to obtain models that solve GSM8K in Romanian, German, Dutch, and Japanese, surpassing all endpoint models by 10–20%. The framework further evolves a single multilingual model from four language-specific endpoints that outperforms each constituent model by up to 19% on ARC-Challenge, demonstrating genuine cross-lingual positive transfer rather than a mere compromise. This application extends the scope of the thesis from the vision domain to large language models.

11.2 Overarching Themes

Reading across the contributions of this thesis, three overarching themes emerge.

Making weight space legible. For most of the history of deep learning, the weight space of a neural network has been treated as fundamentally illegible: a high-dimensional numerical object whose internal organization is an accidental byproduct of optimization, meaningful only in the aggregate sense that the network computes something useful. Model merging quietly challenged this assumption: the very act of adding task vectors and observing coherent behavior implied that weight space had structure, that directions in it were interpretable, that arithmetic over parameters was not arbitrary. What chapter 5 and chapter 6 together establish is precisely this legibility in formal terms. Task vectors are not arbitrary displacements; they are approximate gradient directions, and their geometry in parameter space reflects the

geometry of the loss landscape. The dominant singular vectors of a per-layer task matrix are not random directions; they are the axes along which a task has reorganized the pre-trained representations, and they are interpretable enough that decoding them through a language model yields accurate semantic descriptions of the task’s visual content. This last observation, demonstrated in chapter 7, where task singular vectors decoded as text recover phrases like *image of a car* or *close-up of a textured mesh*, is a particularly suggestive finding: it indicates that the structure uncovered by the theory corresponds to interpretable properties of the learned representations.

Structure begets efficiency. A second theme is that exploiting the geometric and algebraic structure of task vectors yields algorithms that are simultaneously more accurate and more efficient than their structure-agnostic counterparts. The low-rank structure of task matrices enables compression that is essentially lossless. The singular vector subspace geometry enables routing without labels or training. The linearity of merging enables IRT-based performance estimation that reduces evolutionary search costs by $50\times$. In each case, the structural insight is not merely a theoretical observation but the direct source of computational gain.

Accessibility as a scientific goal. A third, perhaps less conventional theme is the deliberate pursuit of accessibility. The democratization already underway in the community (evidenced by the fraction of top-ranked models that are the product of merging rather than training) demands that the tools for doing merging well be as widely accessible as the practice of merging itself. This thesis argues, and attempts to demonstrate, exactly that. The MERGE3 framework makes evolutionary merging feasible on hardware available to any individual researcher. The compression guarantees of TSV-COMPRESS make multi-task expert libraries practical at modest storage budgets. The data-free routing of MASS makes adaptive inference available without training supervision or access to task data. Together, these contributions strive to keep merging the accessible tool it was meant to be.

11.3 Limitations and Open Questions

An honest account of the contributions in this thesis must also acknowledge their limits. The single-task alignment methods of part II have been validated on convolutional and MLP architectures; their behavior on large transformer networks, where the interaction between attention and MLP layers creates more complex symmetry structures, remains to be established. The multi-task methods of part III assume homologous source models sharing the same pre-trained backbone, a condition that holds in the fine-tuning regime but excludes the increasingly common scenario of merging models with different architectures, tokenizers, or pre-training distributions. The gradient equivalence of chapter 5 is derived under full-batch gradient descent with a fixed step size, and while its empirical implications transfer to stochastic settings, a formal analysis under SGD with momentum and learning rate schedules remains an open problem.

More broadly, the field still lacks a predictive theory of mergeability, a cheaply computable quantity, analogous to a scaling law, that would tell a practitioner in advance whether two given models can be merged effectively and what performance to expect. The tools developed in this thesis (gradient alignment, spectral interference analysis) each offer partial windows onto this question, but a unified predictive framework has not yet emerged. Developing such a theory would transform model merging from a largely empirical practice into a fully principled engineering discipline.

Closing Remarks

The thesis has argued that the weights of a neural network are not merely the byproduct of an optimization process to be discarded once training is complete. They are a structured encoding of knowledge, one that can be decomposed, composed, routed, and compared, enabling capabilities that go beyond those of any single model. Achieving this requires understanding the geometry and algebra of weight space, and this thesis has taken concrete steps in that direction.

As model repositories continue to grow at an exponential rate, and as the computational cost of training from scratch becomes increasingly prohibitive for most of the world's researchers, the ability to reuse, combine, and compose existing models will only grow in importance. The contributions of this thesis aim to make such composition not only practically effective, but grounded in principled theoretical understanding.

Part V

Appendix



“The devil is in the details, but so is salvation.”

— Hyman G. Rickover

Appendix A

Additional material on Chapter 3

A.1 Additional details

Here we report in-depth explanations and additional experimental details. In particular, section A.1.1 extensively outlines the most related works, section A.1.2 shows the `Frank-Wolfe` algorithm for the pairwise case, while section A.1.3 describes the `MergeMany` procedure presented in [2] for merging multiple models. Finally, we show how the matching algorithm empirically converges in section A.1.4.

A.1.1 Extended related work

We report here a thorough review of works that are relevant to our research, providing a comprehensive understanding of the context of our work.

Linear mode connectivity Mode connectivity is interested in modes, i.e., model parameters at convergence. In this regard, Frankle et al. [53] first studied the connectivity of the parameters of models that were trained for a few epochs from the same initialization, while Garipov et al. [59] investigated whether these can be connected through a high-accuracy path without requiring the same initialization. Simultaneously, Draxler et al. [46] proposed an algorithm to find a *Minimum Energy Path* (MEP) between two modes of a neural network, showing that these paths are mostly flat in both the training and test landscapes. This implies that many minima actually live in a shared low-loss valley rather than in distinct basins. From a different perspective, Zhou et al. [189] proposed to study a class of neural functionals which are permutation-equivariant by design. Recent research proposes to study model behavior in the weight space beyond linear mode connectivity: Lubana et al. [105] show that different “mechanisms” in related models prevent simple paths of low loss in the weight space, while Zhou et al. [193] studied the linear connections between the linear features of each layer of differently trained models.

Model merging Model merging [2, 65, 83, 144, 149, 168] has seen a surge of interest in the last years as a means to ensemble models without incurring the added computational cost. One of the first works in this direction is Singh and Jaggi [144], who proposed an optimal-transport based weight-matching procedure. Later, Ainsworth et al. [2] proposed three matching methods, one of which being data-free. Closer to our global optimization, Guerrero-Peña et al. [65] proposed a gradient-descent based procedure that iteratively updates soft permutation matrices maintaining their bistochasticity via a differentiable Sinkhorn routine. When the models to match have been trained on different tasks, Stoica et al. [149] introduce a more general “zip” operation that accounts for features that may be task-specific and further allow obtaining multi-headed models. Most recently, Navon et al. [116] proposed aligning models in the embedding space of a deep weight-space architecture. Finally, weight merging proved

useful for large language models [83] and robotics [168]. For a complete survey of mode connectivity and model merging, we refer the reader to [98].

Cycle consistency Cycle consistency is a recurrent idea in computer vision and pattern recognition, where it appears under different names (e.g., “synchronization”, “loop constraints”, or “multi-way matching”) depending on the task. In the area of multi-view 3D reconstruction, Zach et al. [185] were probably the first to make an explicit attempt at finding solutions meeting the cycle-consistency requirement, although without ensuring theoretical guarantees on the result. In geometry processing, Cosmo et al. [32] ensured cycle-consistent alignment of collections of 3D shapes using an n -fold extension of the Gromov-Wasserstein distance with sparsity constraints. Overall, cycle consistency is a recurring idea in the computer vision [7, 167, 185] graph matching [125, 141, 152] and geometry processing literature [14, 32, 75].

A.1.2 Pairwise Frank-Wolfe algorithm

As introduced in section 3.2, we optimize a layer-global objective by iteratively optimizing its linear approximation via the Frank-Wolfe algorithm [51]. We compute the gradient of eq. (2.2) with respect to each permutation P_i , as the sum of two contributions for each ∇_{P_i} : one from permuting the rows of W_i and another from permuting the columns of W_{i+1} :

$$\nabla_{P_i} f = \underbrace{W_i^A P_{i-1} (W_i^B)^\top}_{\text{from permuting rows}} + \underbrace{(W_{i+1}^A)^\top P_{i+1} W_{i+1}^B}_{\text{from permuting columns}}. \quad (\text{A.1})$$

We report in Algorithm 7 the Frank-Wolfe algorithm for the pairwise case.

Algorithm 7 Frank-Wolfe for pairwise Weight Matching

Require: Weights of two models A and B with L layers, tolerance $\epsilon > 0$

Ensure: Approximate solution to eq. (2.2)

```

1:  $\mathbf{P}^k \leftarrow$  identity matrices
2: repeat
3:   for  $i = 1$  to  $L$  do
4:      $P_i^k \leftarrow$  permutation acting on rows of  $W_i$ 
5:      $P_{i-1}^k \leftarrow$  permutation acting on columns of  $W_i$ 
6:      $\nabla_{P_i^k} f \mathrel{+}= W_i^A P_{i-1}^k (W_i^B)^\top$ 
7:      $\nabla_{P_{i-1}^k} f \mathrel{+}= (W_i^A)^\top P_i^k W_i^B$ 
8:   end for
9:   for  $P_i^k \in \mathbf{P}^k$  do
10:     $\Pi_i \leftarrow \text{LAP}(\nabla_{P_i^k} f)$ 
11:   end for
12:    $\alpha \leftarrow \text{LINESEARCH}(f, \mathbf{P}^k, \Pi)$ 
13:   for  $P_i^k \in \mathbf{P}^k$  do
14:      $P_i^{k+1} = (1 - \alpha)P_i^k + \alpha \Pi_i$ 
15:   end for
16: until  $\|f(A, B, \mathbf{P}^{k+1}) - f(A, B, \mathbf{P}^k)\| < \epsilon$ 
17: return  $\mathbf{P}^k$ 

```

A.1.3 MergeMany algorithm

Algorithm 8 reports the MergeMany procedure originally proposed by Ainsworth et al. [2] for merging multiple models, mainly consisting in alternating matching and aggregation until

convergence. In practice, at each iteration, the procedure picks a reference model at random and matches all the other models to it. Then, they are all aggregated by averaging the weights.

Algorithm 8 MERGEMANY

Require: Model weights $\theta_1, \dots, \theta_N$

Ensure: A merged set of parameters $\tilde{\theta}$.

```

1: repeat
2:   for  $i \in \text{RANDOMPERMUTATION}(1, \dots, N)$  do
3:      $\theta' \leftarrow \frac{1}{N-1} \sum_{j \in \{1, \dots, N\} \setminus \{i\}} \theta_j$ 
4:      $\pi \leftarrow \text{PERMUTATIONCOORDINATEDDESCENT}(\theta', \theta_i)$ 
5:      $\theta_i \leftarrow \pi(\theta_i)$ 
6:   end for
7: until convergence
8: return  $\frac{1}{N} \sum_{j=1}^N \theta_j$ 

```

A.1.4 Convergence and efficiency

We report here the convergence of our matching algorithm. In particular, fig. A.1 shows the objective values during the optimization, exhibiting the expected monotonic increase, while fig. A.2 shows the step sizes result of the line search at each iteration. Interestingly, fig. A.2a shows that the step sizes are generally decreasing, but descend in an alternating manner. This is likely due to the fact that the permutations are obtained as consecutive interpolations, where even steps result in a soft permutation matrix that is the average of the current and next permutation matrix, while odd steps generally result in a hard permutation matrix with entries in $[0, 1]$. Figure A.3 finally shows the intermediate permutation values during the optimization: at each step, the entries of the permutation matrix are the linear interpolation of the current solution and the projected gradient with factor α given by the step size. The red values in the figure represent entries currently being updated, which are neither 1 (blue) nor 0 (yellow).

We report in section A.1.4 the wall-clock time when merging $n = 2, 3$ RESNET20 models having $1\times, 2\times, 4\times, 8\times$ and $16\times$ width, together with their number of parameters.

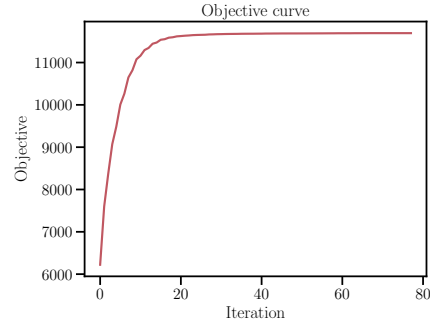


FIGURE A.1: Objective values during the optimization. As guaranteed by the Frank-Wolfe algorithm, the objective value increases monotonically.

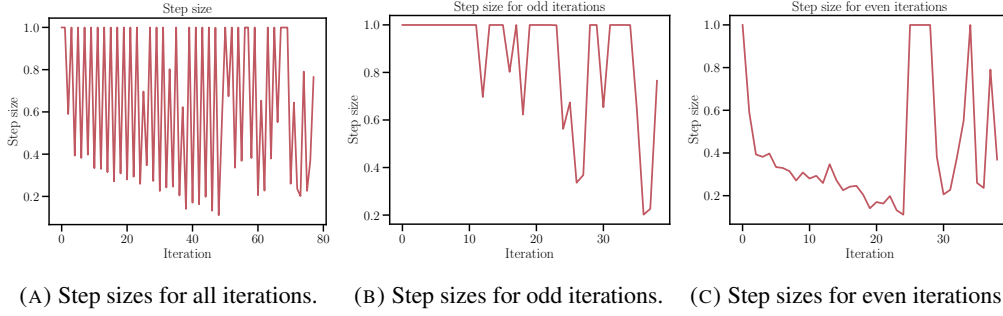


FIGURE A.2: Step sizes during the optimization.

	1x	2x	4x	8x	16x
# params	292k	1.166m	4.655m	18.600m	74.360m
n=2					
C^2M^3	33.4s	33.5s	40.5s	80.8s	367.8s
MergeMany	0.24s	0.4s	3.4s	8.9s	59.4s
n=3					
C^2M^3 time	32.9s	83.18s	91.0s	162.0s	715.8s
MergeMany	1.2s	4.1s	19.5s	105.8s	892.3s

TABLE A.1: Wall-clock time for merging $n = 3$ ResNet20 models with different widths.

As can be inferred from the table, the scaling laws depend on the complexity of the resulting matching problem and cannot be predicted merely from the number of parameters, with a 4-fold increase in parameters resulting in no increase in runtime for the first three columns, a double increase in the second-last column and a 5-fold increase in the last. Compared to MergeMany, our approach enjoys a milder increase in running time when increasing the number of parameters. For simpler settings, however, MergeMany is significantly faster. Being the two approaches on the same order of magnitude and given the one-time nature of model merging, we believe this aspect to be of secondary importance, especially considering merging to be, in many cases, an alternative to training a model from scratch.

A.1.5 Architectural details

We report here the architectural details of all the architectures we have used in the experiments.

Multi-Layer Perceptrons We use a simple MLP mapping input to a 256-dimensional space followed by 3 hidden layers of 512, 512 and 256 units respectively, followed by an output layer mapping to the number of classes. We use *ReLU* activations for all layers except the output layer, where we use a *log_softmax* activation.

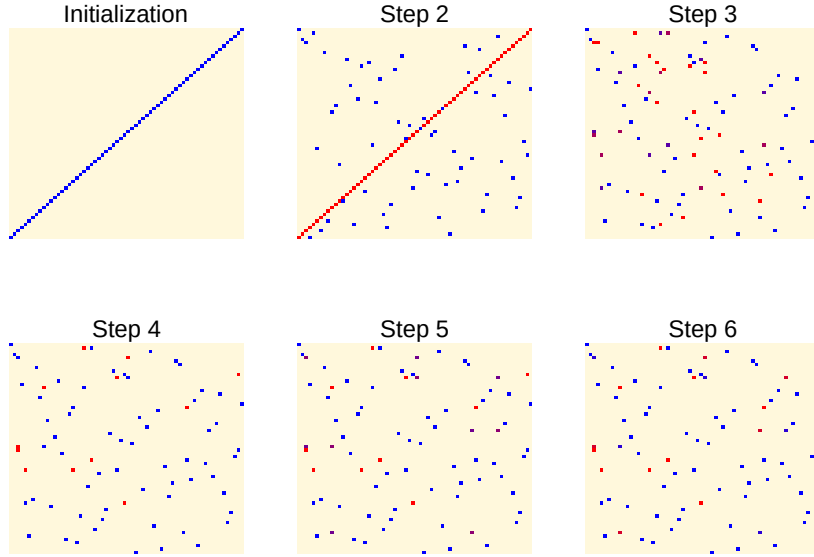


FIGURE A.3: First 6 steps of algorithm 7 for one permutation matrix. At each step, the new solution is given by the linear interpolation of the current solution and the gradient of eq. (2.2).

ResNet We consider a RESNET20 [67] architecture composed by three ResNet block groups, each containing three residual blocks. The model starts with an initial convolutional layer followed by normalization and *ReLU* activation. It then passes through the three block groups with increasing channel sizes (determined by the widen factor) and varying strides, followed by global average pooling and a fully connected layer that outputs class logits. As normalization layers, we consider both the most commonly used *BatchNorm* [79] and, for the sake of comparing with `Git Re-Basin`, also *LayerNorm* [8]. The results in the main manuscript are all obtained with *LayerNorm*, while we report the results with *BatchNorm* in section A.2.1.

VGG We employ a VGG16 [143] architecture with *LayerNorm* [8] normalization layers. The model has the following convolutional layer dimensions, with “M” indicating the presence of a max-pooling layer

$$64, 64, M, 128, 128, M, 256, 256, 256, M, 512, 512, 512, M, 512, 512, 512, M \quad (\text{A.2})$$

The convolutional layers are organized in 5 blocks, each containing 2 or 3 convolutional layers, followed by a max-pooling layer. The final classifier is composed of three fully connected layers with 512 hidden dimension and ReLU activations.

A.1.6 Datasets, hyperparameters and hardware details

We employ the most common datasets for image classification tasks: MNIST [42], CIFAR-10 [93], EMNIST [31] and CIFAR-100 [93], having 10, 10, 26 and 100 classes respectively. We use the standard train-test splits provided by *torchvision* for all datasets.

We use the same hyperparameters as `Git Re-Basin` where possible to ensure a fair comparison. In particular, we train most of our models with a batch size of 100 for 250 epochs, using SGD with momentum 0.9, a learning rate of 0.1, and a weight decay of 10^{-4} . We use a cosine annealing learning rate scheduler with a warm restart period of 10 epochs and a minimum learning rate of 0. We report each and every one of the hyperparameters used

for each experiment, as well as all the trained models, in the project’s WandB dashboard (available in the released codebase).

All of the experiments were carried out using consumer hardware, in particular mostly on a 32GiB RAM machine with a *12th Gen Intel(R) Core(TM) i7-12700F* processor and an *Nvidia RTX 3090* GPU, except for some of the experiments that were carried out on a 2080. Our modular and reusable codebase is based on *PyTorch*, leveraging *PyTorch Lightning* to ensure reproducible results and modularity and *NN-Template*¹ to easily bootstrap the project and enforce best practices.

A.1.7 Proofs

Theorem A.1.1. *The gradient of the objective function*

$$\sum_{p=1}^{n-1} \sum_{q=p+1}^n \sum_{\ell=1}^L \langle (P_\ell^p)^\top W_\ell^p P_{\ell-1}^p, (P_\ell^q)^\top W_\ell^q P_{\ell-1}^q \rangle$$

is Lipschitz continuous, implying our algorithm obtains a stationary point at a rate of $\mathcal{O}(1/\sqrt{t})$ [94].

Proof. We recall that, for each layer permutation $P^A = \{P_1^A, P_2^A, \dots, P_L^A\}$ of model A , we can define the gradient of our objective function relatively to the model B we are matching towards:

$$\begin{aligned} f(P_\ell^A) &= \nabla_{P_\ell^A}^{\text{rows}} + \nabla_{P_\ell^A}^{\text{cols}} + \nabla_{P_\ell^A}^{\text{rows}, \leftrightarrow} + \nabla_{P_\ell^A}^{\text{cols}, \leftrightarrow} = \\ &\quad \left[W_\ell^A P_{\ell-1}^A (P_{\ell-1}^B)^\top (W_\ell^B)^\top + (W_{\ell+1}^A)^\top P_{\ell+1}^A (P_{\ell+1}^B)^\top W_{\ell+1}^B \right] P_\ell^B + \\ &\quad \left[W_\ell^B P_{\ell-1}^B (P_{\ell-1}^A)^\top (W_\ell^A)^\top + (W_{\ell+1}^B)^\top P_{\ell+1}^B (P_{\ell+1}^A)^\top W_{\ell+1}^A \right] P_\ell^A \end{aligned}$$

To prove Lipschitz continuity, we need to show there exists a constant C such that $\forall p = 1, \dots, n, \ell = 1, \dots, L \quad \|f(P_\ell^p) - f(Q_\ell^p)\| \leq C \|P_\ell^p - Q_\ell^p\|$. To simplify passages, we only consider a fixed ℓ and perform a generic analysis. We begin by observing that

$$\begin{aligned} f(P_\ell^p) - f(Q_\ell^p) &= \\ &\sum_{q \in [1, n] \setminus \{p\}} \left[W_\ell^p P_{\ell-1}^p (P_{\ell-1}^q)^\top (W_\ell^q)^\top + (W_{\ell+1}^p)^\top P_{\ell+1}^p (P_{\ell+1}^q)^\top W_{\ell+1}^q \right] (P_\ell^q - Q_\ell^q) + \\ &\quad \left[W_\ell^q P_{\ell-1}^q (P_{\ell-1}^p)^\top (W_\ell^p)^\top + (W_{\ell+1}^q)^\top P_{\ell+1}^q (P_{\ell+1}^p)^\top W_{\ell+1}^p \right] (P_\ell^p - Q_\ell^p) \end{aligned}$$

The last form of the above equation can be rewritten as a sum of the two sums:

$$\begin{aligned} &\sum_{q \in [1, n] \setminus \{p\}} \left[W_\ell^p P_{\ell-1}^p (P_{\ell-1}^q)^\top (W_\ell^q)^\top + (W_{\ell+1}^p)^\top P_{\ell+1}^p (P_{\ell+1}^q)^\top W_{\ell+1}^q \right] (P_\ell^q - Q_\ell^q) + \\ &\sum_{q \in [1, n] \setminus \{p\}} \left[W_\ell^q P_{\ell-1}^q (P_{\ell-1}^p)^\top (W_\ell^p)^\top + (W_{\ell+1}^q)^\top P_{\ell+1}^q (P_{\ell+1}^p)^\top W_{\ell+1}^p \right] (P_\ell^p - Q_\ell^p) \end{aligned}$$

Since the first term does not depend on either P_ℓ^p or Q_ℓ^p , we assume as a worst case that its norm is 0. Then, we remove transposes (since $\|M\| = \|M^\top\|$) and apply the triangle inequality and the sub-multiplicative property of matrix norms:

$$\begin{aligned} \|f(P_\ell^p) - f(Q_\ell^p)\| &\leq \\ &\sum_{q \in [1, n] \setminus \{p\}} \|P_\ell^p - Q_\ell^p\| (\|W_\ell^q\| \|P_{\ell-1}^q\| \|P_{\ell-1}^p\| \|W_\ell^p\| + \|W_{\ell+1}^q\| \|P_{\ell+1}^q\| \|P_{\ell+1}^p\| \|W_{\ell+1}^p\|) \end{aligned}$$

¹<https://github.com/grok-ai/nn-template>

Matcher	Barrier									
	ResNet _{2x}		ResNet _{4x}		ResNet _{8x}		ResNet _{16x}		VGG16	
	Train	Test	Train	Test	Train	Test	Train	Test	Train	Test
Naive	5.16 ± 1.83	5.45 ± 1.83	2.94 ± 0.27	3.26 ± 0.27	2.12 ± 0.03	2.40 ± 0.03	1.84 ± 0.18	2.12 ± 0.17	1.85 ± 0.00	2.31 ± 0.00
Git Re-Basin	0.73 ± 0.16	0.86 ± 0.17	0.74 ± 0.35	0.80 ± 0.40	0.19 ± 0.03	0.13 ± 0.02	0.17 ± 0.02	0.07 ± 0.02	0.08 ± 0.03	0.24 ± 0.03
Frank-Wolfe	0.73 ± 0.19	0.85 ± 0.19	0.78 ± 0.33	0.81 ± 0.38	0.19 ± 0.03	0.12 ± 0.02	0.16 ± 0.02	0.06 ± 0.02	0.08 ± 0.03	0.25 ± 0.03

TABLE A.3: Mean and standard deviation of the test and train loss barrier for each method when matching $n = 2$ models on CIFAR10.

Let $C = \max_{q \in [1, n] \setminus \{p\}} \{ \|W_\ell^q\| \|P_{\ell-1}^q\| \|P_{\ell-1}^p\| \|W_\ell^p\| + \|W_{\ell+1}^q\| \|P_{\ell+1}^q\| \|P_{\ell+1}^p\| \|W_{\ell+1}^p\| \}$. Then,

$$\|f(P_\ell^p) - f(Q_\ell^p)\| \leq C \sum_{q \in [1, n] \setminus \{p\}} \|P_\ell^p - Q_\ell^p\| = C(n-1) \|P_\ell^p - Q_\ell^p\|$$

we conclude that $f(P_\ell^p)$ is Lipschitz continuous for all models and all layers, with Lipschitz constant $C(n-1)$ depending on both the norm of the weights matrices and the number of models. $\square$

A.2 Additional experiments

We report additional experiments and results in this section. In particular, section A.2.1 presents a complete evaluation of our matching method for the pairwise case, showing it to be generally competitive with the state-of-the-art Git Re-Basin algorithm [2] and to outperform it on architectures employing *BatchNorm* [79] normalization. We then discuss different permutation initialization strategies in section A.2.2.

A.2.1 Pairwise model matching and merging

Matcher		Barrier	
		Train	Test
ResNet _{8x}	Naive	7.00 ± 1.24	8.37 ± 1.23
	Git-Rebasin	1.04 ± 0.10	1.54 ± 0.13
	Frank-Wolfe	0.92 ± 0.06	1.42 ± 0.10
VGG16	Naive	5.79 ± 0.39	7.36 ± 0.38
	Git-Rebasin	0.44 ± 0.03	0.64 ± 0.03
	Frank-Wolfe	0.44 ± 0.05	0.63 ± 0.06

TABLE A.2: Mean and standard deviation of the test and train loss barriers for each method when matching $n = 2$ models on CIFAR100.

As described in section 3.2, our formalization can readily be used to match $n = 2$ models. In this case, the energy is given by eq. (2.2) and the permutations are not factorized. We compare the performance of our approach against the Git Re-Basin algorithm [2] and the naive baseline that aggregates the models by taking an unweighted mean on the original model weights without applying any permutation. From the data presented in table A.2, we observe that the approach is competitive with Git Re-Basin [2], with the two methods exhibiting analogously low test barrier on CIFAR10. Focusing on the ResNet20 architecture, we can see that width plays the same role in both approaches, with the barrier decreasing as it increases. We can also appreciate how, while the same architecture resulted in similar barriers for the two approaches on CIFAR10, the barrier is significantly

lower for Frank-Wolfe in CIFAR100, possibly suggesting that the latter is more robust to the complexity of the dataset.

ResNet with BatchNorm

We also report the results of a ResNet20 with $2\times$ width using *BatchNorm* [79] layers instead of *LayerNorm* [8] ones. This version, as noted in [85], is in fact harder to match but also the one that is commonly used in practice. We can see in table A.4 that the Frank-Wolfe matcher is able to achieve a lower barrier than Git Re-Basin, indicating the approach to be more robust to architectures using different normalization layers.

A.2.2 Initialization strategies

As introduced in algorithm 1, we initialize each N -dimensional permutation to be the $N \times N$ identity matrix. We now compare this strategy against two alternatives that provide doubly stochastic

models	loss barrier ($\downarrow$)		
	id	barycenter	Sinkhorn
(a, b)	0.52	0.47	0.60 ± 0.04
(b, c)	0.65	0.70	0.64 ± 0.06
(a, c)	0.97	0.95	0.92 ± 0.07

TABLE A.5: Test barrier of the interpolations of 3 ResNet20- $2\times$ models using different initializations.

accuracy points.

matrices, *i.e.*, such that their rows and columns sum to one: i) the Sinkhorn initialization [145] that initializes the permutation matrix as the solution of the Sinkhorn-Knopp algorithm [145]; ii) the barycenter of doubly stochastic matrices, *i.e.* the matrix where each element is given by $1/N$. Table A.5 shows the test barrier of the interpolations of three ResNet20- $2\times$ models a , b , and c when using the different strategies over 10 different trials. We can see that the constant initializations (identity and barycenter) work well in general, with the additional benefit of having 0 variance in the results. On the other hand, if computational cost is not a concern, one can still choose to run a pool of trials with different Sinkhorn initializations and finally select the best one, trading this way efficiency with some extra

A.2.3 Variance of the results in Git Re-Basin

As introduced in section 3.3, Git Re-Basin [2] depends on a random choice of layers, resulting in variations of up to 10% in accuracy depending on the optimization seed, while our method shows zero variance. While we have already seen the results for a model pair in fig. 3.4, we report, for completeness, the results of matching and averaging models with Git Re-Basin using different optimization seeds for additional pairs. As can be seen in table A.6, the trend is confirmed over these ones, with results significantly oscillating and our approach always above or on par with their mean.

A.2.4 Large-scale matching: ResNet50s trained over ImageNet

For this experiment, we matched three different ResNet50s trained over ImageNet. We used three publicly available pre-trained checkpoints from *timm*, namely `a1_in1k`², `c1_in1k`³ and `ram_in1k`⁴. As table A.7 shows, C^2M^3 underperforms the baseline in this case. To see why, we report in fig. A.4 the pairwise accuracies obtained using pairwise weight matching over all the ResNet50 checkpoints available in *timm*.

²https://huggingface.co/timm/resnet50.a1_in1k

³https://huggingface.co/timm/resnet50.c1_in1k

⁴https://huggingface.co/timm/resnet50.ram_in1k

Matcher	loss barrier ($\downarrow$)	
	train	test
Naive	4.72 ± 0.86	4.99 ± 0.86
Git Re-Basin	4.33 ± 0.64	4.62 ± 0.65
Frank-Wolfe	3.53 ± 0.58	3.79 ± 0.57

TABLE A.4: Mean and stddev of the test and train loss barriers on 2 ResNet20- $2\times$ models with *BatchNorm* normalization.

models		1	2	3	4	5	6	7	8	9	mean	stddev	max gap	Frank-Wolfe
(1,2)	train	0.76	0.78	0.78	0.80	0.77	0.76	0.78	0.75	0.81	0.78	0.018	0.057	0.78
	test	0.73	0.75	0.75	0.77	0.74	0.73	0.74	0.72	0.78	0.74	0.018	0.060	0.75
(1,3)	train	0.67	0.69	0.69	0.69	0.62	0.69	0.66	0.71	0.68	0.68	0.023	0.085	0.68
	test	0.64	0.66	0.67	0.65	0.60	0.66	0.63	0.67	0.65	0.65	0.020	0.071	0.65
(2,3)	train	0.75	0.74	0.75	0.72	0.76	0.74	0.70	0.73	0.78	0.74	0.020	0.074	0.76
	test	0.70	0.71	0.71	0.68	0.72	0.70	0.67	0.70	0.74	0.70	0.020	0.071	0.72

TABLE A.6: Accuracy of the interpolated model using Git Re-Basin [2] over different pairs of models (1, 2), (1, 3), (2, 3) by changing random seed $i = 1, \dots, 9$ in the weight matching procedure.

Let us focus on the triplet (am, a2, ram) and replace the model names with (a, b, c) for clarity. We see that, while the mergings (a, b) and (b, c) result in high-accuracy models, the merging (a, c) yields poor results. Given the cycle consistency of our method, we inherit the difficulty of the hardest pair, which in this case is (a, c). It is worth noting that this behavior is not present in the other cases we investigated in this thesis, and might be due to the considered models being trained with different training schedules and hyperparameters. Future research could investigate new strategies to handle such cases, *e.g.* by iteratively merging models by following a max-accuracy path in an accuracy weighted graph.

Matcher	Accuracy ($\uparrow$)	Loss ($\downarrow$)
Naive	0.001	6.91
MergeMany	0.001	6.91
MergeMany [†]	0.30	4.87
C^2M^3	0.001	6.91
$C^2M^{3†}$	0.07	6.13

TABLE A.7: Accuracy and loss of the interpolated model using different matchers over three ResNet50 models trained on ImageNet.

A.2.5 Federated learning

We here report the results of a preliminary experiment where we ran our framework in a federated learning scenario. To this end, we have used the state-of-the-art federated learning library Flower⁵ [16] and employed our matching scheme over a set of 10 clients over CIFAR10, each adopting a small CNN model. We observe the following:

- When all the clients start from the same initialization, our approach has no benefit and falls back to standard averaging. In fact, the optimization process quickly returns identity matrices as permutations, suggesting the models already share the same basin.
- When instead we initialize the clients from different random initializations, tables A.8 and A.9 show that our approach visibly outperforms FedAVG. In particular, the benefits get more pronounced when increasing the number of local epochs. This is in line with the intuition that standard averaging becomes less effective when clients drift due to prolonged local training and too infrequent aggregation.

While these results are not sufficient to claim an overall supremacy of the approach for the task due to the limited evaluation and choice of models, they show the approach to be promising for the problem and encourage further research.

⁵<https://flower.ai/>



FIGURE A.4: Pairwise accuracies obtained using Git Re-Basin [2] over different ResNet50 models trained on ImageNet. Models are available from the *timm* library.

Round	Accuracy									
	1	5	10	15	20	25	30	35	40	45
FedAvg	0.0942	0.394	0.4972	0.5517	0.5699	0.5893	0.6018	0.6063	0.6099	0.6136
C^2M^3	0.0941	0.4234	0.5193	0.5555	0.5783	0.5978	0.6077	0.6165	0.618	0.622

TABLE A.8: Accuracy over 10 clients in a federated learning scenario. We report the accuracy for 50 aggregation rounds, with each client training for 20 local epochs. We report one every five rounds for the sake of clarity.

Round	Accuracy									
	1	2	3	4	5	6	7	8	9	10
FedAvg	0.0942	0.2638	0.3543	0.3825	0.4165	0.4505	0.4742	0.4994	0.5169	0.5317
C^2M^3	0.0947	0.3303	0.3899	0.4441	0.4764	0.4968	0.5184	0.5334	0.5434	0.5536

TABLE A.9: Accuracy over 10 clients in a federated learning scenario. We report the accuracy for 10 aggregation rounds, with each client training for 30 local epochs.

A.3 Additional analysis

In this section, we report additional analyses that complement the results presented in the main text. We first analyze in section A.3.1 how mapping to universe affects the similarity of the models; then, we evaluate how the composition of the match set affects the accuracy of the merged model in section A.3.2.

A.3.1 Similarity of models

We analyze here how similar are models before and after being mapped to the universe space, first by comparing their representations and then by comparing their weights.

Representation-level similarity

Figures A.5a and A.5b show the Centered Kernel Alignment (CKA) [91] of the representations of 5 RESNET20 models trained on CIFAR10 with $2\times$ width. The linear version of CKA is defined as

$$\text{CKA}(X, Y) = \frac{\text{HSIC}(X, Y)}{\sqrt{\text{HSIC}(X, X) \text{HSIC}(Y, Y)}}, \quad (\text{A.3})$$

where $\text{HSIC}(X, Y) = \frac{1}{(N-1)^2} \text{tr}(\mathbf{X}\mathbf{H}\mathbf{X}^\top \mathbf{H})$, $\mathbf{H} = \mathbf{I} - \frac{1}{N}\mathbf{1}\mathbf{1}^\top$ is a centering matrix, and $\mathbf{1}$ is a vector of N ones. The denominator is introduced to scale CKA between zero and one, where a value of one indicates equivalent representations. CKA is invariant to orthogonal transformations and isotropic scaling. Being permutations orthogonal transformations, CKA stays exactly the same after mapping the models to the universe. On the contrary, the Euclidean distance of the representations of the models significantly decreases after mapping to the universe, as shown in figs. A.5c and A.5d.

Weight-level similarity

We have seen in fig. 3.5 that the cosine similarity of the weights is higher after mapping the weights to the universe. This suggests that the models are more similar in the universe, which is consistent with the fact that it constitutes a convenient space to merge them. We report here for completeness in fig. A.6 the Euclidean distance of the weights of 5 RESNET20 models trained on CIFAR10 with $2\times$ width, showing the same trend as the cosine similarity.

A.3.2 Merging different subsets

We merge subsets of $k < 5$ models from the set of 5 models a, b, c, d, e to gauge the effect of the match set composition over the accuracy of the merged model. As shown in fig. A.7, we run two different merging schemes: in the former (left column), we globally match all the 5 models jointly and then consider subsets only at the aggregation step. In the second analysis (right column), we instead consider model subsets from the start, therefore running the whole matching procedure on the k models before averaging them. This way, we aim to disentangle the error resulting from imperfect matching from the one naturally resulting from the aggregation. We highlight a few notable aspects:

1. While the accuracies are expectedly higher when matching a subset with permutations expressly optimized for that same subset (right column), this is not the case for $n = 2$, in which the permutations resulting from matching the superset of 5 models yield better results when merging pairs of them. This hints at the added constraint of cycle consistency over a wide number of models adding in some cases an advisable prior over the search space.

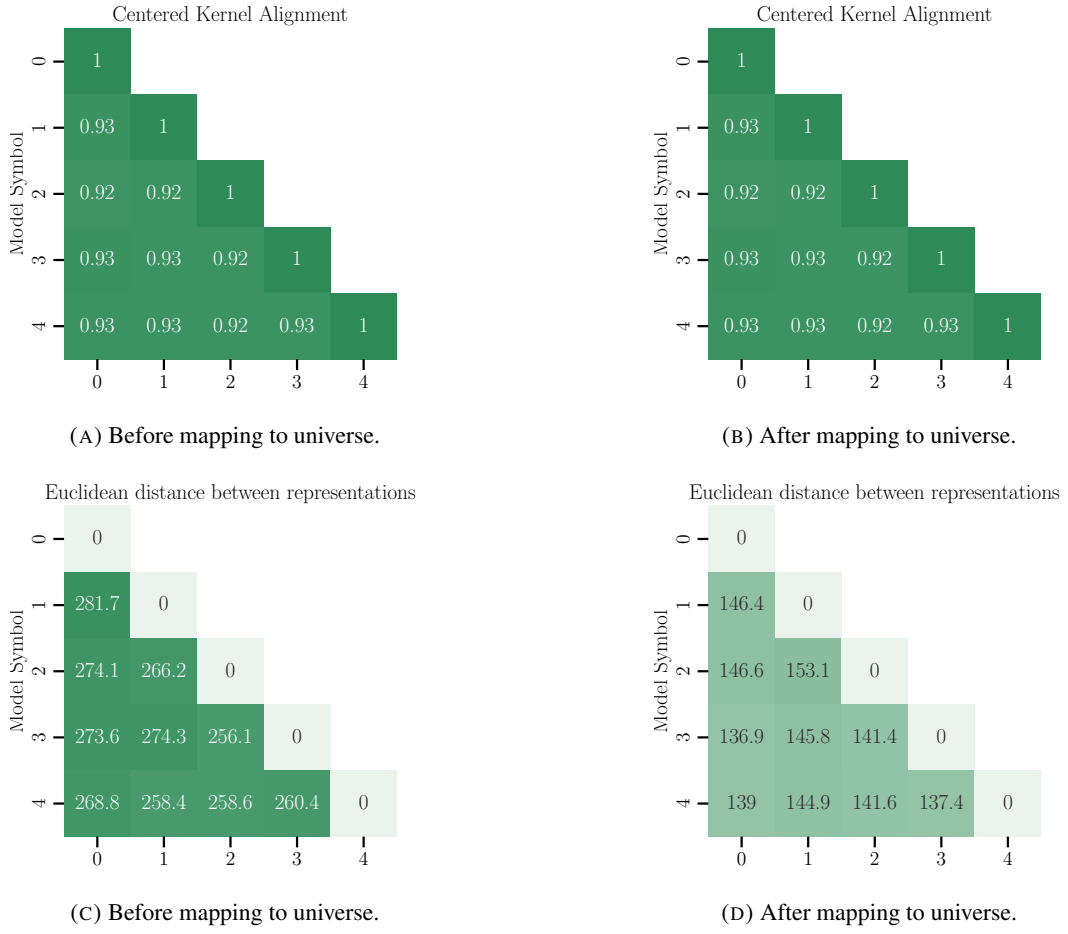


FIGURE A.5: Centered Kernel Alignment and Euclidean distances of the representations of 5 RESNET20 trained on CIFAR10 with $2\times$ width.

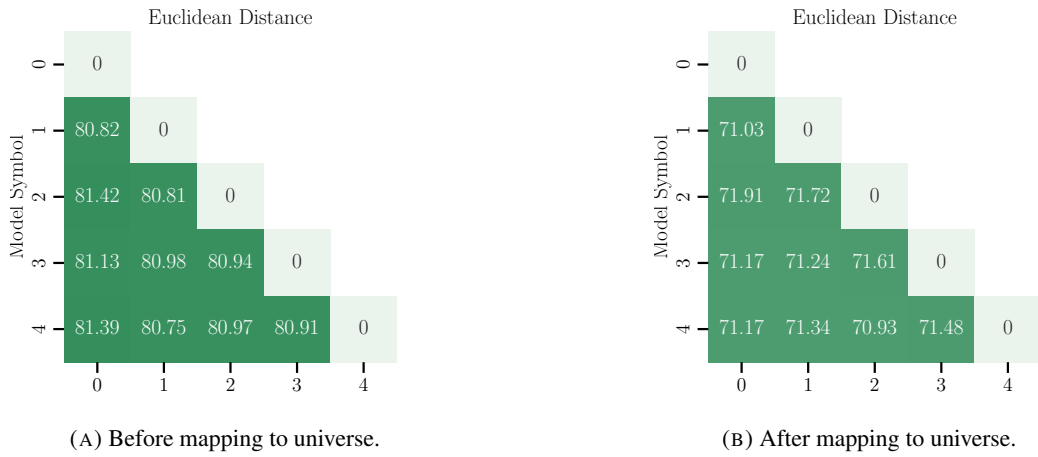


FIGURE A.6: Euclidean distance of the weights of 5 RESNET20 trained on CIFAR10 with $2\times$ width.

2. The particular composition of the match set has a significant impact over the matching and subsequent merge operation, yielding differences of up to ≈ 20 accuracy points for the downstream model.
3. The standard deviations before the repair operation (red semi-transparent bars in the plots) are way lower when optimizing for the permutations over the superset of all 5 models; this suggests that the matching difficulty is spread over all the maps jointly, eventually yielding more stable results.

A.4 Discussion

We discuss in this section the limitations of our work, as well as potential future societal impact.

A.4.1 On the cycle-consistency of C^2M^3

Our method is natively cycle-consistent due to the mathematical formulation of the optimization problem. If we were to not desire cycle consistency, the matching method would fall back to the $n = 2$ Frank-Wolfe (FW) case presented in section 3.2. One would then have to transform the pairwise matching problem to a n -way matching problem, *e.g.* by using the $n = 2$ FW procedure as matching step in the MergeMany [2] algorithm. Results for the $n = 2$ FW matching are reported in table A.2.

A.4.2 Limitations

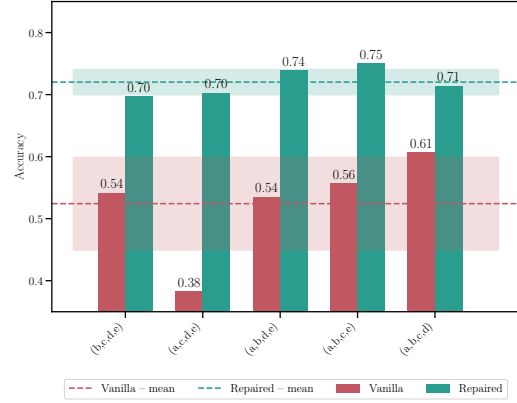
From what we have observed in our experiments, permutations satisfying linear mode connectivity of the models are hard to find for most architectures and datasets. In fact, given that there is no practical way to prove or disprove the conjecture for which most models end up in the same basin modulo permutations of the neurons, we cannot be sure that a certain set of models even allows finding such permutations, let alone that the permutations found are the optimal ones. We therefore encourage the community not to rely on the existence of such permutations in general. However, we have also shown that we can always find permutations that improve the resulting aggregated model, which is a promising practical result for model merging. As for all the existing works concerning linear mode connectivity and model merging, the resulting models that we obtain are sensible to a wide variety of factors, from training hyperparameters to the optimization algorithm used. Several works have already observed the phenomenon in practice: among these, Ainsworth et al. [2] mention among the known failure modes of their approaches models trained with SGD and too low learning rate, or ADAM coupled with too high learning rate. Jordan et al. [85] show that the chosen normalization layer incredibly affects the accuracy of the resulting merged model, while Qu and Horvath [131] observe learning rate, weight decay, and initialization method to play a strong role as well. Being a mostly empirical field, most of the technical choices that we make in our work mirror the ones made in previous works and are not based on a solid theoretical foundation. We therefore release all our code and encourage the community to investigate further on what training and optimization hyperparameters affect linear mode connectivity and model merging.

A.4.3 Societal impact and broader vision

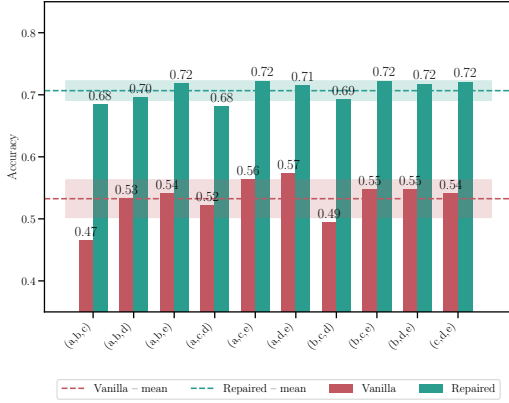
The work presented in this chapter serves as an additional tool for the community to improve the efficiency of deep learning models. By merging models, we can reduce the computational



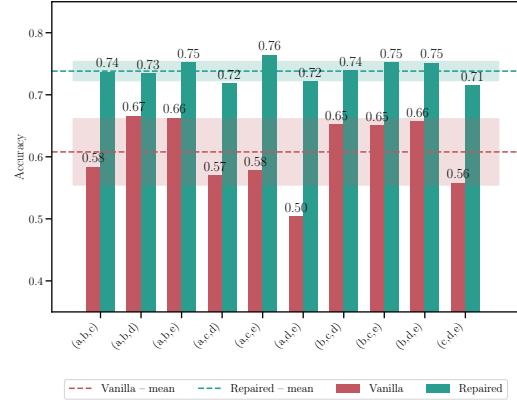
(A) Subsets of 4 out of 5 jointly matched models.



(B) Subsets of 4 matched models out of 5 models.



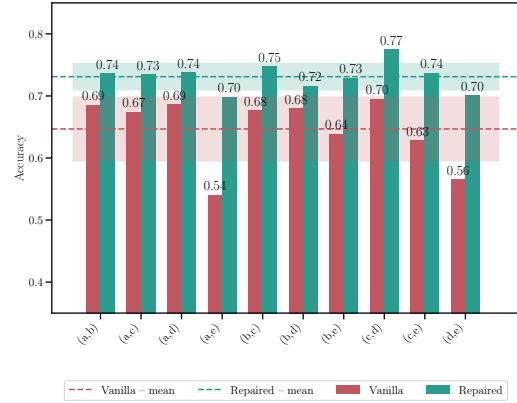
(C) Subsets of 3 out of 5 jointly matched models.



(D) Subsets of 3 matched models out of 5 models.



(E) Subsets of 2 out of 5 jointly matched models.



(F) Subsets of 2 matched models out of 5 models.

FIGURE A.7: Accuracy of the resulting model when merging different model subsets. **(left)** performance of models obtained from aggregating subsets of $k < 5$ models that were matched jointly. **(right)** analogous results for subsets of k models that are instead matched independently, *i.e.*, by only optimizing for the permutations that align those k models and discarding the remaining ones. The semi-transparent bands represent the standard deviation of the accuracy.

cost of training and inference, as well as the memory footprint of the models. In fact, by aggregating the information of a set of models into a single one with the same architecture, practitioners can benefit from the effects of ensembling without incurring its computational cost. Moreover, merging is in many cases a practical necessity to guarantee confidentiality and privacy of user data, as it allows to train models on different subsets of the data, *e.g.* originating from different clients, and then merge them to obtain a single model integrating all the information. This is particularly important in the context of federated learning, where the data is distributed among different clients and cannot be shared. We believe that the work presented in this chapter can be a stepping stone towards more efficient and privacy-preserving deep learning models, and we encourage the community to further investigate the potential of model merging in these contexts.

Appendix B

Additional material on Chapter 5

B.1 Proofs

B.1.1 Proofs of Proposition 5.2.3 and Theorem 5.2.1

In this section, we provide proofs for Proposition 5.2.3 and Theorem 5.2.1. For clarity, we restate both the proposition and theorem. In the Appendix, when summing the individual task losses over all tasks, we sometimes index them by their label $t \in T$ and other times list them as $i = 1, \dots, T$. Both notations denote the exact same aggregate over every task.

Proposition. Let $\{\theta_t^{(k)}\}_{t=1}^{|T|}$ be a set of models obtained by fine-tuning the base model θ_{pre} for k epochs on tasks T using GD with a learning rate η , where fine-tuning task $t \in T$ minimizes the loss $\bar{L}_t(\theta) = \frac{1}{n_t} \sum_{i=1}^{n_t} \ell(x_i, y_i, \theta)$. Additionally, let $\{\tau_t^{(k)}\}_{t=1}^{|T|}$ denote the corresponding set of task vectors, with each $\tau_t^{(k)} = \theta_t^{(k)} - \theta_{pre}$. Let $\tau_{MT}^{(k)}$ be the multi-task vector $\tau_{MT}^{(k)} = \sum_{t \in T} \tau_t^{(k)}$. Finally, let $\theta_{MT}^{(k)}$ represent the model obtained by minimizing the combined loss $\sum_{i=1}^{|T|} \bar{L}_i$ for k epochs using GD with a learning rate of $\alpha\eta$. It holds that

$$\tau_{MT}^{(1)} = -\eta \nabla \sum_{t \in T} \bar{L}_t(\theta_{pre}) \quad (\text{B.1})$$

$$\tau_{MT}^{(k)} = -\eta \sum_{t \in T} \sum_{j=0}^{k-1} \nabla \bar{L}_t(\theta_{MT}^{(j)}) + \eta^2 C(\{\theta_{MT}^{(j)}\}_{j=1}^{k-2}) + O(\eta^3) \quad (\text{B.2})$$

with

$$C(\{\theta_{MT}^{(j)}\}_{j=1}^h) = \sum_{t \in T} \sum_{\ell=0}^h \nabla^2 \bar{L}_t(\theta_{MT}^{(\ell)}) \sum_{m=0}^{\ell} \left[\alpha \sum_{t' \neq t, t' \in T} \nabla \bar{L}_{t'}(\theta_{MT}^{(m)}) + (\alpha - 1) \nabla \bar{L}_t(\theta_{MT}^{(m)}) \right] \quad (\text{B.3})$$

Theorem. Let $\theta_{TA}^{(k)} = \theta_{pre} + \alpha \sum_{t=1}^T \tau_t^{(k)}$ be the model obtained using vanilla task arithmetic. Using the same notation of Theorem 5.2.3, it holds that

$$\theta_{TA}^{(1)} = \theta_{MT}^{(1)} \quad (\text{B.4})$$

$$\theta_{TA}^{(k)} = \theta_{MT}^{(k)} + \eta^2 C(\{\theta_{MT}^{(j)}\}_{j=1}^{k-2}) + O(\eta^3) \quad \text{for } k > 1 \quad (\text{B.5})$$

We recall that $\theta_i^{(k)}$ is the model obtained by fine-tuning on task i for k epochs, and that both the fine-tuning on different tasks and the training on the average loss start from a pre-trained model θ_{pre} .

To prove the statement of the theorem and of the corollary, we need an intermediate result. We introduce the following notation:

$$r_i(\theta, \alpha) = \alpha \sum_{j \neq i} \nabla \bar{L}_j(\theta) + (\alpha - 1) \nabla \bar{L}_i(\theta) = \alpha \sum_{j=1}^{|T|} \nabla \bar{L}_j(\theta) - \nabla \bar{L}_i(\theta) \quad (\text{B.6})$$

$$p_i^k(\theta_{\text{pre}}, \theta_{\text{MT}}^{(1)}, \dots, \theta_{\text{MT}}^{(k)}) = \sum_{j=0}^k r_i(\theta_{\text{MT}}^{(j)}) \quad (\text{B.7})$$

$$s_t^k(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(k)}) := \sum_{j=0}^k \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(j+1)}) [p_i^j(\theta_{\text{pre}}, \theta_{\text{MT}}^{(1)}, \dots, \theta_{\text{MT}}^{(j)})]. \quad (\text{B.8})$$

Since α in this context is fixed, we will not emphasize the dependence on α and we will use the notation $r_i(\theta) = r_i(\theta, \alpha)$. We need the following preliminary Lemma.

Lemma. *Using the notation introduced in Proposition 5.2.3, it holds that*

$$\theta_i^{(1)} = \theta_{\text{MT}}^{(1)} + \eta p_i^0(\theta_{\text{pre}}) \quad (\text{B.9})$$

and for $m \geq 2$

$$\theta_i^{(m+1)} = \theta_{\text{MT}}^{(m+1)} + \eta p_i^m(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m)}) - \eta^2 s_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3) \quad (\text{B.10})$$

Proof. We first verify the statement directly for $m = 1$, then prove the general case $m \geq 2$ by induction. The induction base case is $m = 2$. The induction step then shows that if the statement holds for some m it must also hold for $m + 1$.

$m = 1$. First epoch For each task $i = 1, \dots, |T|$

$$\theta_i^{(1)} = \theta_{\text{pre}} - \eta \nabla \bar{L}_i(\theta_{\text{pre}}) \text{ while } \theta_{\text{MT}}^{(1)} = \theta_{\text{pre}} - \alpha \eta \sum_{i \in T} \nabla \bar{L}_i(\theta_{\text{pre}}).$$

Consequently, it holds that

$$\begin{aligned} \theta_i^1 &= \theta_{\text{MT}}^{(1)} + \eta \left[\alpha \sum_{j \neq i} \nabla \bar{L}_j(\theta_{\text{pre}}) + (\alpha - 1) \nabla \bar{L}_i(\theta_{\text{pre}}) \right] \\ &= \theta_{\text{MT}}^{(1)} + \eta r_i(\theta_{\text{pre}}) = \theta_{\text{MT}}^{(1)} + \eta p_i^0(\theta_{\text{pre}}). \end{aligned}$$

$m = 2$. Second epoch

$$\begin{aligned} \theta_i^{(2)} &= \theta_i^{(1)} - \eta \nabla \bar{L}_i(\theta_i^{(1)}) \\ &= \theta_{\text{MT}}^{(1)} + \eta r_i(\theta_{\text{pre}}) - \eta \nabla \bar{L}_i(\theta_{\text{MT}}^{(1)} + \eta r_i(\theta_{\text{pre}})) \\ &\stackrel{\text{Taylor}}{\approx} \theta_{\text{MT}}^{(1)} + \eta r_i(\theta_{\text{pre}}) - \eta \nabla \bar{L}_i(\theta_{\text{MT}}^{(1)}) - \frac{\eta^2}{2} \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(1)}) r_i(\theta_{\text{pre}}) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(1)} - \eta \nabla \bar{L}_i(\theta_{\text{MT}}^{(1)}) + \eta r_i(\theta_{\text{pre}}) - \frac{\eta^2}{2} \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(1)}) r_i(\theta_{\text{pre}}) + O(\eta^3) \end{aligned}$$

Adding and subtracting $\eta \alpha \sum_{t \in T} \nabla \bar{L}_i(\theta_{\text{MT}}^{(1)})$

$$\begin{aligned} \theta_i^{(2)} &= \theta_{\text{MT}}^{(1)} - \underbrace{\eta \alpha \sum_{t \in T} \nabla \bar{L}_t(\theta_{\text{MT}}^{(1)})}_{= \theta_{\text{MT}}^{(2)}} + \underbrace{\eta \alpha \sum_{t \in T} \nabla \bar{L}_i(\theta_{\text{MT}}^{(1)}) - \eta \nabla \bar{L}_i(\theta_{\text{MT}}^{(1)}) + \eta r_i(\theta_{\text{pre}})}_{\eta r_i(\theta_{\text{MT}}^{(1)})} \\ &\quad - \frac{\eta^2}{2} \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(1)}) r_i(\theta_{\text{pre}}) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(2)} + \eta r_i(\theta_{\text{MT}}^{(1)}) + \eta r_i(\theta_{\text{pre}}) - \frac{\eta^2}{2} \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(1)}) r_i(\theta_{\text{pre}}) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(2)} + \eta p_i^1(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(1)}) - \eta^2 s_i^0(\theta_{\text{pre}}) + O(\eta^3) \end{aligned}$$

Inductive step Let us assume that

$$\theta_i^{(m)} = \theta_{\text{MT}}^{(m)} + \eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) + O(\eta^3)$$

We can derive that

$$\begin{aligned} \theta_i^{(m+1)} &= \theta_i^{(m)} - \eta \nabla \bar{L}_i(\theta_i^{(m)}) \\ &= \theta_{\text{MT}}^{(m)} + \eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) - \eta \nabla \bar{L}_i(\theta_i^{(m)}) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(m)} + \eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) \\ &\quad - \eta \nabla \bar{L}_i \left(\theta_{\text{MT}}^{(m)} + \eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) \right) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(m)} + \eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) \\ &\quad - \eta \nabla \bar{L}_i(\theta_{\text{MT}}^{(m)}) - \frac{\eta}{2} \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(m)}) \left(\eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) \right) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(m)} + \eta p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) - \eta^2 s_i^{m-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-2)}) \\ &\quad - \eta \nabla \bar{L}_i(\theta_{\text{MT}}^{(m)}) - \eta^2 \nabla^2 \bar{L}_i(\theta_{\text{MT}}^{(m)}) p_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(m+1)} + \eta p_i^m(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m)}) - \eta^2 s_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3) \end{aligned}$$

□

Proof Proposition and Theorem. For the first epoch

$$\theta_{\text{TA}}^{(1)} = \theta_{\text{pre}} + \alpha \sum_{i \in T} \tau_i^{(1)} = \theta_{\text{pre}} - \eta \alpha \sum_{i \in T} \nabla \bar{L}_i(\theta_{\text{pre}})$$

while, choosing $\alpha \eta$ as learning rate for the loss $\sum_{i \in T} \bar{L}_i$:

$$\theta_{\text{MT}}^{(1)} = \theta_{\text{pre}} - \alpha \eta \sum_{i \in T} \nabla \bar{L}_i(\theta_{\text{pre}}).$$

So $\theta_{\text{MT}}^{(1)} = \theta_{\text{TA}}^{(1)}$.

For $k \geq 2$, notice that

$$\theta_{\text{MT}}^{(k)} = \theta_{\text{pre}} - \alpha \eta \sum_{j=0}^{k-1} \nabla \sum_{t \in T} \bar{L}_t(\theta_{\text{MT}}^{(j)}). \quad (\text{B.11})$$

Now, using Lemma 5.2.5, we get:

$$\begin{aligned} & -\alpha \eta \sum_{j=0}^{k-1} \nabla \sum_{t \in T} \bar{L}_t(\theta_{\text{MT}}^{(j)}) + \eta p_i^{k-1}(\eta^0, \dots, \eta_{\text{MT}}^{k-1}) \\ &= -\alpha \eta \sum_{j=0}^{k-1} \nabla \sum_{t \in T} \bar{L}_t(\theta_{\text{MT}}^{(j)}) + \sum_{j=0}^{k-1} r_i(\theta_{\text{MT}}^{(j)}) \\ &= -\alpha \eta \sum_{j=0}^{k-1} \nabla \sum_{t \in T} \bar{L}_i(\theta_{\text{MT}}^{(j)}) + \sum_{j=0}^{k-1} \alpha \sum_{j \in T} \nabla \bar{L}_j(\theta_{\text{MT}}^{(j)}) - \nabla \bar{L}_i(\theta_{\text{MT}}^{(j)}) \\ &= -\eta \sum_{j=0}^{k-1} \nabla \bar{L}_i(\theta_{\text{MT}}^{(j)}). \end{aligned}$$

Namely:

$$\begin{aligned} \theta_i^{(m+1)} &= \theta_{\text{MT}}^{(m+1)} + \eta p_i^m(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m)}) - \eta^2 s_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3) \\ &= \theta_{\text{pre}} - \alpha \eta \sum_{j=0}^m \nabla \sum_{t \in T} \bar{L}_i(\theta_{\text{MT}}^{(j)}) + \eta p_i^m(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m)}) - \eta^2 s_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3) \\ &= \theta_{\text{pre}} - \eta \sum_{j=0}^m \nabla \bar{L}_i(\theta_{\text{MT}}^{(j)}) - \eta^2 s_i^{m-1}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(m-1)}) + O(\eta^3) \end{aligned}$$

we can rewrite the tasks vectors as

$$\tau_i^{(k)} = \theta_i^{(k)} - \theta_{\text{pre}} \quad (\text{B.12})$$

$$= -\eta \sum_{j=0}^{k-1} \nabla \bar{L}_i(\theta_{\text{MT}}^{(j)}) - \eta^2 s_i^{k-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(k-2)}) + O(\eta^3) \quad (\text{B.13})$$

Consequently the model obtained with TA is

$$\begin{aligned} \theta_{\text{TA}}^{(k)} &= \theta_{\text{pre}} + \alpha \sum_{i \in T} \tau_i^{(k)} \\ &= \theta_{\text{pre}} - \eta \alpha \sum_{j=0}^{k-1} \sum_{i \in T} \nabla \bar{L}_i(\theta_{\text{MT}}^{(j)}) - \alpha \sum_{i \in T} \eta^2 s_i^{k-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(k-2)}) + O(\eta^3) \\ &= \theta_{\text{MT}}^{(k)} - \alpha \sum_{i \in T} \eta^2 s_i^{k-2}(\theta_{\text{pre}}, \dots, \theta_{\text{MT}}^{(k-2)}) + O(\eta^3). \end{aligned}$$

□

B.1.2 Proofs of Theorem 5.2.6

In this section, we provide the proof for Theorem 5.2.6, which is restated for clarity.

Theorem (Uniform bound on the coefficient C). *Let $C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h)$ be the error term obtained in Theorem 5.2.1, with the condition of the theorem above. Additionally, we add that the tasks are all classification tasks for which we used cross entropy loss. We also assume that the network is a depth- L feed-forward network with weight matrices $W^{(1)}, \dots, W^{(L)}$, with the property that there exist constants $s_\ell > 0$ such that $\|W^{(\ell)}\|_2 \leq s_\ell$. We abbreviate $\Pi := \prod_{\ell=1}^L s_\ell$. We also assume that every input vector satisfies $\|x\|_2 \leq M_x$. We assume that the activation functions have bounded first and second derivatives: there are $\beta_\phi, \gamma_\phi > 0$ such that $\sup_z |\phi'(z)| \leq \beta_\phi$ and $\sup_z |\phi''(z)| \leq \gamma_\phi$. For the ReLU activation we have $\gamma_\phi = 0$.*

(i) *General activations.*

$$\|C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h)\|_2 \leq T \frac{(h+1)(h+2)}{2} |\alpha T - 1| H_{\max} G_{\max},$$

with

$$G_{\max} \leq \sqrt{2} M_x \Pi \beta_\phi^{L-1}, \quad H_{\max} \leq 2 \gamma_\phi M_x^2 \Pi^2 \beta_\phi^{2L-2}.$$

(ii) *ReLU activations ($\gamma_\phi = 0$).*

$$\|C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h)\|_2 \leq T \frac{(h+1)(h+2)}{2} |\alpha T - 1| \frac{1}{2} \sqrt{2} M_x^3 \Pi^3 \beta_\phi^{3L-3}.$$

Proof. We want to bound the 2-norm of the coefficient C in Equation 5.3. Let's start by noticing that

$$\begin{aligned} \|C(\{\theta_{\text{MT}}^{(j)}\}_{j=1}^h)\|_2 &\leq \left\| \sum_{t \in T} \sum_{\ell=0}^h \nabla^2 \bar{L}_t(\theta_{\text{MT}}^{(\ell)}) \right\|_2 \cdot \left\| \sum_{m=0}^{\ell} \left[\sum_{t' \neq t, t' \in T} \alpha \nabla \bar{L}_{t'}(\theta_{\text{MT}}^{(m)}) + (\alpha - 1) \nabla \bar{L}_t(\theta_{\text{MT}}^{(m)}) \right] \right\|_2 \\ &\leq \left(\sum_{t \in T} \sum_{\ell=0}^h \|\nabla^2 \bar{L}_t(\theta_{\text{MT}}^{(\ell)})\|_2 \right) \cdot \left(\sum_{m=0}^{\ell} \sum_{t' \neq t, t' \in T} \|\alpha \nabla \bar{L}_{t'}(\theta_{\text{MT}}^{(m)}) + (\alpha - 1) \nabla \bar{L}_t(\theta_{\text{MT}}^{(m)})\|_2 \right) \end{aligned}$$

Let us denote by G_{max} the max of the gradient and by H_{max} the max of the Hessian, the term C can be upper-bounded by

$$\|C(\{\theta_{MT}^{(j)}\}_{j=1}^h)\|_2 \leq \sum_{t \in T} \sum_{\ell=0}^h H_{max} \sum_{m=0}^{\ell} |\alpha(T+1) - 1| G_{max} \quad (B.14)$$

$$= T \frac{(h+1)(h+2)}{2} (\alpha(T+1) - 1) H_{max} G_{max} \quad (B.15)$$

We are left with bounding H_{max} and G_{max} .

Let us start with G_{max} , the cross entropy loss on a sample (x, y) is $L(x, y, \theta) = -\log(\sigma(f_\theta(x) \cdot e_y))$, with $\sigma(\cdot)$ being the softmax function and e_y being the one-hot vector of class y . We denote $z = f_\theta(x)$, we denote by

$$p = \sigma(z) = \left[\frac{e^{-z_1}}{\sum_{k=1}^K e^{-z_k}}, \dots, \frac{e^{-z_K}}{\sum_{k=1}^K e^{-z_k}} \right].$$

The gradient of the loss with respect to the output of the network z is:

$$g = \nabla_z (-\log(\sigma(z) \cdot e_y)) = p - e_y.$$

$$H_{logits} = \nabla_z^2 (-\log(\sigma(z) \cdot e_y)) = \text{diag}(p) - pp^T$$

K is the number of classes and P the number of parameters. Moreover we denote by $J_\theta f_\theta(x) \in \mathbb{R}^{K \times P}$ the Jacobian of the network with respect to the parameters, $\nabla_\theta^2 f_\theta$ is a 3-dimensional tensor in $\mathbb{R}^{K \times P \times P}$. By chain rule we can obtain that:

$$\nabla_\theta L(x, y, \theta) = \underbrace{\nabla_z L(x, y, \theta)}_{g^T} \underbrace{\nabla_\theta z}_{J_\theta f_\theta(x)} = g^T J_\theta f_\theta(x) \quad (B.16)$$

$$\nabla_\theta^2 L(x, y, \theta) = \nabla_\theta g^T J_\theta f_\theta(x) + g^T \nabla_\theta^2 f_\theta(x) \quad (B.17)$$

$$= [H_{logits} J_\theta f_\theta(x)]^T J_\theta f_\theta(x) + g^T \nabla_\theta^2 f_\theta(x) \quad (B.18)$$

$$= J_\theta f_\theta(x)^T H_{logits} J_\theta f_\theta(x) + g^T \nabla_\theta^2 f_\theta(x) \quad (B.19)$$

Bound on the Jacobian of the network We assume the activation has bounded first and second derivative, more formally we assume the activation function $\phi(\cdot)$ is such that there exists β_ϕ so that $\sup_{x \in \mathbb{R}} \phi'(x) \leq \beta_\phi$ and there exists γ_ϕ so that $\sup_{x \in \mathbb{R}} \phi''(x) \leq \gamma_\phi$. Notice that this covers both the case of ReLU, the hyperbolic tangent or sigmoid activation function.

Indeed

for the points $x \in \mathbb{R}$ where the derivative are defined: $|\text{ReLU}'(x)| \leq 1$ and $\text{ReLU}''(x) = 0$. (B.20)

For the sigmoid activation function

$$\sup_{x \in \mathbb{R}} |\sigma'(x)| \leq \frac{1}{4} \text{ and } \sup_{x \in \mathbb{R}} |\sigma''(x)| \leq \frac{1}{6\sqrt{3}}. \quad (B.21)$$

Finally for the hyperbolic tangent

$$\sup_{x \in \mathbb{R}} |\tanh'(x)| \leq 1 \text{ and } \sup_{x \in \mathbb{R}} |\tanh''(x)| \leq \frac{4}{3\sqrt{3}}. \quad (B.22)$$

Under the hypothesis of activation function with bounded first and second derivatives almost everywhere, plus the hypothesis that the input of the network lies in a bounded set,

$\|x\| \leq M_x$, and that for each layer ℓ there exists s_ℓ so that $\|W^\ell\|_2 \leq s_\ell$. We obtain, by the chain rule:

$$\|J_\theta f_\theta(x)\| \leq M_x \prod_{l=1}^L s_l \beta_\phi^{L-1}.$$

We denote by $\Pi = \prod_{l=1}^L s_l$. From this we can derive the following **bound on the gradient of the loss function**

$$\|\nabla_\theta L\|_2 \leq \|g\|_2 \|J_\theta f_\theta(x)\|_2 \leq \sqrt{2} M_x \Pi \beta_\phi^{L-1}$$

Bound on the Hessian of the loss.

If the activation function is the ReLU function then the network is piecewise linear and $\nabla_\theta^2 f_\theta(x) = 0$ is almost everywhere. So the Hessian of the loss becomes:

$$\nabla_\theta^2 L(x, y, \theta) = J_\theta f_\theta(x)^T H_{logits} J_\theta f_\theta(x).$$

The matrix H_{logits} is the correct covariance of a categorical variable, it is positive semi-definite, so it is diagonalizable with non-negative eigenvalues. For any unit vector $x \in \mathbb{R}^n$,

$$x^T H_{logits} x = \sum_{i=1}^n p_i x_i^2 - \left(\sum_{i=1}^n p_i x_i \right)^2.$$

We can look for the eigenvector corresponding to the maximum eigenvalues in the space orthogonal to the null-space. So, the maximum eigenvalues is given by:

$$\max_{x: \|x\|_2=1} x^T H_{logits} x = \max_{x: \|x\|_2=1} \sum_{i=1}^n p_i x_i^2 - \left(\sum_{i=1}^n p_i x_i \right)^2. \quad (\text{B.23})$$

Now, for $\|x\|_2 = 1$, we can derive that

$$\begin{aligned} \sum_{i=1}^n p_i x_i^2 - \left(\sum_{i=1}^n p_i x_i \right)^2 &= \sum_{i=1}^n p_i x_i^2 - \sum_{i,j} p_i p_j x_i x_j = \sum_{i=1}^n p_i x_i^2 - \sum_i p_i^2 x_i^2 - \sum_{i \neq j} p_i p_j x_i x_j \\ &= \sum_{i=1}^n p_i (1 - p_i) x_i^2 - \sum_{i \neq j} p_i p_j x_i x_j \leq \sum_{i=1}^n p_i (1 - p_i) x_i^2 + \frac{1}{2} \sum_{i \neq j} p_i p_j (x_i^2 + x_j^2) \\ &= \sum_{i=1}^n p_i (1 - p_i) x_i^2 + \sum_{i \neq j} p_i p_j x_i^2 = \sum_{i=1}^n p_i (1 - p_i) x_i^2 + \sum_i p_i (1 - p_i) x_i^2 \\ &= 2 \sum_{i=1}^n p_i (1 - p_i) x_i^2 \leq 2 \frac{1}{4} \sum_{i=1}^n x_i^2 = \frac{1}{2} \end{aligned}$$

So the maximum eigenvalue for H_{logits} is $\frac{1}{2}$. Consequently, when the activation function is ReLU:

$$\|\nabla_\theta^2 L(x, y, \theta)\|_2 \leq \frac{1}{2} M_x^2 \Pi^2 \beta_\phi^{2L-2}.$$

In the other cases we also have to include the terms coming from the hessian of the network with respect to the parameters. We have to bound that term.

Bound Hessian of the Network We start by noticing the structure of the Jacobian blocks. We use the following layer-wise notation:

$$h^{(\ell)} := W^{(\ell)} a^{(\ell-1)}, \quad a^{(\ell)} := \phi(h^{(\ell)}), \quad a^{(0)} := x.$$

Let Θ collect *all* scalar parameters and denote a single one by θ_p . We consider θ_p^r is an entry of $W^{(r)}$ with $1 \leq r \leq k$.

The product splits at layer r :

$$\partial_{\theta_p^r} a^{(k)} = (D^{(k)} W^{(k)} \dots D^{(r)}) (\partial_{\theta_p^r} W^{(r)}) a^{(r-1)}$$

where $D^{(\ell)} = \text{diag}(\phi'(h^{(\ell)}))$.

For any parameter pair $(\theta_p^r, \theta_q^{r'})$ with $1 \leq r \leq r' \leq k$ (the function implemented by the neural network is continuous, therefore the mixed double derivatives are equal) :

$$\begin{aligned} \partial_{\theta_q^{r'}} \partial_{\theta_p^r} a^{(k)} &= \underbrace{\partial_{\theta_q^{r'}} (D^{(k)} W^{(k)} \dots D^{(r)}) (\partial_{\theta_p^r} W^{(r)}) a^{(r-1)}}_{(A)} + \\ &\quad \underbrace{D^{(k)} W^{(k)} \dots D^{(r)} (\partial_{\theta_p^r} W^{(r)}) \partial_{\theta_q^{r'}} a^{(r-1)}}_{=0} + \\ &\quad \underbrace{D^{(k)} W^{(k)} \dots D^{(r)} \partial_{\theta_q^{r'}} (\partial_{\theta_p^r} W^{(r)}) a^{(r-1)}}_{(B)}. \end{aligned}$$

$$\begin{aligned} \partial_{\theta_q^{r'}} \partial_{\theta_p^r} a^{(k)} &= \underbrace{(\partial_{\theta_q^{r'}} Z_{k \leftarrow r}) (\partial_{\theta_p^r} W^{(r)}) a^{(r-1)}}_{(A)} + \\ &\quad \underbrace{Z_{k \leftarrow r} \partial_{\theta_q^{r'}} (\partial_{\theta_p^r} W^{(r)}) a^{(r-1)}}_{(B)}, \end{aligned}$$

where

$$Z_{k \leftarrow r} := D^{(k)} W^{(k)} \dots D^{(r)}, \quad D^{(\ell)} = \text{diag}(\phi'(h^{(\ell)})).$$

We recall that

$$\|D^{(\ell)}\|_2 \leq \beta, \quad \|\partial_{\theta} W^{(\ell)}\|_2 = 1, \quad \|W^{(\ell)}\|_2 \leq s_{\ell}, \quad \|a^{(r-1)}\|_2 \leq M_x.$$

$$\text{Define } \Pi = \prod_{\ell=1}^k s_{\ell}, \quad \Pi_{1:r-1} := \prod_{\ell=1}^{r-1} s_{\ell}, \quad \Pi_{r+1:k} := \prod_{\ell=r+1}^k s_{\ell}.$$

Bound of the term (A)

$$\begin{aligned} \partial_{\theta_q^{r'}} Z_{k \leftarrow r} &= \sum_{\ell=r'}^k [D^{(k)} W^{(k)} \dots (\partial_{\theta_q^{r'}} D^{(\ell)}) \dots D^{(r)} + D^{(k)} W^{(k)} \dots (\partial_{\theta_q^{r'}} W^{(\ell)}) \dots D^{(r)}] = \\ &= D^{(k)} W^{(k)} \dots (\partial_{\theta_q^{r'}} W^{(r')}) \dots D^{(r)} + \sum_{\ell=r'+1}^k D^{(k)} W^{(k)} \dots (\partial_{\theta_q^{r'}} D^{(\ell)}) \dots D^{(r)}, \end{aligned}$$

with

$$\begin{aligned} \partial_{\theta_q^{r'}} D^{(\ell)} &= \text{diag}(\phi''(h^{(\ell)})) \partial_{\theta_q^{r'}} h^{(\ell)} = \\ &= \text{diag}(\phi''(h^{(\ell)})) W^{(\ell)} D^{(\ell-1)} \dots D^{(r')} \partial_{\theta_q^{r'}} W^{(r')} a^{(r'-1)} \end{aligned}$$

Hence

$$\|\partial_{\theta_q^{r'}} D^{(\ell)}\|_2 \leq \gamma \beta^{\ell-r'} M_x \Pi_{r'+1:\ell},$$

and therefore

$$\|(A)\|_2 \leq M_x [\beta^{k-r+1} \Pi_{r:r'} \Pi_{r'+1:k} + \sum_{\ell=r'+1}^k \gamma \beta^{\ell-r'} M_x \Pi_{r'+1:\ell}].$$

Bound of the term (B)

$$\partial_{\theta_q^{r'}}(\partial_{\theta_p^r} W^{(r)}) = 0$$

Bound Hessian

$$\|\partial_{\theta_q^{r'}} \partial_{\theta_p^r} a^{(k)}\|_2 \leq M_x [\beta^{k-r+1} \Pi_{r:r'} \Pi_{r'+1:k} + \sum_{\ell=r'+1}^k \gamma \beta^{\ell-r'} M_x \Pi_{r'+1:\ell}]$$

□

B.1.3 Upper bounding the multi-task loss

In this section, we explore the relationship between the PA-ATM loss, defined as the mean of average losses over all tasks, and the loss of a model trained jointly on all the datasets. We conduct this analysis using GD, rather than SGD, thus removing stochasticity and enabling a clearer analysis while retaining the key insights. Denoting with t both the task and its corresponding dataset with cardinality n_t . The total number of samples for all tasks is given by $N = \sum_{t \in T} n_t$.

By Theorem 5.2.3, when merging occurs after one step of fine-tuning on each dataset, the PA-ATM update is given by:

$$\theta_{\text{pre}}^{(k+1)} = \theta_{\text{pre}}^{(k)} + \frac{\alpha}{|T|} \sum_{t \in T} \tau_t^{(k)} = \theta_{\text{pre}}^{(k)} - \alpha \eta \nabla \left(\frac{1}{|T|} \sum_{t \in T} \bar{L}_t \right), \quad (\text{B.24})$$

which corresponds to performing a GD step over the loss $L_{\text{ATM}} = \frac{1}{|T|} \sum_{t \in T} \bar{L}_t$.

Having established that one step of PA-ATM in GD minimizes $L_{\text{PA-ATM}}$, a crucial question arises: under what conditions does minimizing L_{ATM} imply the minimization of L_{target} ? In other words, when can we be certain that optimizing the PA-ATM loss will also minimize the loss associated with training jointly on all datasets?

To study if optimizing the PA-ATM loss will also minimize the loss associated with training jointly on all datasets we first note that $L_{\text{PA-ATM}}$ is an unweighted average of the individual dataset losses, while the target loss is a weighted average:

$$L_{\text{target}}(\theta) = \frac{\sum_{t \in T} n_t \bar{L}_t(\theta)}{\sum_{t \in T} n_t}. \quad (\text{B.25})$$

We now analyze the parameter update from $\theta^{(k)}$ to $\theta^{(k+1)}$. For both PA-ATM and target methods, we denote the change in loss, L_{method} , as $\Delta L_{\text{method}} = L_{\text{method}}(\theta^{(k)}) - L_{\text{method}}(\theta^{(k+1)})$. In the following theorem, we prove that if the drop in ATM loss exceeds a threshold δ , the target loss will also decrease. The value of δ depends on the size of the largest dataset with a decreasing loss and the smallest dataset with an increasing loss. In particular, if the former dataset is larger than the latter, a reduction in PA-ATM loss reduces the target loss. In practice, this is ensured when the loss is reduced on the largest dataset.

Theorem B.1.1. *Let $D = \{t \mid \Delta \bar{L}_t > 0\}$ be the set of datasets where the loss decreases after a parameter update, and $I = \{t \mid \Delta \bar{L}_t \leq 0\}$ be the set of datasets where the loss increases or remains unchanged. If the reduction in the PA-ATM loss satisfies $\Delta L_{\text{PA-ATM}} > \delta$, where*

$$\delta = \frac{1}{|T|} \left(1 - \frac{\min_{t \in I} n_t}{\max_{t \in D} n_t} \right) \sum_{t \in I} |\Delta \bar{L}_t|,$$

then the target loss L_{target} will also decrease, i.e., $\Delta L_{\text{target}} > 0$.

Proof. Suppose that when transitioning from parameters $\theta^{(i)}$ to parameters $\theta^{(i+1)}$, the change in the average loss for each dataset D_k is given by $\Delta \bar{L}_k = \bar{L}_k(\theta^{(i)}) - \bar{L}_k(\theta^{(i+1)})$. We denote by P the set of tasks for which the delta of the loss is positive and by N the set for which it is negative, namely $P = \{k \in T \text{ s.t. } \Delta \bar{L}_k > 0\}$ and $N = \{k \in T \text{ s.t. } \Delta \bar{L}_k \leq 0\}$. The set $\{1, \dots, n\} = P \cup N$. In the following formulas we will use the symbol $|\cdot|$ for different purposes. For sets, like D_i , $|D_i|$ denotes the cardinality of the set, while for scalars, such as $\Delta \bar{L}_k$, $|\Delta \bar{L}_k|$ denotes their absolute value. Since the tasks in N have negative $\Delta \bar{L}_k$, it holds that

$$\sum_{j \in N} |D_j| \Delta \bar{L}_j = - \sum_{j \in N} |D_j| |\Delta \bar{L}_j|.$$

Under the hypothesis that $\Delta L_{\text{PA-ATM}} > \delta$, this implies $\sum_{j=1}^n \Delta_j > n\delta$. We have that $\sum_{j \in P} \Delta \bar{L}_j + \sum_{j \in N} \Delta \bar{L}_j = \sum_{j \in P} \Delta \bar{L}_j - \sum_{j \in N} |\Delta \bar{L}_j| > n\delta$, namely $\sum_{j \in P} \Delta \bar{L}_j > n\delta + \sum_{j \in N} |\Delta \bar{L}_j|$.

We want to prove that $\Delta L_{\text{target}} > 0$. Let us now consider $\Delta L_{\text{target}} > 0$ iff $\sum_{j=1}^n |D_j| \Delta_j > 0$.

$$\begin{aligned} \sum_{j=1}^n |D_j| \Delta_j &= \sum_{j \in P} |D_j| \Delta \bar{L}_j + \sum_{j \in N} |D_j| \Delta \bar{L}_j \\ &= \sum_{j \in P} |D_j| \Delta \bar{L}_j - \sum_{j \in N} |D_j| |\Delta \bar{L}_j| \\ &> \min_{j \in P} |D_j| \sum_{j \in P} \Delta \bar{L}_j - \max_{j \in N} |D_j| \sum_{j \in N} |\Delta \bar{L}_j| \\ &> \min_{j \in P} |D_j| \left[n\delta + \sum_{j \in N} |\Delta \bar{L}_j| \right] - \max_{j \in N} |D_j| \sum_{j \in N} |\Delta \bar{L}_j| \\ &= n \min_{j \in P} |D_j| \delta + (\min_{j \in P} |D_j| - \max_{j \in N} |D_j|) \sum_{j \in N} |\Delta \bar{L}_j|. \end{aligned}$$

The last line of the previous equation is positive by hypothesis, since we assumed $\delta > \frac{1}{n} (1 - \frac{\min_{j \in N} |D_j|}{\max_{j \in P} |D_j|}) \sum_{j \in N} |\Delta \bar{L}_j|$. $\square$

Remark B.1.2. If we choose the target loss to be the maximum of the average loss across all datasets $L_{\text{target}} = \max_{t \in T} \bar{L}_t$, by leveraging the equivalence between the L_1 -norm and the max norm, we obtain the bound $L_{\text{target}} \leq T \cdot L_{\text{PA-ATM}}$.

Appendix C

Additional material on Chapter 6

C.1 Illustrative example: merging two tasks with rank-1 approximation

Consider merging two distinct tasks by selecting only the first singular vector and singular value from the SVD for each task. This setting yields the following setup for each layer ℓ :

$$\begin{bmatrix} | & | \\ u_{1_\ell} & u_{2_\ell} \\ | & | \end{bmatrix} \begin{bmatrix} \sigma_{1_\ell} & 0 \\ 0 & \sigma_{2_\ell} \end{bmatrix} \begin{bmatrix} - & v_{1_\ell}^T & - \\ - & v_{2_\ell}^T & - \end{bmatrix}$$

In this formulation, u_{1_ℓ} originates from task 1 and u_{2_ℓ} from task 2, with analogous assignments for the singular vectors v and singular values σ .

To elucidate the interaction between tasks, we examine three distinct cases, considering a single layer, thereby omitting the layer index ℓ :

1. **Orthogonal Singular Vectors:** when u_1 and u_2 (respectively v) are orthogonal, the similarity matrix $U^\top U$ (respectively $V^\top V$) is given by:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

The zeroes in the off-diagonal elements indicate no interference between the tasks. Consequently, the orthogonal components derived from different tasks operate independently, ensuring that each task does not affect the other.

2. **Collinear Singular Vectors:** when u_1 and u_2 (respectively v) are collinear, either aligned in the same direction (angle of 0 degrees) or in the opposite direction (angle of 180 degrees), the similarity matrix $U^\top U$ (respectively $V^\top V$) takes the form:

$$\begin{bmatrix} 1 & \langle u, \pm u \rangle \\ \langle \pm u, u \rangle & 1 \end{bmatrix}$$

If the singular vectors are perfectly aligned (0 degrees), then $u_1 = u_2 = u$, simplifying the diagonal elements to $\langle u, u \rangle = \|u\|^2 = 1$. Conversely, if the singular vectors are oppositely aligned (180 degrees), $u_1 = -u_2$, resulting in $\langle u, -u \rangle = -1$. Thus, the similarity matrices becomes:

$$\begin{bmatrix} 1 & \pm 1 \\ \pm 1 & 1 \end{bmatrix}$$

This structure reveals complete interference between the tasks: a double scaling effect when the vectors agree and complete cancellation when they disagree.

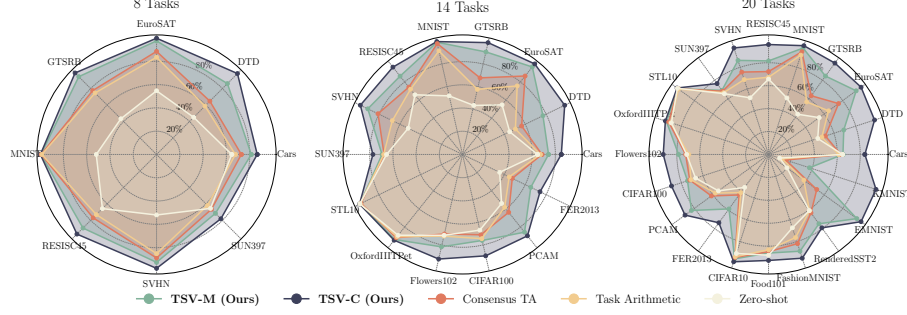


FIGURE C.1: Absolute accuracy of a ViT-B-16 merged over 8, 14, and 20 tasks, respectively.

3. **Partially Collinear Singular Vectors:** when u_1 and u_2 (respectively v) are partially collinear, with the angle between them ranging from slightly greater than 0 degrees to less than 90 degrees or slightly more than 90 degrees to less than 180 degrees, the similarity matrices are expressed as:

$$\begin{bmatrix} 1 & \langle u_1, u_2 \rangle \\ \langle u_2, u_1 \rangle & 1 \end{bmatrix}$$

In this case, the overlap between singular vectors induces a partial interaction between the tasks. The degree of interference, whether it is constructive or destructive, is proportional to the cosine of the angle between the singular vectors. This partial collinearity leads to subtle interplay, where the tasks influence each other to a degree dictated by their vector alignment.

This example underscores the critical role of singular vector alignment in model merging, highlighting how orthogonality ensures independent task performance, collinearity leads to maximal interference and partial collinearity results in an intermediate level of task interaction.

C.2 Additional details

C.2.1 Implementation details and computational resources

Normalized Accuracy To address the varying difficulties of the task, we report both normalized and absolute accuracies in our results. The normalized accuracy provides a relative performance metric by comparing the accuracy of the multi-task model to that of individually fine-tuned models. Specifically, the normalized accuracy is calculated as:

$$\text{Normalized Accuracy} = \frac{1}{T} \sum_{i=1}^T \frac{\text{accuracy}(\theta_{\text{MT}}, t_i)}{\text{accuracy}(\theta_{f_{t_i}}, t_i)} \quad (\text{C.1})$$

where T is the total number of tasks, θ_{MT} represents the multi-task model and θ_i denotes the individually fine-tuned model for task t_i . This metric allows for a more fair comparison by adjusting for the baseline performance of each task.

Datasets for tasks All benchmarks were performed by integrating the codebase provided by Wang et al. [166]. In line with the principles of PEFT, we reused the already existing model checkpoints in the codebase for both the models and classification heads without additional fine-tuning.

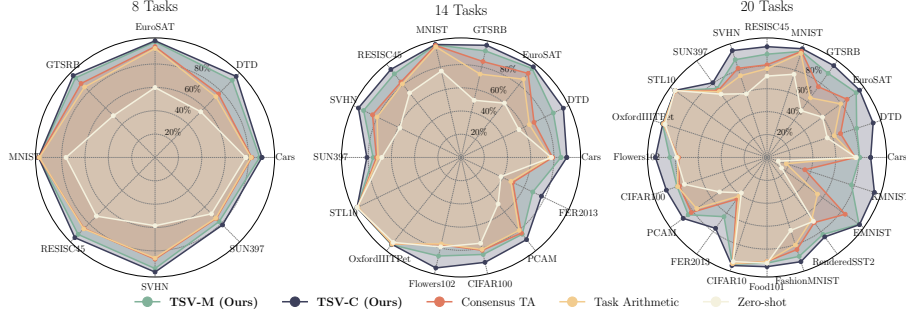


FIGURE C.2: Absolute accuracy of a ViT-L-14 merged over 8, 14, and 20 tasks, respectively.

Implementation Our method utilizes the SVD, a matrix decomposition technique applicable to two-dimensional matrices. For layers that are not represented as matrices (e.g., normalization layers) we default to standard `Task Arithmetic`. In particular, we employ Knuth’s algorithm [170] to compute the average efficiently. This ensures that all fine-tuned model task layers, regardless of their structure, are appropriately integrated into the merged model.

Compute Resources We utilize PyTorch as deep learning framework. All the merging and evaluations were conducted on an NVIDIA 4060Ti GPU with 16GB of memory, and an Intel i7-6800K CPU equipped with 64GB of RAM. For experiments that need more than 64GB of RAM, we resort to a shared HTCondor cluster equipped with NVIDIA P6000 GPUs.

C.2.2 Hyperparameter settings

Following `Task Arithmetic` [78] and `Consensus TA` [166], we apply a single scaling factor, α , to adjust the multi-task vector within the model merging techniques outlined in Table 6.1. This scaling factor is optimized, when feasible, over the range $\{0.0, 0.1, \dots, 2.9, 3.0\}$, with the optimal value selected based on the average validation performance across all tasks. However, as discussed in Section 6.5.3, our experimental findings indicate that the proposed `TSV-Merge` method does not strictly depend on this hyperparameter, as the marginal performance gains from tuning α do not justify the computational resources required for the evaluation on the validation datasets. Consequently, this allows us to eliminate the necessity for validation datasets and the corresponding labels from the prerequisites of the method, further simplifying the practicality and resource usage of the approach. The evaluation is performed on a batch of 32 images. To produce fig. 6.7 we selected the following subsets of 8 tasks from the 20 available, using the whitening method to speed up computation, the number in the image is the index indicating the subset:

0. SVHN, SUN397, STL10, OXFORDIIITPET, FLOWERS102, CIFAR100, PCAM, FER2013
1. PCAM, FER2013, CIFAR10, FOOD101, FASHIONMNIST, RENDEREDSST2, EMNIST, KMNIST
2. EUROSAT, GTSRB, MNIST, RESISC45, SVHN, SUN397, STL10, OXFORDIIITPET
3. CARS, DTD, EUROSAT, GTSRB, MNIST, RESISC45, SVHN, SUN397

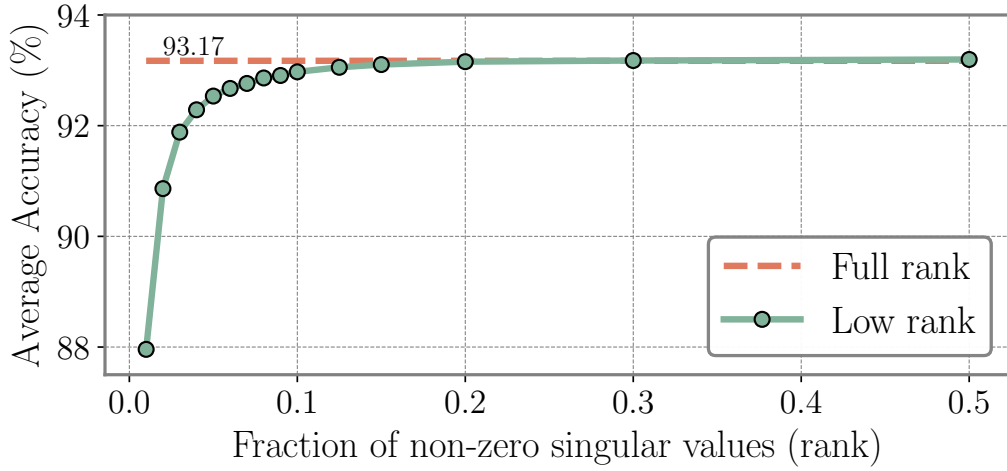


FIGURE C.3: Mean absolute accuracy of the ViT-B-16 model across increasing fractions of retained singular components, averaged over 20 tasks. The red line represents the average accuracy of the original fine-tuned models with full-rank task matrices, while the green line shows the accuracies using low-rank approximations.

4. CARS, DTD, EUROSAT, GTSRB, FASHIONMNIST, RENDEREDSST2, EMNIST, KMNIST
5. MNIST, RESISC45, SVHN, SUN397, STL10, OXFORDIIITPET, FLOWERS102, CIFAR100
6. STL10, OXFORDIIITPET, FLOWERS102, CIFAR100, PCAM, FER2013, CIFAR10, FOOD101

C.2.3 Storage cost calculation

Suppose we have a neural network comprising L two-dimensional layers, each of dimension $d \times m$, and N one-dimensional layers of size c . The total number of parameters in the network is therefore:

$$\text{Params}(\text{NN}) = L \times (d \times m) + N \times c.$$

In standard `Task Arithmetic`, one must store the same number of parameters to obtain a task vector. In contrast, our approach provides the flexibility to select the number of parameters to preserve based on storage constraints or the desired performance level, to adhere to the chosen constraints. Under the above assumptions, our method applies the truncated SVD to each two-dimensional layer. This decomposition yields two matrices of singular vectors, U and V , and a vector of singular values, σ , specifically:

- U of size $d \times k$,
- V of size $k \times m$,
- σ of size k ,

where $k = \min(d, m)$. We select a reduced rank $k' \ll k$ to approximate each layer's task matrix. Consequently, the total number of parameters for TSV becomes:

$$\text{Params}(\text{TSV}) = L \times ((d \times k') + k' + (k' \times m)) + N \times c$$

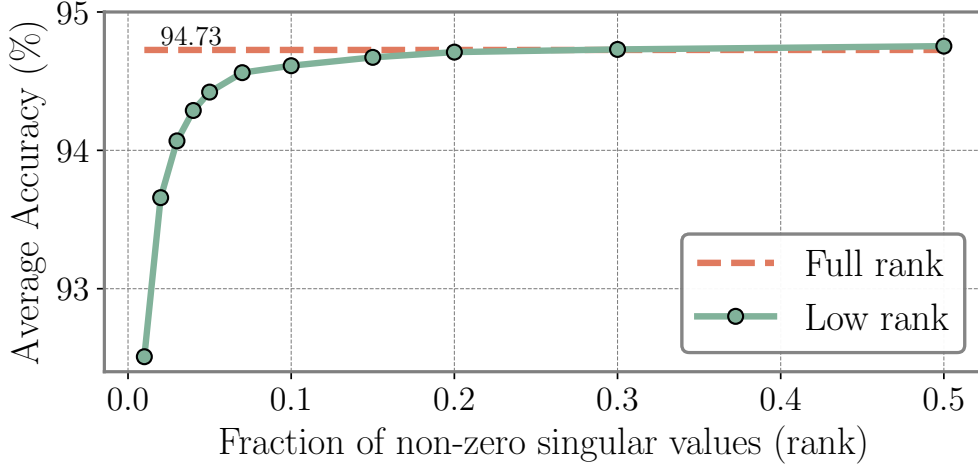


FIGURE C.4: Mean absolute accuracy of the ViT-L-14 model across increasing fractions of retained singular components, averaged over 20 tasks. The red line represents the average accuracy of the original fine-tuned models with full-rank task matrices, while the green line shows the accuracies using low-rank approximations.

To demonstrate that our method results in fewer stored parameters than the original parameter count, we require that $k' < \frac{d \times m}{d+m+1}$. This condition ensures:

$$\text{Params(NN)} > \text{Params(TSV)}$$

Substituting the expressions, yields:

$$L \times (d \times m) + N \times c > L \times ((d \times k') + k' + (k' \times m)) + N \times c$$

Simplifying, we obtain:

$$\begin{aligned} L \times (d \times m) &> L \times ((d \times k') + k' + (k' \times m)) \\ (d \times m) &> ((d \times k') + k' + (k' \times m)) \\ d \times m &> k' \times (d + 1 + m) \\ k' &< \frac{d \times m}{d + m + 1}. \end{aligned} \tag{C.2}$$

This inequality confirms that our method reduces the storage requirements of a task vector when $k' < \frac{d \times m}{d+m+1}$. Empirical evidence from Figures C.3, C.4 and 6.2 suggests that selecting $k' < 0.1 \times k = 0.1 \times \min(d, m)$ is sufficient to preserve most of the task performance, preserving the main requirement of performance. Furthermore, it is easy to prove that when choosing $k' = \frac{k}{T}$, the inequality is always satisfied for $T \geq 3$, respecting the main requirement of limited storage usage.

C.3 Proofs

We hereby prove the claims outlined in the main manuscript.

C.3.1 Characterization of the similarity matrices

Proposition C.3.1. *The matrix $\hat{U}^\top \hat{U}$ is positive definite.*

Proof. We define $\hat{U}^\top \hat{U}$, where $\hat{U}$ is a generic $d \times k$ rectangular matrix. Consequently, $\hat{U}^\top \hat{U}$ is a $k \times k$ square matrix. To establish that $\hat{U}^\top \hat{U}$ is positive definite, it suffices to demonstrate that for all non-zero vectors $x \in \mathbb{R}^k$, the following inequality holds:

$$x^\top \hat{U}^\top \hat{U} x > 0.$$

This expression can be rewritten as:

$$x^\top (\hat{U}^\top \hat{U}) x = (\hat{U} x)^\top (\hat{U} x) = \|\hat{U} x\|^2.$$

Here, $\|\hat{U} x\|^2$ denotes the squared Euclidean norm of the vector $\hat{U} x$, which is always non-negative. Moreover, assuming that $\hat{U}$ has full column rank, the norm $\|\hat{U} x\|^2$ is strictly positive for any non-zero vector x . Therefore, we have:

$$\|\hat{U} x\|^2 > 0 \quad \text{for all } x \in \mathbb{R}^k, x \neq 0.$$

This implies that:

$$x^\top \hat{U}^\top \hat{U} x > 0 \quad \text{for all } x \in \mathbb{R}^k, x \neq 0,$$

which confirms that $\hat{U}^\top \hat{U}$ is positive definite. $\square$

Corollary C.3.2. *Since $\hat{U}^\top \hat{U}$ is positive definite, then $\hat{U}^\top \hat{U}$ is invertible.*

Proof. From Proposition C.3.1, we have established that $\hat{U}^\top \hat{U}$ is a positive definite matrix. A positive definite matrix, by definition, has all its eigenvalues strictly positive. Let $\lambda_1, \lambda_2, \dots, \lambda_k$ denote the eigenvalues of $\hat{U}^\top \hat{U}$. Therefore, we have:

$$\lambda_i > 0 \quad \text{for all } i = 1, 2, \dots, k.$$

The determinant of $\hat{U}^\top \hat{U}$ is the product of its eigenvalues:

$$\det(\hat{U}^\top \hat{U}) = \prod_{i=1}^k \lambda_i.$$

Since each λ_i is positive, their product is also positive:

$$\det(\hat{U}^\top \hat{U}) > 0.$$

A matrix is invertible if and only if its determinant is non-zero. Given that $\det(\hat{U}^\top \hat{U}) > 0$, it follows that $\hat{U}^\top \hat{U}$ is invertible.

Therefore, $\hat{U}^\top \hat{U}$ is invertible. $\square$

Low-rank approx.	Interf. reduction	ViT-B-16		
		8 tasks	14 tasks	20 tasks
×	×	79.6 (+0.0)	75.9 (+0.0)	70.8 (+0.0)
✓	×	79.6 (+0.0)	74.9 (-1.0)	70.0 (-0.8)
×	✓	84.8 (+5.2)	79.0 (+4.1)	73.2 (+3.2)
✓	✓	93.9 (+9.1)	91.0 (+12.0)	86.5 (+13.3)

TABLE C.1: Comparison of different versions of Task Arithmetic, comprising either the low-rank approximation step, the interference reduction step, or both. The method performing both corresponds to the proposed TSV-Merge.

C.3.2 Observations

Since $\hat{U}^\top \hat{U}$ is a real symmetric matrix, it admits an eigendecomposition of the form

$$\hat{U}^\top \hat{U} = Q\Lambda Q^{-1} = Q\Lambda Q^\top,$$

where:

- Λ is a diagonal matrix containing the real eigenvalues of $\hat{U}^\top \hat{U}$,
- Q is an orthogonal matrix whose columns are the orthonormal eigenvectors of $\hat{U}^\top \hat{U}$, satisfying $Q^\top = Q^{-1}$.

The inverse of $\hat{U}^\top \hat{U}$ exists (see Corollary C.3.2), and can be expressed using its eigendecomposition as

$$(\hat{U}^\top \hat{U})^{-1} = Q\Lambda^{-1}Q^{-1} = Q\Lambda^{-1}Q^\top.$$

Additionally, since Λ is a diagonal matrix with non-zero diagonal entries (Proposition C.3.1), its inverse Λ^{-1} is straightforward to compute, with each diagonal element given by

$$\Lambda^{-1} = \text{diag}\left(\frac{1}{\lambda_i}\right),$$

where λ_i are the eigenvalues of $\hat{U}^\top \hat{U}$.

Furthermore, the eigenvalues of $(\hat{U}^\top \hat{U})^{-1}$ are $\frac{1}{\lambda_i}$, each of which is positive since $\lambda_i > 0$ for all i (following by the definition in Proposition C.3.1). Consequently, $(\hat{U}^\top \hat{U})^{-1}$ is also a positive definite matrix.

These observations confirm that not only is $\hat{U}^\top \hat{U}$ positive definite, but its inverse inherits this property due to the positivity of its eigenvalues.

Low-rank approx.	Interf. reduction	ViT-L-14		
		8 tasks	14 tasks	20 tasks
×	×	88.6 (+0.0)	84.0 (+0.0)	78.1 (+0.0)
✓	×	87.9 (-0.7)	83.4 (-0.6)	77.2 (-0.9)
×	✓	92.1 (+4.2)	86.8 (+3.4)	81.0 (+3.8)
✓	✓	97.0 (+4.9)	94.4 (+7.6)	92.5 (+11.5)

TABLE C.2: Comparison of different versions of Task Arithmetic, comprising either the low-rank approximation step, the interference reduction step, or both. The method performing both corresponds to the proposed TSV-Merge.

C.3.3 Proof of Theorem 6.5.1

Theorem. Let $T \in \mathbb{N}$ such that $T > 4$. Define $U = [U_1, \dots, U_T]$ as the matrix obtained by concatenating T orthogonal matrices U_i , each of shape $n \times n$. Let $\hat{U} = [\hat{U}_1, \dots, \hat{U}_T]$ be the matrix formed by truncating each U_i to its first k columns. Denote by X and $\hat{X}$ the matrices resulting from Procrustes orthonormalization of U and $\hat{U}$, respectively. If $k \leq n \frac{T-2\sqrt{T}}{T}$, then

$$\|U - X\|_F \geq \|\hat{U} - \hat{X}\|_F.$$

Proof. Let us consider the SVD decomposition of U and $\hat{U}$: $U = P_U \Sigma_U P_V$ and $\hat{U} = R_U \hat{\Sigma}_U R_V$. X and $\hat{X}$ obtain as $X = P_U P_V^\top$, $\hat{X} = R_U R_V^\top$ respectively. We first consider the Frobenius norm of $\|X - U\|_F$. Notice that the singular values of U are the square root of the eigenvalues of $\Sigma_U = U U^\top$.

$U U^\top = \sum_{i=1}^N U_i U_i^\top = T I_n$. As a consequence, the eigenvalues of $U U^\top$ are all equal to T and the singular values are all equal to $\sqrt{T}$.

$$\begin{aligned} \|X - U\|_F &= \|P_U P_V^\top - P_U \Sigma_U P_V^\top\|_F \\ &= \|P_U (I - \Sigma_U) P_V^\top\|_F \\ &= \|I_n - \Sigma_U\|_F \\ &= \|I_n - \sqrt{T} I_n\|_F \\ &= \sqrt{n}(\sqrt{T} - 1). \end{aligned}$$

We are now left to compute $\|\hat{X} - \hat{U}\|_F$. In this case, we are not able to compute the exact norm without other assumptions, but we can provide an upper bound that gives us a sufficient condition to prove our statement. As before

$$\begin{aligned} \|\hat{X} - \hat{U}\|_F &= \|I_n - \hat{\Sigma}_U\|_F \\ &= \sqrt{\sum_{i=1}^n (\hat{\sigma}_i - 1)^2}. \end{aligned}$$

where $\hat{\sigma}$ are the singular values of $\hat{U}$.

Notice that $\sum_{i=1}^n \hat{\sigma}_i^2 = \text{tr}(\hat{U} \hat{U}^\top) = \text{tr}(\hat{U}^\top \hat{U})$ and $\hat{U}^\top \hat{U}$ is a $Tk \times Tk$ matrix with diagonal elements equal to one, so $\text{tr}(\hat{U}^\top \hat{U}) = kT$.

Moreover,

$$\sum_{i=1}^n \hat{\sigma}_i = \sum_{i=1}^n \sqrt{\lambda_i(\hat{U} \hat{U}^\top)} \quad (\text{C.3})$$

$$\geq \sqrt{\sum_{i=1}^n \lambda_i(\hat{U} \hat{U}^\top)} \quad (\text{C.4})$$

$$= \sqrt{\text{tr}(\hat{U} \hat{U}^\top)} = \sqrt{kT} \quad (\text{C.5})$$

Notice that this upper bound is tight, indeed the sum of the singular values of $\hat{U}$ must lie within: $\sqrt{kT} \leq \sum_{i=1}^n \hat{\sigma}_i \leq kT$. The minimum $\sqrt{kT}$ is achieved if all matrices U_i are equals, on the other end, the maximum kT is achieved if the kT columns are orthonormal.

Putting everything together,

$$\|\hat{X} - \hat{U}\|_F = \sqrt{\sum_{i=1}^n (\hat{\sigma}_i - 1)^2} \quad (\text{C.6})$$

$$= \sqrt{n + \sum_{i=1}^n \hat{\sigma}_i^2 - 2 \sum_{i=1}^n \hat{\sigma}_i} \quad (\text{C.7})$$

$$= \sqrt{n + kT - 2 \sum_{i=1}^n \hat{\sigma}_i} \quad (\text{C.8})$$

$$\leq \sqrt{n + kT - 2\sqrt{kT}}. \quad (\text{C.9})$$

So we have to check for what values of k it holds that $\sqrt{n + kT - 2\sqrt{kT}} \leq \sqrt{n}(\sqrt{T} - 1)$. We have that

$$\sqrt{n + kT - 2\sqrt{kT}} \leq \sqrt{n + kT} \leq \sqrt{n}(\sqrt{T} - 1) \quad (\text{C.10})$$

eq. (C.10) is satisfied if

$k \leq n \frac{T - 2\sqrt{T}}{T}$. This concludes the proof. Since k is a positive number, the inequality is meaningful only for $T > 4$. $\square$

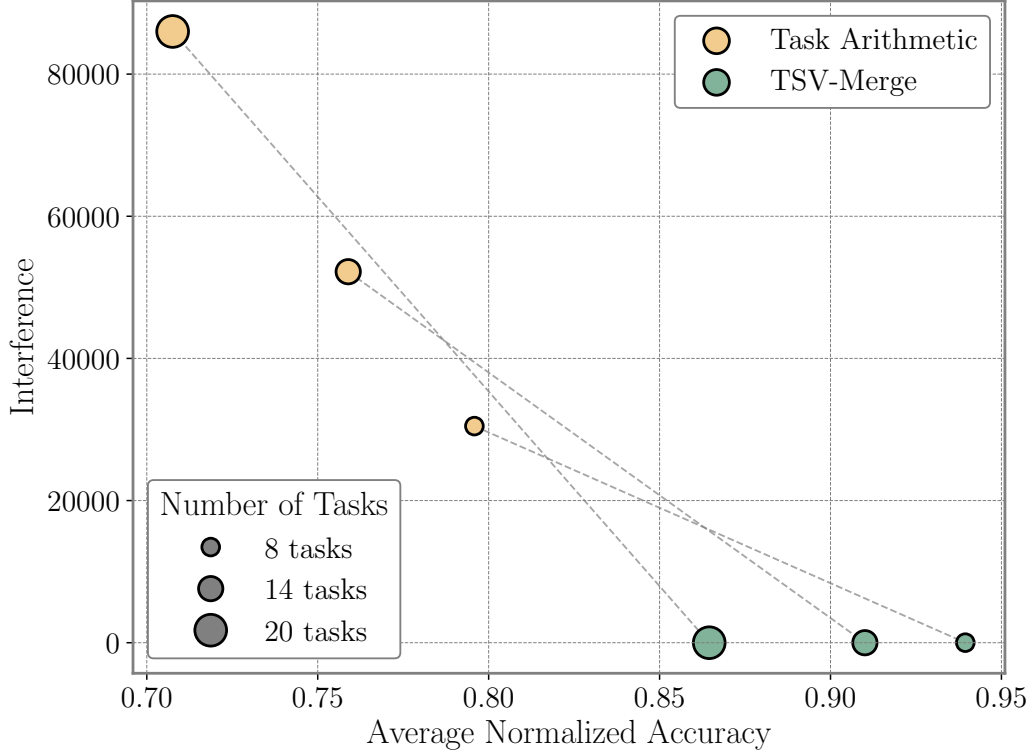


FIGURE C.5: Singular Task Interference (STI) and average normalized accuracy for Task Arithmetic and TSV-Merge on the ViT-B-16 model, evaluated across merges of 8, 14, and 20 tasks.

C.4 Additional experimental results

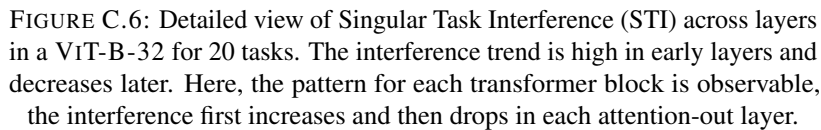
C.4.1 Per-dataset performance metrics

In Section 6.4.1, we present comprehensive results for individual tasks using the ViT-B-32 model. Here we include analogous radar plots for the ViT-B-16 model in Figure C.1 and the ViT-L-14 model in Figure C.2. The analyses of these models reveal findings consistent with those reported for ViT-B-32 in the main text.

C.4.2 Extended analysis

Whitening vs. SVD

As we have seen in Section 6.3.2, applying a whitening transformation to the matrices of task singular vectors is mathematically equivalent to solving the Orthogonal Procrustes problem.


$$\Lambda^{-\frac{1}{2}} = \text{diag} \left(\frac{1}{\sqrt{|\lambda_i|} + \epsilon} \right)$$

Impact of rank

Extended ablation study

1. **Baseline Task Arithmetic:** the standard TA method without any modification.
2. **Low-Rank Approximation:** apply only low-rank approximation to task matrices without any interference reduction step.

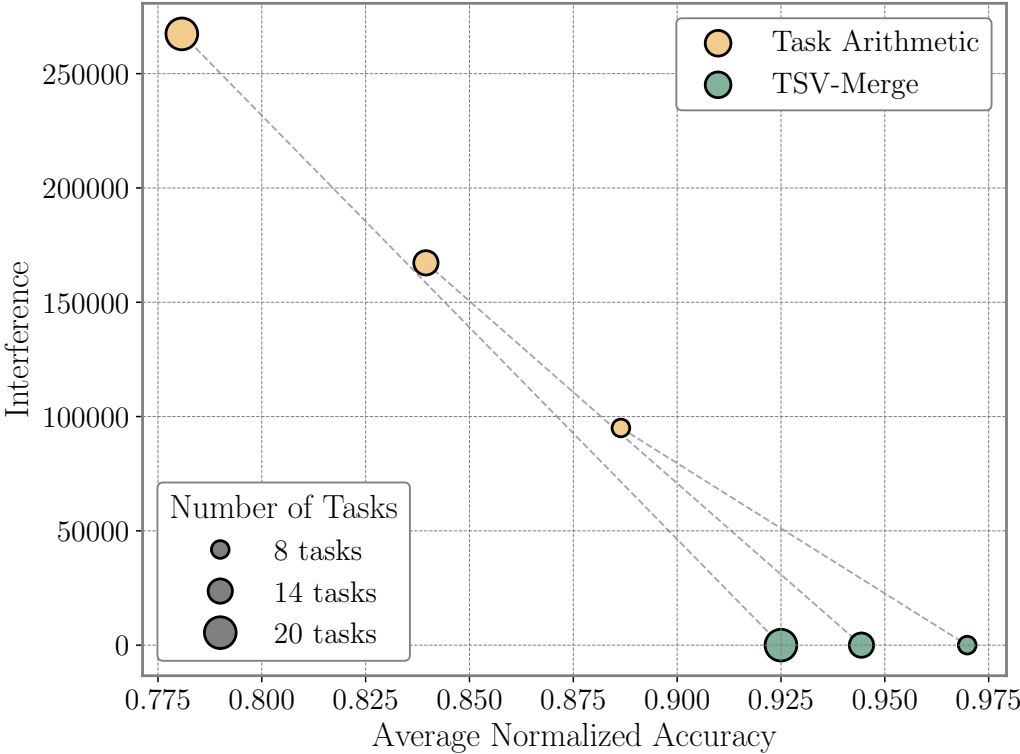


FIGURE C.7: Singular Task Interference (STI) and average normalized accuracy for Task Arithmetic and TSV-Merge on the ViT-L-14 model, evaluated across merges of 8, 14, and 20 tasks.

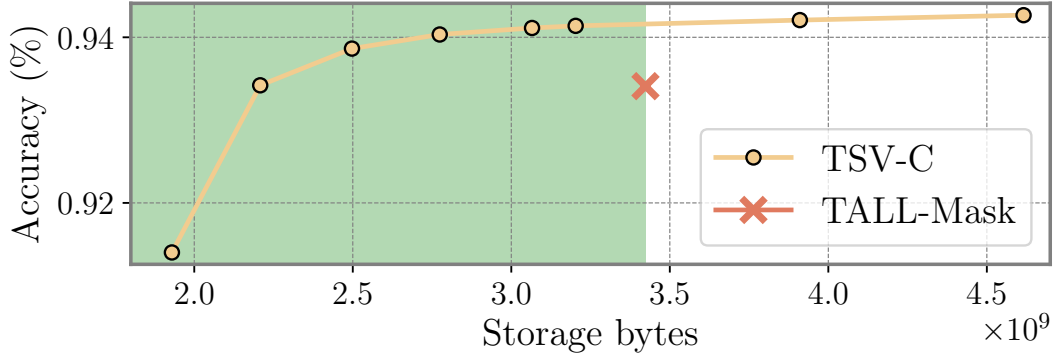


FIGURE C.8: Accuracy with varying compression budgets for ViT-L-14 across 14 tasks.

3. **Interference Reduction:** apply interference reduction to the full-rank task matrices without any pre-step of low-rank approximation.
4. **TSV-Merge:** Combining both low-rank approximation and interference reduction.

Effect of task interference

We provide here the same plots shown in Figure 6.8, for the ViT-B-16 we show it in Figure C.5 and respectively for ViT-L-14 in Figure C.7. The finding remains valid for these models as well: in all instances, a significant gain in accuracy is observed when the interference is removed.

Detailed per-layer task interference

We show in fig. C.6 the per-layer task interference, extending the block-level analysis in Figure 6.6 in the main manuscript.

Compression analysis

Our experimental results (see Table 6.2 in main manuscript) demonstrate that TSV outperforms TALL-Mask on the large-scale ViT-L-14 model for 14 and 20 tasks benchmarks, signaling a scaling advantage. With a fixed-budget analysis, we show in Figure C.8 that unlike TALL-Mask, which has a fixed requirement defined by model size and number of tasks, we allow flexible compression rates by allowing rank selection. This enables more aggressive compression, as highlighted in the green region in the figure.

Test-time adaptation

We compare our method with AdaMerging [180] for test-time adaptation. On a subset of 7 tasks from the 8 task benchmark, AdaMerging achieves an accuracy of 85.43%, while our TSV-M attains 88.93%, an improvement of approximately 3.5% without requiring any test-time adaptation. Additionally, when integrating an AdaMerging-style test-time adaptation into our framework, the accuracy increases to 89.87%, demonstrating the complementary benefits of combining TSV-M with test-time adaptation techniques.

C.5 Theoretical motivations and analysis

C.5.1 Theoretical foundation – empirical design

The TSV-C method is grounded in the well-established framework of low-rank approximation for compression (e.g., [43]). Instead, TSV-M is motivated by more empirical foundations: it is designed to achieve noise reduction through low-rank approximation and to eliminate interference via orthogonalization. Low-rank truncation serves to filter out insignificant variations, while orthogonalization ensures that task-specific singular vectors remain independent, preserving individual task performance.

C.5.2 Heuristic interference measure

Given that a formal definition of interference in model merging is not yet established, we adopt an operational definition: interference is any cross-task interaction that hinders the merging process. Our proposed Singular Task Interference measure is empirically validated by the consistent performance improvements observed when its value is minimized. Furthermore, we examine the relationship between overlapping singular vectors and knowledge sharing. Unlike multi-task learning (MTL), which enables coordinated knowledge sharing through joint training, the independent task-wise fine-tuning in model merging may evolve in destructive overlaps in the activations, resulting in interference rather than beneficial knowledge sharing. By orthogonalizing the singular vectors, our approach effectively mitigates these overlaps, reducing interference and enhancing the performance of the merged model.

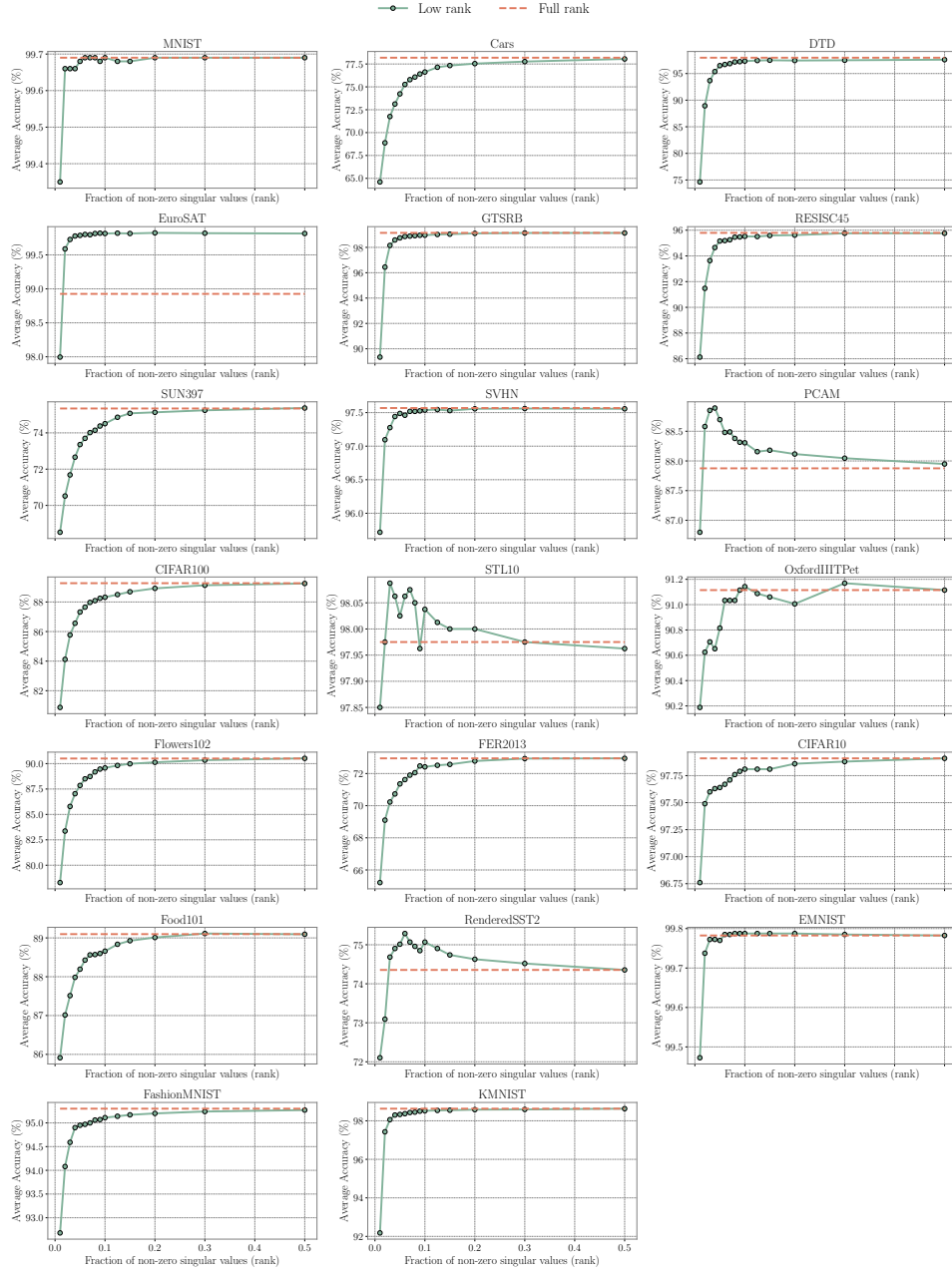


FIGURE C.9: Absolute accuracy of the ViT-B-32 model across increasing fractions of retained singular components, for each task. The red line represents the accuracy of the original fine-tuned models with full-rank task matrices, while the green line shows the accuracies using low-rank approximations.

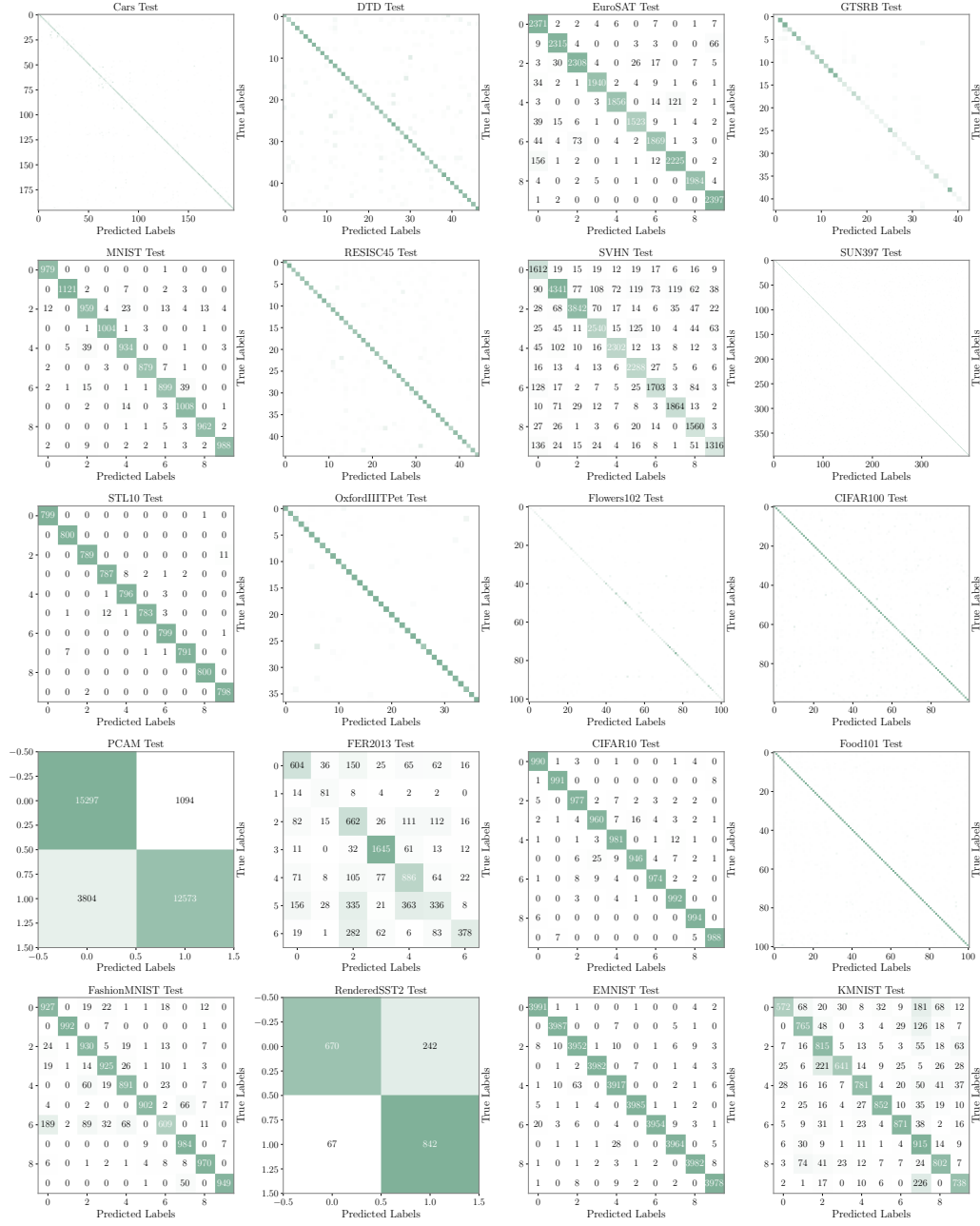


FIGURE C.10: Breakdown of classification accuracy in confusion matrices of a single merged ViT-L-14 model over 20 tasks. The numbers are omitted when they are too small to display.

Appendix D

Additional material on Chapter 7

D.1 Extended related work

D.1.1 Model merging

Model Merging has recently gained traction as a computationally efficient alternative to ensembling. Early methods, motivated by linear mode connectivity [50, 53, 59, 113], primarily aligned models trained with different optimization seeds. This was achieved by finding neuron permutations matching these ones before the aggregation [2, 65, 70, 85, 116, 144, 149] and chapter 3. More recent merging approaches aggregate instead multiple fine-tuned models derived from a shared pre-trained backbone [35, 39, 74, 78, 107, 122, 150, 157, 166, 172, 176, 179, 183, 191], chapter 6 and chapter 8. These methods incorporate various strategies to improve the merging, such as finding the optimal combination of task vectors [180], mitigating sign disagreement [176], randomly dropping a fraction of the updates [183], or employing evolutionary strategies [3] and chapter 8. The most recent line of work considers task vectors at a layer level, accounting this way for the natural structure of the layers and significantly improving the merging outcome [36, 150] and chapter 6. Our approach builds upon the latter methodologies by introducing adaptivity through input-driven routing, significantly narrowing the performance gap between the fine-tuned endpoints and their resulting merge.

D.1.2 MoErging

Model Merging with Mixture-of-Experts, often referred to as “MoErging” [11, 22, 24, 68, 81, 104, 115, 123, 151, 155, 156, 177, 188], explores how independently trained experts—potentially contributed by a decentralized community—can be combined within a single adaptive model by dynamically selecting which expert(s) should handle a given input. In the LLM domain, specialized modules (e.g., LoRAs or adapters) are merged via parametric or data-driven routers that match the incoming prompt to the most relevant module [11, 22, 24, 81, 115, 123, 156, 188]; some approaches, such as PHATGOOSE [115], require fine-tuning additional routing parameters, whereas others, like weight-ensembling MoE [156], may need to train the router at test time. Sharing a similar two-pass pipeline, *Transformer*² [151] modifies the fine-tuning procedure to yield task-aligned singular vectors, allowing for expert routing at test time. Differently from the latter, MASS works on any independently fine-tuned models, requiring no ad hoc training routines. A similar strategy was independently introduced by SMILE [155], which, like our method, enables data-free merging of multiple experts. However, SMILE leaves the pre-trained backbone intact and selectively *adds* task-specific low-rank updates at inference, whereas MASS begins with a single model containing all updates and *deactivates* any irrelevant subspaces via a router. The fact that both lines of work arrived at a similar subspace-activation concept, despite differing motivations, highlights the versatility and broad appeal of such an approach. Lastly, *TwinMerging* [104] similarly merges a shared expert and multiple task-specific experts via a gating function. However, *TwinMerging* relies on flat task arithmetic [78] and requires per-task labeled data to train its router, reducing its applicability. In contrast, MASS operates in a fully data-free and training-free regime by design.

D.2 Additional details

We here describe the details required to implement and reproduce our results. The code is publicly available. In particular, section D.2.2 describes implementation details, section D.2.4 specifies the employed evaluation metrics, section D.2.5 reports the employed architecture and section D.3.1 specifies the hyperparameters and how they were chosen.

	Method	ViT-B-32			ViT-B-16			ViT-L-14		
		8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks
MoE	WeMoE	5.06	8.06	11.05	5.12	8.16	11.20	5.78	9.31	12.84
	SMILE-1	1.61	2.07	2.52	1.62	2.09	2.55	1.47	1.82	2.18
	SMILE-2	3.05	4.60	6.14	3.09	4.67	6.24	2.59	3.78	4.96
	MASS	2.00	2.00	2.00	2.00	2.00	2.00	2.00	2.00	2.00

TABLE D.1: Relative parameter increase with respect to the base model.

D.2.1 Storage and compute overhead

The $\sim 2\times$ **compute** figure is an approximation: we run the backbone twice (fixed TSV-M (chapter 6) pass + routed pass) and the router itself adds $< 1\%$ FLOPs. Since model-merging saves training and storage rather than inference time, this modest overhead is acceptable. MASS routes per sample, so each incurs this cost; however, batching over the same task can reduce it. For example, with 32 samples, routing costs only $1.06\times$ forward equivalents. The **storage overhead** is exactly $2\times$: alongside the merged model, we store $1/T$ TSVs per task. Summed across tasks and layers, these form a second full set of weights, effectively doubling storage. A full comparison with the overhead incurred by the baselines is provided in table D.1.

D.2.2 Implementation

We used the same model checkpoints as `Consensus TA` [166], except for the one for the EMNIST dataset which we had to re-finetune due to an inconsistency in image orientation between EMNIST and MNIST. Shortly, the `torchvision`¹ version yields rotated and flipped images, spuriously yielding extremely similar models (same classes, roughly same dataset statistics) that performed very poorly when interchanged. Simply re-rotating and flipping the EMNIST images to match the orientation of MNIST solves the issue. We further used a single classification head for STL10 and CIFAR10 due to their 9 shared classes. The final head has the shared classes plus the two dataset-specific ones, *i.e.* monkey and frog.

D.2.3 Benchmarks and datasets

The 8-task benchmark, introduced in [78], comprises the following datasets: CARS, DTD, EURO-SAT, GTSRB, MNIST, RESISC45, SUN397, and SVHN. Moving to 14 tasks, we add CIFAR100, STL10, FLOWERS102, OXFORDIIITPET, PCAM, and FER2013. The 20-task suite further includes EMNIST, CIFAR10, FOOD101, FASHIONMNIST, RENDEREDSST2, and KMNIST. We provide the specific dataset details in table D.2.

D.2.4 Evaluation measures

To account for differences in task difficulty, we report both *absolute* and *normalized* accuracy in our results. The normalized accuracy serves as a relative performance measure by comparing the multi-task model’s accuracy to that of individual fine-tuned models. It is computed as:

$$\text{Normalized Accuracy} = \frac{1}{T} \sum_{i=1}^T \frac{\text{accuracy}(\theta_{\text{MT}}, t_i)}{\text{accuracy}(\theta_{ft_i}, t_i)} \quad (\text{D.1})$$

where T represents the total number of tasks, θ_{MT} is the multi-task model, and θ_{ft_i} corresponds to the fine-tuned model for task t_i . By normalizing accuracy in this way, we ensure a fairer comparison that accounts for variations in baseline task performance.

¹<https://pytorch.org/vision/stable/index.html>

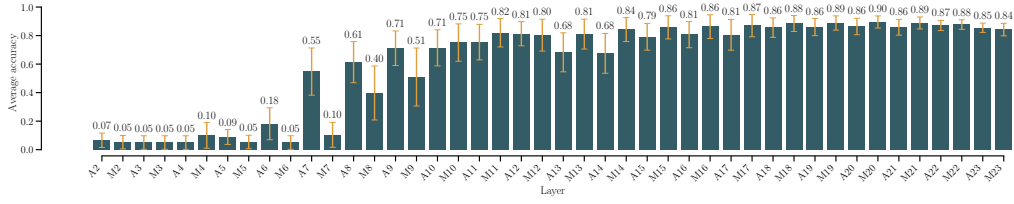


FIGURE D.1: ViT-L-14 per-layer task accuracies.

D.2.5 Architectures

We use CLIP from the *OpenClip* library², using the three different versions described in table D.3. For the router, we use a small two-layer MLP with a hidden dimension of 1024. It accepts a 512-dimensional embedding vector, applies a linear transformation, a ReLU activation, and dropout with a probability of 0.5, and outputs logits corresponding to task selection probabilities.

D.2.6 Adapting oracle MoErging methods

As extensively discussed, we argue that MoErging methods should not rely on an oracle head, since their pipelines implicitly assume that the task label is unknown at inference time (otherwise, merging the correct task vector would be trivial). To respect this assumption, we introduced a routing procedure that exploits the routers already implemented in each method. For SMILE, we extracted the mode of the tokens at each layer and applied a naïve majority-voting scheme across layers to select the head for each sample, which was then used for the final classification. Analogously, for WeMoE, we applied the same logic, implementing a straightforward head-selection strategy that leverages the existing gates and coefficients. All results reported for MoE methods were obtained under this unified setting.

Radar charts on 8- and 14-task benchmarks. In Section 7.3, we present comprehensive results for the approach using the 20 task benchmark. Figure D.3 and Figure D.4 respectively display the normalized accuracies for our method on 8 and 14 tasks across all three model sizes (ViT-B-32, ViT-B-16, and ViT-L-14). In both cases, the approach retains a high fraction of each fine-tuned model’s performance, with normalized accuracies often above 80–90%. Notably, the method scales gracefully as the number of tasks increases from 8 to 14.

D.3 Additional experiments and results

In this section, we provide detailed per-task and per-layer accuracy plots, along with further examples of decoded task vectors, to complement the results presented in the main paper.

D.3.1 Hyperparameter settings

Following the recommendation of chapter 6, we use α as a single scaling factor with the suggested value of 1.0 for the TSV-M merging configurations in both algorithm 9 and algorithm 4. Consistent with TSV, the compression rate assigned to each task space is set to $\frac{1}{T}$. We optimized the similarity threshold ε over the range $\{0.1, 0.2, \dots, 0.9\}$ and determined the router’s selection threshold η via a Bayesian search within the interval $[0.05, 0.5]$. As illustrated in Figures D.1 and 7.4a, we identify the optimal projection layer by focusing on

²https://github.com/mlfoundations/open_clip/

Algorithm 9 Fixed Merging Step

Require: Pretrained model weights θ_{pre} , task-specific updates $\{\Delta_i\}_{i=1}^T$, user-specified threshold ε

Ensure: Fixed merged model weights θ_{MT}

```

1: Accounting for redundant directions (section 7.2.2)
2:  $\mathcal{M} = \{\}$ 
3: for  $i = 1, \dots, T$  do
4:    $\delta_i \leftarrow \text{vec}(\Delta_i)$ 
5:   if  $\max_{j \in \mathcal{M}} \text{sim}(\delta_i, \delta_j) < \varepsilon$  then
6:      $\mathcal{M} \leftarrow \mathcal{M} \cup \{i\}$ 
7:   end if
8: end for
9: Merging step using TSV-M (chapter 6) on the  $\{\Delta_i\}_{i \in \mathcal{M}}$ 
10: for  $i \in \mathcal{M}$  do
11:    $\Delta_i = U_i \Sigma_i V_i^\top$ 
12:    $\tilde{U}_i \leftarrow U_{i[:, 1:k]}, \tilde{\Sigma}_i \leftarrow \Sigma_{i[1:k, 1:k]}, \tilde{V}_i \leftarrow V_{i[:, 1:k]}$ 
13: end for
14:  $U \leftarrow [\tilde{U}_1 \mid \tilde{U}_2 \mid \dots \mid \tilde{U}_T]$ 
15:  $\Sigma \leftarrow \text{block\_diag}(\tilde{\Sigma}_1, \tilde{\Sigma}_2, \dots, \tilde{\Sigma}_T)$ 
16:  $V \leftarrow [\tilde{V}_1 \mid \tilde{V}_2 \mid \dots \mid \tilde{V}_T]$ 
17:  $U_\perp \leftarrow \text{orthogonalize}(U)$ 
18:  $V_\perp \leftarrow \text{orthogonalize}(V)$ 
19:  $\hat{\Delta} \leftarrow U_\perp \Sigma V_\perp^\top$ 
20:  $\theta_{\text{MT}} \leftarrow \theta_{\text{pre}} + \alpha \hat{\Delta}$ 
21: return  $\theta_{\text{MT}}$ 

```

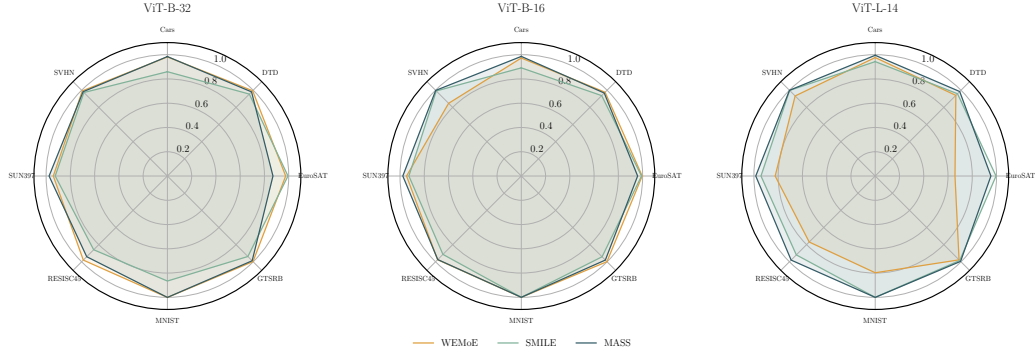


FIGURE D.3: Normalized task accuracies over models ViT-B-32, ViT-B-16 and ViT-L-14 for the 8 tasks benchmark.

those revealing the highest task accuracy. Specifically, for ViT-B-32 and ViT-B-16 models, we select the attention and MLP layers within the range $\{7, \dots, 11\}$, while for the ViT-L-14 model, the chosen layers fall in the range $\{19, \dots, 23\}$. The temperature parameter for tuning the behavior of the softmax function at line 6 in algorithm 4 is set to 1.

Sensitivity to Hyperparameters The merging coefficient is set to 1 as in TSV-M (chapter 6). Figure D.2 shows how accuracy changes with routing threshold η and top- K . At low η (0.05) and large K , too many tasks are merged, causing interference. At high η (≥ 0.3), only the top task is selected, making performance insensitive to K (it is effectively an argmax). The best accuracy occurs in a broad middle range, peaking at $\eta = 0.2$, $K = 3$, where the router balances selectivity and coverage. We also vary the cosine threshold ε used to discard similar task updates before merging. Due to the high dimensionality of the Δ s, large thresholds ($\varepsilon \geq 0.4$) retain all updates, leaving redundancy unaddressed (accuracy 93.5). Small ones ($\varepsilon \leq 0.05$) instead remove even distinct directions, significantly harming accuracy (≤ 88.6). Intermediate values ($\varepsilon \approx 0.2$) offer a robust filtering, improving performance (≥ 93.9) by suppressing redundancy.

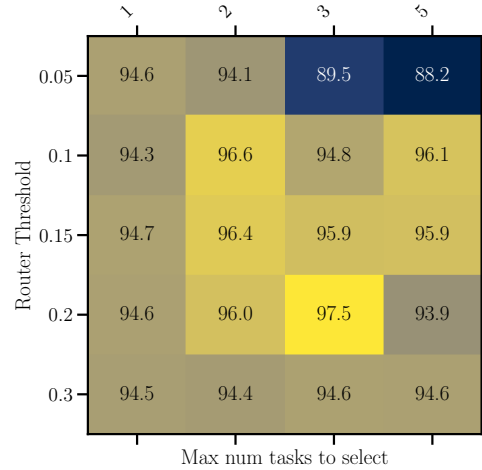


FIGURE D.2: Hyperparameter sensitivity.

Layer-wise accuracies for individual datasets. Figures D.5 and D.6 show per-layer accuracies for ViT-B-32 on different subsets of the 8-task benchmark:

- Figure D.5 focuses on CARS, DTD, EUROSAT, and GTSRB.
- Figure D.6 displays results for MNIST, RESISC45, SUN397, and SVHN.

Again, the top-performing layer is not shared across the tasks, confirming what we observed in the main paper.

Layer-wise accuracies for ViT-L-14. Figure D.1 reports average per-layer task accuracy for the larger ViT-L-14, showing that in this case the most predictive layer for routing

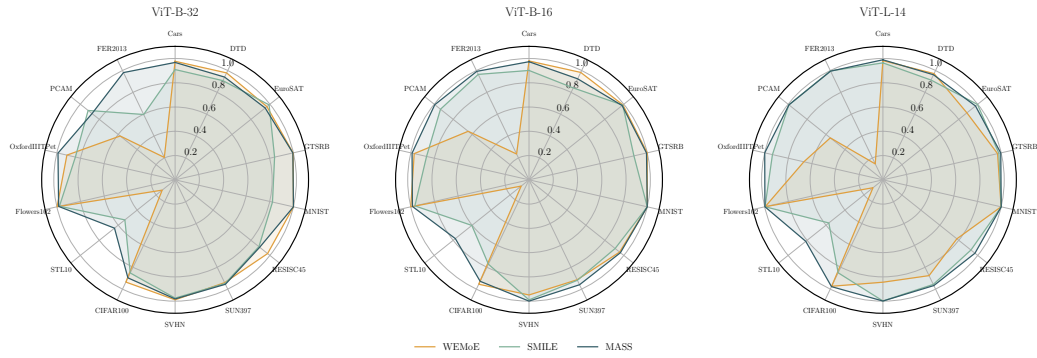
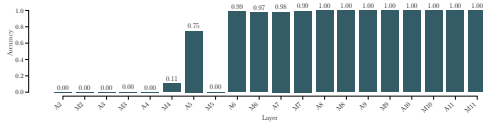
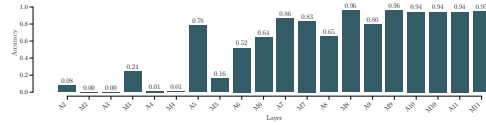


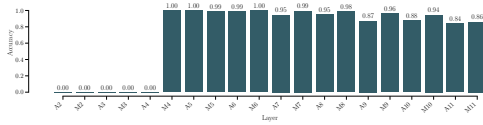
FIGURE D.4: Normalized task accuracies over models ViT-B-32, ViT-B-16 and ViT-L-14 for the 14 tasks benchmark.



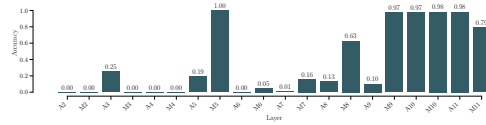
(A) CARS.



(B) DTD.

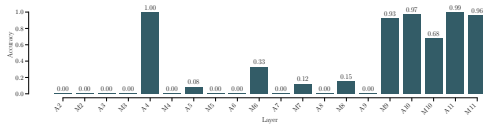


(C) EUROSAT.

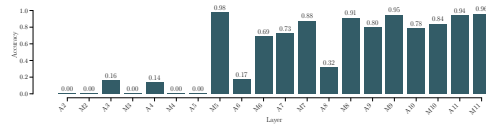


(D) GTSRB.

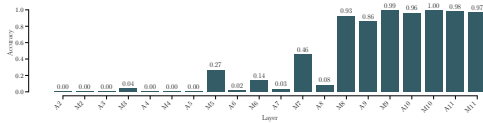
FIGURE D.5: Per-layer task accuracies for ViT-B-32 on CARS, DTD, EUROSAT, and GTSRB.



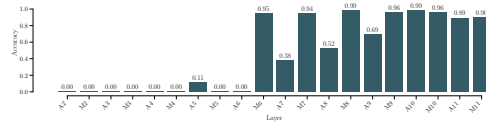
(A) MNIST.



(B) RESISC45.



(C) SUN397.



(D) SVHN.

FIGURE D.6: Per-layer task accuracies for ViT-B-32 on MNIST, RESISC45, SUN397, and SVHN.

is $\ell = 20$. Since ViT-L-14 has 24 layers (compared to the 12 layers in ViT-B-32 and ViT-B-16), the most predictive layer is roughly at the same relative depth.

Visualizations for additional datasets. Following the approach in section 7.3.4, Figure D.7 shows examples of decoded task vectors for datasets like SVHN, GTSRB, SUN397, and RESISC45. Here, we see textual prompts such as “An image of the number 4” or “Aerial view of an industrial area”, which align with each dataset’s distinct domain. This reaffirms that our singular vectors capture domain-specific transformations while preserving high-level semantic alignment to the pre-trained model.



FIGURE D.7: Captions obtained by decoding task singular vectors as text for datasets SVHN, GTSRB, SUN397, and RESISC45 as described in section 7.3.4, accompanied by three representative images for each dataset.

D.4 Propositions and proofs

Proposition D.4.1 (Optimality of Orthogonal Projection). *Let $V \in \mathbb{R}^{d \times k}$ have orthonormal columns spanning a subspace $\mathcal{S} \subseteq \mathbb{R}^d$, and let $\mathbf{a} \in \mathbb{R}^d$. Then the unique minimizer of $\|\mathbf{a} - \mathbf{w}\|_2^2$ over all $\mathbf{w} \in \mathcal{S}$ is*

$$\hat{\mathbf{w}} = V V^\top \mathbf{a}.$$

Proof. Any $\mathbf{w} \in \mathcal{S}$ can be written as $V \boldsymbol{\alpha}$ for some $\boldsymbol{\alpha} \in \mathbb{R}^k$. The problem

$$\min_{\mathbf{w} \in \mathcal{S}} \|\mathbf{a} - \mathbf{w}\|_2^2 \iff \min_{\boldsymbol{\alpha} \in \mathbb{R}^k} \|\mathbf{a} - V \boldsymbol{\alpha}\|_2^2$$

has a strictly convex objective, so its global minimizer is found by setting the gradient to zero. A short calculation shows

$$\boldsymbol{\alpha} = V^\top \mathbf{a} \implies \hat{\mathbf{w}} = V (V^\top \mathbf{a}) = V V^\top \mathbf{a}.$$

Uniqueness follows from the strict convexity, and $\|\mathbf{a} - \widehat{\mathbf{w}}\|_2$ is necessarily the smallest possible distance in $\mathcal{S}$. Equivalently, $\mathbf{a} - \widehat{\mathbf{w}}$ is orthogonal to $\mathcal{S}$, so no further reduction in norm is possible. $\square$

Proposition D.4.2 (§7.2.1). *Let $z_\ell \in \mathbb{R}^d$ be a feature vector, and for each task i , decompose it as*

$$z_\ell = V_i V_i^\top z_\ell + \varepsilon_i, \quad \varepsilon_i = (I - V_i V_i^\top) z_\ell.$$

Assume $\varepsilon_i \sim \mathcal{N}(0, \sigma^2 I)$. Then the maximum a posteriori estimate of the task reduces to

$$\hat{i}_{\text{MAP}} = \arg \max_i p(\text{task} = i \mid z_\ell) = \arg \min_i \|\varepsilon_i\|_2^2.$$

Thus, under these assumptions, selecting the task with the smallest squared Euclidean residual is exactly equivalent to maximizing the posterior.

Proof. By assumption, $p(\varepsilon_i) = (2\pi\sigma^2)^{-d/2} \exp(-\|\varepsilon_i\|_2^2/(2\sigma^2))$, so $-\log p(\varepsilon_i) \propto \|\varepsilon_i\|_2^2$. Since $V_i V_i^\top z_\ell$ is a deterministic shift, the likelihood $p(z_\ell \mid \text{task} = i)$ depends only on ε_i . With a uniform prior over tasks,

$$\hat{i}_{\text{MAP}} = \arg \max_i p(z_\ell \mid \text{task} = i) = \arg \max_i p(\varepsilon_i) = \arg \min_i \|\varepsilon_i\|_2^2.$$

Hence, minimizing the ℓ_2 residual is exactly equivalent to maximizing the posterior. $\square$

Dataset	image size	# train	# val	# test
CARS [92]	varies	7330	814	8041
DTD [26]	varies	1692	188	1880
EUROSAT [69]	64×64	21600	2700	2700
GTSRB [148]	varies	23976	2664	12630
MNIST [97]	28×28	55000	5000	10000
RESISC45 [23]	256×256	17010	1890	6300
SUN397 [175]	varies	17865	1985	19850
SVHN [117]	32×32	68257	5000	26032
CIFAR100 [93]	32×32	45000	5000	10000
STL10 [29]	96×96	4500	500	8000
FLOWERS102 [121]	varies	918	102	6149
OXFORDIIITPET [126]	varies	3312	368	3669
PCAM [164]	96×96	257144	5000	32768
FER2013 [63]	48×48	25839	2870	7178
EMNIST [31]	28×28	235000	5000	40000
CIFAR10 [93]	32×32	45000	5000	10000
FOOD101 [19]	512×512	70750	5000	25250
FASHIONMNIST [174]	28×28	55000	5000	10000
RENDEREDSST2 [146]	varies	6228	692	1821
KMNIST [27]	28×28	55000	5000	10000

TABLE D.2: Image sizes, and numbers of train, validation, and test samples for the considered datasets.

Model	Layers	Hidden Dimension	Heads	Patch Size	Parameters
ViT-B-32	12	768	12	32×32	$\sim 86\text{M}$
ViT-B-16	12	768	12	16×16	$\sim 86\text{M}$
ViT-L-14	24	1024	16	14×14	$\sim 307\text{M}$

TABLE D.3: Comparison of ViT-B-32, ViT-B-16, and ViT-L-14 architectures.

MASS +	ViT-B-32			ViT-B-16		
	8 tasks	14 tasks	20 tasks	8 tasks	14 tasks	20 tasks
nn	92.7	89.6	89.4	92.7	90.0	90.2
mlp	96.8	95.8	95.8	97.1	94.8	96.5
proj _{PRE}	98.2	88.6	79.4	98.7	92.0	81.2
proj _{TSV-M}	96.5	93.2	90.9	98.0	96.1	88.7

TABLE D.4: Average normalized accuracy for different routers.

Appendix E

Additional material on Chapter 8

E.1 Mergenetic

Each experiment was run using a library developed specifically for this work, which will be released as open-source software, called *Mergenetic* [112]. This library allows for defining a merging problem as either a single-objective or multi-objective optimization problem, where one only needs to specify the merging method, a fitness function, and select the hyperparameters for a chosen evolutionary algorithm.

The implementation relies on *Mergekit* [61] for merging the models, *Pymoo* [17] for optimizing the objective function through evolutionary algorithms, and *Lm-Evaluation-Harness* [57] for implementing some of the fitness functions. In table E.1 we outline the supported merging methods, while in table E.2 we outline the currently available evolutionary algorithms.

We believe this library is a significant contribution as it facilitates evolutionary model merging and aligns well with the chapter’s approach, which aims to reduce computational burden. It can be a valuable tool for the community and for users interested in cross-lingual transfer or creating multilingual models for target low-resource languages.

E.2 Additional Details

This section provides additional implementation and experimental details that were not included in the main paper.

E.2.1 IRT Fitting Details

As previously stated, we used the implementation from Polo et al. [129] and adopted their configuration settings. Specifically, we used $\gamma_m \sim N(\mu_\gamma \mathbf{1}_d, 1/u_\gamma I_d)$, $a_i \sim N(\mu_a \mathbf{1}_d, 1/u_a I_d)$,

TABLE E.1: Overview of supported merging methods in Mergenetic.

Method	Multi-Model	Uses Base Model
Linear (Model Soups)	Yes	No
SLERP	No	Yes
Task Arithmetic	Yes	Yes
TIES	Yes	Yes
DARE (TIES)	Yes	Yes
DARE (Task Arithmetic)	Yes	Yes

TABLE E.2: Overview of supported Pymoo's evolutionary algorithms in Mergenic.

Algorithm	Class	Objective(s)	Constraints
Genetic Algorithm	GA	single	x
Differential Evolution	DE	single	x
Biased Random Key GA	BRKGA	single	x
Nelder Mead	NelderMead	single	x
Pattern Search	PatternSearch	single	x
CMAES	CMAES	single	
Evolutionary Strategy	ES	single	
SRES	SRES	single	x
ISRES	ISRES	single	x
NSGA-II	NSGA2	multi	x
R-NSGA-II	RNSGA2	multi	x
NSGA-III	NSGA3	many	x
U-NSGA-III	UNSGA3	many	x
R-NSGA-III	RNSGA3	many	x
MOEAD	MOEAD	many	
AGE-MOEA	AGEMOEA	many	
C-TAEA	CTAEA	many	x
SMS-EMOA	SMS-EMOA	many	x
RVEA	RVEA	many	x

and $\beta_i \sim N(\mu_\beta, 1/u_\beta)$. Following Polo et al. [129], we also applied (hyper)priors to the prior parameters using the software for fitting hierarchical Bayesian models [96]: $\mu_\gamma \sim N(0, 10)$, $u_\gamma \sim \Gamma(1, 1)$, $\mu_a \sim N(0, 10)$, $u_a \sim \Gamma(1, 1)$, $\mu_\beta \sim N(0, 10)$, and $u_\beta \sim \Gamma(1, 1)$. For both the model and example-specific parameters γ_m , a_i , and β_i , we take their point estimates as the means of their respective variational distributions. The γ model dimensionality is set to 15 following the parameter choice suggested by Polo et al. [129].

TABLE E.3: Mistral-based models with shortened column headers and names. Role can be either E, M or B, referring to endpoint, merge or base model respectively. Spec refers instead to specialization, with *math*, *ger*, *ita*, *jpn*, *dut* and *gen* referring to Math, German, Italian, Japanese, Dutch and General respectively. We finally have the author and model ID as per the Huggingface.

Role	Spec	Author	Model
E	<i>math</i>	<i>upaya07</i>	ARITHMO2-MISTRAL-7B
E	<i>math, jp</i>	<i>SakanaAI</i>	EvoLLM-JP-v1-7B
E	<i>math</i>	<i>GAIR</i>	ABEL-7B-002
E	<i>math</i>	<i>meta-math</i>	METAMATH-MISTRAL-7B
B	<i>gen</i>	<i>mistralai</i>	MISTRAL-7B-v0.1
E	<i>ger</i>	<i>jphme</i>	EM_GERMAN_MISTRAL_v01
E	<i>ger</i>	<i>LeoLM</i>	LEO-MISTRAL-HESSIANAI-7B
E	<i>ita</i>	<i>DeepMount00</i>	MISTRAL-ITA-7B
E	<i>jpn</i>	<i>augmxnt</i>	SHISA-GAMMA-7B-v1
E	<i>dut</i>	<i>BramVanroy</i>	GEITJE-7B-ULTRA
E	<i>ro</i>	<i>OpenLLM-Ro</i>	RoMISTRAL-7B-INSTRUCT
M	<i>gen</i>	<i>chlee10</i>	T3Q-MERGE-MISTRAL7B
E	<i>gen</i>	<i>liminerity</i>	M7-7B
E	<i>gen</i>	<i>yam-peleg</i>	EXPERIMENT26-7B
M	<i>gen</i>	<i>PracticeLLM</i>	SOLAR-TAIL-10.7B-MERGE-v1.0
E	<i>gen</i>	<i>upstage</i>	SOLAR-10.7B-v1.0
E	<i>gen</i>	<i>Yhyu13</i>	LMCOCKTAIL-10.7B-v1
M	<i>gen</i>	<i>FuseAI</i>	FUSECHAT-7B-SLERP
M	<i>gen</i>	<i>FuseAI</i>	FUSECHAT-7B-TA
E	<i>gen</i>	<i>FuseAI</i>	OPENCHAT-3.5-7B-MIXTRAL
E	<i>gen</i>	<i>FuseAI</i>	OPENCHAT-3.5-7B-SOLAR
M	<i>gen</i>	<i>jan-hq</i>	SUPERMARIO-SLERP-v3
E	<i>gen</i>	<i>jan-hq</i>	SUPERMARIO-SLERP-v2
E	<i>gen</i>	<i>jan-hq</i>	SUPERMARIO-v2
M	<i>gen</i>	<i>superlazycoder</i>	NEURALPIPE-7B-SLERP
E	<i>gen</i>	<i>OpenPipe</i>	MISTRAL-FT-OPTIMIZED-1218
E	<i>gen</i>	<i>mlabonne</i>	NEURALHERMES-2.5-MISTRAL-7B

E.2.2 Experimental Details

Models. One key assumption of model merging is that the endpoint models lie within the same basin [78]. This means that merging arbitrary models is not feasible; rather, **all models involved must be fine-tuned versions of the same base model**. To satisfy this requirement, we selected several fine-tuned models from the Hugging Face Hub that originated from the same base model. Specifically, we focused on models fine-tuned starting from MISTRAL-7B [82], following common best practices in the community [3]. Table E.3 lists all the models used in our experiments, along with their corresponding names on the Hugging Face Hub. A total of 27 models were considered for our experiments.

Number of models. The initialization step, shared across all evolutionary algorithms, involves sampling merging configurations (e.g., interpolation coefficients) and applying them to merge the endpoint models. Consequently, MERGE³ requires the same number of models as any standard merging approach.

In the rest of the section, we provide further details for reproducing the experiments in section 8.4.2 and section 8.4.4 of the main paper.

Cross-lingual transfer

In the cross-lingual transfer evolutionary merging, we evolved four merged models with mathematical capabilities in different languages: Japanese, Romanian, German, and Dutch. In each of these experiments, we deployed an ad-hoc genetic algorithm for single-objective optimization. We employed the Simulated Binary Crossover [41] operator to generate offspring solutions by combining parent solutions. To maintain diversity and explore the search space, we applied Polynomial Mutation [41], which introduces small perturbations to offspring solutions and enhances the algorithm’s ability to escape local optima. This combination of SBX and PM effectively balances exploration and exploitation, facilitating efficient convergence toward optimal solutions.

Furthermore, guided by empirical tests, we decided to deploy SLERP to evolve solutions for the Romanian and Dutch problems, while we used a combination of TIES and DARE for the Japanese and the German ones. We deployed four different sizes of the fitness datasets for Japanese, namely 20, 30, 50, and 100, in order to obtain a more detailed analysis of the method for comparison with the work of [3]. On the other hand, we kept the fitness dataset size fixed to 20 for all other aforementioned experiments. The fitness dataset was extracted from the test set of GSM8K, and we used the remaining, non-overlapping samples as test set for evaluating the model. To get the language-specific versions of GSM8K, we used UNBABEL/TOWERINSTRUCT-7B-V0.2 [4] to translate the datasets. In each experiment, the population size was fixed to 25 and the number of iterations to 7.

To check the correctness of the solution, following Akiba et al. [3], we used a regex to extract the last numerical value returned in the model’s answer and compare it with the ground truth. The solution is also checked to be in the correct language with the language identifier from fastText [86].

The mathematical models used in combination with TIES and DARE were ABEL-7B-002 and ARITHMO2-MISTRAL-7B, whereas we used METAMATH-MISTRAL-7B in combination with SLERP. Moreover, we employed the following language-specialized models: SHISA-GAMMA-7B-V1, EM_GERMAN_MISTRAL_V01, GEITJE-7B-ULTRA, and ROMISTRAL-7B-INSTRUCT. More information about these models can be found in table E.3.

Lastly, we evaluated EVOLLM-JP-v1-7B [3] under the same conditions as MERGE³ to assess its accuracy, following the prompting structure outlined by Akiba et al. [3].

Multi-lingual transfer

In this experiment, we tackle the ARC dataset in multiple languages (Italian, Dutch, German, and English)¹ [159] using a multi-objective evolutionary merging procedure based this time on NSGA-II [40]. We configure the population size to 25 and the number of evolutionary iterations to 7. We deployed a combination of TIES and DARE as merging strategy. As in previous settings, both the fitness function and the test metrics operate by extracting the final model-generated choice via a regex, but this time they look for an instance from the set {A, B, C, D} rather than a number. On top of this, we employed a dataset composed by 20 datapoints for each language from the relative translation of ARC to compute the fitness, and we extracted the test set as for the previous experiments. Furthermore, unlike the single-objective approach described earlier, here we explicitly optimize multiple objectives simultaneously. This time, the employed models are MISTRAL-ITA-7B, GEITJE-7B-ULTRA, LEO-MISTRAL-HESSIANAI-7B, and the base model MISTRAL-7B-V0.1.

Ability and Performance Estimator

In these experiments (reported in section 8.4.1 and section 8.4.1) we used the test set of the standard version of GSM8K, HELLASWAG, ARC, WINOGRANDE, and TRUTHFULQA. Furthermore, we used 6 different models to test the different performance of the ability and performance estimator: SOLAR-TAIL-10.7B-MERGE-V1.0, FUSECHAT-7B-SLERP, NEURALPIPE-7B-SLERP, T3Q-MERGE-MISTRAL7B, FUSECHAT-7B-TA, and SUPERMARIO-SLERP-V3. These models were chosen as already available on the Open LLM Leaderboard.

E.3 Additional Experiments

We report here additional experiments and analyses.

E.3.1 Extract step

In the extract step outlined in section 8.3.1, random sampling has been proposed as the main method to subsample the dataset $\bar{D} \subset D$. While we explored various dataset subsampling strategies, we ultimately opted for uniform random sampling, as our experiments showed that more complex approaches offered no significant advantage over this simpler method. In this section we report some of the experiments behind this decision and the two alternative methods tried in the extraction step: IRT Clustering (IRT), introduced by Polo et al. [129], and a custom Representation Clustering (RC) method.

IRT clustering

Given a dataset D and the parameter of a fitted IRT model a and β , one can define a low-dimensional embedding of each datapoint $i \in D$ by $E_i = [a_i || \beta_i]$. Therefore, IRT-clustering obtains a representative subset by first obtaining a clustering over this embedding space through K -Means, and then choosing the points closest to the centroids as representative samples.

Representation clustering

Let $\{m_j\}_{j=1}^M$ be the set of endpoint models, and let $D = \{x_i\}_{i=1}^N$ be our full dataset. For each sample x_i , we first encode it into a high-dimensional vector by concatenating model-specific

¹We used the dataset on the Hugging Face Hub from openGPT-X/arcx

TABLE E.4: Notation used in chapter 8.

Notation	Description
D	Full dataset.
$\bar{D}$	Reduced subset of the dataset.
D_i	Subdataset for task i .
γ_m	Latent abilities of model m .
Γ_m	True latent abilities of model m .
$\gamma_m^{\{\text{p,gp}\}-\text{IRT}}$	Latent abilities of model m via P-IRT ability estimator.
$\gamma_m^{\{\text{mp,gmp}\}-\text{IRT}}$	Latent abilities of model m via MP-IRT ability estimator.
a_i, β_i	IRT parameters related to dataset item i .
ξ	Interpolation coefficients for latent abilities.
$\hat{\xi}, \hat{\gamma}, \hat{a}, \hat{\beta}$	MLE of the aforementioned parameters.
$p_{i,m}$	IRT model for datapoint i and model m .
$\hat{p}_{i,m}$	IRT model for datapoint i and model m parametrized by MLE estimators of a, β, γ, ξ .
$\tilde{m}$	Merged language model.
$Y_{i,m}$	Sample-level correctness of model m for example i .
$\hat{Z}^{\text{MP-IRT}}$	Merged performance estimator MP-IRT.
$\hat{Z}^{\text{GMP-IRT}}$	Generalized merged performance estimator GMP-IRT.
$F(m)$	Fitness value of a model m .
θ	Parameters being optimized in evolutionary search.
$P_{\bar{F}_D}$	Pareto front defined by function's set $\bar{F}$ and data D
θ^*	Global optimum on D .
$\hat{\theta}$	Global optimum on $\bar{D}$.
N	Number of samples in the dataset.

embeddings. Concretely, we compute:

$$E_{i,j} = \frac{1}{T_i} \sum_{t=1}^{T_i} E_{i,j,t} \in \mathbb{R}^d,$$

where $E_{i,j,t}$ is the embedding of the t -th token of sample x_i under model m_j , and T_i is the number of tokens in x_i . We form the concatenated representation:

$$E_i = [E_{i,1} \| E_{i,2} \| \cdots \| E_{i,M}] \in \mathbb{R}^{M \cdot d}.$$

Since E_i can be very high-dimensional, we apply Principal Component Analysis (PCA) to project E_i onto a lower-dimensional space:

$$\tilde{E}_i = \text{PCA}_k(E_i) \in \mathbb{R}^k, \quad k \ll M \cdot d.$$

Next, we apply k -means clustering to the reduced embeddings $\{\tilde{E}_i\}_{i=1}^N$:

$$\min_{\{\mathbf{c}_k\}_{k=1}^K} \sum_{i=1}^N \min_{1 \leq k \leq K} \|\tilde{E}_i - \mathbf{c}_k\|^2,$$

where $\mathbf{c}_k$ is the centroid of the k -th cluster. This partitions the dataset into K clusters, each capturing a distinct region of the representation space. From each cluster k , we select the representative sample $x_{i_k^*}$ whose embedding $\tilde{E}_{i_k^*}$ is closest to the centroid $\mathbf{c}_k$:

$$i_k^* = \arg \min_{x_i \in C_k} \|\tilde{E}_i - \mathbf{c}_k\|,$$

where C_k is the set of samples assigned to cluster k . To approximate the full-dataset metrics from the selected subset $\bar{D} = \{x_{i_k^*}\}_{k=1}^K$, we assign a weight to each representative sample. Since the size of the cluster C_k indicates how prevalent that region of representation space is, we define $w_{i_k^*} = \frac{|C_k|}{|D|}$. These weights ensure that the contribution of each representative sample to the overall metric reflects the true proportion of samples that it represents in the original dataset. By evaluating a new model m only on $\bar{D}$ and using $\{w_{i_k^*}\}$ to calculate a weighted average, we approximate m 's performance on the full dataset D at a fraction of the computational cost.

A schematic overview of the full process is outlined in algorithm 10.

Experiments

To compare the performance of the Sample Extractors, we followed a procedure similar to that described in section 8.4.1, computing the absolute estimation error for each extractor. For random sampling, the accuracy estimator was obtained via uniform averaging, whereas for IRT and RC it was obtained via weighted averaging. We evaluated the estimator in two different settings: (1) merging a math model with a language-tuned model (similar to the cross-lingual setting of section 8.4.2) for several languages (Italian, German, Romanian, Dutch) and testing the extractor on the corresponding translations of GSM8K (see fig. E.1), and (2) merging several math models and testing the extractor on the English version of GSM8K (see fig. E.2).

Focusing on fig. E.1, we see that performance variability is somewhat higher (larger error bars) due to different language-specific datasets. Even so, Random sampling never falls behind IRT or RC, especially for small sample sizes. By the time the subset size reaches 50 or more examples, all three methods converge to comparable accuracy-error levels, underscoring the robustness of Random sampling. Instead, in fig. E.2, the trends are broadly similar for RC and Random sampling, while slightly worse for IRT. Again, as the dataset sample size grows, overall error drops and the gap among methods narrows.

Algorithm 10 Representation Clustering Extractor**Require:** Dataset D , Endpoint Models $m_1, \dots, m_n$, Desired subset size K **Ensure:** Subset of size K with weights w_i

```

1: for  $i$  in  $D$  do
2:    $E_i \leftarrow []$ 
3:   for  $m$  in  $\{m_1, \dots, m_n\}$  do
4:      $E_{im} \leftarrow \text{embed } i \text{ with model } m$ 
5:      $E_i \leftarrow E_i \cup E_{im}$ 
6:   end for
7: end for
8:  $\{\mathbf{E}_i\}_{i \in D} \leftarrow \text{PCA}(\{\mathbf{E}_i\}_{i \in D})$ 
9: Apply k-means clustering to  $\{\mathbf{E}_i\}_{i \in D}$ , obtaining  $K$  centroids  $\{\mathbf{c}_k\}_{k=1}^K$ 
10: For each cluster  $k$ , select the closest example  $i_k^* = \arg \min_{i \in D} \|\mathbf{E}_i - \mathbf{c}_k\|_2$ 
11: Let  $C_k = \{i \in D \mid \arg \min_{c \in \{\mathbf{c}_k\}_{k=1}^K} \|\mathbf{E}_i - c\|_2 = \mathbf{c}_k\}$  be the set of examples in cluster  $k$ 
12: Assign weights  $w_{i_k^*} = \frac{|C_k|}{|D|}$  for  $k = 1, \dots, K$ 
13: return  $\{i_k^*, w_{i_k^*}\}_{k=1}^K$ 

```

To sum up, the Random sampler can sometimes lag slightly behind the more sophisticated IRT and RC. Nevertheless, neither of these methods has a clear advantage over the others. Given its simplicity and negligible overhead, the Random strategy stands out as a highly practical choice for dataset subsampling, especially when the marginal improvements of more complex methods do not clearly justify their added complexity.

E.3.2 Estimation step**Additional experiment for ability estimator**

We report in fig. E.3 the Euclidean distance between the estimated and ground-truth ability vectors across different sample sizes. The results are consistent with the case $n = 10, 20$ seen in fig. 8.3, with our estimated ability vector being significantly closer to the ground-truth one compared to the ability vector estimated by pIRT and gp-IRT. Similarly, we report the corresponding cosine similarity in fig. E.4, confirming much higher similarity in our case.

Additional experiment for performance estimator

We report in fig. E.6 the evaluation of performance estimators across WINOGRANDE and HELLASWAG, extending the results in fig. 8.2.

Hyperparameter analysis for performance estimator

We now analyse the optimal choice of the scalar c required in GMP-IRT and GP-IRT (see eq. (8.5)). In the experiments reported in the main paper (section 8.4.1) and above (section E.3.2), we used as a heuristic $c = \frac{1}{2}$. Despite its empirical effectiveness, this uniform interpolation may not be optimal across all model pairs and data regimes. Therefore, we introduce a grid-search-based strategy to estimate an improved value of c and empirically validate its potential to reduce estimation error.

Methodology. We propose a two-step approach to selecting the optimal interpolation coefficient c for use in the GMP-IRT estimator. The procedure is as follows:

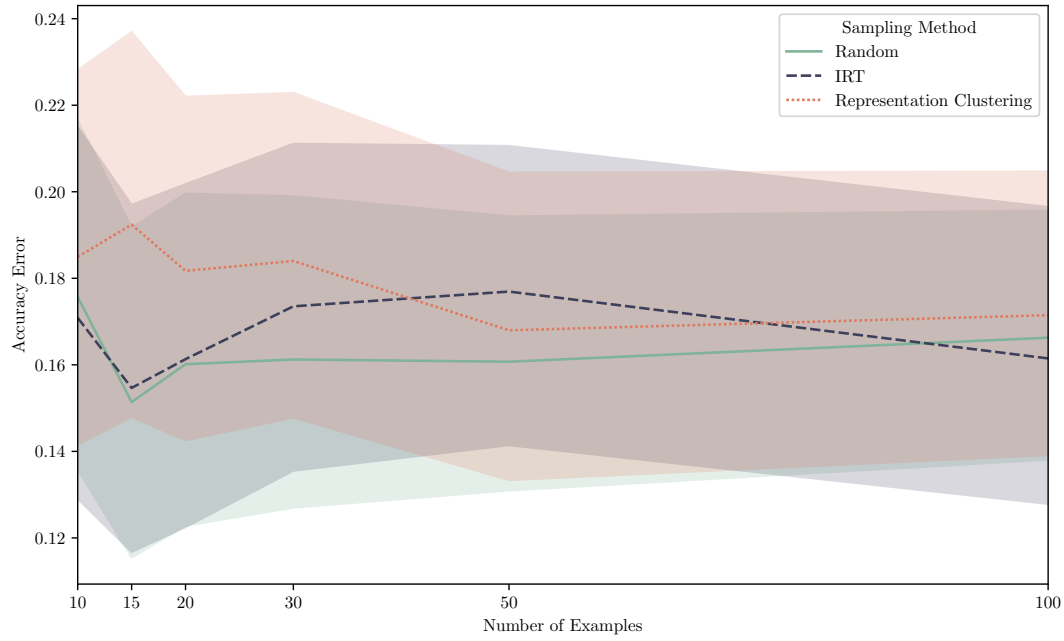


FIGURE E.1: Absolute error of the estimated accuracy of Sample Extractors, averaged across merges of language-specific and English Math fine-tunings of MISTRAL-7B-v0.1, evaluated on translations of GSM8K and presented as a function of the number of dataset samples.

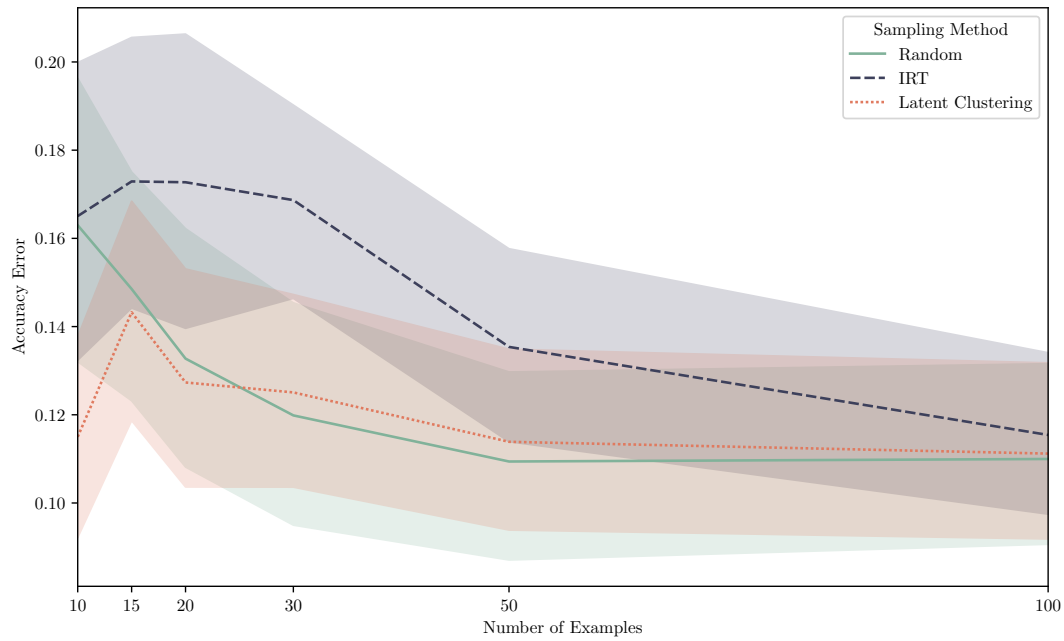


FIGURE E.2: Absolute error of the estimated accuracy of Sample Extractors, averaged across merges of English Math models based on MISTRAL-7B-v0.1, evaluated on GSM8K and presented as a function of the dataset sample size.

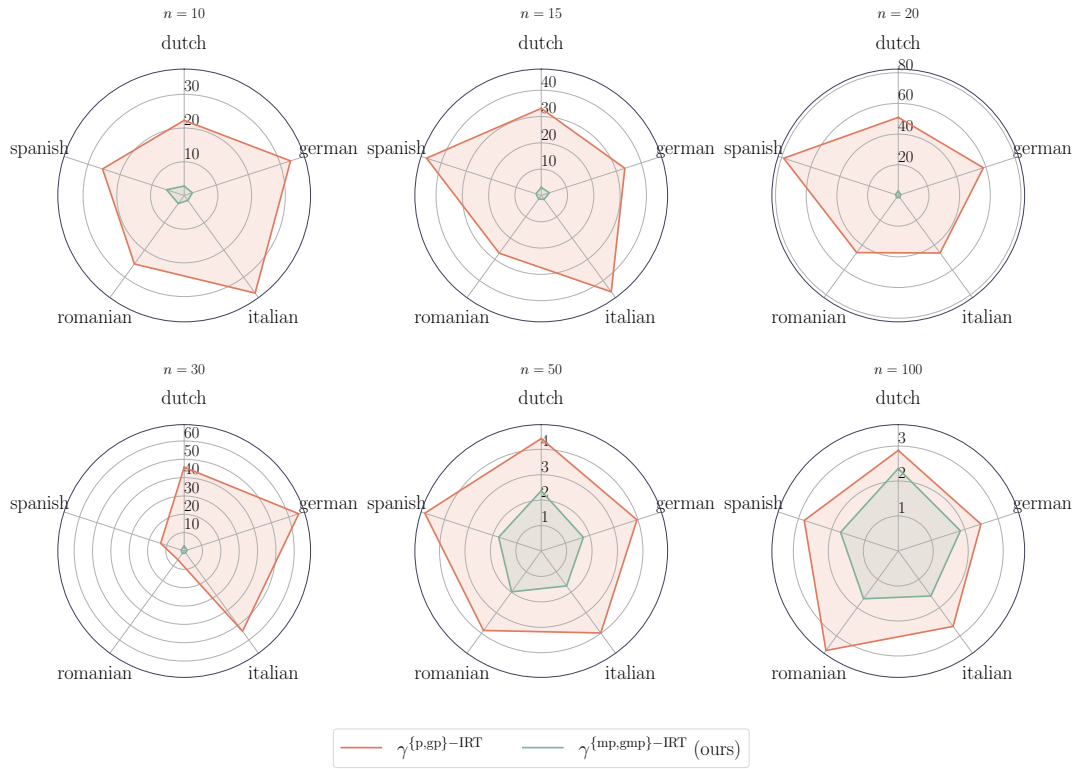


FIGURE E.3: Euclidean distance (lower is better) between estimated and true abilities for different languages.

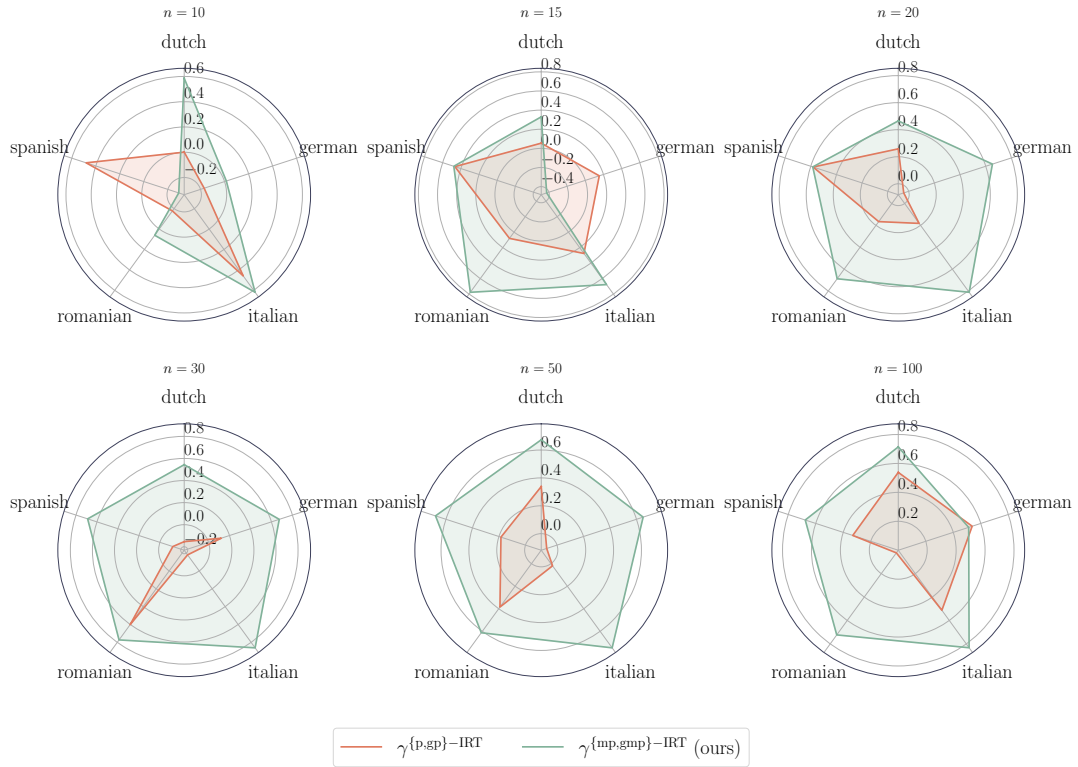


FIGURE E.4: Cosine similarity (higher is better) between estimated and true abilities for different languages.

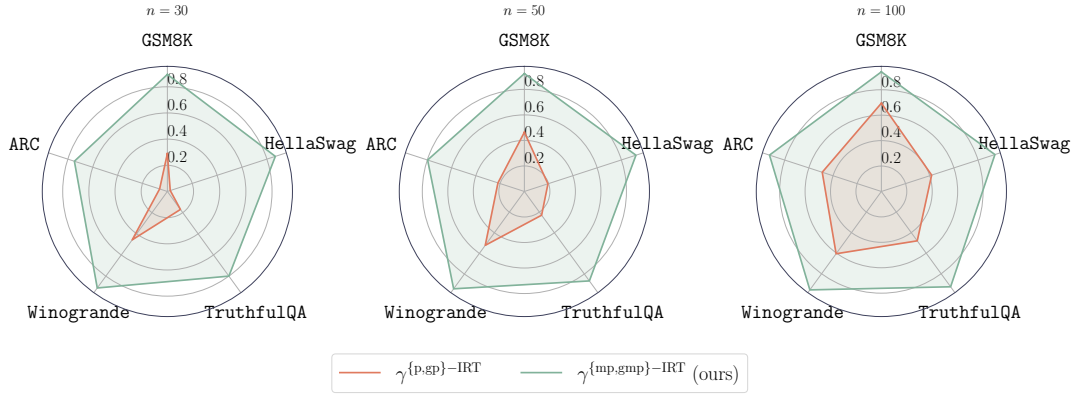


FIGURE E.5: Cosine similarity (higher is better) between estimated and true abilities for different tasks.

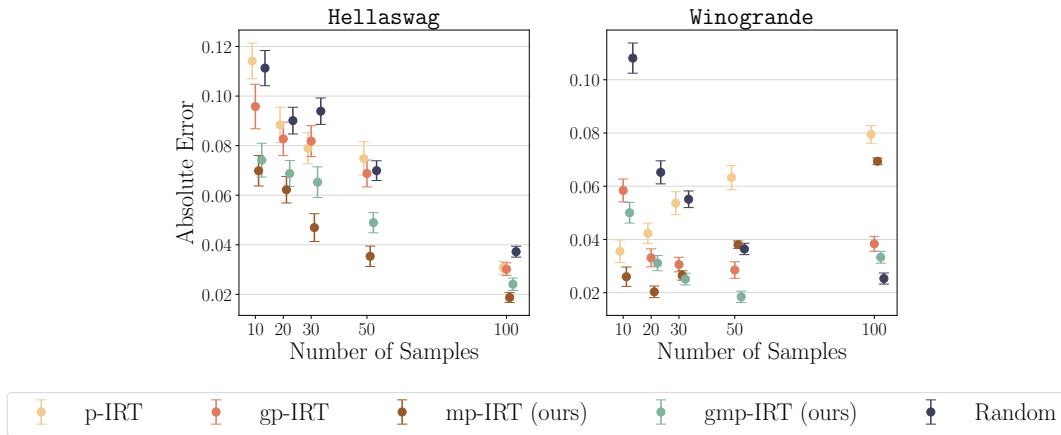


FIGURE E.6: Absolute error of various estimators as a function of sample size (lower is better). gmp-IRT consistently achieves lower error.

1. **Optimizing c for Endpoint Models** For a given dataset $\bar{D}$ ², we find a $c \in [0, 1]$ by minimizing the absolute estimation error of GP-IRT on each individual model. This yields a set of optimal coefficients $\{c_1, \dots, c_M\}$, one for each endpoint model M .
2. **Averaging c for the Merged Model.** To obtain a suitable interpolation coefficient for GMP-IRT, we compute the average of the optimal values across the endpoint models, $\bar{c} = \frac{1}{M} \sum_{m=1}^M c_m$, and use this as the parameter for the merged model’s GMP-IRT.

Results Discussion We evaluate the absolute estimation error across five benchmarks, comparing the adaptive averaging strategy for c (denoted by $\star$) to the baseline fixed heuristic $c = \frac{1}{2}$. Results for GMP-IRT $\star$ and GP-IRT $\star$, along with the baseline models GMP-IRT, GP-IRT, MP-IRT, and P-IRT, are reported in table E.5.

TABLE E.5: Absolute estimation error across datasets. The $\star$ symbol indicates the adaptive c strategy described above. Lower is better.

Dataset	GMP-IRT $\star$	GMP-IRT	GP-IRT $\star$	GP-IRT	MP-IRT
ARC	0.035	0.040	0.046	0.049	0.048
Winogrande	0.018	0.031	0.032	0.037	0.036
GSM8K	0.057	0.057	0.074	0.064	0.062
HellaSwag	0.046	0.056	0.077	0.071	0.047
TruthfulQA	0.040	0.045	0.062	0.055	0.044

Across most datasets, the adaptive coefficient strategy yields consistent improvements in both GMP-IRT and GP-IRT, with the largest gains observed on Winogrande and HellaSwag. Notably, GMP-IRT with a tuned c performs on par or better than any other method across all benchmarks.

Spearman correlation

Following White et al. [171], we report the Spearman correlation between the ground-truth ranking and the ranking induced by each estimator’s predictions. We compare the best merging-specific estimator, GMP-IRT, against the strongest vanilla baseline, GP-IRT. The results, averaged across dataset sizes and types, are shown in fig. E.7. Notably, GMP-IRT achieves the highest correlation in each setting, further underscoring the benefits of using estimators specifically designed for the model merging context.

E.3.3 Evolve step

Comparison with in-context learning

A natural question when evaluating merging-based methods is how their performance compares to inference-time adaptation strategies such as *In-Context Learning* (ICL). In particular, given access to a small validation set, one might ask whether directly providing these examples as input context at evaluation time can match the performance achieved through evolutionary merging.

To investigate this, we evaluate a 20-shot ICL setup in the multilingual transfer setting introduced in section 8.4.4. Prompts are constructed using 20 validation examples and

²Such a dataset is available because it relies solely on the correctness of the endpoint models’ answers, rather than on the unknown answers of the merged model.

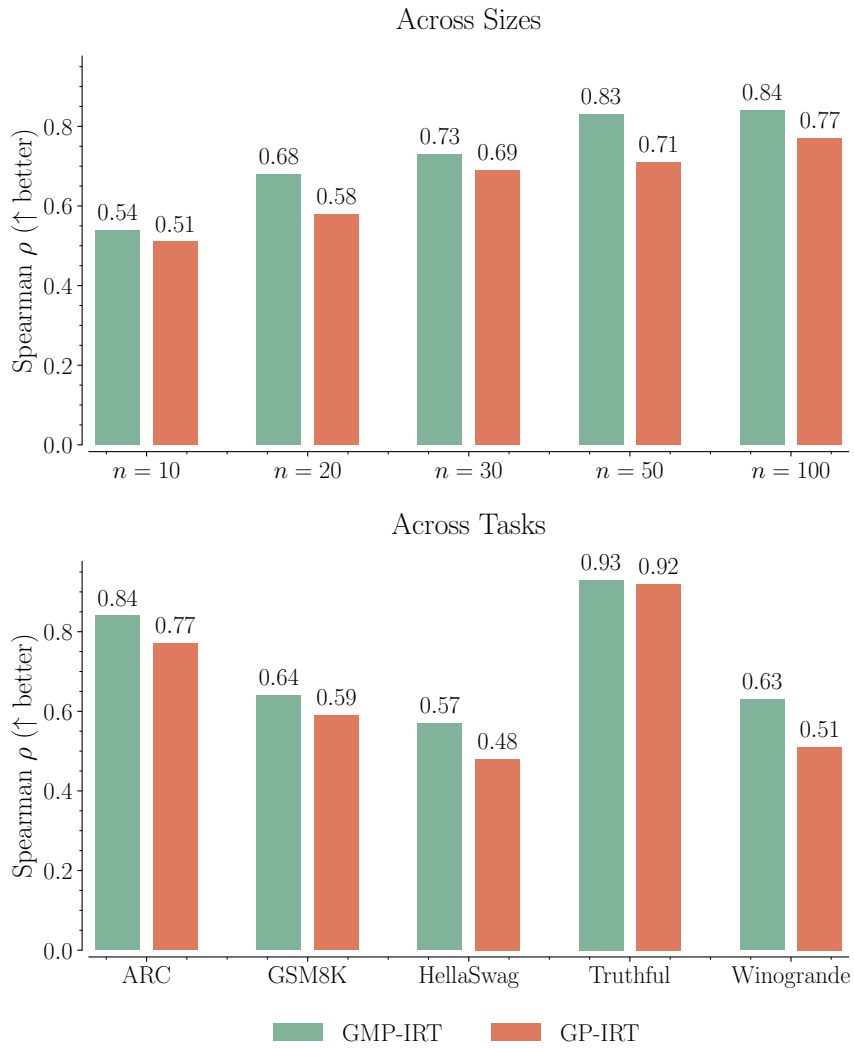


FIGURE E.7: Spearman rank correlations using the section E.3.2 setup. Higher is better. GMP-IRT shows substantially higher correlation than GP-IRT.

prepended to the input at inference time. We apply this setup to two merging baselines, TIES-DARE and Task Arithmetic, and compare the results against our method using GMP-IRT with a fitness dataset of size 20. Across all languages, MERGE³ significantly

TABLE E.6: Accuracy on multilingual ARC using few-shot ICL (20-shot) versus MERGE³ (GMP-IRT-20).

Method	DE	IT	NL	EN
TIES-DARE Few-shot (20)	0.227	0.226	0.227	0.226
Task Arithmetic Few-shot (20)	0.427	0.406	0.491	0.566
MERGE ³ (GMP-IRT-20)	0.720	0.690	0.690	0.790

outperforms the ICL-augmented baselines. While ICL can provide moderate improvements over the original models, it increases inference-time memory usage and latency due to the expanded context. In contrast, the merged models produced by our method operate without additional overhead and are immediately deployable as standalone networks.

Analyzing negative transfer

While our main analysis focused on cross-lingual transfer of MERGE³ (section 8.4.2), we did not explicitly examine the phenomenon of *negative transfer*; that is, cases where merging degrades performance on specific inputs. In this subsection, we formally define negative transfer in the context of multiple-choice questions (MCQs) and introduce a framework for measuring its prevalence in merged multilingual models. We then present an analysis of negative transfer in GSM8K.

Methodology We consider a multiple-choice question (MCQ) evaluation setup, where knowledge is operationalized as a model’s ability to correctly answer a question. Let $m_1, m_2, \dots, m_K$ denote the set of K endpoint models, and let $\tilde{m}$ represent the merged model resulting from their combination. Following our earlier notation (see table E.4), correctness for a given sample i is defined as a binary variable:

- $Y_{i,m_j} \in \{0, 1\}$: indicates whether endpoint model m_j answers sample i correctly,
- $Y_{i,\tilde{m}} \in \{0, 1\}$: indicates whether the merged model answers sample i correctly.

We define negative transfer on example i as occurring when at least one of the base models answers correctly, but the merged model fails:

$$\exists j \in \{1, \dots, K\} \text{ such that } Y_{i,m_j} = 1 \text{ and } Y_{i,\tilde{m}} = 0.$$

To track this, we introduce a binary indicator variable n_i for each input:

$$n_i = \begin{cases} 1, & \text{if } (\exists j : Y_{i,m_j} = 1) \text{ and } Y_{i,\tilde{m}} = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Finally, we compute the *Negative Transfer Rate (NTR)* as the proportion of examples exhibiting negative transfer among those for which at least one base model answered correctly:

$$\text{NTR} = \frac{\sum_{i=1}^N n_i}{\sum_{i=1}^N \mathbf{1} \{ \exists j \in \{1, \dots, K\} : Y_{i,m_j} = 1 \}}.$$

This metric provides a task-level perspective on the potential degradation introduced by merging and complements the aggregate performance measures reported in the main paper.

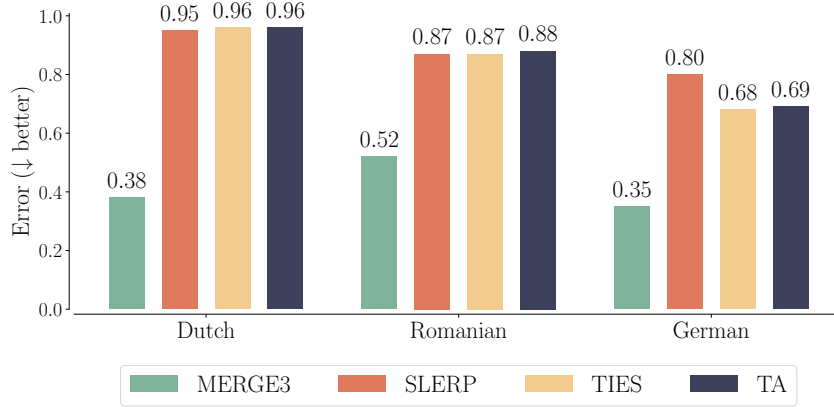


FIGURE E.8: Negative Transfer Rate across languages. Lower is better. MERGE³ shows substantially less negative transfer than standard baselines.

Results Discussion We compute the NTR using the same experimental setting described in section 8.4.2. As shown in fig. E.8, MERGE³ consistently yields substantially lower negative transfer than SLERP, TIES, and Task Arithmetic across all languages. This indicates that MERGE³ not only improves average accuracy but also preserves correct knowledge from its component models, thereby maintaining per-example competence during merging.

Time & FLOPs requirements for evolutionary merging

Hardware Setting. To compare the efficiency of different model evaluation strategies, we measured the time required to evolve merged LLM models using a single NVIDIA 4090 with 24GB of VRAM, and report the Throughput R in table E.8. We also benchmark evaluation and merging times across three GPU models (3090, 4090, V100) to illustrate practical runtimes for MERGE³ on both modern and older hardware. We report the results in table E.9

TABLE E.7: Comparison of Evolve methods by number of trials, estimated total time on a single NVIDIA 4090, sample size used for Fitness computation, and final accuracy on GSM8K. The number of trials is the result of population size $\times$ iterations, parameters of the genetic algorithms of each method, and represents the total number of merged models evaluated during the entire Evolve run.

Method	N_{models}	Estimated total time	Sample size	Accuracy
EvoLLM-JP-7B	1000	62 days	1000	0.49
MERGE ₁₀₀ ³	175	21h	100	0.42
MERGE ₅₀ ³	175	12h 20m	50	0.38
MERGE ₃₀ ³	175	10h 30m	30	0.38
MERGE ₂₀ ³	175	10h 15m	20	0.34

Results Discussion. Over a 12-hour period, we were able to evaluate 8 models on 1000 samples of GSM8K with a single NVIDIA 4090, allowing us to estimate that evaluating 1000 models would take approximately 62 days under similar conditions. In contrast, MERGE³ enabled the evaluation of a larger number of merged models in significantly less time by using a reduced dataset. These results suggest that researchers and practitioners could leverage

consumer-grade GPUs for efficient LLM merging and evaluation, making rapid experimentation of model merging methods more accessible. We report in table E.7 the estimated total time of the Evolve runs, which we calculated using the following formula:

$$T(N_{\text{models}}) = \frac{N_{\text{models}}}{R_{\text{Dataset Size}}}$$

Lastly, table E.9 shows that MERGE³ maintains practical runtimes across a range of GPUs. While the 4090 offers the fastest evaluation, older hardware like the V100 still supports feasible experimentation, highlighting the framework’s accessibility and the generalizability of results across different consumer GPUs.

TABLE E.8: Throughput (R) in models per hour for different sample sizes per fitness evaluation on GSM8K. These estimates are based on 12-hour Evolve runs on a single NVIDIA 4090 with 24GB of VRAM.

Sample size	1000	100	50	30	20
Throughput (Models/Hour)	0.67	8.33	14.17	16.67	17.08

TABLE E.9: Evaluation and merge time across different GPU models using Mistral-7B on 10 examples (4-bit, SLERP).

GPU Model	Eval Time (s)	Merge Time (s)
NVIDIA 3090 24GB	65	135
NVIDIA 4090 24GB	45	160
NVIDIA V100 32GB	80	220

FLOPs Calculation. We provide a Jupyter Notebook that describes the FLOPs calculations for our experiments in the supplementary material, based on the *calc-flops* library³. This script has been used to estimate the FLOPs for the experiment in Figure 8.1.

E.4 Mathematical proofs

We outline in table E.4 a scheme of the notation used throughout this chapter.

E.4.1 Proof of Theorem 8.5.2

Proof. Let $m := F(\theta^*; D)$ and $\hat{m} := F(\hat{\theta}; \bar{D})$. We must show that $|m - \hat{m}| \leq \epsilon$.

1. By ϵ -stability, for all $\theta \in \Theta$:

$$|F(\theta; D) - F(\theta; \bar{D})| \leq \epsilon.$$

In particular, for $\theta = \theta^*$,

$$|F(\theta^*; D) - F(\theta^*; \bar{D})| \leq \epsilon.$$

³<https://github.com/MrYxJ/calculate-flops.pytorch>.

Hence

$$F(\theta^*; \bar{D}) \geq F(\theta^*; D) - \epsilon$$

and

$$F(\theta^*; \bar{D}) \leq F(\theta^*; D) + \epsilon.$$

2. Since $\hat{\theta}$ is the minimizer of $F(\cdot; \bar{D})$, we have

$$F(\hat{\theta}; \bar{D}) \leq F(\theta^*; \bar{D}).$$

Because θ^* is the minimizer of $F(\cdot; D)$,

$$F(\hat{\theta}; D) \geq F(\theta^*; D).$$

3. To bound $\hat{m} - m$, we can add and subtract $F(\theta^*; \bar{D})$ to have

$$\begin{aligned} \hat{m} - m &= \left(F(\hat{\theta}; \bar{D}) - F(\theta^*; \bar{D}) \right) \\ &\quad + \left(F(\theta^*; \bar{D}) - F(\theta^*; D) \right). \end{aligned}$$

The first term is ≤ 0 (since $\hat{\theta}$ is a minimizer on $\bar{D}$), and the second term is $\leq \epsilon$. Hence

$$\hat{m} - m \leq 0 + \epsilon = \epsilon.$$

4. Analogously, to bound $m - \hat{m}$, we can rewrite

$$\begin{aligned} m - \hat{m} &= \left(F(\theta^*; D) - F(\hat{\theta}; D) \right) \\ &\quad + \left(F(\hat{\theta}; D) - F(\hat{\theta}; \bar{D}) \right). \end{aligned}$$

The first term is ≤ 0 (since θ^* is a minimizer on D), and the second term is $\leq \epsilon$. Thus,

$$m - \hat{m} \leq 0 + \epsilon = \epsilon.$$

5. Combining these inequalities:

$$-\epsilon \leq \hat{m} - m \leq \epsilon \implies |m - \hat{m}| \leq \epsilon.$$

Hence $|F(\theta^*; D) - F(\hat{\theta}; \bar{D})| \leq \epsilon$, completing the proof.

□

E.4.2 Proof of Theorem 8.5.4

Proof. By hypothesis, for every $\theta \in \Theta$,

$$\mathbb{E}_{\bar{D}}[|F(\theta; D) - F(\theta; \bar{D})|] \leq \epsilon.$$

Using Jensen's inequality for the absolute value,

$$\begin{aligned} &|\mathbb{E}_{\bar{D}}[F(\theta; D) - F(\theta; \bar{D})]| \\ &\leq \mathbb{E}_{\bar{D}}[|F(\theta; D) - F(\theta; \bar{D})|] \leq \epsilon. \end{aligned}$$

Hence,

$$-\epsilon \leq \mathbb{E}_{\bar{D}}[F(\theta; \bar{D}) - F(\theta; D)] \leq \epsilon$$

for each fixed θ . It thus follows that

$$\mathbb{E}_{\bar{D}}[F(\theta; \bar{D})] \leq F(\theta; D) + \epsilon$$

and

$$\mathbb{E}_{\bar{D}}[F(\theta; \bar{D})] \geq F(\theta; D) - \epsilon.$$

Consequently,

$$\min_{\theta \in \Theta} \mathbb{E}_{\bar{D}}[F(\theta; \bar{D})] \leq \min_{\theta \in \Theta} [F(\theta; D) + \epsilon] = m^* + \epsilon,$$

where $m^* := \min_{\theta \in \Theta} F(\theta; D)$. Meanwhile, by a min-versus-expectation (Jensen-type) inequality,

$$\mathbb{E}_{\bar{D}}[\min_{\theta \in \Theta} F(\theta; \bar{D})] \geq \min_{\theta \in \Theta} \mathbb{E}_{\bar{D}}[F(\theta; \bar{D})].$$

Hence,

$$\begin{aligned} \mathbb{E}_{\bar{D}}[\hat{m}(\bar{D})] &= \mathbb{E}_{\bar{D}}[\min_{\theta \in \Theta} F(\theta; \bar{D})] \\ &\geq \min_{\theta \in \Theta} \mathbb{E}_{\bar{D}}[F(\theta; \bar{D})] \geq m^* - \epsilon. \end{aligned}$$

Combining these two bounds results in

$$m^* - \epsilon \leq \mathbb{E}_{\bar{D}}[\hat{m}(\bar{D})] \leq m^* + \epsilon$$

and, therefore,

$$\left| m^* - \mathbb{E}_{\bar{D}}[\hat{m}(\bar{D})] \right| \leq \epsilon.$$

□

E.4.3 Proof of Proposition 8.5.5

Proof. We must show that

$$|\mathbb{E}[\hat{Z}_{jl} \mid Y_{i_0l}, \dots, Y_{i_kl}] - \mathbb{E}[Z_{jl} \mid Y_{i_0l}, \dots, Y_{i_kl}]| \rightarrow 0$$

in probability as $|\hat{I}| \rightarrow \infty$. Under the assumptions of the proposition (including linear inheritance of abilities, $\hat{\xi} \rightarrow \xi$ in probability, and bounded $\|a_i\|$), we may bound this difference as follows:

$$\begin{aligned} &|\mathbb{E}[\hat{Z}_{jl} \mid Y_{i_0l}, \dots, Y_{i_kl}] - \mathbb{E}[Z_{jl} \mid Y_{i_0l}, \dots, Y_{i_kl}]| \\ &\leq \frac{1 - \hat{\xi}}{|I_j \setminus \hat{I}_j|} \sum_{i \in I_j \setminus \hat{I}_j} \left| \sigma((\hat{\xi}_1 \theta_{l_1} + \hat{\xi}_2 \theta_{l_2})^\top a_i - \beta_i) \right. \\ &\quad \left. - \sigma(\theta_{l_m}^\top a_i - \beta_i) \right|. \end{aligned}$$

Since σ is $1/4$ -Lipschitz on $\mathbb{R}$, we have

$$\begin{aligned} &\leq \frac{1}{|I_j|} \sum_{i \in \hat{I}_j} \left| ((\hat{\xi}_1 \theta_{l_1} + \hat{\xi}_2 \theta_{l_2}) - \theta_{l_m})^\top a_i \right| \\ &\leq \frac{1}{|I_j|} \sum_{i \in \hat{I}_j} \|a_i\|_2 \|(\hat{\xi}_1 \theta_{l_1} + \hat{\xi}_2 \theta_{l_2}) - \theta_{l_m}\|_2. \end{aligned}$$

Since $\sup_{i \in I_j} \|a_i\|_2 \leq c$, it follows that

$$\leq c \|(\hat{\xi}_1 - \xi_1) \theta_{l_1} + (\hat{\xi}_2 - \xi_2) \theta_{l_2}\|_2 \rightarrow 0$$

in probability as $|\hat{I}| \rightarrow \infty$. (The last step uses $\hat{\xi} \rightarrow \xi$ in probability, with $\theta_{l_1}, \theta_{l_2}$ fixed in $\mathbb{R}^d$.) Hence $\hat{Z}_{jl}$ converges in probability to Z_{jl} , completing the proof. $\square$

E.4.4 Proof of Theorem 8.5.2 via Proposition 8.5.5

Proof. By Proposition 8.5.5, $\hat{Z}^{\text{mp-IRT}}$ becomes arbitrarily close (in probability) to Z as $|\bar{D}| \rightarrow \infty$. Under standard regularity conditions, this implies

$$|Z(\theta; D) - \hat{Z}^{\text{mp-IRT}}(\theta; \bar{D})| \leq \epsilon$$

in expectation, for all sufficiently large $|\bar{D}|$, hence $\hat{Z}^{\text{mp-IRT}}$ is ϵ -stable in expectation. Applying Theorem 8.5.4 completes the argument. $\square$

Bibliography

- [1] Akshit Acharya, Tatiana Gaintseva, Mateo Mahaut, Pritish Chakraborty, Viktor Stenby Johansson, Melih Barsbey, Emanuele Rodolà, and Donato Crisostomi. Multi-way representation alignment, 2026. URL <https://arxiv.org/abs/2602.06205>.
- [2] Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha S. Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *Proc. of ICLR*. OpenReview.net, 2023. URL <https://openreview.net/pdf?id=CQsmMYmlP5T>.
- [3] Takuya Akiba, Makoto Shing, Yujin Tang, Qi Sun, and David Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence*, 2025. ISSN 2522-5839.
- [4] Duarte M. Alves, José Pombal, Nuno M. Guerreiro, Pedro H. Martins, João Alves, Amin Farajian, Ben Peters, Ricardo Rei, Patrick Fernandes, Sweta Agrawal, Pierre Colombo, José G. C. de Souza, and André F. T. Martins. Tower: An open multilingual large language model for translation-related tasks, 2024. URL <https://arxiv.org/abs/2402.17733>.
- [5] Xinming An and Yiu-Fai Yung. Item response theory: What it is and how you can use the IRT procedure to apply it. *SAS Institute Inc.*, 2014.
- [6] Sanjeev Arora, Rong Ge, Behnam Neyshabur, and Yi Zhang. Stronger generalization bounds for deep nets via a compression approach. In *Proc. of ICLR*. OpenReview.net, 2018. URL <https://openreview.net/forum?id=S1lDV3RcKm>.
- [7] Federica Arrigoni and Andrea Fusiello. Synchronization problems in computer vision with closed-form solutions. *International Journal of Computer Vision*, 128, 01 2020. doi: 10.1007/s11263-019-01240-x.
- [8] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *ArXiv preprint*, abs/1607.06450, 2016. URL <https://arxiv.org/abs/1607.06450>.
- [9] Peter L. Bartlett, Dylan J. Foster, and Matus J. Telgarsky. Spectrally-normalized margin bounds for neural networks. In *Proc. of NeurIPS*, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/b22b257ad0519d4500539571f698c4d8-Abstract.html>.
- [10] Lorenzo Basile, Valentino Maiorca, Luca Bortolussi, Emanuele Rodolà, and Francesco Locatello. Residual transformer alignment with spectral decomposition. *ArXiv preprint*, abs/2411.00246, 2024. URL <https://arxiv.org/abs/2411.00246>.
- [11] Joshua Belofsky. Token-level adaptation of lora adapters for downstream task generalization. In *Proceedings of the 2023 6th Artificial Intelligence and Cloud Computing Conference*, 2023.

- [12] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- [13] Gregory W. Benton, Wesley J. Maddox, Sanae Lotfi, and Andrew Gordon Wilson. Loss surface simplexes for mode connecting volumes and fast ensembling. In *Proc. of ICML*, volume 139 of *Proceedings of Machine Learning Research*. PMLR, 2021. URL <http://proceedings.mlr.press/v139/benton21a.html>.
- [14] Florian Bernard, Johan Thunberg, Paul Swoboda, and Christian Theobalt. Hipp: Higher-order projected power iterations for scalable multi-matching. In *2019 IEEE/CVF International Conference on Computer Vision, ICCV 2019, Seoul, Korea (South), October 27 - November 2, 2019*. IEEE, 2019. URL <https://doi.org/10.1109/ICCV.2019.01038>.
- [15] Florian Bernard, Daniel Cremers, and Johan Thunberg. Sparse quadratic optimisation over the stiefel manifold with application to permutation synchronisation. In *Proc. of NeurIPS*, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/d4bad256c73a6b25b86cc9c1a77255b1-Abstract.html>.
- [16] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchu Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Kwing Hei Li, Titouan Parcollet, Pedro Porto Buarque de Gusmão, et al. Flower: A friendly federated learning research framework. *ArXiv preprint*, abs/2007.14390, 2020. URL <https://arxiv.org/abs/2007.14390>.
- [17] Julian Blank and Kalyanmoy Deb. pymoo: Multi-objective optimization in Python. *IEEE Access*, 8:89497–89509, 2020. doi: 10.1109/ACCESS.2020.2990567.
- [18] Oliver Bolton, Arash Ahmadian, Sara Hooker, Marzieh Fadaee, Beyza Ermiş, et al. Simmerge: Learning to select merge operators from similarity signals. *arXiv preprint arXiv:2601.09473*, 2026.
- [19] Lukas Bossard, Matthieu Guillaumin, and Luc Van Gool. Food-101 – Mining Discriminative Components with Random Forests. In *Computer Vision – ECCV 2014*. Springer International Publishing, 2014.
- [20] Justyna Brzezinska. Item response theory models in the measurement theory. *Communications in Statistics – Simulation and Computation*, 2020.
- [21] Li Cai and Mark D. Reckase. *Item Response Theory*. Annual Review of Statistics and Its Application, 2016.
- [22] Feng Cheng, Ziyang Wang, Yi-Lin Sung, Yan-Bo Lin, Mohit Bansal, and Gedas Bertasius. Dam: Dynamic adapter merging for continual video qa learning. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*. IEEE Computer Society, 2025.
- [23] Gong Cheng, Junwei Han, and Xiaoqiang Lu. Remote Sensing Image Scene Classification: Benchmark and State of the Art. *Proceedings of the IEEE*, 105(10), 2017. ISSN 1558-2256. URL <https://ieeexplore.ieee.org/document/7891544/?arnumber=7891544>. Conference Name: Proceedings of the IEEE.
- [24] Alexandra Chronopoulou, Matthew Peters, Alexander Fraser, and Jesse Dodge. Adapter-Soup: Weight averaging to improve generalization of pretrained language models. In *Findings of the Association for Computational Linguistics: EACL 2023*. Association for

- Computational Linguistics, 2023. URL <https://aclanthology.org/2023.findings-eacl.153>.
- [25] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70), 2024.
- [26] Mircea Cimpoi, Subhransu Maji, Iasonas Kokkinos, Sammy Mohamed, and Andrea Vedaldi. Describing textures in the wild. In *2014 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2014, Columbus, OH, USA, June 23-28, 2014*. IEEE Computer Society, 2014. URL <https://doi.org/10.1109/CVPR.2014.461>.
- [27] Tarin Clanuwat, Mikel Bober-Irizar, Asanobu Kitamoto, Alex Lamb, Kazuaki Yamamoto, and David Ha. Deep Learning for Classical Japanese Literature, 2018. URL <https://arxiv.org/abs/1812.01718>.
- [28] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the AI2 reasoning challenge. *ArXiv preprint*, abs/1803.05457, 2018. URL <https://arxiv.org/abs/1803.05457>.
- [29] Adam Coates, Andrew Ng, and Honglak Lee. An Analysis of Single-Layer Networks in Unsupervised Feature Learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics. JMLR Workshop and Conference Proceedings*, 2011. URL <https://proceedings.mlr.press/v15/coates11a.html>. ISSN: 1938-7228.
- [30] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *ArXiv preprint*, abs/2110.14168, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [31] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. Emnist: an extension of mnist to handwritten letters, 2017.
- [32] Luca Cosmo, Emanuele Rodolà, Andrea Albarelli, Facundo Mémoli, and Daniel Cremers. Consistent partial matching of shape collections via sparse modeling. *Computer Graphics Forum*, 36(1), 2017.
- [33] Donato Crisostomi, Alessandro Zirilli, Antonio Andrea Gargiulo, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, Iacopo Masi, and Emanuele Rodolà. Mass: Merging through adaptive subspace selection. In *UniReps: 3rd Edition of the Workshop on Unifying Representations in Neural Models*.
- [34] Donato Crisostomi, Marco Fumero, Daniele Baieri, Florian Bernard, and Emanuele Rodolà. C²M³: Cycle-consistent multi-model merging. In *Advances in Neural Information Processing Systems*, volume 37, 2025.
- [35] Nico Daheim, Thomas Möllenhoff, Edoardo M. Ponti, Iryna Gurevych, and Mohammad Emtiyaz Khan. Model merging by uncertainty-based gradient matching. In *Proc. of ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=D7KJmfEDQP>.

- [36] Marczak Daniel, Magistri Simone, Cygert Sebastian, Twardowski Bartłomiej, D Bagdanov Andrew, and Joost van de Weijer. No task left behind: Isotropic model merging with common and task-specific subspaces. *ArXiv preprint*, 2025.
- [37] C Daniel Freeman and Joan Bruna. Topology and geometry of Half-Rectified network optimization. November 2016.
- [38] Dipankar Dasgupta and Zbigniew Michalewicz. *Evolutionary Algorithms in Engineering Applications*. Springer, 1997.
- [39] MohammadReza Davari and Eugene Belilovsky. Model breadcrumbs: Scaling multi-task model merging with sparse masks. In *European Conference on Computer Vision*. Springer, 2025.
- [40] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: Nsga-ii. *IEEE Transactions on Evolutionary Computation*, 6(2), 2002.
- [41] Kalyanmoy Deb, Karthik Sindhya, and Tatsuya Okabe. Self-adaptive simulated binary crossover for real-parameter optimization. In *Proceedings of the 9th Annual Conference on Genetic and Evolutionary Computation, GECCO '07*. Association for Computing Machinery, 2007. URL <https://doi.org/10.1145/1276958.1277190>.
- [42] Li Deng. The mnist database of handwritten digit images for machine learning research. *IEEE Signal Processing Magazine*, 29(6), 2012.
- [43] Emily L. Denton, Wojciech Zaremba, Joan Bruna, Yann LeCun, and Rob Fergus. Exploiting linear structure within convolutional networks for efficient evaluation. In *Proc. of NeurIPS*, 2014. URL <https://proceedings.neurips.cc/paper/2014/hash/2afe4567e1bf64d32a5527244d104cea-Abstract.html>.
- [44] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. of NAACL-HLT*. Association for Computational Linguistics, 2019. URL <https://aclanthology.org/N19-1423>.
- [45] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *Proc. of ICLR*. OpenReview.net, 2021. URL <https://openreview.net/forum?id=YicbFdNTTy>.
- [46] Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred A. Hamprecht. Essentially no barriers in neural network energy landscape. In *Proc. of ICML*, volume 80 of *Proceedings of Machine Learning Research*. PMLR, 2018. URL <http://proceedings.mlr.press/v80/draxler18a.html>.
- [47] Carl Eckart and G. Marion Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1, 1936. URL <https://api.semanticscholar.org/CorpusID:10163399>.
- [48] A.E. Eiben and J.E. Smith. *Introduction to Evolutionary Computing*. Springer, 2015.
- [49] David Eigen, Marc’Aurelio Ranzato, and Ilya Sutskever. Learning factored representations in a deep mixture of experts, 2014. URL <https://arxiv.org/abs/1312.4314>.

- [50] Rahim Entezari, Hanie Sedghi, Olga Saukh, and Behnam Neyshabur. The role of permutation invariance in linear mode connectivity of neural networks. In *Proc. of ICLR*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=dNigytemkL>.
- [51] Marguerite Frank and Philip Wolfe. An algorithm for quadratic programming. *Naval Research Logistics Quarterly*, 3(1-2), 1956.
- [52] Jonathan Frankle and Michael Carbin. The lottery ticket hypothesis: Finding sparse, trainable neural networks. In *International Conference on Learning Representations*.
- [53] Jonathan Frankle, Gintare Karolina Dziugaite, Daniel Roy, and Michael Carbin. Linear mode connectivity and the lottery ticket hypothesis. In *Proc. of ICML*, volume 119 of *Proceedings of Machine Learning Research*. PMLR, 2020. URL <http://proceedings.mlr.press/v119/frankle20a.html>.
- [54] Marco Fumero, Marco Pegoraro, Valentino Maiorca, Francesco Locatello, and Emanuele Rodolà. Latent functional maps: a spectral framework for representation alignment. In *Advances in Neural Information Processing Systems*, volume 37, 2024.
- [55] Yossi Gandelsman, Alexei A. Efros, and Jacob Steinhardt. Interpreting clip’s image representation via text-based decomposition. In *Proc. of ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=5Ca9sSzuDp>.
- [56] Rohit Gandikota, Joanna Materzynska, Timber Shi, and David Bau. Concept sliders: LoRA adaptors for precise control in diffusion models. In *Proc. of ECCV*, 2024.
- [57] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 12 2023. URL <https://zenodo.org/records/10256836>.
- [58] Antonio Andrea Gargiulo, Donato Crisostomi, Maria Sofia Bucarelli, Simone Scardapane, Fabrizio Silvestri, and Emanuele Rodolà. Task singular vectors: Reducing task interference in model merging. In *Proc. CVPR*, 2025.
- [59] Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry P. Vetrov, and Andrew Gordon Wilson. Loss surfaces, mode connectivity, and fast ensembling of dnns. In *Proc. of NeurIPS*, 2018. URL <https://proceedings.neurips.cc/paper/2018/hash/be3087e74e9100d4bc4c6268cdbc8456-Abstract.html>.
- [60] Matan Gavish and David L. Donoho. The optimal hard threshold for singular values is $4/\sqrt{3}$. *IEEE Transactions on Information Theory*, 60(8), 2014.
- [61] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vlad Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. Arcee’s MergeKit: A toolkit for merging large language models, 2024. URL <https://arxiv.org/abs/2403.13257>.
- [62] Noah Golowich, Alexander Rakhlin, and Ohad Shamir. Size-independent sample complexity of neural networks. In *Proc. of COLT*, volume 75 of *Proceedings of Machine Learning Research*. PMLR, 2018. URL <http://proceedings.mlr.press/v75/golowich18a.html>.

- [63] Ian J. Goodfellow, Dumitru Erhan, Pierre Luc Carrier, Aaron Courville, Mehdi Mirza, Ben Hamner, Will Cukierski, Yichuan Tang, David Thaler, Dong-Hyun Lee, Yingbo Zhou, Chetan Ramaiah, Fangxiang Feng, Ruifan Li, Xiaojie Wang, Dimitris Athanasakis, John Shawe-Taylor, Maxim Milakov, John Park, Radu Ionescu, Marius Popescu, Cristian Grozea, James Bergstra, Jingjing Xie, Lukasz Romaszko, Bing Xu, Zhang Chuang, and Yoshua Bengio. Challenges in Representation Learning: A Report on Three Machine Learning Contests. In *Neural Information Processing*. Springer, 2013.
- [64] Ian J Goodfellow, Oriol Vinyals, and Andrew M Saxe. Qualitatively characterizing neural network optimization problems. December 2014.
- [65] Fidel A. Guerrero-Peña, Heitor Rapela Medeiros, Thomas Dubail, Masih Aminbeidokhti, Eric Granger, and Marco Pedersoli. Re-basin via implicit sinkhorn differentiation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023*. IEEE, 2023. URL <https://doi.org/10.1109/CVPR52729.2023.01938>.
- [66] Aapo Hämmäläinen et al. Diffusion soup: Model merging for text-to-image diffusion models. In *Proc. of ECCV*, 2024.
- [67] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2016, Las Vegas, NV, USA, June 27-30, 2016*. IEEE Computer Society, 2016. URL <https://doi.org/10.1109/CVPR.2016.90>.
- [68] Shwai He, Run-Ze Fan, Liang Ding, Li Shen, Tianyi Zhou, and Dacheng Tao. Merging experts into one: Improving computational efficiency of mixture of experts. In *Proc. of EMNLP*. Association for Computational Linguistics, 2023. URL <https://aclanthology.org/2023.emnlp-main.907>.
- [69] Patrick Helber, Benjamin Bischke, Andreas Dengel, and Damian Borth. EuroSAT: A Novel Dataset and Deep Learning Benchmark for Land Use and Land Cover Classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 12(7), 2019. ISSN 2151-1535. URL <https://ieeexplore.ieee.org/document/8736785/?arnumber=8736785>. Conference Name: IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing.
- [70] Stefan Horoi, Albert Manuel Orozco Camacho, Eugene Belilovsky, and Guy Wolf. Harmony in diversity: Merging neural networks with canonical correlation analysis. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=hLuNVjRnY3>.
- [71] Stefan Horoi, Guy Wolf, and Eugene Belilovsky. Less is more: On the role of redundancy in model merging, 2025. URL <https://arxiv.org/abs/2502.05868>.
- [72] Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *International Conference on Learning Representations*.
- [73] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. In *Proc. of ICLR*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.

- [74] Chenyu Huang, Peng Ye, Tao Chen, Tong He, Xiangyu Yue, and Wanli Ouyang. Emrmerging: Tuning-free high-performance model merging. In *Proc. of NeurIPS*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/dda5cac5272a9bcd4bc73d90bc725ef1-Abstract-Conference.html.
- [75] Qixing Huang, Fan Wang, and Leonidas Guibas. Functional map networks for analyzing and exploring large shape collections. *ACM Trans. Graph.*, 33(4), jul 2014. ISSN 0730-0301. doi: 10.1145/2601097.2601111. URL <https://doi.org/10.1145/2601097.2601111>.
- [76] Yerlan Idelbayev and Miguel Á. Carreira-Perpiñán. Low-rank compression of neural nets: Learning the rank of each layer. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*. IEEE, 2020. URL <https://doi.org/10.1109/CVPR42600.2020.00807>.
- [77] Gabriel Ilharco, Mitchell Wortsman, Samir Yitzhak Gadre, Shuran Song, Hannaneh Hajishirzi, Simon Kornblith, Ali Farhadi, and Ludwig Schmidt. Patching open-vocabulary models by interpolating weights. *Advances in Neural Information Processing Systems*, 35:29262–29277, 2022.
- [78] Gabriel Ilharco, Marco Túlio Ribeiro, Mitchell Wortsman, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *Proc. of ICLR*. OpenReview.net, 2023. URL <https://openreview.net/pdf?id=6t0Kwf8-jrj>.
- [79] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proc. of ICML*, volume 37 of *JMLR Workshop and Conference Proceedings*. JMLR.org, 2015. URL <http://proceedings.mlr.press/v37/ioffe15.html>.
- [80] Dong-Hwan Jang, Sangdoo Yun, and Dongyoon Han. Model Stock: All we need is just a few fine-tuned models, 2024.
- [81] Joel Jang, Seungone Kim, Seonghyeon Ye, Doyoung Kim, Lajanugen Logeswaran, Moontae Lee, Kyungjae Lee, and Minjoon Seo. Exploring the benefits of training expert language models over instruction tuning. In *Proc. of ICML*, volume 202 of *Proceedings of Machine Learning Research*. PMLR, 2023. URL <https://proceedings.mlr.press/v202/jang23a.html>.
- [82] A.Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D.S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L.R. Lavaud, M.-A. Lachaux, P. Stock, T. Le Scao, T. Lavril, T. Wang, T. Lacroix, and W. El Sayed. Mistral 7b. *ArXiv preprint*, abs/2310.06825, 2023. URL <https://arxiv.org/abs/2310.06825>.
- [83] Xisen Jin, Xiang Ren, Daniel Preotiuc-Pietro, and Pengxiang Cheng. Dataless knowledge fusion by merging weights of language models. In *Proc. of ICLR*. OpenReview.net, 2023. URL <https://openreview.net/pdf?id=FCnohuR6AnM>.
- [84] Keller Jordan. Muon: An optimizer for hidden layers in neural networks. <https://github.com/KellerJordan/Muon>, 2024.
- [85] Keller Jordan, Hanie Sedghi, Olga Saukh, Rahim Entezari, and Behnam Neyshabur. REPAIR: renormalizing permuted activations for interpolation repair. In *Proc. of ICLR*. OpenReview.net, 2023. URL <https://openreview.net/pdf?id=gU5sJ6ZggcX>.

- [86] Armand Joulin, Edouard Grave, Piotr Bojanowski, and Tomas Mikolov. Bag of tricks for efficient text classification. In *Proc. of EACL*, volume 2, pages 427–431. Association for Computational Linguistics, 2017. URL <https://aclanthology.org/E17-2068/>.
- [87] Ioannis Kalogeropoulos, Giorgos Bouritsas, and Yannis Panagakis. Scale equivariant graph metanetworks. *Advances in neural information processing systems*, 37:106800–106840, 2024.
- [88] Nikhil Kandpal, Brian Lester, Mohammed Muqeeth, Anisha Mascarenhas, Monty Evans, Vishal Baskaran, Tenghao Huang, Haokun Liu, and Colin Raffel. Git-Theta: A Git extension for collaborative development of machine learning models. In *Proc. of ICML*, 2023.
- [89] Neal M. Kingston and Gail H. Dorans. The feasibility of using item response theory as a psychometric model for the GRE aptitude test. *ETS Research Report Series*, 1982.
- [90] Miltiadis Kofinas, Boris Knyazev, Yan Zhang, Yunlu Chen, Gertjan J Burghouts, Efstratios Gavves, Cees GM Snoek, and David W Zhang. Graph neural networks for learning equivariant representations of neural networks. In *The Twelfth International Conference on Learning Representations*.
- [91] Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey E. Hinton. Similarity of neural network representations revisited. In *Proc. of ICML*, volume 97 of *Proceedings of Machine Learning Research*. PMLR, 2019. URL <http://proceedings.mlr.press/v97/kornblith19a.html>.
- [92] Jonathan Krause, Michael Stark, Jia Deng, and Li Fei-Fei. 3D Object Representations for Fine-Grained Categorization. In *2013 IEEE International Conference on Computer Vision Workshops*. IEEE, 2013. URL <http://ieeexplore.ieee.org/document/6755945/>.
- [93] Alex Krizhevsky and Geoffrey Hinton. Learning multiple layers of features from tiny images. Technical Report 0, University of Toronto, 2009. URL <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [94] Simon Lacoste-Julien. Convergence rate of frank-wolfe for non-convex objectives. *ArXiv preprint*, abs/1607.00345, 2016. URL <https://arxiv.org/abs/1607.00345>.
- [95] John P. Lalor, Hao Wu, and Hong Yu. Building an evaluation scale using item response theory. In *Proc. of NAACL-HLT*, 2016.
- [96] John Patrick Lalor and Pedro Rodriguez. py-irt: A scalable item response theory library for python. *INFORMS Journal on Computing*, 35(1), 2023.
- [97] Y. Lecun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 1998.
- [98] Weishi Li, Yong Peng, Miao Zhang, Liang Ding, Han Hu, and Li Shen. Deep model fusion: A survey, 2023.
- [99] Yunshui Li, Yiyuan Ma, Shen Yan, Chaoyi Zhang, Jing Liu, Jianqiao Lu, Ziwen Xu, Mengzhao Chen, Minrui Wang, Shiyi Zhan, et al. Model merging in pre-training of large language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

- [100] Derek Lim, Haggai Maron, Marc T Law, Jonathan Lorraine, and James Lucas. Graph metanetworks for processing diverse neural architectures. In *The Twelfth International Conference on Learning Representations*.
- [101] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proc. of ACL*. Association for Computational Linguistics, 2022. URL <https://aclanthology.org/2022.acl-long.229/>.
- [102] Kaizhao Liu, Bo Liu, Qiang Liu, et al. Muon is scalable for LLM training. *ArXiv preprint*, abs/2502.16982, 2025.
- [103] Frederic M. Lord and Melvin R. Novick. *Statistical Theories of Mental Test Scores*. Addison-Wesley, 1968.
- [104] Zhenyi Lu, Chenghao Fan, Wei Wei, Xiaoye Qu, Dangyang Chen, and Yu Cheng. Twin-merging: Dynamic integration of modular expertise in model merging. In *Proc. of NeurIPS*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/8fcd17eb91bae20d9826786d7d6be799-Abstract-Conference.html.
- [105] Ekdeep Singh Lubana, Eric J. Bigelow, Robert P. Dick, David Scott Krueger, and Hidenori Tanaka. Mechanistic mode connectivity. In *Proc. of ICML*, volume 202 of *Proceedings of Machine Learning Research*. PMLR, 2023. URL <https://proceedings.mlr.press/v202/lubana23a.html>.
- [106] Davide Marincione, Donato Crisostomi, Roberto Dessi, Emanuele Rodolà, and Emanuele Rossi. Model merging improves zero-shot generalization in bioacoustic foundation models. In *The Thirty-Ninth Annual Conference on Neural Information Processing Systems workshop: AI for non-human animal communication*.
- [107] Michael Matena and Colin Raffel. Merging models with fisher-weighted averaging. In *Proc. of NeurIPS*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/70c26937fbf3d4600b69a129031b66ec-Abstract-Conference.html.
- [108] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*, pages 1273–1282. Pmlr, 2017.
- [109] Tommaso Mencattini, Adrian Robert Minut, Donato Crisostomi, Andrea Santilli, and Emanuele Rodolà. Merge³: Efficient evolutionary merging on consumer-grade gpus. *arXiv preprint arXiv:2502.10436*, 2025.
- [110] Benjamin Minixhofer, Edoardo M Ponti, and Ivan Vulić. Zero-shot tokenizer transfer. *Advances in Neural Information Processing Systems*, 37:46791–46818, 2024.
- [111] Adrian Robert Minut, Tommaso Mencattini, Andrea Santilli, Donato Crisostomi, and Emanuele Rodolà. Mergenetic: a simple evolutionary model merging library. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 572–582, 2025.
- [112] Adrian Robert Minut, Tommaso Mencattini, Andrea Santilli, Donato Crisostomi, and Emanuele Rodolà. Mergenetic: a simple evolutionary model merging library, 2025. URL <https://arxiv.org/abs/2505.11427>.

- [113] Seyed-Iman Mirzadeh, Mehrdad Farajtabar, Dilan Görür, Razvan Pascanu, and Hassan Ghasemzadeh. Linear mode connectivity in multitask and continual learning. In *Proc. of ICLR*. OpenReview.net, 2021. URL https://openreview.net/forum?id=Fmg_fQYUejf.
- [114] Mohammed Muqeeth, Haokun Liu, and Colin Raffel. Soft merging of experts with adaptive routing. *Transactions on Machine Learning Research*.
- [115] Mohammed Muqeeth, Haokun Liu, Yufan Liu, and Colin Raffel. Learning to route among specialized experts for zero-shot generalization. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=r0qcGcFL4U>.
- [116] Aviv Navon, Aviv Shamsian, Ethan Fetaya, Gal Chechik, Nadav Dym, and Haggai Maron. Equivariant deep weight space alignment. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=nBPnmk6EeO>.
- [117] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y. Ng. Reading digits in natural images with unsupervised feature learning. In *NIPS Workshop on Deep Learning and Unsupervised Feature Learning 2011*, 2011. URL http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf.
- [118] Behnam Neyshabur, Hanie Sedghi, and Chiyuan Zhang. What is being transferred in transfer learning? *Advances in neural information processing systems*, 33:512–523, 2020.
- [119] Dang Nguyen, Trang Nguyen, Khai Nguyen, Dinh Q. Phung, Hung Hai Bui, and Nhat Ho. On cross-layer alignment for model fusion of heterogeneous neural networks. In *IEEE International Conference on Acoustics, Speech and Signal Processing ICASSP 2023, Rhodes Island, Greece, June 4-10, 2023*. IEEE, 2023. URL <https://doi.org/10.1109/ICASSP49357.2023.10097156>.
- [120] Konstantinos E. Nikolakakis, Faisal Alam, Karthik Nandakumar, and Dionysios S. Kalogerias. Beyond linear subspace operations for efficient multi-task model merging, 2025. URL <https://arxiv.org/abs/2502.05891>.
- [121] Maria-Elena Nilsback and Andrew Zisserman. Automated Flower Classification over a Large Number of Classes. In *2008 Sixth Indian Conference on Computer Vision, Graphics & Image Processing*, 2008. URL <https://ieeexplore.ieee.org/stampPDF/getPDF.jsp?tp=&arnumber=4756141&ref=aHR0cHM6Ly9pZWVleHBsb3JlLm1lZWUub3JnL2Fic3RyYWN0L2RvY3VtZW50LzQ3NTYxNDY2FzYV90b2t1b3JlFOU44YkVRRWw0TUFBQUFBOLZtNDYzMdK0dFNBOXNkbnpemZNbkYtSE>
- [122] Guillermo Ortiz-Jiménez, Alessandro Favero, and Pascal Frossard. Task arithmetic in the tangent space: Improved editing of pre-trained models. In *Proc. of NeurIPS*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/d28077e5ff52034cd35b4aa15320caea-Abstract-Conference.html.
- [123] Oleksiy Ostapenko, Zhan Su, Edoardo M. Ponti, Laurent Charlin, Nicolas Le Roux, Lucas Caccia, and Alessandro Sordoni. Towards modular llms by building and reusing a library of loras. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=0ZFWfeVsaD>.

- [124] Maks Ovsjanikov, Mirela Ben-Chen, Justin Solomon, Adrian Butscher, and Leonidas Guibas. Functional maps: A flexible representation of maps between shapes. *ACM Trans. Graph.*, 31(4), 2012.
- [125] Deepti Pachauri, Risi Kondor, and Vikas Singh. Solving the multi-way matching problem by permutation synchronization. In C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/3df1d4b96d8976ff5986393e8767f5b2-Paper.pdf.
- [126] Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman, and C. V. Jawahar. Cats and dogs. In *2012 IEEE Conference on Computer Vision and Pattern Recognition, Providence, RI, USA, June 16-21, 2012*. IEEE Computer Society, 2012. URL <https://doi.org/10.1109/CVPR.2012.6248092>.
- [127] Nancy S. Petersen, Marco D. Kolen, and H. D. Hoover. Scaling, norming and equating. *Educational Measurement: Issues and Practice*, 1982.
- [128] Alain Petrowski and Sana Ben-Hamida. *Evolutionary Algorithms*. John Wiley & Sons, 2017.
- [129] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinybenchmarks: evaluating llms with fewer examples. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=qAml3FpfhG>.
- [130] B. T. Polyak and A. B. Juditsky. Acceleration of stochastic approximation by averaging. *SIAM Journal on Control and Optimization*, 30(4):838–855, 1992. doi: 10.1137/0330046. URL <https://doi.org/10.1137/0330046>.
- [131] Xingyu Qu and Samuel Horvath. Rethinking model re-basin and linear mode connectivity, 2024. URL <https://arxiv.org/abs/2402.05966>.
- [132] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proc. of ICML*, volume 139 of *Proceedings of Machine Learning Research*. PMLR, 2021. URL <http://proceedings.mlr.press/v139/radford21a.html>.
- [133] Adir Rahamim, Asaf Yehudai, Boaz Carmeli, Leshem Choshen, Yosi Mass, and Yonatan Belinkov. Will it merge? on the causes of model mergeability. *arXiv preprint arXiv:2601.06672*, 2026.
- [134] Alexandre Ramé, Matthieu Kirchmeyer, Thibaud Rahier, Alain Rakotomamonjy, Patrick Gallinari, and Matthieu Cord. Diverse weight averaging for out-of-distribution generalization. In *Proc. of NeurIPS*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/46108d807b50ad4144eb353b5d0e8851-Abstract-Conference.html.
- [135] Filippo Rinaldi, Giacomo Capitani, Lorenzo Bonicelli, Donato Crisostomi, Federico Bolelli, ELISA FICARRA, Emanuele Rodolà, Simone Calderara, and Angelo Porrello. Update your transformer to the latest release: Re-basin of task vectors. In *Forty-second International Conference on Machine Learning*.

- [136] Filippo Rinaldi, Aniello Panariello, Giacomo Salici, Fengyuan Liu, Marco Ciccone, Angelo Porrello, and Simone Calderara. Gradient-sign masking for task vector transport across pre-trained models, 2025. URL <https://arxiv.org/abs/2510.09658>.
- [137] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proc. of AAAI*, volume 34, pages 8732–8740. AAAI Press, 2020. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6399>.
- [138] Sunny Sanyal, Atula Tejaswi Neerkaje, Jean Kaddour, Abhishek Kumar, et al. Early weight averaging meets high learning rates for llm pre-training. In *First Conference on Language Modeling*.
- [139] Konstantin Schürholt, Giorgos Bouritsas, Eliahu Horwitz, Derek Lim, Yoav Gelberg, Bo Zhao, Allan Zhou, Damian Borth, and Stefanie Jegelka. Neural network weights as a new data modality. In *ICLR 2025 Workshop Proposals*, 2024. URL <https://openreview.net/forum?id=Bz6wEdobY7>.
- [140] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc V. Le, Geoffrey E. Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *Proc. of ICLR*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=BlckMDqlg>.
- [141] Xinchu Shi, Haibin Ling, Weiming Hu, Junliang Xing, and Yanning Zhang. Tensor power iteration for multi-graph matching. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5062–5070, 2016. doi: 10.1109/CVPR.2016.547.
- [142] Ken Shoemake. Animating rotation with quaternion curves. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '85)*, pages 245–254. ACM, 1985. doi: 10.1145/325165.325242. URL <https://doi.org/10.1145/325165.325242>.
- [143] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *Proc. of ICLR*, 2015. URL <http://arxiv.org/abs/1409.1556>.
- [144] Sidak Pal Singh and Martin Jaggi. Model fusion via optimal transport. In *Proc. of NeurIPS*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/fb2697869f56484404c8ceee2985b01d-Abstract.html>.
- [145] Richard Sinkhorn and Paul Knopp. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21, 1967. URL <https://api.semanticscholar.org/CorpusID:50329347>.
- [146] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proc. of EMNLP*. Association for Computational Linguistics, 2013. URL <https://aclanthology.org/D13-1170>.
- [147] Rishi Sonthalia, Michael Murray, and Guido Montufar. Low rank gradients and where to find them. In *High-dimensional Learning Dynamics 2025*.

- [148] Johannes Stallkamp, Marc Schlipsing, Jan Salmen, and Christian Igel. The German Traffic Sign Recognition Benchmark: A multi-class classification competition. In *The 2011 International Joint Conference on Neural Networks*, 2011. URL <https://ieeexplore.ieee.org/document/6033395/?arnumber=6033395>. ISSN: 2161-4407.
- [149] George Stoica, Daniel Bolya, Jakob Bjorner, Pratik Ramesh, Taylor Hearn, and Judy Hoffman. Zipit! merging models from different tasks without training. In *Proc. of ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=LEYUkvDUhQ>.
- [150] George Stoica, Pratik Ramesh, Boglarka Ecsedi, Leshem Choshen, and Judy Hoffman. Model merging with svd to tie the knots. In *Proc. of ICLR*, 2024.
- [151] Qi Sun, Edoardo Cetin, and Yujin Tang. Transformer-squared: Self-adaptive llms. In *Proc. of ICLR*, 2025.
- [152] Paul Swoboda, Dagmar Kainmüller, Ashkan Mokarian, Christian Theobalt, and Florian Bernard. A convex relaxation for multi-graph matching. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 11148–11157, 2019. doi: 10.1109/CVPR.2019.01141.
- [153] Derek Tam, Mohit Bansal, and Colin Raffel. Merging by matching models in task parameter subspaces. *Transactions on Machine Learning Research*.
- [154] Chuanqi Tan, Fuchun Sun, Tao Kong, Wenchang Zhang, Chao Yang, and Chunfang Liu. A survey on deep transfer learning. In *Artificial Neural Networks and Machine Learning–ICANN 2018: 27th International Conference on Artificial Neural Networks, Rhodes, Greece, October 4-7, 2018, Proceedings, Part III* 27. Springer, 2018.
- [155] Anke Tang, Li Shen, Yong Luo, Shuai Xie, Han Hu, Lefei Zhang, Bo Du, and Dacheng Tao. Smile: Zero-shot sparse mixture of low-rank experts construction from pre-trained foundation models. *ArXiv preprint*, abs/2408.10174, 2024. URL <https://arxiv.org/abs/2408.10174>.
- [156] Anke Tang, Li Shen, Yong Luo, Nan Yin, Lefei Zhang, and Dacheng Tao. Merging multi-task models via weight-ensembling mixture of experts. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=nLRKnO74RB>.
- [157] Anke Tang, Li Shen, Yong Luo, Yibing Zhan, Han Hu, Bo Du, Yixin Chen, and Dacheng Tao. Parameter-efficient multi-task model fusion with partial linearization. In *Proc. of ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=iynRvVVAH>.
- [158] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. *Advances in neural information processing systems*, 30, 2017.
- [159] Klaudia Thellmann, Bernhard Stadler, Michael Fromm, Jasper Schulze Buschhoff, Alex Jude, Fabio Barth, Johannes Leveling, Nicolas Flores-Herr, Joachim Köhler, René Jäkel, and Mehdi Ali. Towards cross-lingual llm evaluation for european languages, 2024.
- [160] Changxin Tian, Jiapeng Wang, Qian Zhao, Kunlong Chen, Jia Liu, Ziqi Liu, Jiaxin Mao, Wayne Xin Zhao, Zhiqiang Zhang, and Jun Zhou. Wsm: decay-free learning rate schedule via checkpoint merging for llm pre-training. *ICLR 2026*, 2026.

- [161] Michael E Tipping and Christopher M Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 61(3), 1999.
- [162] Wim J. van der Linden. *Handbook of Item Response Theory*. Chapman and Hall/CRC, 3rd edition, 2018.
- [163] Clara Vania, Phu Mon Htut, William Huang, Dhara Mungra, Richard Yuanzhe Pang, Jason Phang, Haokun Liu, Kyunghyun Cho, and Samuel R. Bowman. Comparing test sets with item response theory. In *Proc. of EMNLP*, 2021.
- [164] Bastiaan S. Veeling, Jasper Linmans, Jim Winkens, Taco Cohen, and Max Welling. Rotation Equivariant CNNs for Digital Pathology. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018*. Springer International Publishing, 2018.
- [165] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *Proc. of ICLR*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=rJ4km2R5t7>.
- [166] Ke Wang, Nikolaos Dimitriadis, Guillermo Ortiz-Jiménez, François Fleuret, and Pascal Frossard. Localizing task information for improved model merging and compression. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=DWT9uiGjxT>.
- [167] Lanhui Wang and Amit Singer. Exact and stable recovery of rotations for robust synchronization. *Information and Inference: A Journal of the IMA*, 2(2):145–193, 2013.
- [168] Lirui Wang, Kaiqing Zhang, Allan Zhou, Max Simchowitz, and Russ Tedrake. Fleet policy learning via weight merging and an application to robotic tool-use, 2023.
- [169] Runzhong Wang, Junchi Yan, and Xiaokang Yang. Neural graph matching network: Learning lawler’s quadratic assignment problem with extension to hypergraph and multiple-graph matching. *ArXiv preprint*, abs/1911.11308, 2019. URL <https://arxiv.org/abs/1911.11308>.
- [170] B. P. Welford. Note on a method for calculating corrected sums of squares and products. *Technometrics*, 4(3), 1962. URL <https://www.tandfonline.com/doi/abs/10.1080/00401706.1962.10490022>.
- [171] Colin White, Arber Zela, Robin Ru, Yang Liu, and Frank Hutter. How powerful are performance predictors in neural architecture search? In *Proc. of NeurIPS*, 2021. URL <https://proceedings.neurips.cc/paper/2021/hash/ef575e8837d065a1683c022d2077d342-Abstract.html>.
- [172] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, Ali Farhadi, Hongseok Namkoong, and Ludwig Schmidt. Robust fine-tuning of zero-shot models, 2021.
- [173] Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *Proc.*

- of *ICML*, volume 162 of *Proceedings of Machine Learning Research*. PMLR, 2022. URL <https://proceedings.mlr.press/v162/wortsman22a.html>.
- [174] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms, 2017. URL <https://arxiv.org/abs/1708.07747>.
- [175] Jianxiong Xiao, Krista A. Ehinger, James Hays, Antonio Torralba, and Aude Oliva. SUN Database: Exploring a Large Collection of Scene Categories. *International Journal of Computer Vision*, 119(1), 2016. ISSN 1573-1405. URL <https://doi.org/10.1007/s11263-014-0748-y>.
- [176] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A. Raffel, and Mohit Bansal. Ties-merging: Resolving interference when merging models. In *Proc. of NeurIPS*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/1644c9af28ab7916874f6fd6228a9bcf-Abstract-Conference.html.
- [177] Prateek Yadav, Colin Raffel, Mohammed Muqeeth, Lucas Caccia, Haokun Liu, Tianlong Chen, Mohit Bansal, Leshem Choshen, and Alessandro Sordoni. A survey on model moerging: Recycling and routing among specialized experts for collaborative learning, 2024.
- [178] Yajat Yadav, Zhiyuan Zhou, Andrew Wagenmaker, Karl Pertsch, and Sergey Levine. Robust finetuning of vision-language-action robot policies via parameter merging, 2025. URL <https://arxiv.org/abs/2512.08333>.
- [179] Enneng Yang, Li Shen, Zhenyi Wang, Guibing Guo, Xiaojun Chen, Xingwei Wang, and Dacheng Tao. Representation surgery for multi-task model merging. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=Sbl2keQEML>.
- [180] Enneng Yang, Zhenyi Wang, Li Shen, Shiwei Liu, Guibing Guo, Xingwei Wang, and Dacheng Tao. Adamerging: Adaptive model merging for multi-task learning. In *Proc. of ICLR*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=nZP6NgD3QY>.
- [181] Dmitry Yarotsky. Error bounds for approximations with deep ReLU networks. *Neural Networks*, 94:103–114, 2017.
- [182] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? In *Proc. of NeurIPS*, 2014. URL <https://proceedings.neurips.cc/paper/2014/hash/375c71349b295fbe2dcdca9206f20a06-Abstract.html>.
- [183] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Proc. of ICML*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=fq0NaiU8Ex>.
- [184] David Yunis, Kumar Kshitij Patel, Pedro Henrique Pamplona Savarese, Gal Vardi, Jonathan Frankle, Matthew Walter, Karen Livescu, and Michael Maire. On convexity and linear mode connectivity in neural networks. In *OPT 2022: Optimization for Machine Learning (NeurIPS 2022 Workshop)*, 2022. URL <https://openreview.net/forum?id=TZQ3PKL3fPr>.

- [185] Christopher Zach, Manfred Klopschitz, and Marc Pollefeys. Disambiguating visual relations using loop constraints. In *The Twenty-Third IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2010, San Francisco, CA, USA, 13-18 June 2010*. IEEE Computer Society, 2010. URL <https://doi.org/10.1109/CVPR.2010.5539801>.
- [186] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proc. of ACL*, pages 4791–4800. Association for Computational Linguistics, 2019. URL <https://aclanthology.org/P19-1472/>.
- [187] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. Galore: Memory-efficient llm training by gradient low-rank projection. In *Forty-first International Conference on Machine Learning*.
- [188] Ziyu Zhao, Leilei Gan, Guoyin Wang, Wangchunshu Zhou, Hongxia Yang, Kun Kuang, and Fei Wu. LoraRetriever: Input-aware LoRA retrieval and composition for mixed tasks in the wild. In *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024. URL <https://aclanthology.org/2024.findings-acl.263/>.
- [189] Allan Zhou, Kaien Yang, Kaylee Burns, Adriano Cardace, Yiding Jiang, Samuel Sokota, J. Zico Kolter, and Chelsea Finn. Permutation equivariant neural functionals. In *Proc. of NeurIPS*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/4e9d8aeeab6120c3c83ccf95d4c211d3-Abstract-Conference.html.
- [190] Luca Zhou, Daniele Solombrino, Donato Crisostomi, Maria Sofia Bucarelli, Giuseppe Alessio D’Inverno, Fabrizio Silvestri, and Emanuele Rodolà. On task vectors and gradients. In *UniReps: 3rd Edition of the Workshop on Unifying Representations in Neural Models*.
- [191] Luca Zhou, Daniele Solombrino, Donato Crisostomi, Maria Sofia Bucarelli, Fabrizio Silvestri, and Emanuele Rodolà. Atm: Improving model merging by alternating tuning and merging, 2024. URL <https://arxiv.org/abs/2411.03055>.
- [192] Luca Zhou, Bo Zhao, Rose Yu, and Emanuele Rodolà. Demystifying mergeability: Interpretable properties to predict model merging success. *arXiv preprint arXiv:2601.22285*, 2026.
- [193] Zhanpeng Zhou, Yongyi Yang, Xiaojiang Yang, Junchi Yan, and Wei Hu. Going beyond linear mode connectivity: The layerwise linear feature connectivity. In *Proc. of NeurIPS*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/bf3ee5a5422b0e2a88b0c9c6ed3b6144-Abstract-Conference.html.
- [194] Yufan Zhuang, Liam Li, Eric Xing, and Vincent Liu. Efficiently measuring the cognitive ability of LLMs: An adaptive testing perspective. In *Proc. of NeurIPS*, 2023.